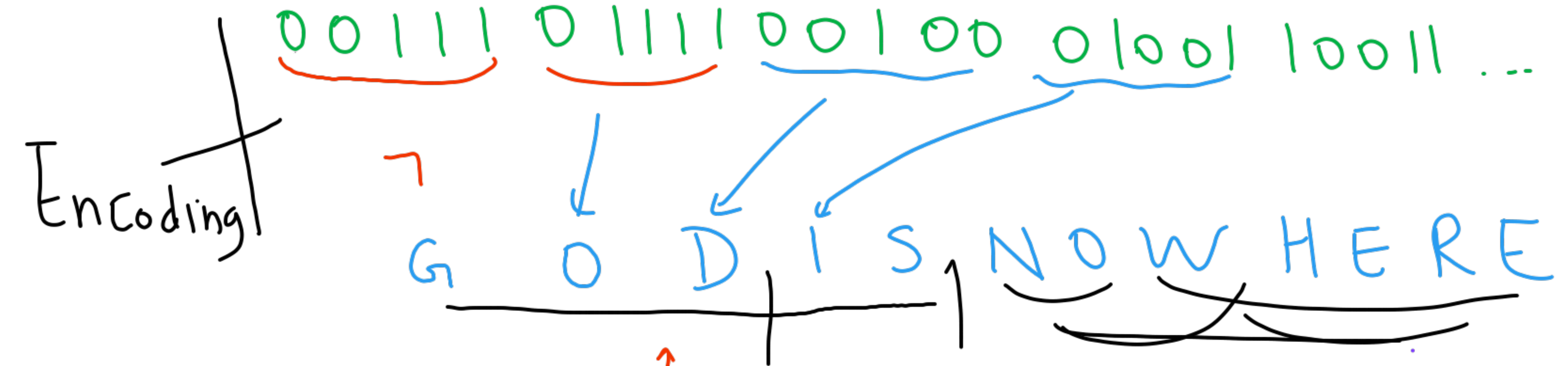


30.8.2025

Bits / Alphabet

CS 310

→ G. Sivakumar
Siva@iitb.ac.in



EnCoding

"stand on the shoulders of giants"

Natural Language
Formal → we define the Syntax, Semantics

AMBIGUITY

3 models
 Finite State Automata FSA
 Push Down Turing machine PDA TM
 What



Science & Engineering

Turing test

Abacus

Slide rule

Casio FX

mobile phone

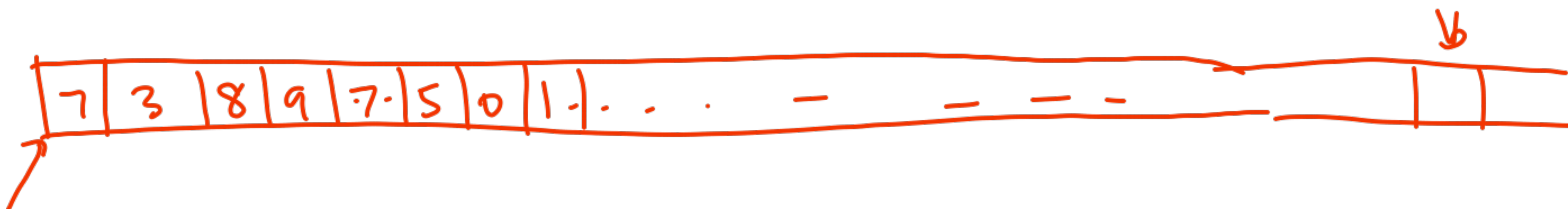
·
·
·
·

Hilbert's 10th problem $3x^2 - 5x + 7 = 0$

→ $7x^3y^2 - 9x^2y^3 + 13xyz$
 $+ \dots = 0$

undecidable

Input Type



Is it
→ divisible by 7?

→ Give a solution (diagram)
that any 10-year old
can use to answer
in "real-time"



A hand-drawn green box containing the text "FSA". A green arrow points from the left towards the box.

PDA

Input Tape

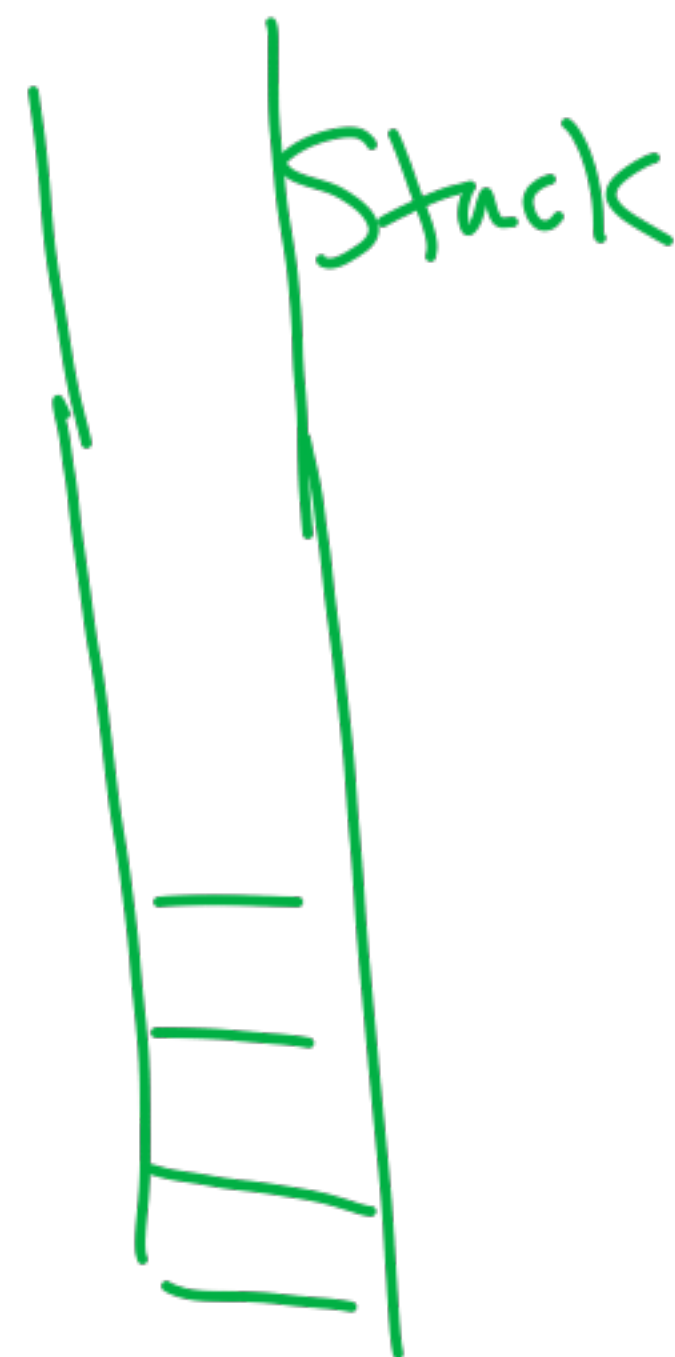
Is the string of the form $7^n 9^n$?

$n \geq 1$



Is it
a palindrome?

→ Non-determinism
(guess)



Turing machine

turingmachinesimulator.io

Input
tape

Read/write



① replace
a digit by some symbol.

② move 1-step L or R q_k

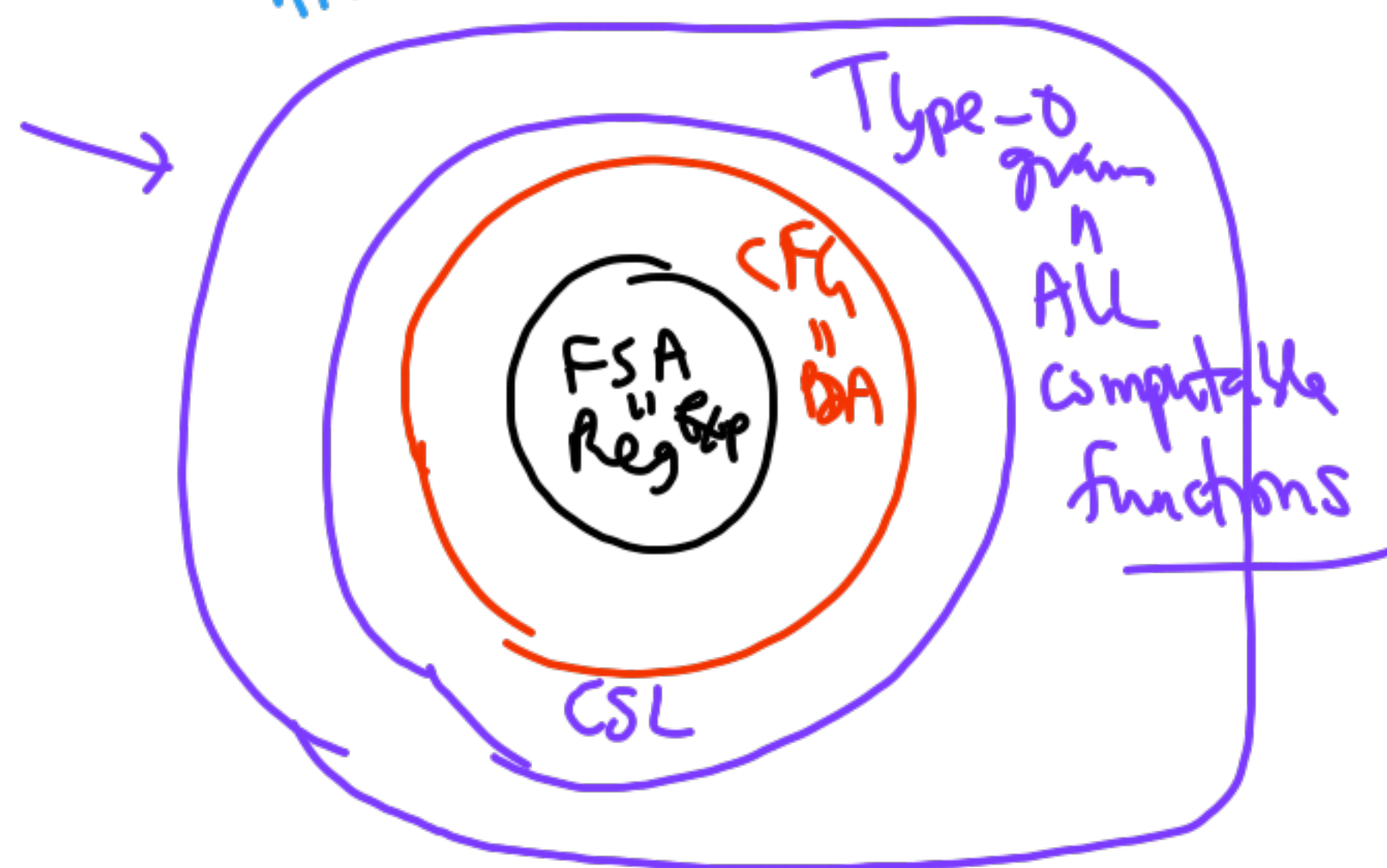
Is n prime?

Chomsky
hierarchy

Machines	Language
FSA	Regular Expression
PDA	Context free grammar
TM	Type 0 grammar

$(0+00)^*1^*$

$S \rightarrow aSb$
 $S \rightarrow ab$
 ~~$B \rightarrow cb$~~



lhs
Can have
any
string

$a^n b^n c^n$
 $aSc \rightarrow Sbc$

a^n
 S^n

CS 310: Automata Theory

Krishna. S

DFA Equivalence and Minimization

- ▶ Let $A = (Q, \Sigma, \delta, q_0, F)$ be a DFA
- ▶ Let $L(A, q) = \{w \in \Sigma^* \mid \hat{\delta}(q, w) \in F\}$ (recall that $\hat{\delta}$ is the extended transition function, $\hat{\delta} : Q \times \Sigma^* \rightarrow Q$)
- ▶ $L(A) = L(A, q_0)$

DFA Equivalence and Minimization

- ▶ Let $A = (Q, \Sigma, \delta, q_0, F)$ be a DFA
- ▶ Let $L(A, q) = \{w \in \Sigma^* \mid \hat{\delta}(q, w) \in F\}$ (recall that $\hat{\delta}$ is the extended transition function, $\hat{\delta} : Q \times \Sigma^* \rightarrow Q$)
- ▶ $L(A) = L(A, q_0)$
- ▶ Two states q_1, q_2 in A are equivalent if $L(A, q_1) = L(A, q_2)$. A state q_1 in DFA A_1 is equivalent to state q_2 in DFA A_2 if $L(A_1, q_1) = L(A_2, q_2)$.
- ▶ Two DFAs A_1, A_2 are equivalent if $L(A_1) = L(A_2)$

DFA Equivalence and Minimization

- ▶ Let $A = (Q, \Sigma, \delta, q_0, F)$ be a DFA
- ▶ Let $L(A, q) = \{w \in \Sigma^* \mid \hat{\delta}(q, w) \in F\}$ (recall that $\hat{\delta}$ is the extended transition function, $\hat{\delta} : Q \times \Sigma^* \rightarrow Q$)
- ▶ $L(A) = L(A, q_0)$
- ▶ Two states q_1, q_2 in A are equivalent if $L(A, q_1) = L(A, q_2)$. A state q_1 in DFA A_1 is equivalent to state q_2 in DFA A_2 if $L(A_1, q_1) = L(A_2, q_2)$.
- ▶ Two DFAs A_1, A_2 are equivalent if $L(A_1) = L(A_2)$

DFA Equivalence

For every DFA, there exists a unique (upto state naming) minimal DFA.

Minimizing DFAs

Two observations:

- ▶ Unreachable states can be removed. This does not change the language accepted.
- ▶ Merging equivalent states. This also does not change the language accepted.

Minimizing DFAs

Two observations:

- ▶ Unreachable states can be removed. This does not change the language accepted.
- ▶ Merging equivalent states. This also does not change the language accepted.

Algorithms:

1. BFS or DFS to identify reachable states and pruning out the rest
2. Table-filling algorithm by E.F.Moore

Table-filling Algorithm

- ▶ Two states are *distinguishable* if they are not equivalent
- ▶ Formally, states q_1, q_2 are *distinguishable* if there exists $w \in \Sigma^*$ such that exactly one of $\hat{\delta}(q_1, w), \hat{\delta}(q_2, w)$ is an accepting state.

Table-filling Algorithm

- ▶ Two states are *distinguishable* if they are not equivalent
- ▶ Formally, states q_1, q_2 are *distinguishable* if there exists $w \in \Sigma^*$ such that exactly one of $\hat{\delta}(q_1, w), \hat{\delta}(q_2, w)$ is an accepting state.
- ▶ Table-filling is recursive discovery of distinguishable pairs of states.

Table-filling Algorithm

- ▶ Two states are *distinguishable* if they are not equivalent
- ▶ Formally, states q_1, q_2 are *distinguishable* if there exists $w \in \Sigma^*$ such that exactly one of $\hat{\delta}(q_1, w), \hat{\delta}(q_2, w)$ is an accepting state.
- ▶ Table-filling is recursive discovery of distinguishable pairs of states.
- ▶ Base case : (p, q) is distinguishable if $p \in F$ and $q \notin F$ or $p \notin F, q \in F$. (why?)

Table-filling Algorithm

- ▶ Two states are *distinguishable* if they are not equivalent
- ▶ Formally, states q_1, q_2 are *distinguishable* if there exists $w \in \Sigma^*$ such that exactly one of $\hat{\delta}(q_1, w), \hat{\delta}(q_2, w)$ is an accepting state.
- ▶ Table-filling is recursive discovery of distinguishable pairs of states.
- ▶ Base case : (p, q) is distinguishable if $p \in F$ and $q \notin F$ or $p \notin F, q \in F$. (why?)
- ▶ Inductive hypothesis : (p, q) is distinguishable if $\delta(p, a)$ and $\delta(q, a)$ are distinguishable for some $a \in \Sigma$. (why?)

Table-filling Algorithm

1. Distinguishable = $\{(p, q) \mid p \in F, q \notin F\}$
2. Repeat while no new pair is added
 - ▶ for every $a \in \Sigma$
add (p, q) to Distinguishable if $(\delta(p, a), \delta(q, a)) \in \text{Distinguishable}$.
3. Return Distinguishable.

Example on Board

Table-filling Algorithm

1. Distinguishable = $\{(p, q) \mid p \in F, q \notin F\}$
2. Repeat while no new pair is added
 - ▶ for every $a \in \Sigma$
add (p, q) to Distinguishable if $(\delta(p, a), \delta(q, a)) \in \text{Distinguishable}$.
3. Return Distinguishable.

Example on Board

Correctness

1. If two states are distinguished by the table-filling algorithm then they are not equivalent (obvious).
2. If two states are not distinguished by the table-filling algorithm then they are equivalent.

Correctness

- ▶ Assume (p, q) is a pair which is not distinguished by the algorithm, but they are not equivalent.

Correctness

- ▶ Assume (p, q) is a pair which is not distinguished by the algorithm, but they are not equivalent.
- ▶ That is, (p, q) is distinguishable, but the algorithm did not find it
- ▶ Call such (p, q) a bad pair. There must be some $w \in \Sigma^*$ that distinguishes (p, q) .

Correctness

- ▶ Assume (p, q) is a pair which is not distinguished by the algorithm, but they are not equivalent.
- ▶ That is, (p, q) is distinguishable, but the algorithm did not find it
- ▶ Call such (p, q) a bad pair. There must be some $w \in \Sigma^*$ that distinguishes (p, q) .
- ▶ Take the shortest such distinguishing word w among all bad pairs, and consider the corresponding bad pair (p, q) .

Correctness

- ▶ Assume (p, q) is a pair which is not distinguished by the algorithm, but they are not equivalent.
- ▶ That is, (p, q) is distinguishable, but the algorithm did not find it
- ▶ Call such (p, q) a bad pair. There must be some $w \in \Sigma^*$ that distinguishes (p, q) .
- ▶ Take the shortest such distinguishing word w among all bad pairs, and consider the corresponding bad pair (p, q) .
 - ▶ $w \neq \epsilon$ (why?)

Correctness

- ▶ Assume (p, q) is a pair which is not distinguished by the algorithm, but they are not equivalent.
- ▶ That is, (p, q) is distinguishable, but the algorithm did not find it
- ▶ Call such (p, q) a bad pair. There must be some $w \in \Sigma^*$ that distinguishes (p, q) .
- ▶ Take the shortest such distinguishing word w among all bad pairs, and consider the corresponding bad pair (p, q) .
 - ▶ $w \neq \epsilon$ (why?)
 - ▶ Let $w = ax$. As p, q are distinguishable, exactly one of $\hat{\delta}(p, ax), \hat{\delta}(q, ax)$ is accepting.
 - ▶ Then $p' = \delta(p, a)$ and $q' = \delta(q, a)$ are distinguished by x .

Correctness

- ▶ Assume (p, q) is a pair which is not distinguished by the algorithm, but they are not equivalent.
- ▶ That is, (p, q) is distinguishable, but the algorithm did not find it
- ▶ Call such (p, q) a bad pair. There must be some $w \in \Sigma^*$ that distinguishes (p, q) .
- ▶ Take the shortest such distinguishing word w among all bad pairs, and consider the corresponding bad pair (p, q) .
 - ▶ $w \neq \epsilon$ (why?)
 - ▶ Let $w = ax$. As p, q are distinguishable, exactly one of $\hat{\delta}(p, ax), \hat{\delta}(q, ax)$ is accepting.
 - ▶ Then $p' = \delta(p, a)$ and $q' = \delta(q, a)$ are distinguished by x .
 - ▶ If (p', q') was discovered by the algorithm, then (p, q) would have been discovered as well.

Correctness

- ▶ Assume (p, q) is a pair which is not distinguished by the algorithm, but they are not equivalent.
- ▶ That is, (p, q) is distinguishable, but the algorithm did not find it
- ▶ Call such (p, q) a bad pair. There must be some $w \in \Sigma^*$ that distinguishes (p, q) .
- ▶ Take the shortest such distinguishing word w among all bad pairs, and consider the corresponding bad pair (p, q) .
 - ▶ $w \neq \epsilon$ (why?)
 - ▶ Let $w = ax$. As p, q are distinguishable, exactly one of $\hat{\delta}(p, ax), \hat{\delta}(q, ax)$ is accepting.
 - ▶ Then $p' = \delta(p, a)$ and $q' = \delta(q, a)$ are distinguished by x .
 - ▶ If (p', q') was discovered by the algorithm, then (p, q) would have been discovered as well.
 - ▶ If (p', q') is not discovered by the algorithm, then (p', q') is a bad pair with a shorter distinguishing word, a contradiction.

Minimization of DFAs

- ▶ Let A be a DFA with no unreachable states
- ▶ Let $\approx \subseteq Q \times Q$ be the state equivalence relation (computed say by the table-filling algorithm)

$$p \approx q \Leftrightarrow \forall x \in \Sigma^* (\hat{\delta}(p, x) \in F \Leftrightarrow \hat{\delta}(q, x) \in F)$$

Minimization of DFAs

- ▶ Let A be a DFA with no unreachable states
- ▶ Let $\approx \subseteq Q \times Q$ be the state equivalence relation (computed say by the table-filling algorithm)

$$p \approx q \Leftrightarrow \forall x \in \Sigma^* (\hat{\delta}(p, x) \in F \Leftrightarrow \hat{\delta}(q, x) \in F)$$

- ▶ $\approx$ is an equivalence relation with finitely many classes
- ▶ Let $[q] = \{q' \mid q' \approx q\}$

Minimization of DFAs

- ▶ Let A be a DFA with no unreachable states
- ▶ Let $\approx \subseteq Q \times Q$ be the state equivalence relation (computed say by the table-filling algorithm)

$$p \approx q \Leftrightarrow \forall x \in \Sigma^* (\hat{\delta}(p, x) \in F \Leftrightarrow \hat{\delta}(q, x) \in F)$$

- ▶ $\approx$ is an equivalence relation with finitely many classes
- ▶ Let $[q] = \{q' \mid q' \approx q\}$
- ▶ Given DFA $A = (Q, \Sigma, \delta, q_0, F)$, we can minimize A to the DFA $A_{min} = (Q', \Sigma, \delta', q'_0, F')$ called the *Quotient Automata* where
 - ▶ $Q' = \{[q] \mid q \in Q\}$
 - ▶ $\delta'([q], a) = [\delta(q, a)]$ for all $a \in \Sigma$,

Minimization of DFAs

- ▶ Let A be a DFA with no unreachable states
- ▶ Let $\approx \subseteq Q \times Q$ be the state equivalence relation (computed say by the table-filling algorithm)

$$p \approx q \Leftrightarrow \forall x \in \Sigma^* (\hat{\delta}(p, x) \in F \Leftrightarrow \hat{\delta}(q, x) \in F)$$

- ▶ $\approx$ is an equivalence relation with finitely many classes
- ▶ Let $[q] = \{q' \mid q' \approx q\}$
- ▶ Given DFA $A = (Q, \Sigma, \delta, q_0, F)$, we can minimize A to the DFA $A_{min} = (Q', \Sigma, \delta', q'_0, F')$ called the *Quotient Automata* where
 - ▶ $Q' = \{[q] \mid q \in Q\}$
 - ▶ $\delta'([q], a) = [\delta(q, a)]$ for all $a \in \Sigma$,
If $p \approx q$ then $\delta(p, a) \approx \delta(q, a)$. That is, if $[p] = [q]$, then $[\delta(p, a)] = [\delta(q, a)]$
 - ▶ $q'_0 = [q_0]$
 - ▶ $F' = \{[q] \mid q \in F\}$

Minimization of DFAs

- ▶ Let A be a DFA with no unreachable states
- ▶ Let $\approx \subseteq Q \times Q$ be the state equivalence relation (computed say by the table-filling algorithm)

$$p \approx q \Leftrightarrow \forall x \in \Sigma^* (\hat{\delta}(p, x) \in F \Leftrightarrow \hat{\delta}(q, x) \in F)$$

- ▶ $\approx$ is an equivalence relation with finitely many classes
- ▶ Let $[q] = \{q' \mid q' \approx q\}$
- ▶ Given DFA $A = (Q, \Sigma, \delta, q_0, F)$, we can minimize A to the DFA $A_{min} = (Q', \Sigma, \delta', q'_0, F')$ called the *Quotient Automata* where
 - ▶ $Q' = \{[q] \mid q \in Q\}$
 - ▶ $\delta'([q], a) = [\delta(q, a)]$ for all $a \in \Sigma$,
If $p \approx q$ then $\delta(p, a) \approx \delta(q, a)$. That is, if $[p] = [q]$, then $[\delta(p, a)] = [\delta(q, a)]$
 - ▶ $q'_0 = [q_0]$
 - ▶ $F' = \{[q] \mid q \in F\}$
 $q \in F$ iff $[q] \in F'$. If $p \approx q$ and $q \in F$, then $p \in F$. Each class is either inside F or disjoint from F .

Minimization of DFAs

$$L(A_{min}) = L(A)$$

1. $x \in L(A_{min})$ iff $\hat{\delta}'(q'_0, x) \in F'$ iff

Minimization of DFAs

$$L(A_{min}) = L(A)$$

1. $x \in L(A_{min})$ iff $\hat{\delta}'(q'_0, x) \in F'$ iff
2. $\hat{\delta}'([q_0], x) \in F'$ iff

Minimization of DFAs

$$L(A_{min}) = L(A)$$

1. $x \in L(A_{min})$ iff $\hat{\delta}'(q'_0, x) \in F'$ iff
2. $\hat{\delta}'([q_0], x) \in F'$ iff
3. $[\hat{\delta}(q_0, x)] \in F'$ iff

Minimization of DFAs

$$L(A_{min}) = L(A)$$

1. $x \in L(A_{min})$ iff $\hat{\delta}'(q'_0, x) \in F'$ iff
2. $\hat{\delta}'([q_0], x) \in F'$ iff
3. $[\hat{\delta}(q_0, x)] \in F'$ iff
4. $\hat{\delta}(q_0, x) \in F$ iff

Minimization of DFAs

$$L(A_{min}) = L(A)$$

1. $x \in L(A_{min})$ iff $\hat{\delta}'(q'_0, x) \in F'$ iff
2. $\hat{\delta}'([q_0], x) \in F'$ iff
3. $[\hat{\delta}(q_0, x)] \in F'$ iff
4. $\hat{\delta}(q_0, x) \in F$ iff
5. $x \in L(A)$

Minimization of DFAs

$$L(A_{min}) = L(A)$$

1. $x \in L(A_{min})$ iff $\hat{\delta}'(q'_0, x) \in F'$ iff
2. $\hat{\delta}'([q_0], x) \in F'$ iff
3. $[\hat{\delta}(q_0, x)] \in F'$ iff
4. $\hat{\delta}(q_0, x) \in F$ iff
5. $x \in L(A)$

Claim

1. No two distinct states in A_{min} are equivalent.
2. A_{min} is the minimum and unique (upto state renaming) DFA equivalent to A .

Minimality of A_{min}

- ▶ Assume there is a DFA B with smaller number of states than A_{min} , such that $L(B) = L(A)$.

Minimality of A_{min}

- ▶ Assume there is a DFA B with smaller number of states than A_{min} , such that $L(B) = L(A)$.
- ▶ Using the table-filling algorithm, compute equivalent states of B and A_{min} .

Minimality of A_{min}

- ▶ Assume there is a DFA B with smaller number of states than A_{min} , such that $L(B) = L(A)$.
- ▶ Using the table-filling algorithm, compute equivalent states of B and A_{min} .
- ▶ The initial states of B and A_{min} must be equivalent (why?)

Minimality of A_{min}

- ▶ Assume there is a DFA B with smaller number of states than A_{min} , such that $L(B) = L(A)$.
- ▶ Using the table-filling algorithm, compute equivalent states of B and A_{min} .
- ▶ The initial states of B and A_{min} must be equivalent (why?)
- ▶ After reading any $w \in \Sigma^*$ from their initial states, both DFAs enter equivalent states (why?)

Minimality of A_{min}

- ▶ Assume there is a DFA B with smaller number of states than A_{min} , such that $L(B) = L(A)$.
- ▶ Using the table-filling algorithm, compute equivalent states of B and A_{min} .
- ▶ The initial states of B and A_{min} must be equivalent (why?)
- ▶ After reading any $w \in \Sigma^*$ from their initial states, both DFAs enter equivalent states (why?)
- ▶ For every state of A_{min} , there is an equivalent state in B
- ▶ Since the number of states in B are $<$ than those in A_{min} , there are at least two states p, q in A_{min} equivalent to some state in B .
- ▶ That is, p, q are equivalent, a contradiction.

A minimal DFA directly from the language

Given a regular language R , can we define the minimal DFA for R directly from it?

Given a language L (not necessarily regular), and words $u, v \in L$, define $u \sim_L v$ if for all $w \in \Sigma^*$, $uw \in L \Leftrightarrow vw \in L$.

- ▶ $\sim_L \subseteq \Sigma^* \times \Sigma^*$ is an equivalence relation on words
- ▶ Consider the equivalence classes of $\sim_L$
- ▶ When can $\sim_L$ have only finitely many equivalence classes?

Properties of $\sim_L$

- ▶ $\sim_L$ is a right congruence : that is, for any $a \in \Sigma$,
 $x \sim_L y \Rightarrow xa \sim_L ya$
- ▶ $\sim_L$ refines L : that is, $x \sim_L y \Rightarrow x \in L \Leftrightarrow y \in L$

Myhill-Nerode relations

An equivalence relation on Σ^* is called a *Myhill-Nerode* relation for $L \subseteq \Sigma^*$ if it is a right congruence refining L , and has finitely many equivalence classes.

The relation was proposed by John Myhill and Anil Nerode.

An Example

Consider $L = \{a^n b^n \mid n \geq 0\}$. Recall $\sim_L$.

- ▶ Consider the set of words $\{a, a^2, \dots, a^i, \dots\}$.
- ▶ If $\sim_L$ has finitely many classes, then there exists a^m, a^n $m \neq n$ such that $a^m \sim_L a^n$. That is, for all $w \in \Sigma^*$, $a^m.w \in L$ iff $a^n.w \in L$.
- ▶ However, $a^m b^m \in L$ but $a^n b^m \notin L$ if we choose $w = a^m$
- ▶ Hence $[a^m] \neq [a^n]$ for $m \neq n$.
- ▶ Infinitely many equivalence classes.

An Example

Consider $L = a^*b^*$. Recall $\sim_L$. What are the classes of $\sim_L$?

An Example

Consider $L = a^*b^*$. Recall $\sim_L$. What are the classes of $\sim_L$?

- ▶ $[\epsilon] = a^*$
- ▶ $[b] = a^*b^+$
- ▶ $[ba] = a^*b^+a\Sigma^*$

Myhill-Nerode Relations

If L is regular, then there exists a Myhill-Nerode relation $\sim_A$ for L defined from the DFA A for L .

Myhill-Nerode Relations

If L is regular, then there exists a Myhill-Nerode relation $\sim_A$ for L defined from the DFA A for L .

- ▶ Let $A = (Q, \Sigma, \delta, q_0, F)$ be the DFA with no inaccessible states such that $L = L(A)$.
- ▶ Define $x \sim_A y$ iff $\hat{\delta}(q_0, x) = \hat{\delta}(q_0, y)$.
- ▶ It can be seen that $\sim_A$ is a right congruence refining L .
- ▶ $[x] = \{y \mid \hat{\delta}(q_0, x) = \hat{\delta}(q_0, y)\}$. There is one equivalence class corresponding to each state in Q . Hence, finitely many classes.
- ▶ Hence, $\sim_A$ is Myhill-Nerode.

Myhill-Nerode Relations

If language L has a Myhill-Nerode relation $\sim$, then L is regular.

Myhill-Nerode Relations

If language L has a Myhill-Nerode relation $\sim$, then L is regular.

- ▶ We know $\sim$ is a right congruence refining L , with finitely many equivalence classes $[x]$.

Myhill-Nerode Relations

If language L has a Myhill-Nerode relation $\sim$, then L is regular.

- ▶ We know $\sim$ is a right congruence refining L , with finitely many equivalence classes $[x]$.
- ▶ From $\sim$, define a DFA $A_{\sim} = (Q, \Sigma, \delta, s, F)$ where
$$Q = \{[x] \mid x \in \Sigma^*\}$$
$$s = [\epsilon]$$
$$F = \{[x] \mid x \in L\}$$
$$\delta([x], a) = [xa].$$
- ▶ Is $A_{\sim}$ well-defined?
- ▶ $x \in L$ iff $[x] \in F$. If $[x] \in F$, then all words in $[x]$ are in L as $\sim$ refines L .
- ▶ $L(A_{\sim}) = L$. $x \in L(A_{\sim})$ iff $\hat{\delta}([\epsilon], x) \in F$ iff $[x] \in F$ iff $x \in L$.

Two constructions

We did two things.

- (1) Given a DFA A accepting L with no inaccessible state, we defined a Myhill-Nerode relation $\sim_A$ from A .
 - (2) Given a language L with a Myhill-Nerode relation $\sim$, we constructed the DFA $A_\sim$ for L .
- (1), (2) are inverses upto isomorphism. That is,
- ▶ Myhill-Nerode relation $\sim \rightarrow$ DFA $A_\sim \rightarrow$ Myhill-Nerode relation $\sim_{A_\sim}$ would mean $\sim = \sim_{A_\sim}$.
 - ▶ DFA A for language L to Myhill-Nerode relation $\sim_A$ to DFA $A_{\sim_A}$ would imply A is isomorphic to $A_{\sim_A}$.

Myhill-Nerode and the Minimal DFA

- ▶ A relation $\sim_1$ is said to refine another relation $\sim_2$ if $\sim_1 \subseteq \sim_2$ when considered as sets of ordered pairs.
- ▶ That is, $x \sim_1 y \Rightarrow x \sim_2 y$.
- ▶ In other words, $[x]_1 \subseteq [x]_2$

Myhill-Nerode and the Minimal DFA

- ▶ A relation $\sim_1$ is said to refine another relation $\sim_2$ if $\sim_1 \subseteq \sim_2$ when considered as sets of ordered pairs.
- ▶ That is, $x \sim_1 y \Rightarrow x \sim_2 y$.
- ▶ In other words, $[x]_1 \subseteq [x]_2$
- ▶ The Myhill-Nerode relation $\sim_L$ for a language L refines the relation which has 2 classes L and $\Sigma^* \setminus L$.
- ▶ If $\sim_1$ refines $\sim_2$, then $\sim_2$ is coarser than $\sim_1$ while $\sim_1$ is finer than $\sim_2$
- ▶ Any set U , has a finest and coarsest equivalence relation : finest is the identity and coarsest is universal relation $\{(x, y) \mid x, y \in U\}$.

Myhill-Nerode and the Minimal DFA

Let L be some language. Recall the relation $\sim_L$ we defined on L . $\sim_L$ is a right congruence refining L and is the coarsest such relation.

- ▶ We already know $\sim_L$ is a right congruence refining L .
- ▶ Show that $\sim_L$ is coarsest. Let $\equiv$ be a right congruence on L refining L . Then $\equiv$ refines $\sim_L$:
 - ▶ $x \equiv y \Rightarrow \forall z(xz \equiv yz)$ (why?)
 - ▶ $\forall z(xz \equiv yz) \Rightarrow \forall z(xz \in L \Leftrightarrow yz \in L)$ (why?)
 - ▶ $\forall z(xz \in L \Leftrightarrow yz \in L) \Rightarrow x \sim_L y$ (definition of $\sim_L$)

Myhill-Nerode Theorem

The following are equivalent :

1. L is regular
2. there exists a Myhill-Nerode relation $\sim_L$ for L
3. $\sim_L$ is of finite index, that is, has finitely many equivalence classes.

Since $\sim_L$ is the coarsest, the DFA it produces for L has the fewest states among all DFAs for L .

The table-filling algorithm gives the above minimal DFA. Show that the relation $\sim_A$ computed from the collapsed DFA A is same as $\sim_L$.

Table-filling and the Myhill-Nerode DFA

Assume $A = (Q, \Sigma, \delta, q_0, F)$ is a DFA for L which has been collapsed by the table-filling algorithm. Recall the equivalence computed by the table-filling algorithm

$$p \approx q \Leftrightarrow \forall x \in \Sigma^* (\hat{\delta}(p, x) \in F \Leftrightarrow \hat{\delta}(q, x) \in F)$$

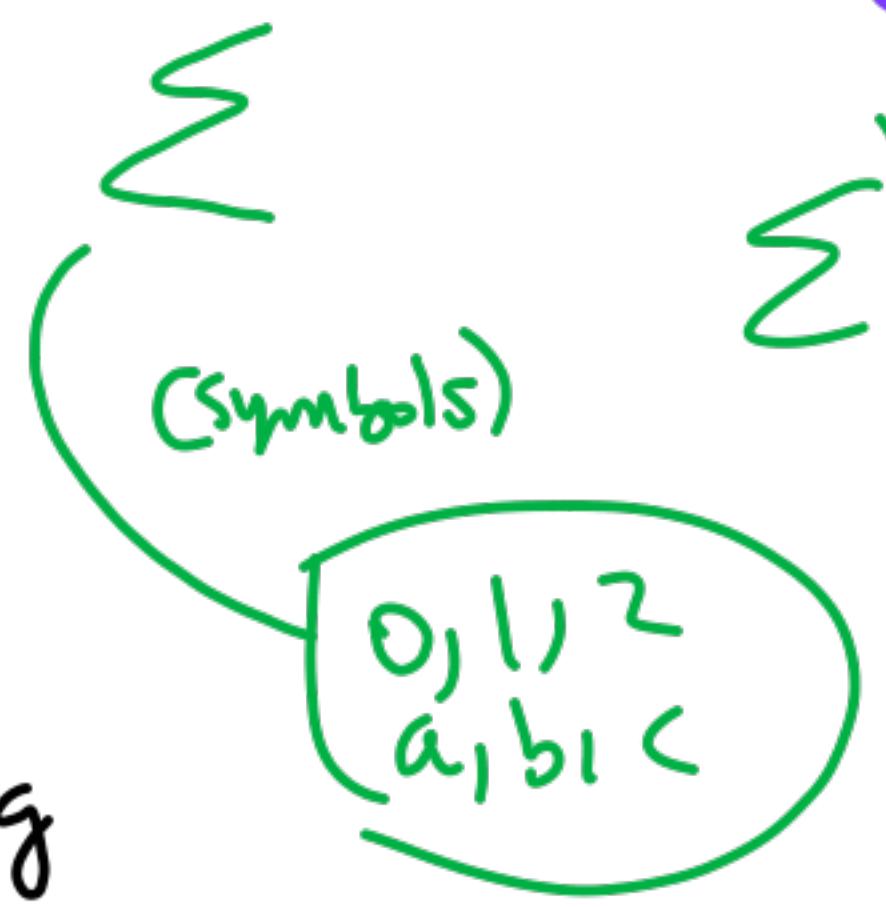
- ▶ If A is the collapsed automaton produced for L by the table-filling algorithm, we show that $\sim_L$ and $\sim_A$ are the same, where $\sim_A$ is the Myhill-Nerode relation constructed from A .
- ▶ $x \sim_L y$ iff $\forall z (xz \in L \Leftrightarrow yz \in L)$
- ▶ $\Leftrightarrow \forall z (\hat{\delta}(q_0, xz) \in F \Leftrightarrow \hat{\delta}(q_0, yz) \in F)$
- ▶ $\Leftrightarrow \hat{\delta}(q_0, x) \approx \hat{\delta}(q_0, y)$
- ▶ $\Leftrightarrow \hat{\delta}(q_0, x) = \hat{\delta}(q_0, y)$ as A is collapsed
- ▶ $\Leftrightarrow x \sim_A y$

8.8.2025

Agenda

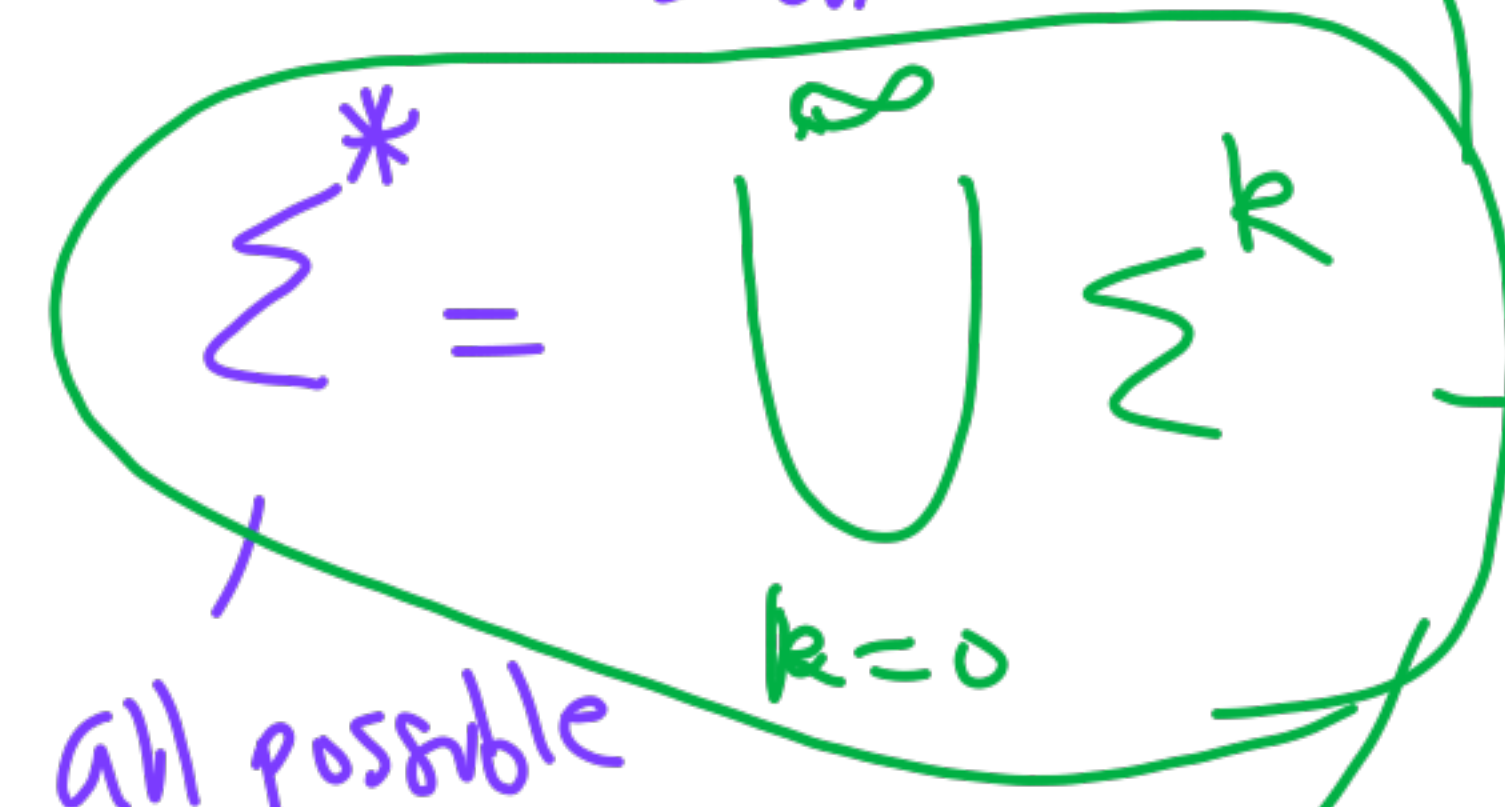
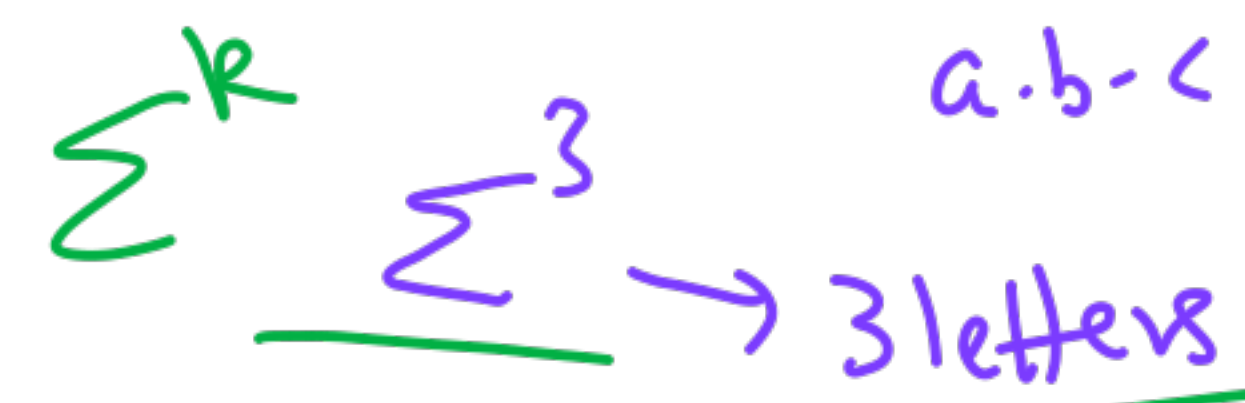
1. Languages
2. Enumeration/
Encoding
3. Define (How to?)
4. Grammars
5. Properties $\left\{ \begin{array}{l} \text{Decision} \\ \text{closure} \end{array} \right.$

Alphabet

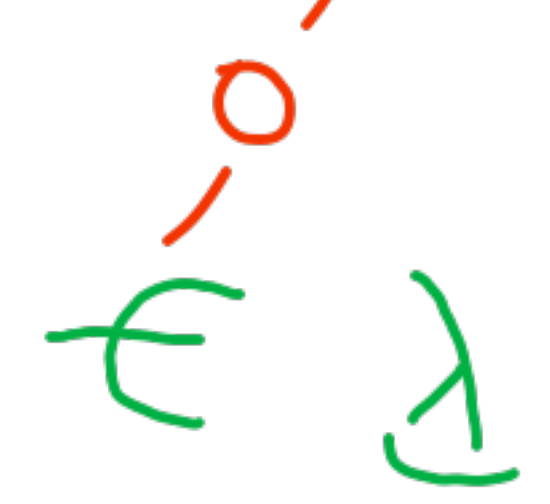


Strings
Words

Concatenation .



all possible
strings
length



- Moodle post
- Assign 1

→ Sentence

LLM

Satan-Cantor

Enumeration

$$\Sigma = \{a, b, c\}$$

length	N		Σ^*
$\langle 1 \rangle$	1	—	ϵ — 0
$\langle 1, 2 \rangle$	2	—	a } 1
$\langle 3 \rangle$	3	—	b }
$\langle 3, 1, 5 \rangle$	4	—	c }
	5	—	} 2

Pairs $N \times N \leftrightarrow N$ Bijection



$$N \times N \times N \leftrightarrow N$$

$$N \leftrightarrow \Sigma^*$$

Dovetailing

$$N \leftrightarrow N^{\aleph_1} \leftarrow \text{unknown (unk)}$$

$\Sigma \quad \Sigma^*$

Define

Language



Can we enumerate
all languages?

$\Sigma = \{a\}$

$\Sigma^N = \{a^1, a^2, \dots, a^N\}$

		a	a^2	a^3	$\dots$	a^{100}	$\dots$
1 -	S_1	0	1	1	0	$\dots$	
2 -	S_2	1	0	1	0	0	$\dots$
3 -	S_3	1	1	1	$\dots$		
4 -	S_4	1	0	1	0		
.	.						

diagonal

$S_d = \{a^i \mid a^i \neq S_i\}$

Terminals Non Terminals Grammars

(to define languages)

$G = \langle \Sigma, V, S, P \rangle$

Set of
Productions
Rules

Productions

lhs $\rightarrow$ rhs

$\alpha \rightarrow \beta$

alphabet

a, b, c
0, 1

Variables

$V = \{ \textcircled{S}, A, B, P, R, L \}$

~~$x \leftarrow x + 1$~~

AA

BB

Startup
Variable

aA

$(\Sigma \cup V)^* aBcA$
 $\downarrow \quad \downarrow$
 $AAcB^*$

1) Type 0
(unrestricted)

Context

$\alpha, \beta \in (\Sigma \cup V)^* (\Sigma \cup V)^*$

$+ |\alpha| \geq |\beta|$

2) CSL
 $\swarrow$ $\searrow$
 ensitive any

3)

$$\alpha \rightarrow \beta$$

① Type-0
(Unrestricted) $\alpha, \beta \in (\Sigma \cup V)^*$

② Context-Sensitive (CSG) $|\alpha| \geq |\beta|$
lhs at least as long as rhs

③ Context-Free Grammar (C.F.G.)

lhs $\in V$
(single variable) linear left-linear right-linear

Reg. Expr

④

Right

Linear

Left

$\alpha \in V, \beta \rightarrow$ has at most one variable.

$S \rightarrow aa$

$S \rightarrow abc$
 $S \rightarrow Acc$
 $S \rightarrow ccBa$

$S \rightarrow abc$

Derivations

$Xb \rightarrow bbY$

S



$abc \underbrace{Xb} aYD$



$abc \underline{bbY} aYD$

$$L(G) = \{ w \in \Sigma^* \mid S \Rightarrow^* w \}$$

$bbYa \rightarrow aa$

"1-step replacement"

Equal # of a's and b's
 $L = \{w \in \Sigma^* \mid n_a(w) = n_b(w)\}$
 $\Sigma = \{a, b\}$
 $L_1 = \{a^n b^n \mid n \geq 1\}$

$a^n b^{2n}$
 $a^{10} b^{20}$

Write a CFG G $L(G) = L_1$

Technique
 Induction or Recursion
 Base
 Build-up

$S \rightarrow ab$
 $S \rightarrow a \underline{S} b$

1. $S \rightarrow abb$
2. $S \rightarrow aSbb$
3. $S \rightarrow bbb$

All Good strings
 $a \underline{S} bb$
 $a a \underline{S} bb bb$
 No BAD String

$$L = \{ a^n b^n c^n \mid n \geq 1 \}$$

↙
Type 0

$$L = \{ a^{i^2} \mid i \geq 1 \}$$

13.8.25

0. Languages

1. Grammars

$\langle \Sigma, V, S, \underline{P} \rangle$

2. Machines

$\begin{array}{l} \text{FSA} \\ \text{PDA (stack)} \\ \text{TM} \end{array}$

$L \subseteq \Sigma^*$

Generate-and-test

Complete
Soundness

Termination

$\Sigma = \{0, 1\}$

$L = \{w \in (0+1)^* \mid w \text{ is a prime}\}$

$L = \{ \langle w_1, w_2, w_3 \rangle \mid w_i \in (0+1)^* \text{ and } w_1, w_2, w_3 \text{ are the roots of } 7x^3 - 3y^2z + \dots \}$

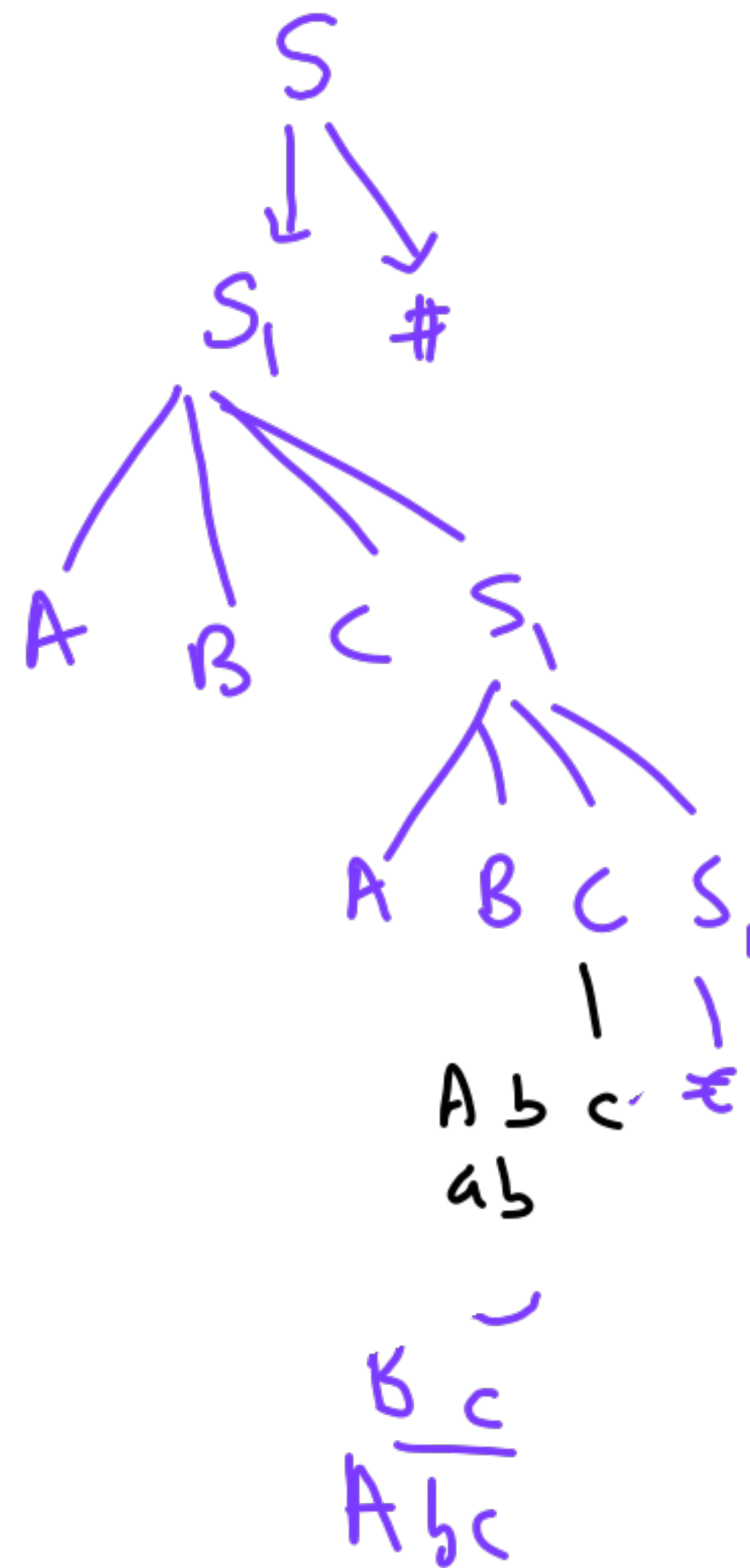
$$L = \{a^n b^n c^n \mid n \geq 1\}$$

$$L(G) = \{ w \mid w \in \Sigma^*, S \xRightarrow{*} w \}$$

Derivation-free

 ~~$S \rightarrow S_1 \#_R$ extra~~
$$S_1 \rightarrow ABCS_1$$
$$S_1 \rightarrow \epsilon$$
$$CA \rightarrow AC$$
$$CB \rightarrow BC$$
$$C \# \rightarrow e$$

$C \rightarrow CC$

$$BA \rightarrow AB$$
$$B_c \rightarrow bc$$
$$Bb \rightarrow bb$$
$$\underline{Ab} \rightarrow ab$$
$$Aa \rightarrow aa$$
$$(a \rightarrow a) \in C$$
$$cb \rightarrow bc$$
$$Ba \rightarrow aB$$


$$L = \{a^n b^n c^n \mid n \geq 1\}$$

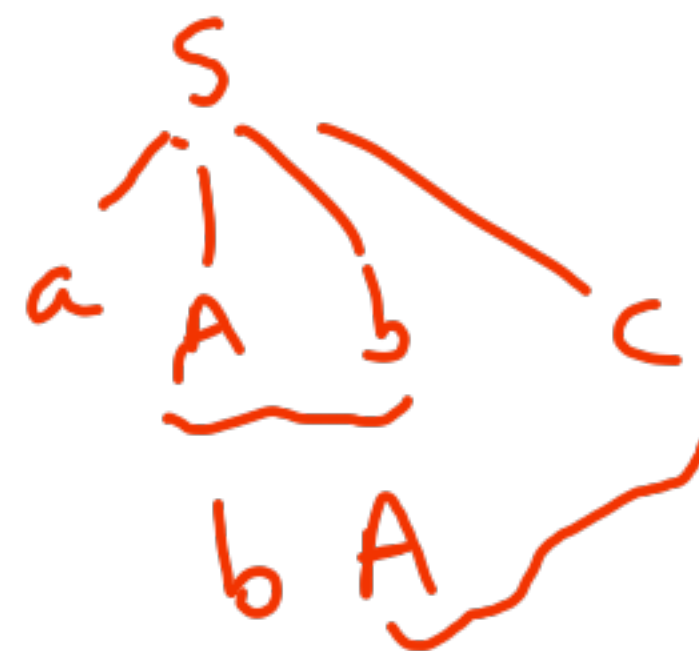
Base case 1. $S \rightarrow abc$

longer 2. $S \rightarrow a A b c$

go find 3. $\left\{ \begin{array}{l} \underline{A}b \rightarrow bA \\ A\underline{c} \rightarrow B\underline{b}c \end{array} \right.$

go back 5. $bB \rightarrow Bb$
 — 6. $aB \rightarrow aa$
7. $aB \rightarrow aa\underline{A}$

5
 2 $a \downarrow \underline{A} b c$
 3 $a b \downarrow \underline{A} c$
 4 $a b \underline{B} b c c$
 5 $\downarrow$
 5 $\underline{a} B b b c c$
 7 $\downarrow$
 7 $a a A b b c c$
 8 $\downarrow$



ambiguity

1

2 different

derivation

trees.

non-isomorphic

~~Type 0~~

CFG

$$L = \{ w \in (a+b)^* \mid n_a(w) = n_b(w) \}$$

Kolmogorov
Complexity

1

$$S \rightarrow \epsilon$$

$$S \rightarrow a S b$$

$$S \rightarrow b S a$$

$$S \rightarrow SS$$

2

$$S' \rightarrow \epsilon$$

$$S \rightarrow a B$$

$$S \rightarrow b A$$

$$B \rightarrow b S$$

$$B \rightarrow a B B$$

$$A \rightarrow a S$$

$$A \rightarrow b A A$$

Closure Properties

$$L = \{ a^i b^j \mid i > j \}$$

$$S \rightarrow S_1 / S_2$$

Concatenation

$$S \rightarrow S_1 C$$

$$L = \{ a^i b^j c^k \mid i=j \text{ or } j=k \}$$

$$\begin{matrix} S_1 \rightarrow \epsilon \\ S_1 \rightarrow a^i b^j \end{matrix}$$

$$\begin{matrix} C \rightarrow \epsilon \\ C \rightarrow cC \end{matrix}$$



$$L_1 \cup L_2$$

Union

$$\begin{matrix} S \rightarrow S_1 \\ S \rightarrow S_2 \end{matrix}$$

Left-linear
Right-linear

Regular Grammars

Regular Expression

$$L = \{ w \in (a+b)^* \mid \begin{array}{l} n_a(w) \text{ is even} \\ \text{AND} \\ n_b(w) \text{ is odd} \end{array} \}$$

FSA

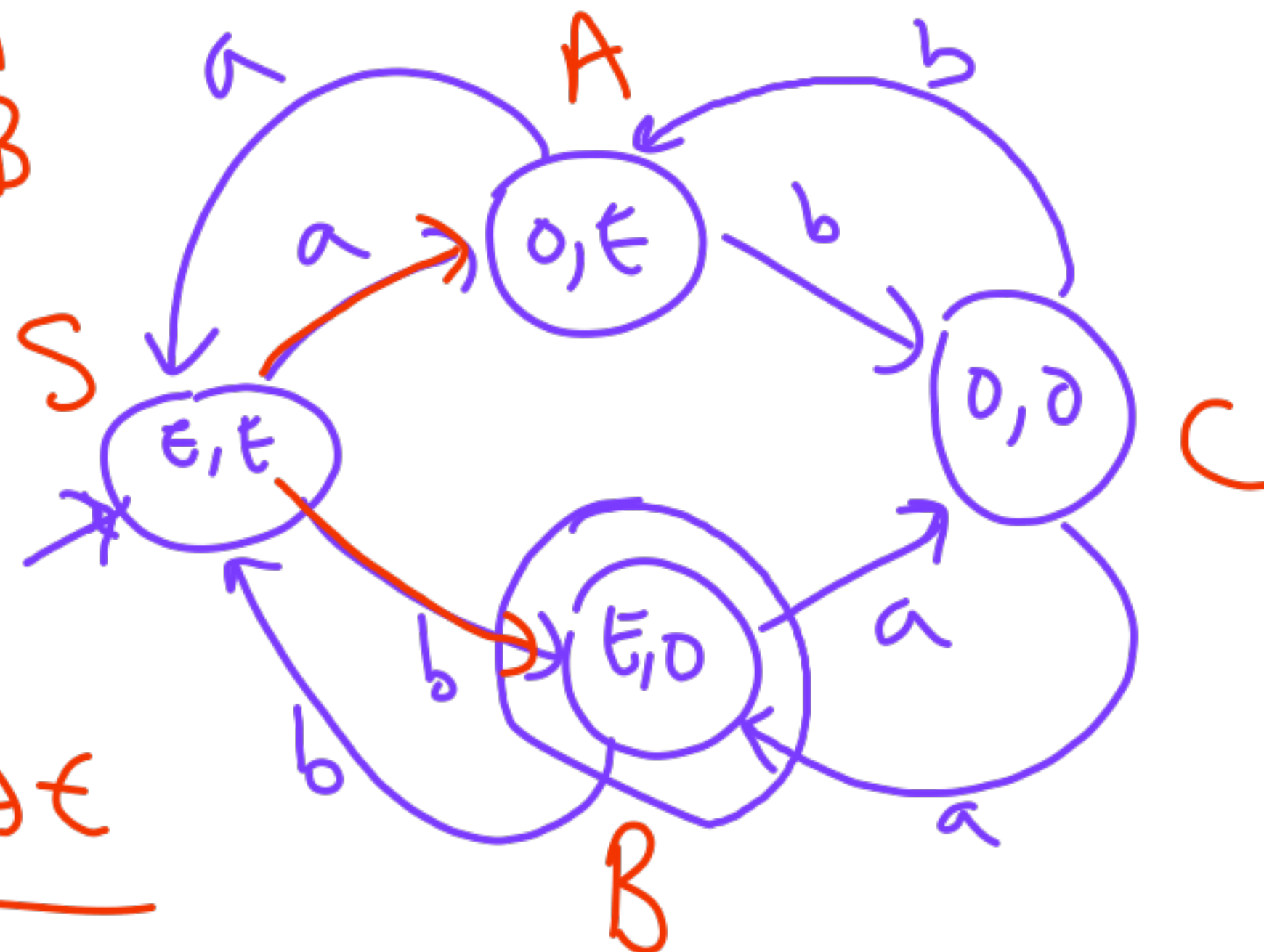
Det

Non-Det

$S \rightarrow aA$
 $S \rightarrow bB$
 $A \rightarrow aS$
 $A \rightarrow bC$
 $B \rightarrow bS$
 $B \rightarrow aC$

$C \rightarrow bA$
 $C \rightarrow aB$

$B \rightarrow \epsilon$



20/8/25

CS310

Alphabet, Σ
Words, w
Lang, $L \subseteq \Sigma^*$

Grammars $\langle \Sigma, V, S, P \rangle$

Machines

Regular	CFG	Type 0
FSA	PDA	TM

Language Acceptors

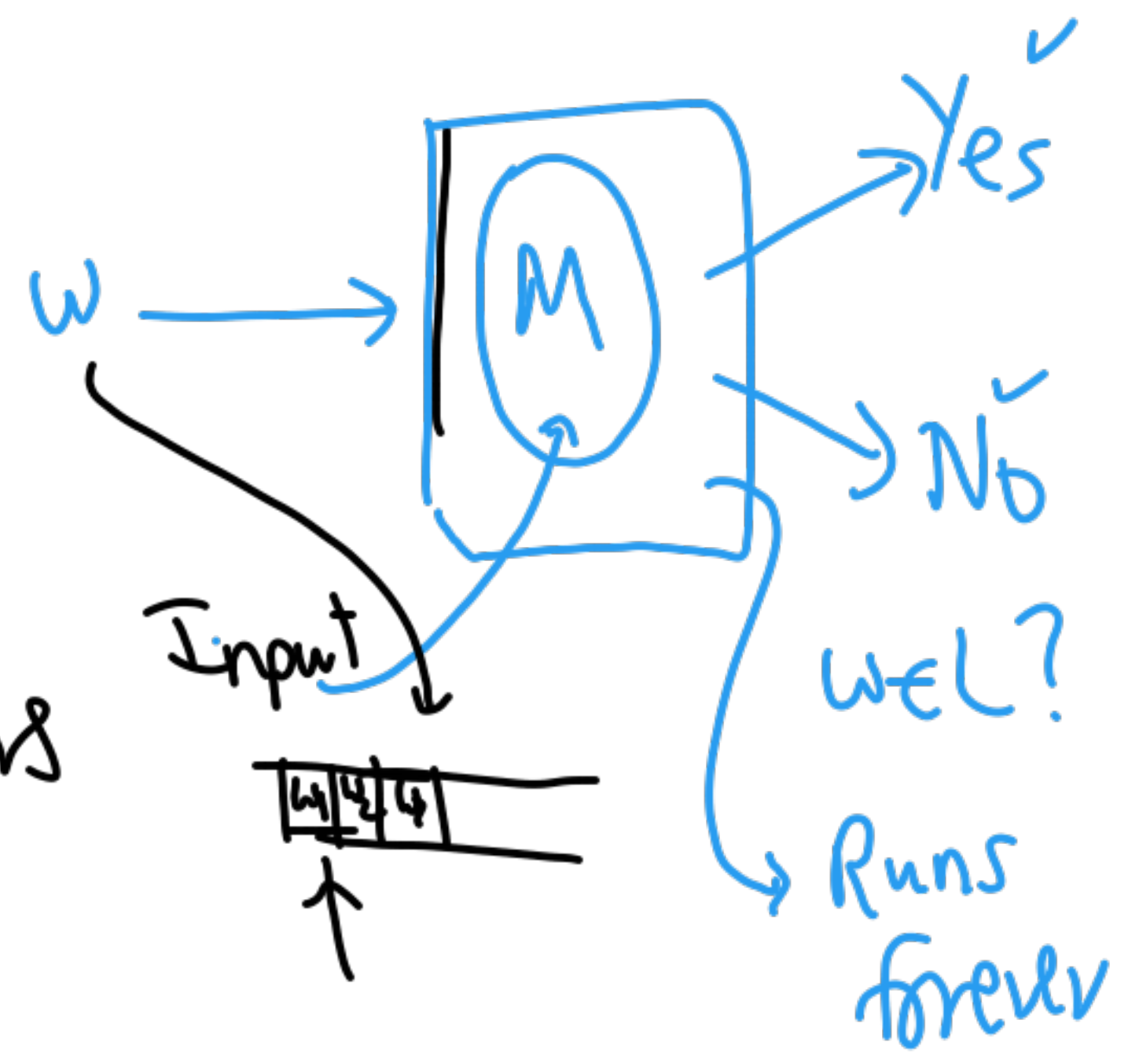
State

Transitions

RUN on an input

(1) Non-Determinism

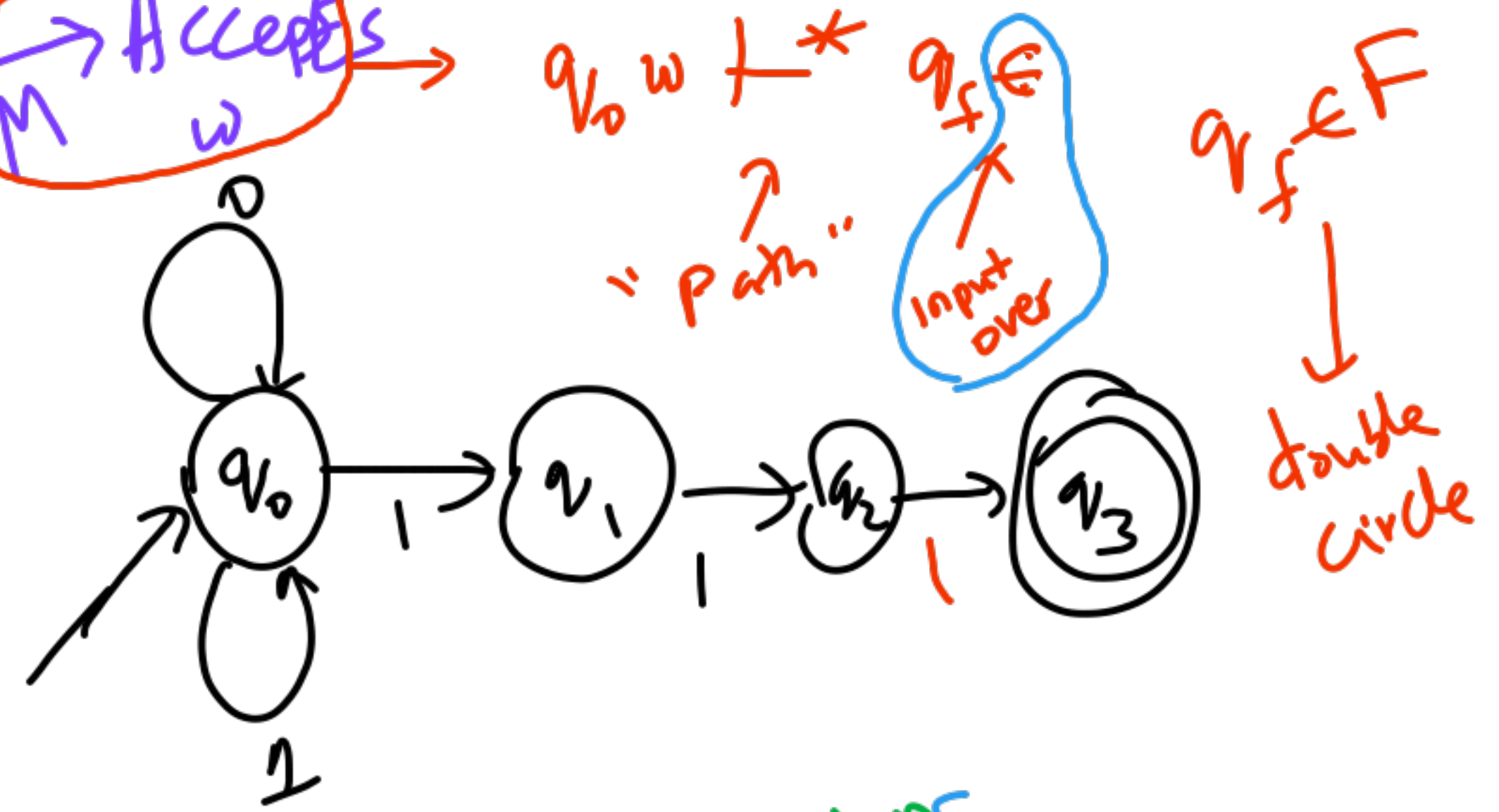
(2) ϵ -moves



Don't know non-det
try all choices

FSA $\rightarrow$ Accepts $M \xrightarrow{w}$

$$L = \{ w \in (0+1)^* \mid w \text{ ends with } 3 \text{ 1s} \}$$



Instantaneous Descriptions

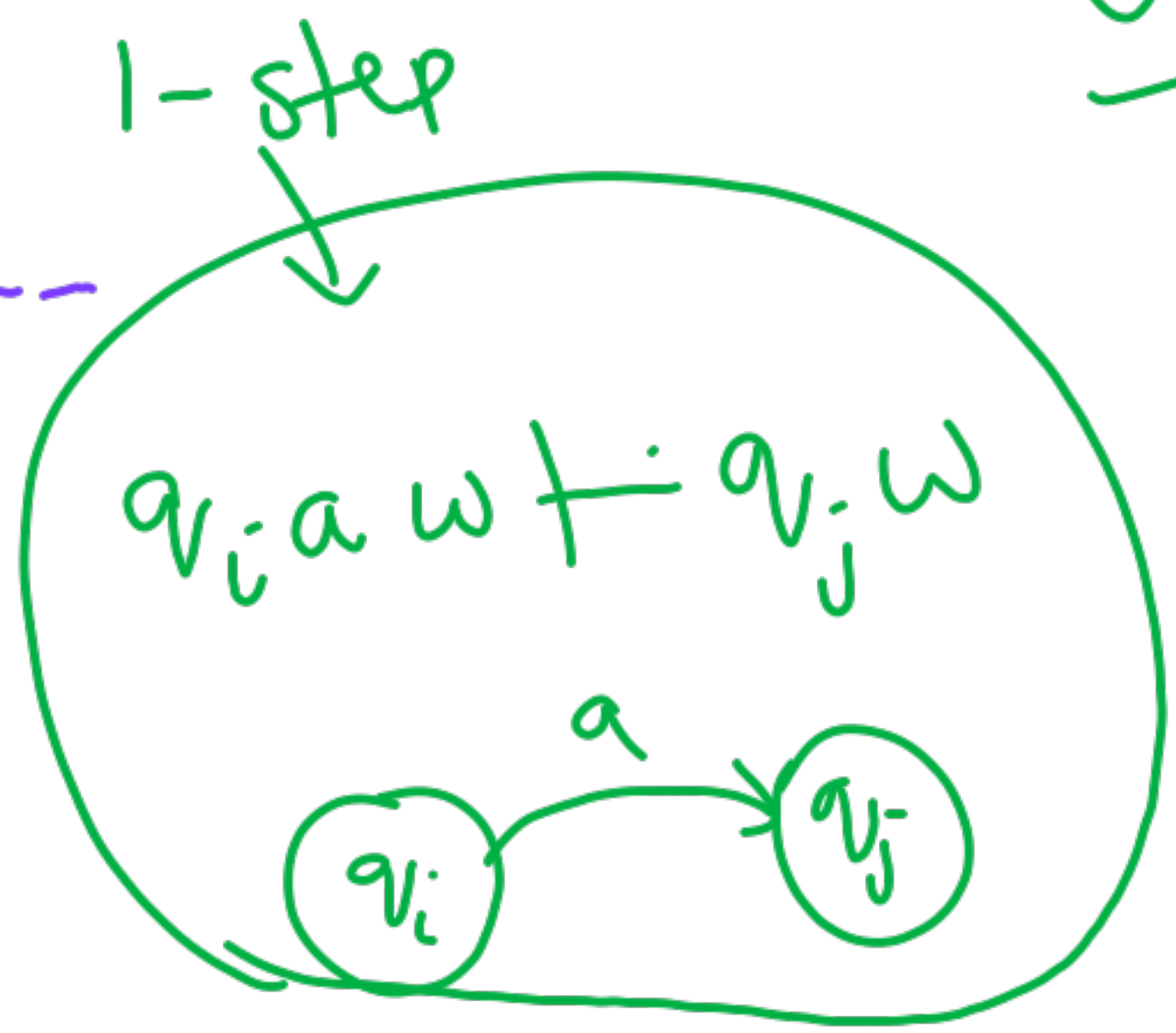


current state q
remaining input w_{rest}

One of the runs

$q_0 1011011$
 $\downarrow$
accepting run?

$q_0 w \vdash^* \dots$



$\vdash q_0 1011$
 $\vdash q_1 011$

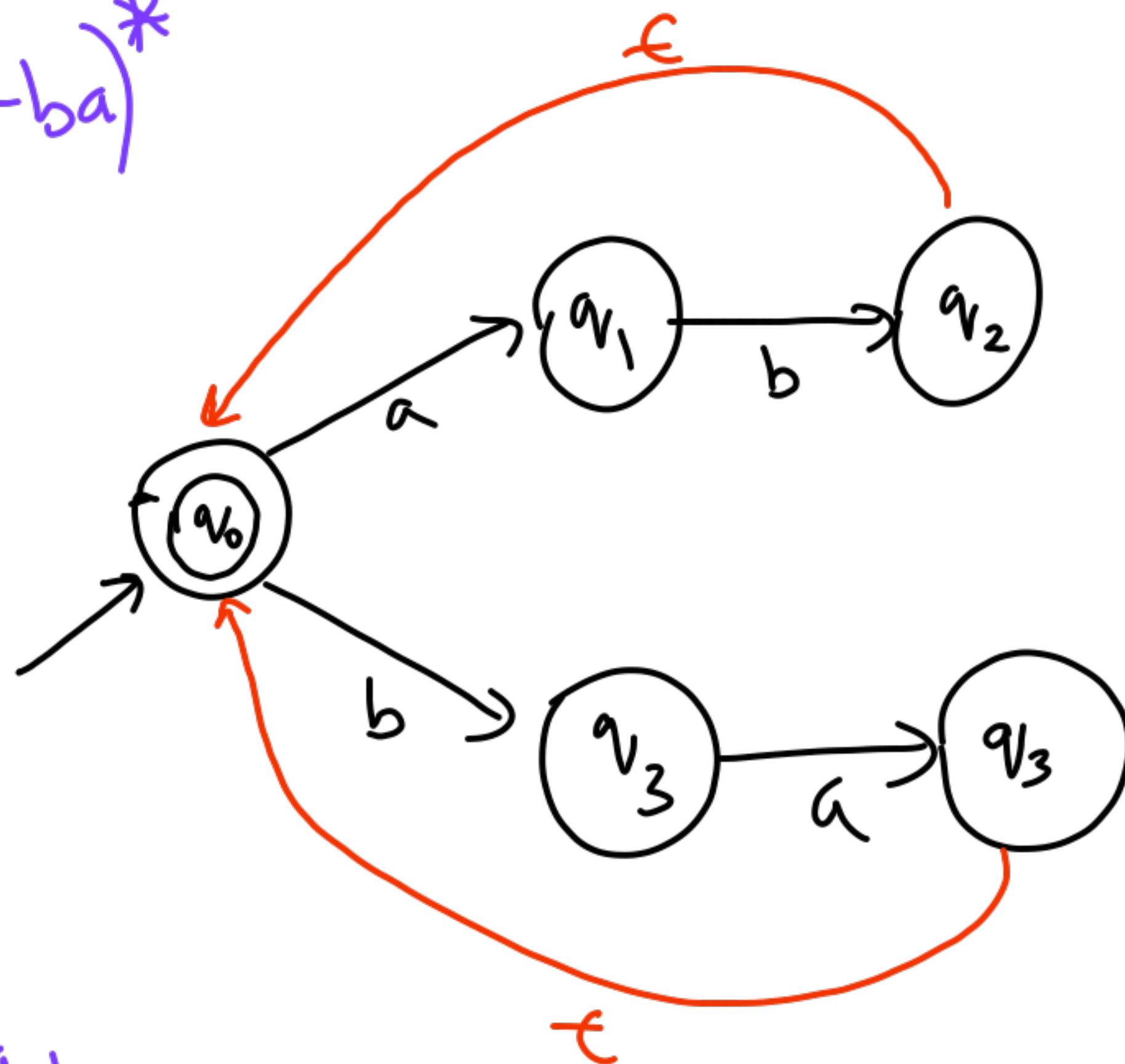
0 or more steps

ϵ -transitions

$$L = (ab+ba)^*$$

DFA

NFA -- ϵ



$$M = \langle \Sigma, Q, \delta, F \rangle$$

States

$q_0 \dots q_n$

final
 $F \subseteq Q$

transitions
"arrows"

labels

$$\Sigma \cup \{\epsilon\}$$

from q_i to q_j
 ϵ

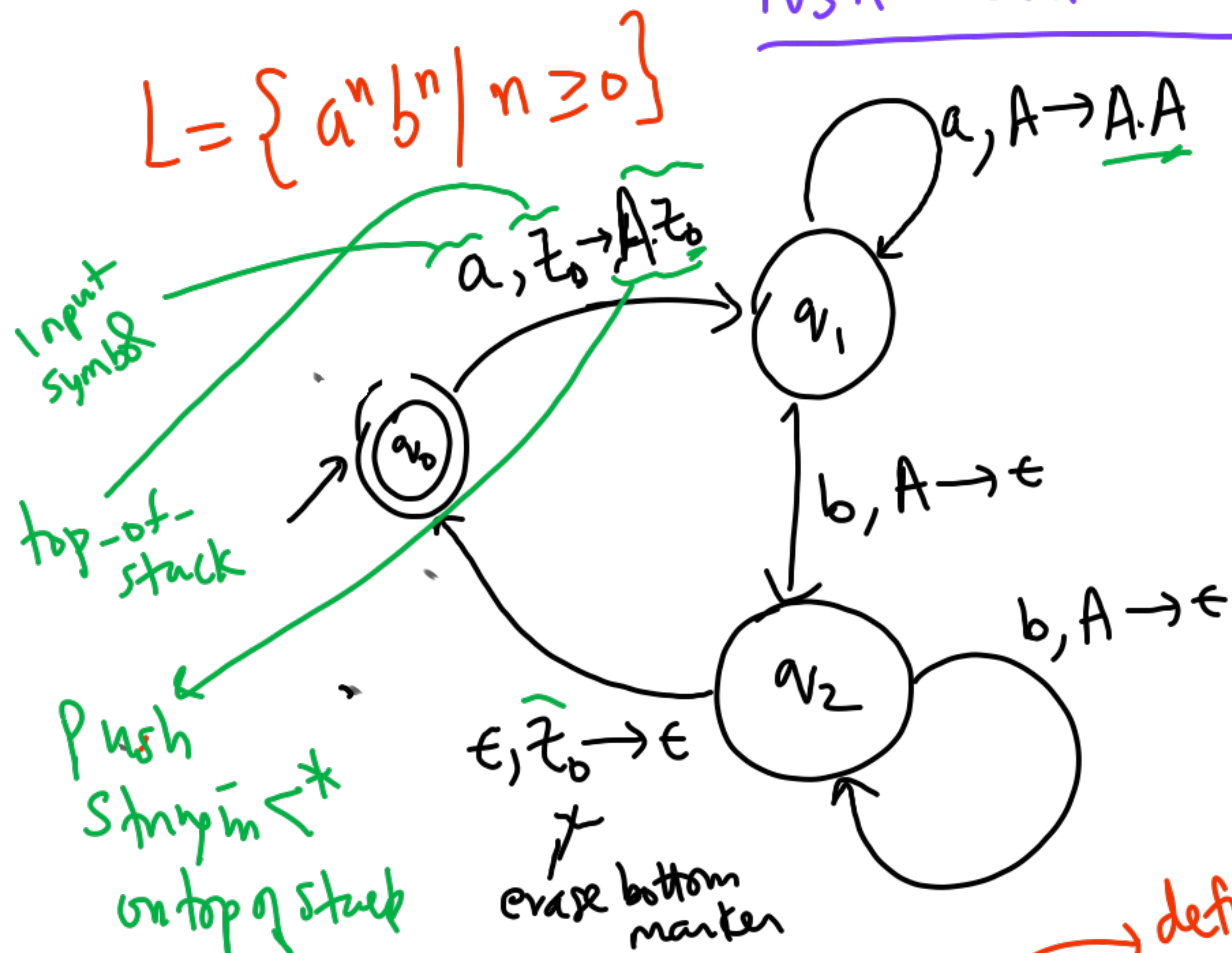
Accepting Run

$q_0 \xrightarrow{a} q_1 \xrightarrow{b} q_2 \xrightarrow{\epsilon} q_0 \xrightarrow{b} q_3 \xrightarrow{a} q_4 \xrightarrow{\epsilon} q_0 \xrightarrow{\epsilon} q_0$
stop

$q_2 \xrightarrow{\epsilon} q_0$ ϵ -tran

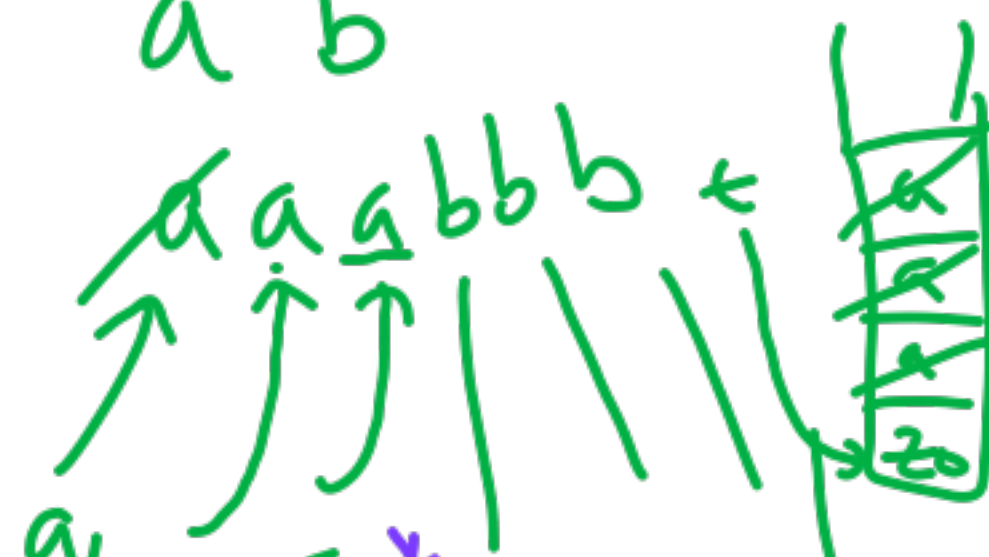
Push Down Automaton

$$L = \{a^n b^n \mid n \geq 0\}$$



$$\{A, B, \epsilon\} \subseteq \Sigma - \{z_0\}$$

Sample $a^3 b^3$



$$(q_i, aw, B\gamma) \vdash (q_j, w, \alpha\gamma)$$

$$\vdash (q_0, w, z_0)$$

Tuple State input stack

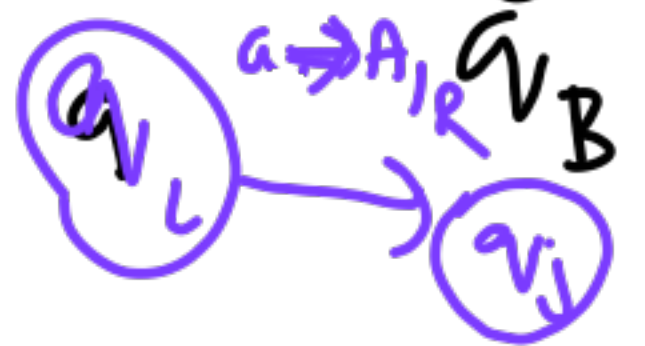
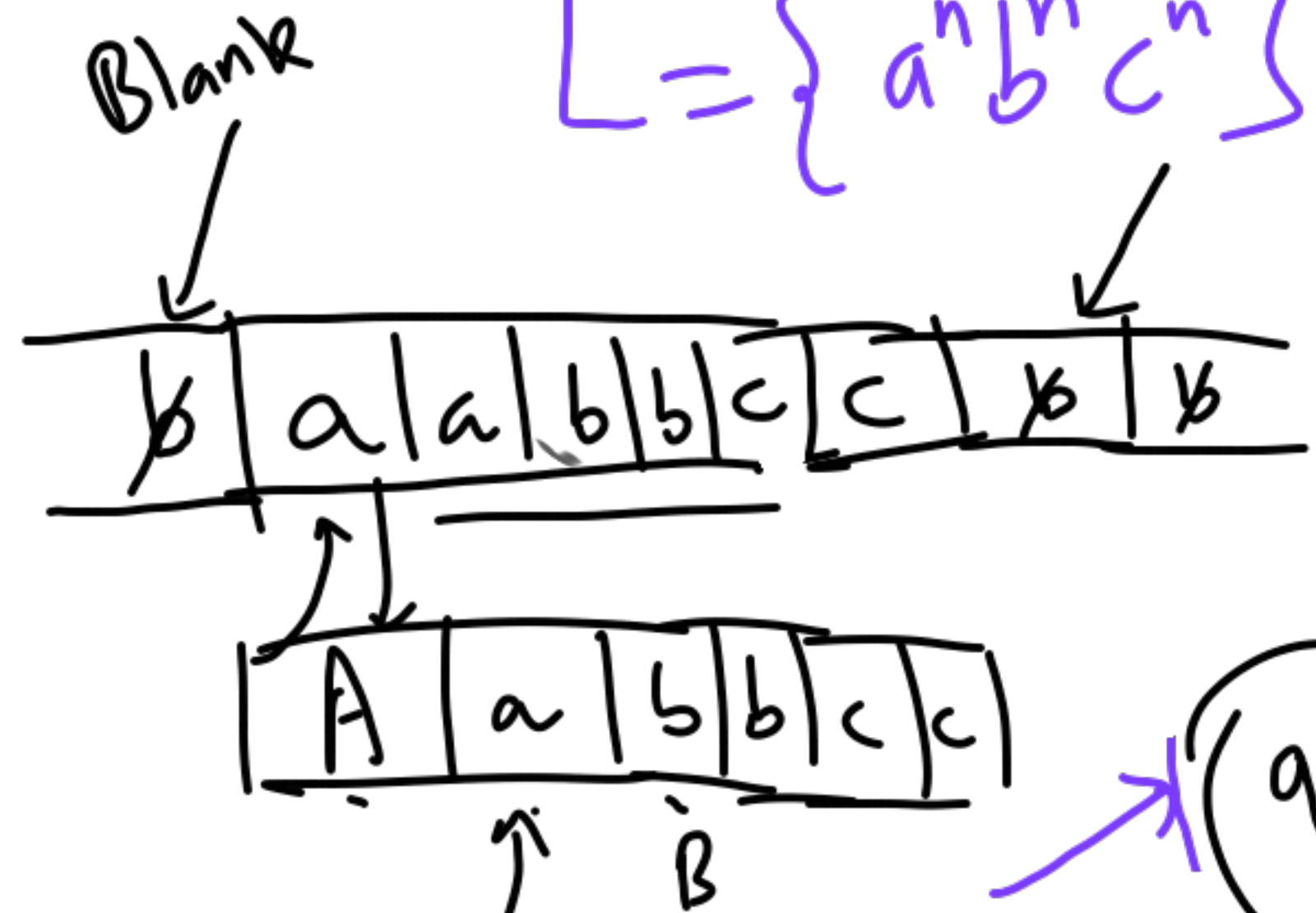
Accept by final state

after all input is consumed

turingmachine.io

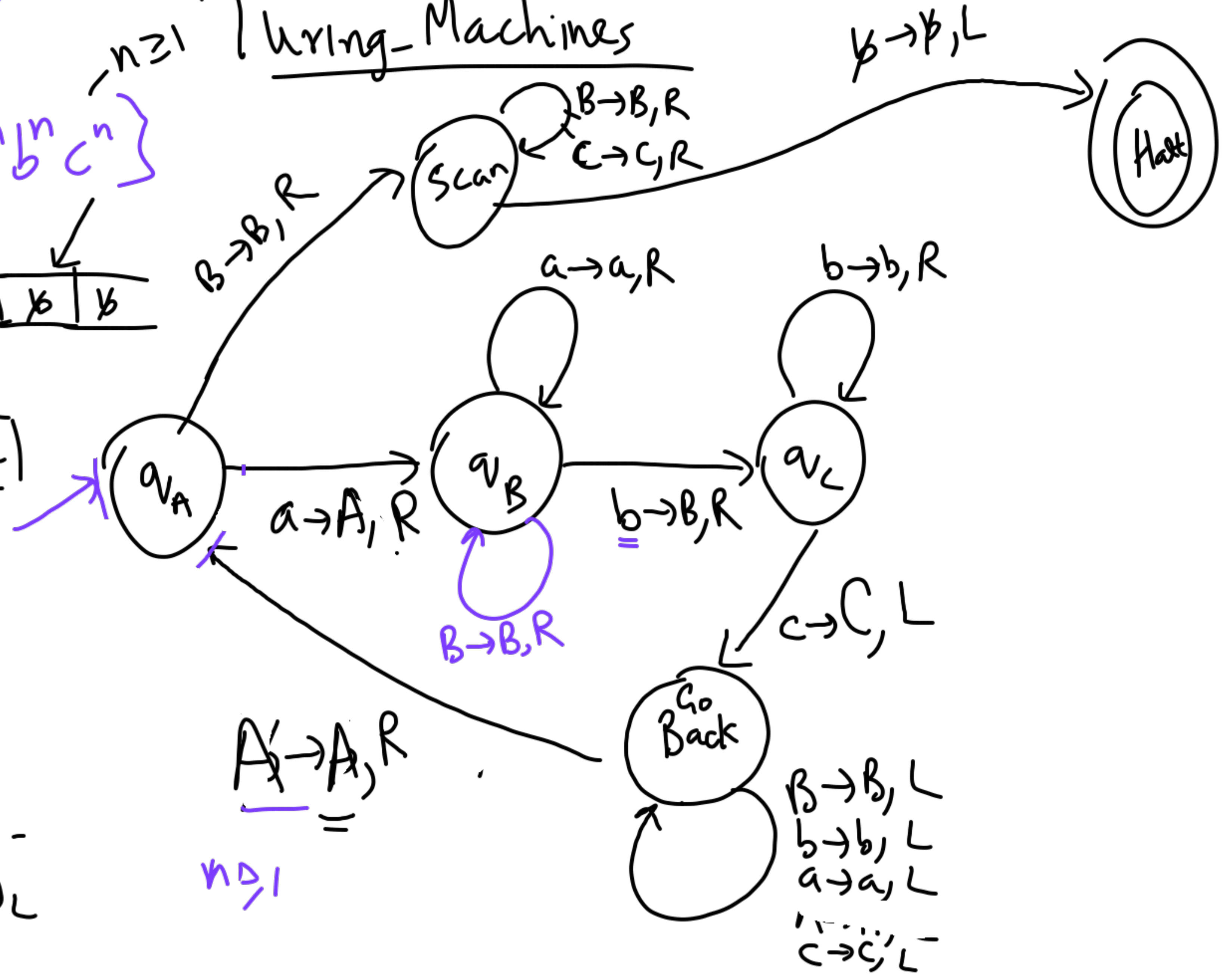
n ≥ 1 Turing-Machines

$$L = \{a^n b^n c^n\}$$



$$\underline{w_L} q_i a \underline{w_L}$$

$$\underline{w_L} A q_i \underline{w_L}$$

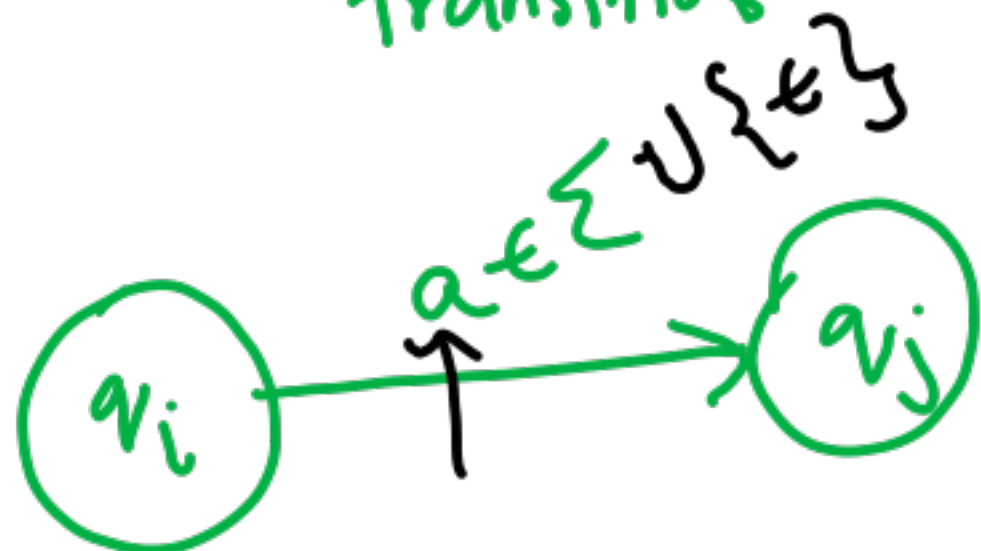


22.8.2025

FSA

$$\langle \Sigma, Q, \delta, q_0, F \rangle$$

transitions
Start



Systematic

NFA + ϵ

$$\delta: Q \times \Sigma \cup \{\epsilon\} \rightarrow Q$$

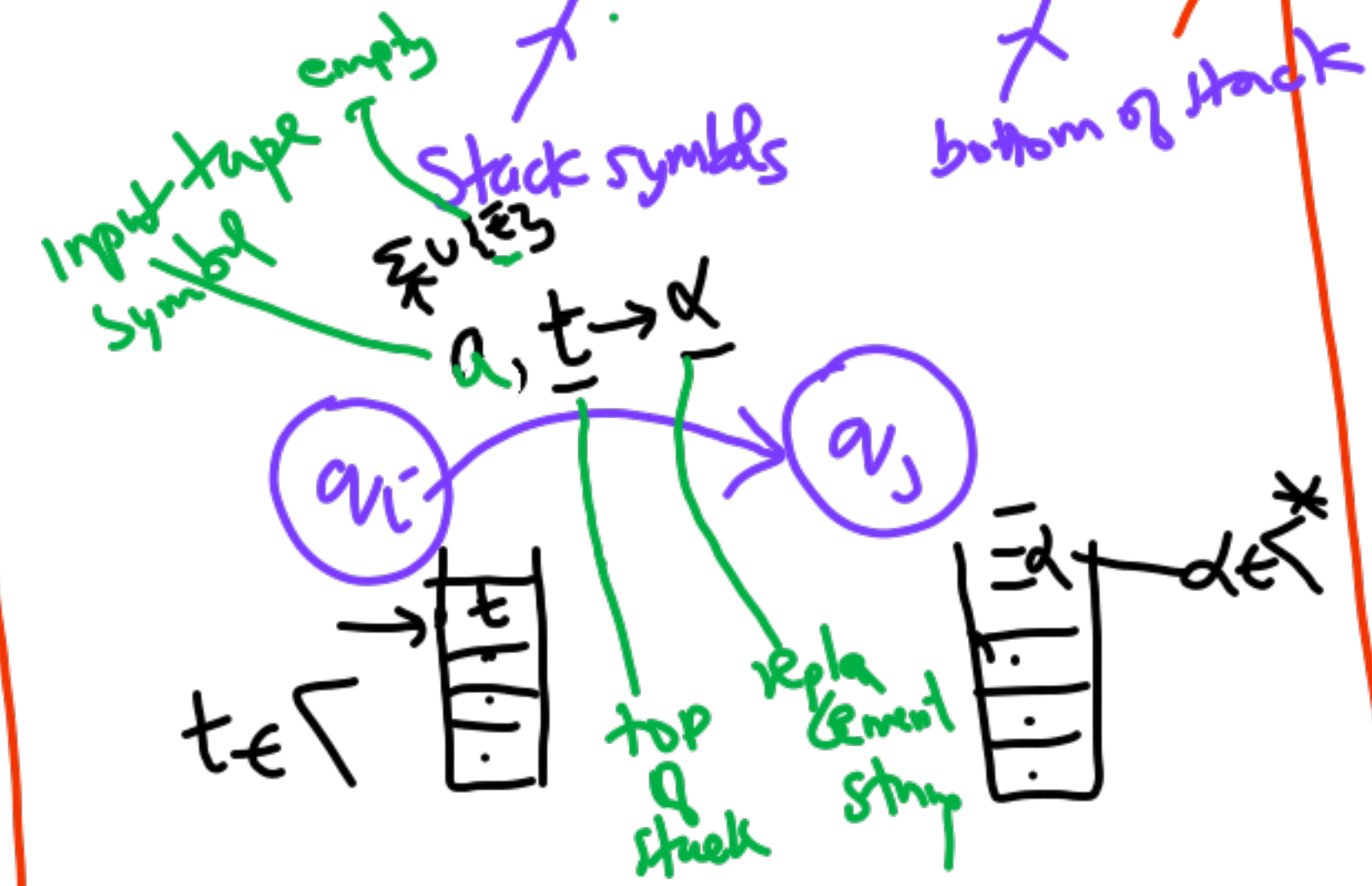
No ϵ -transitions incl. $\emptyset$

function $Q \times \Sigma \rightarrow Q$

exactly one next state

PDA

$$\langle Q, \Sigma, \Gamma, \delta, q_0, z_0, F \rangle$$



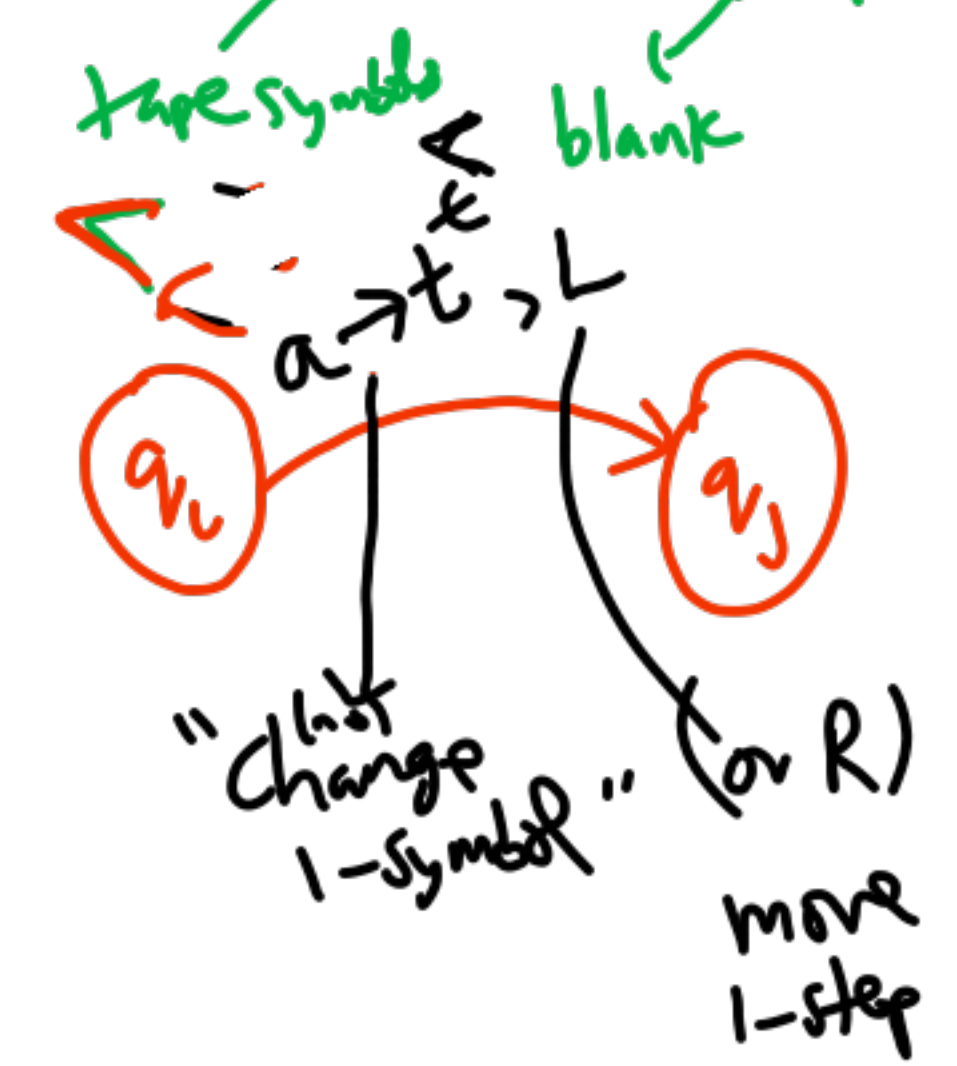
$$\delta: Q \times \Sigma \cup \{\epsilon\} \times \Gamma \rightarrow 2^{Q \times \Gamma^*}$$

at most one next state

no choice $\Rightarrow$ deterministic
no arrow for a, t not

TM

$$\langle Q, \Sigma, \Gamma, \delta, q_0, \epsilon, F \rangle$$



$$\delta: Q \times \Sigma \rightarrow Q \times \Sigma \cup \{\epsilon\}$$

can replace only top symbol

crash if no move!
 $\emptyset =$ possible

Input tape
b | a | ...

PDA M

ID

$\langle q, w, \alpha \rangle$

$\epsilon \epsilon^*$

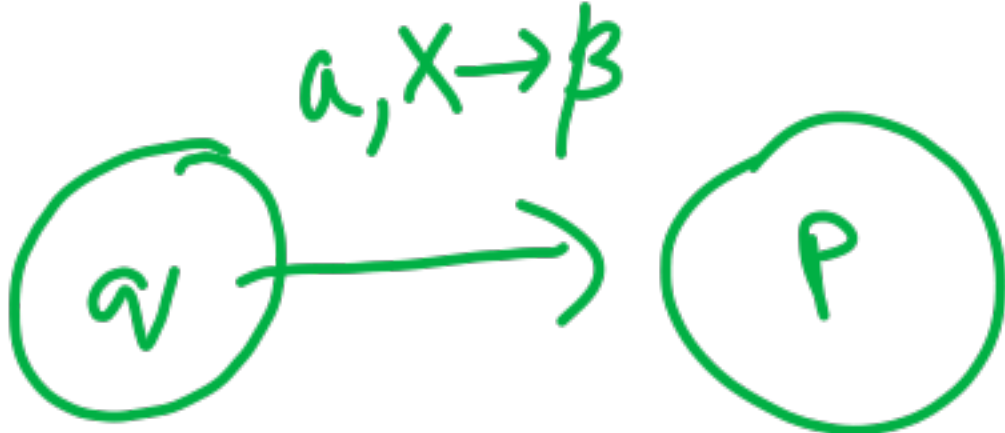
$\epsilon \epsilon^*$

input symbol

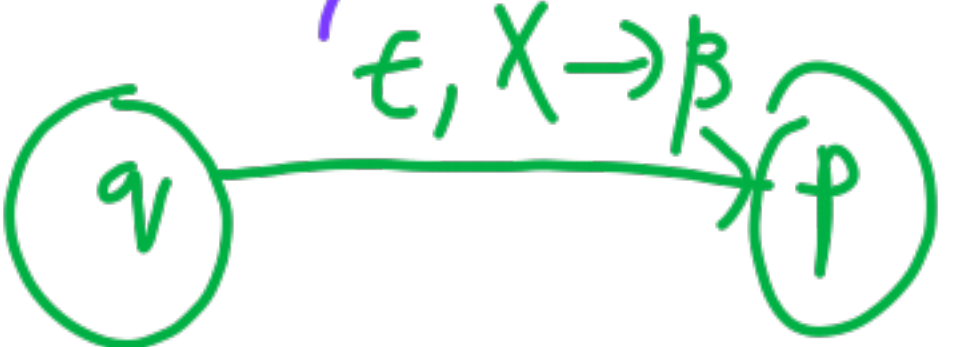
top of stack

(1) $\langle q, aw, X\alpha \rangle$

$\vdash \langle p, w, \beta\alpha \rangle$



(2) $\langle q, aw, X\alpha \rangle \vdash \langle p, aw, \beta\alpha \rangle$



$$L(M) = \{w \mid M \text{ accepts } w\}$$

by final state

$\langle q_0, w, z_0 \rangle$

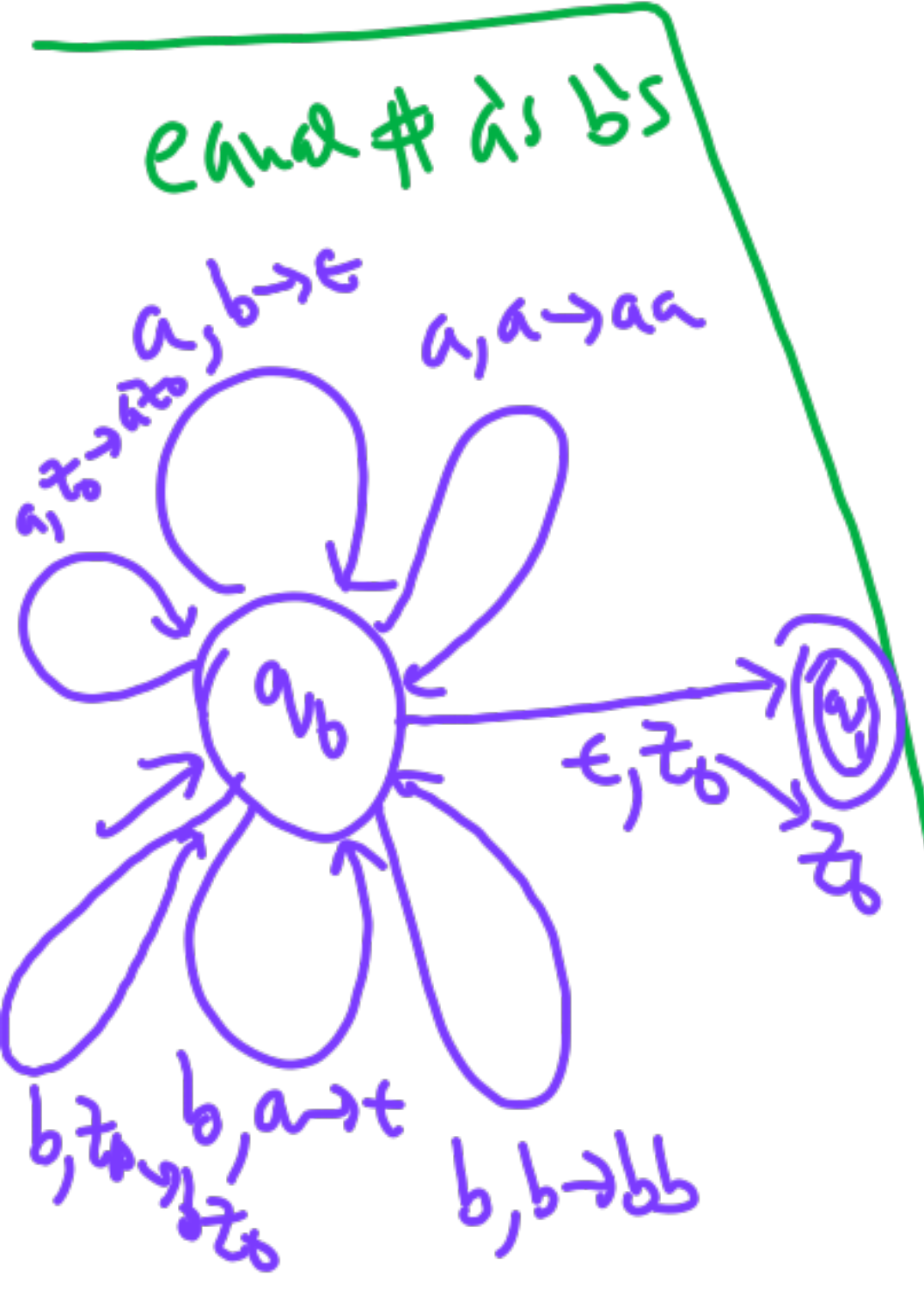
$\vdash^*$

$\langle q_f, \epsilon, \alpha \rangle$

in Γ^*

empty (all input processed)

anything on stack

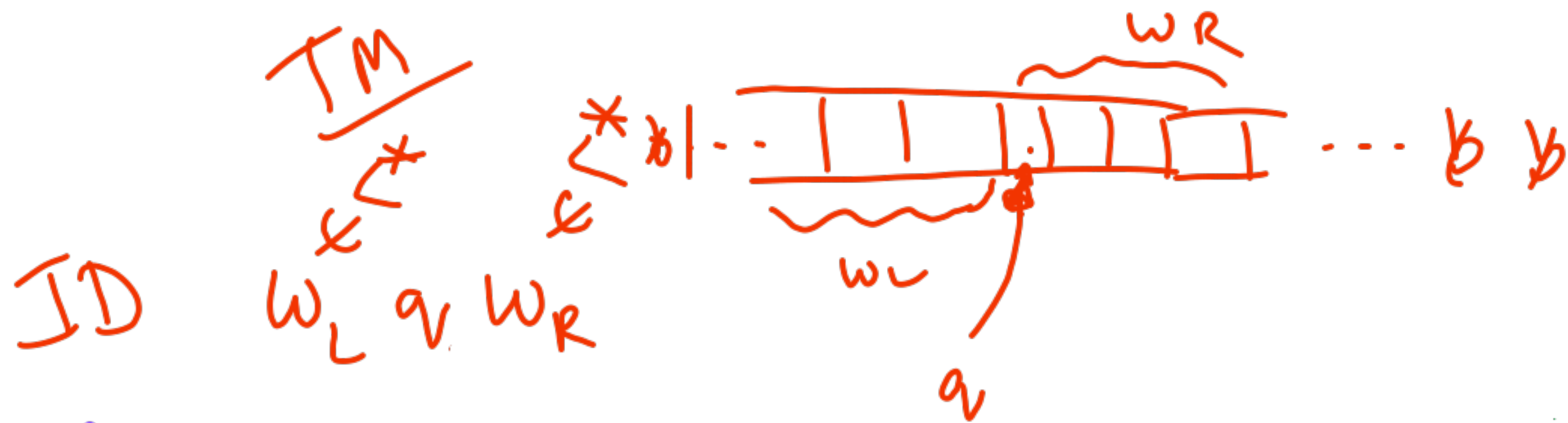


$$\{a^i b^j c^k \mid i=j \text{ or } j=k\}$$



non-bl





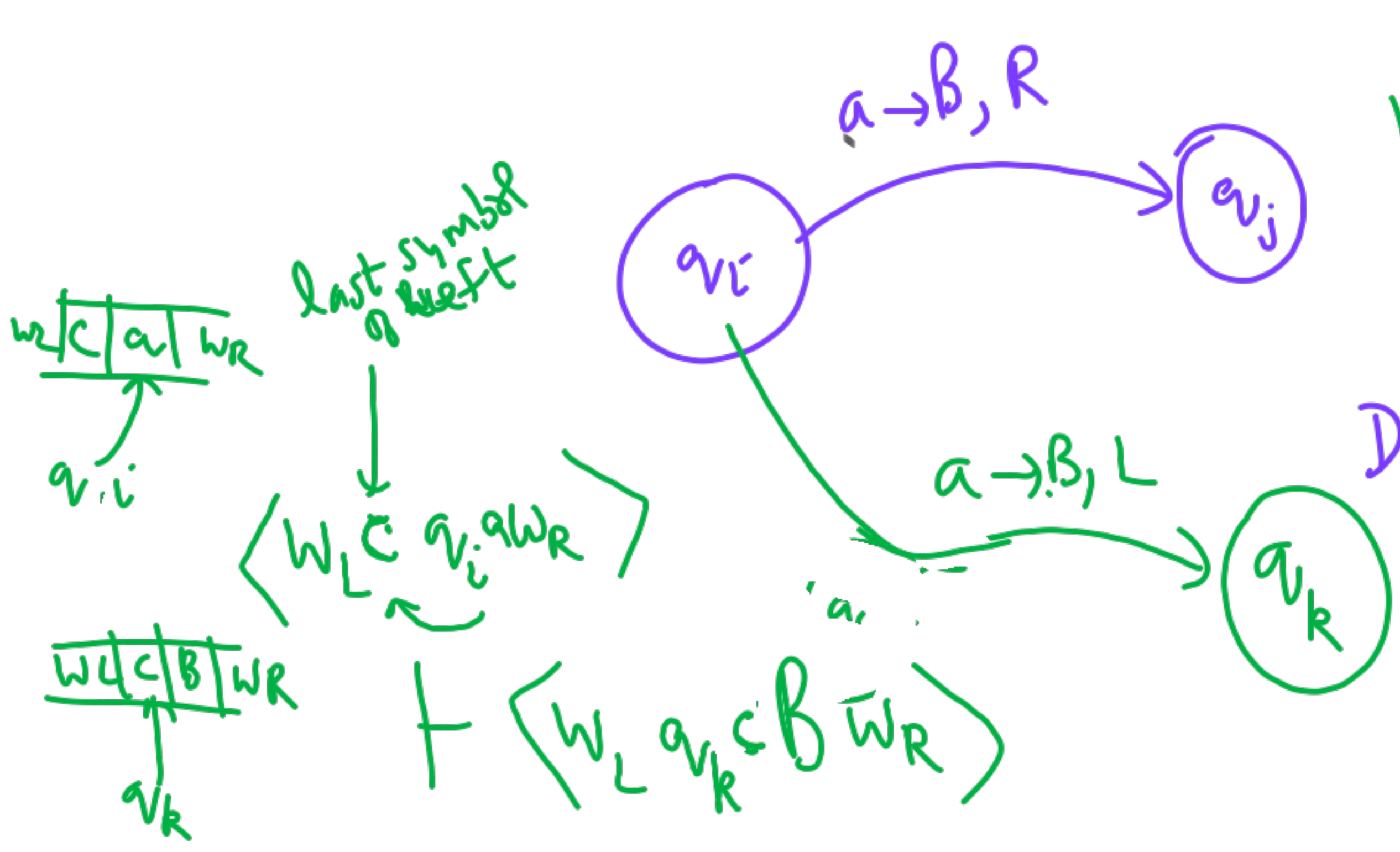
accepts

move $w_L q_i a w_R \vdash w_L B q_j w_R$

$q_0 w \vdash^* \lambda, q_f, \lambda_2$

any string

any string



DO

turing machine	
	a, b
$w w$	
$w \# w$	given

without #

Final state

(Halt)

(no more possible)

29/8/25

CS 310

Given

L_1 L_2

regular

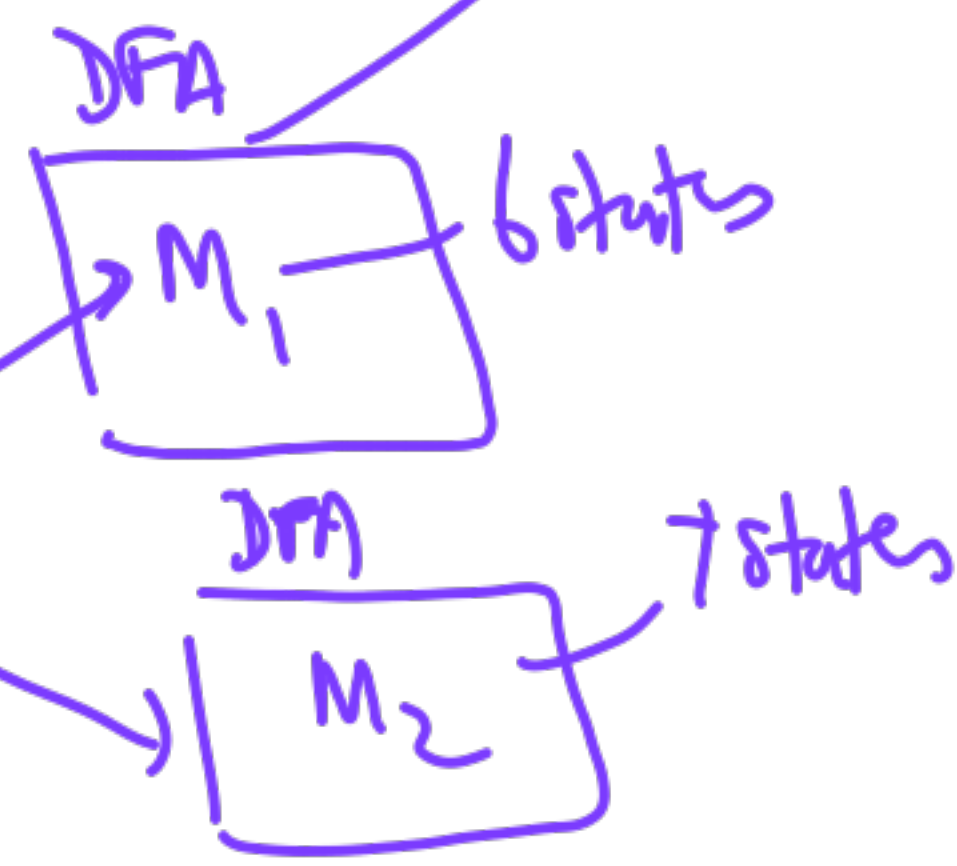
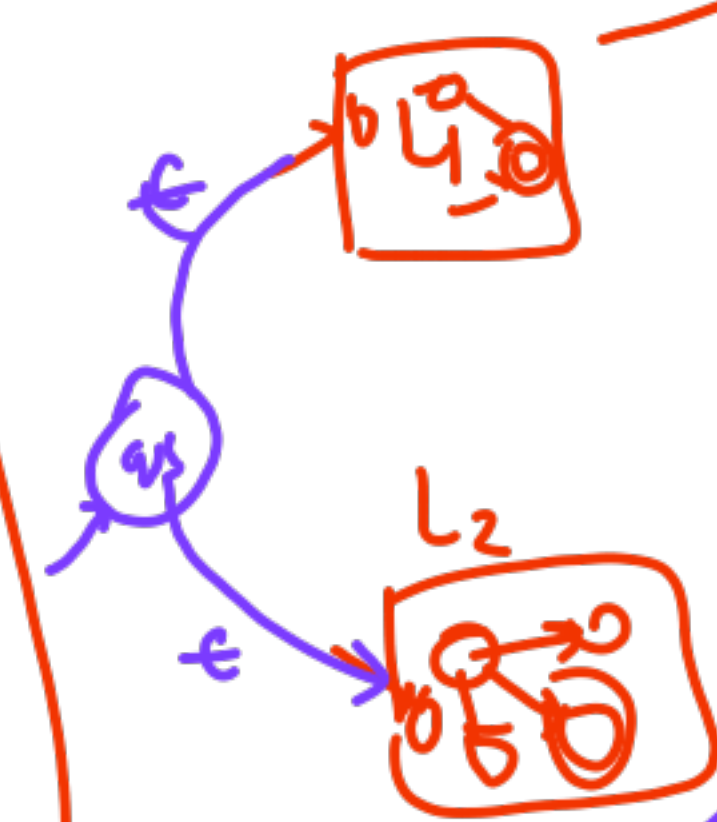
① FSA Closure
Pumping Lemma

② PDA → accept by
empty stack
final state

DPDA vs NPDA

Minimize

Product Automata



✓ Complement $\bar{L}_1$

✓ Union $L_1 \cup L_2$

✓ Intersection $L_1 \cap L_2$

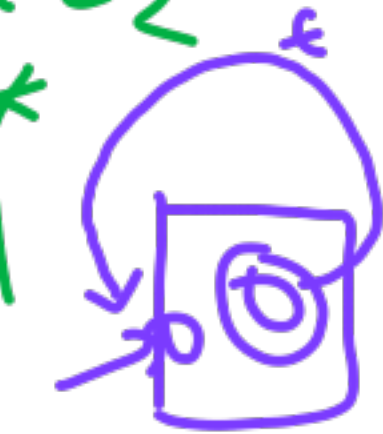
** ✓ Kleene Closure L_1^*

✓ Concat. $L_1 L_2$

Difference $L_1 - L_2$

Reverse L_1^R Symm. Diff -

{ Half
odd
even
Alternate



Pumping Lemma

$L = \{a^n b^n \mid n \geq 0\}$ is not regular

Proof by

Contradiction

(You)

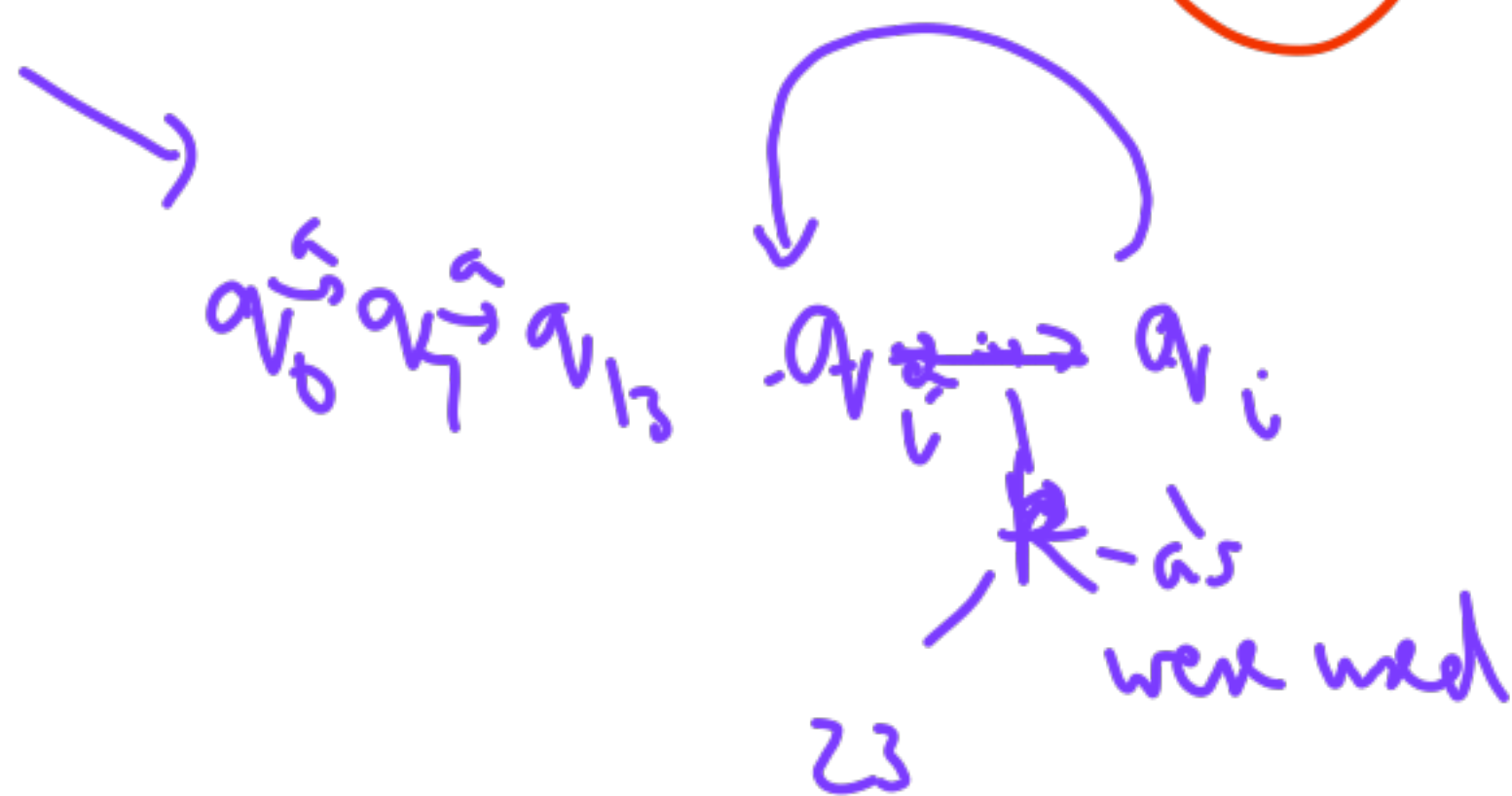
(Adversary)

A

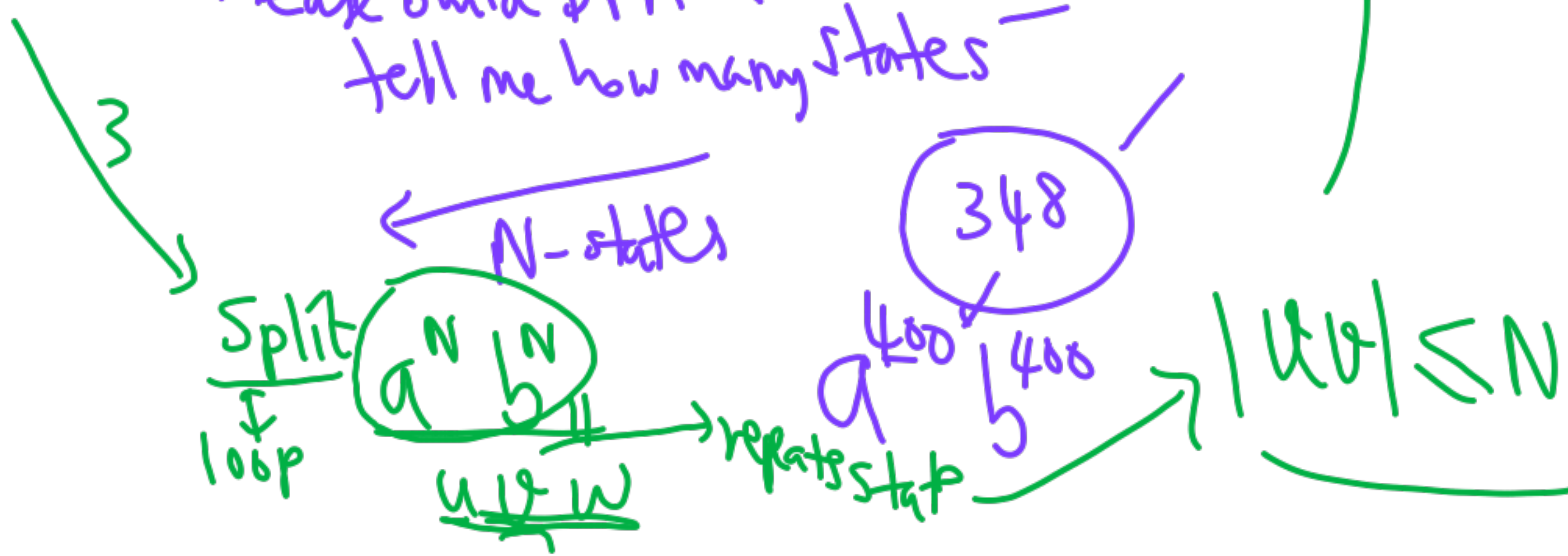
B $\rightarrow$ L is regular
~~Then~~

Then

$uv^i w \in L$
for $i \geq 0$



2 Please build DFA & tell me how many states -

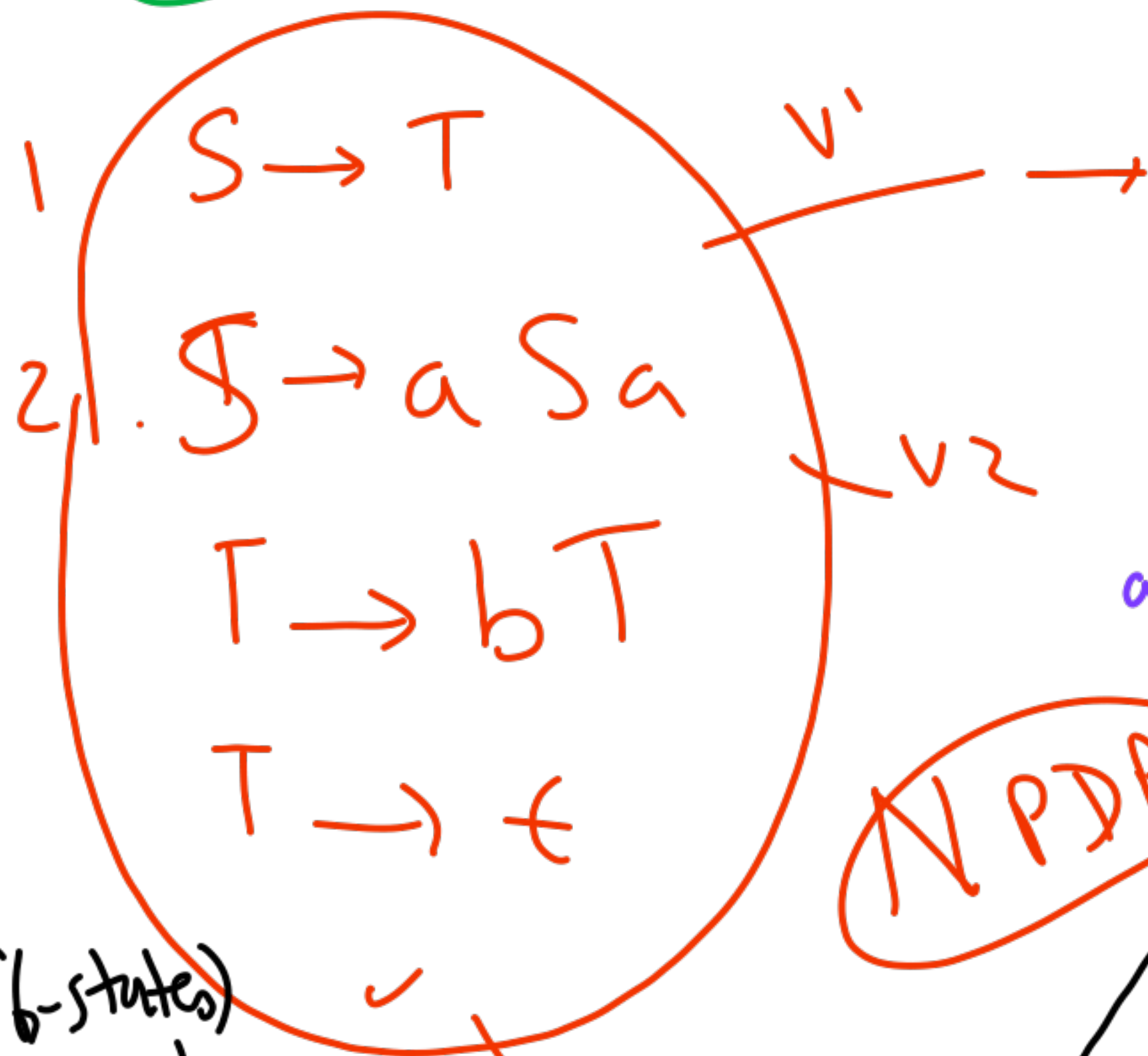


PDA

$$L = \{ a^n b^m a^n \mid n, m \geq 0 \}$$

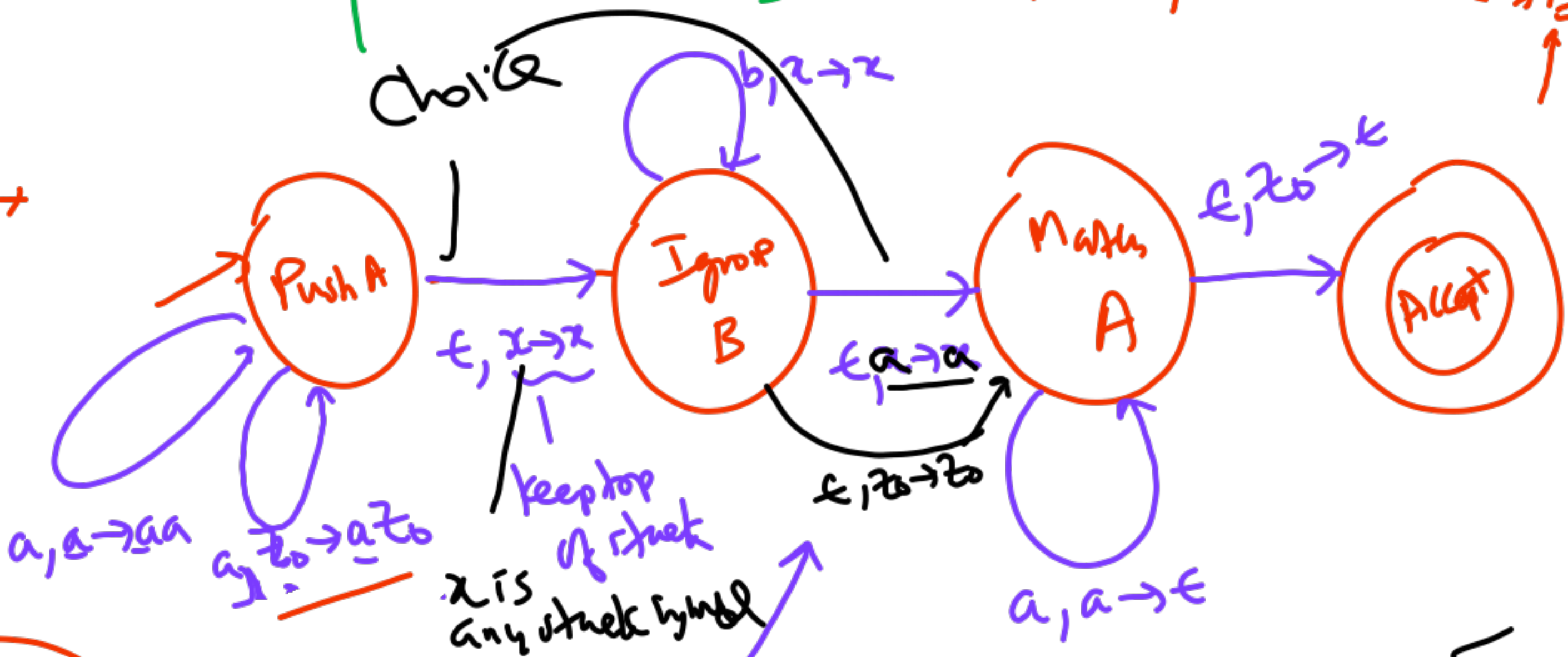
final state

$$(q_0, w, z_0) \vdash \dots \vdash (q_f, \epsilon, \Delta)$$



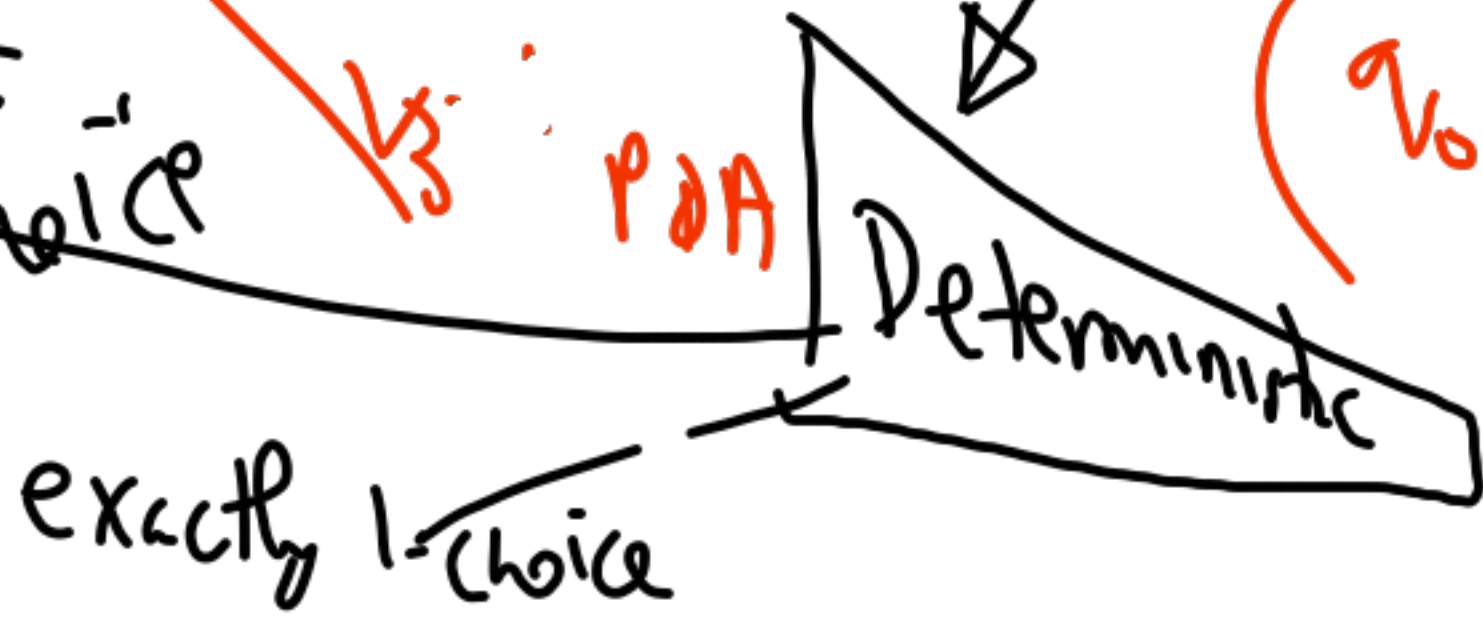
NPDA

Non-deterministic



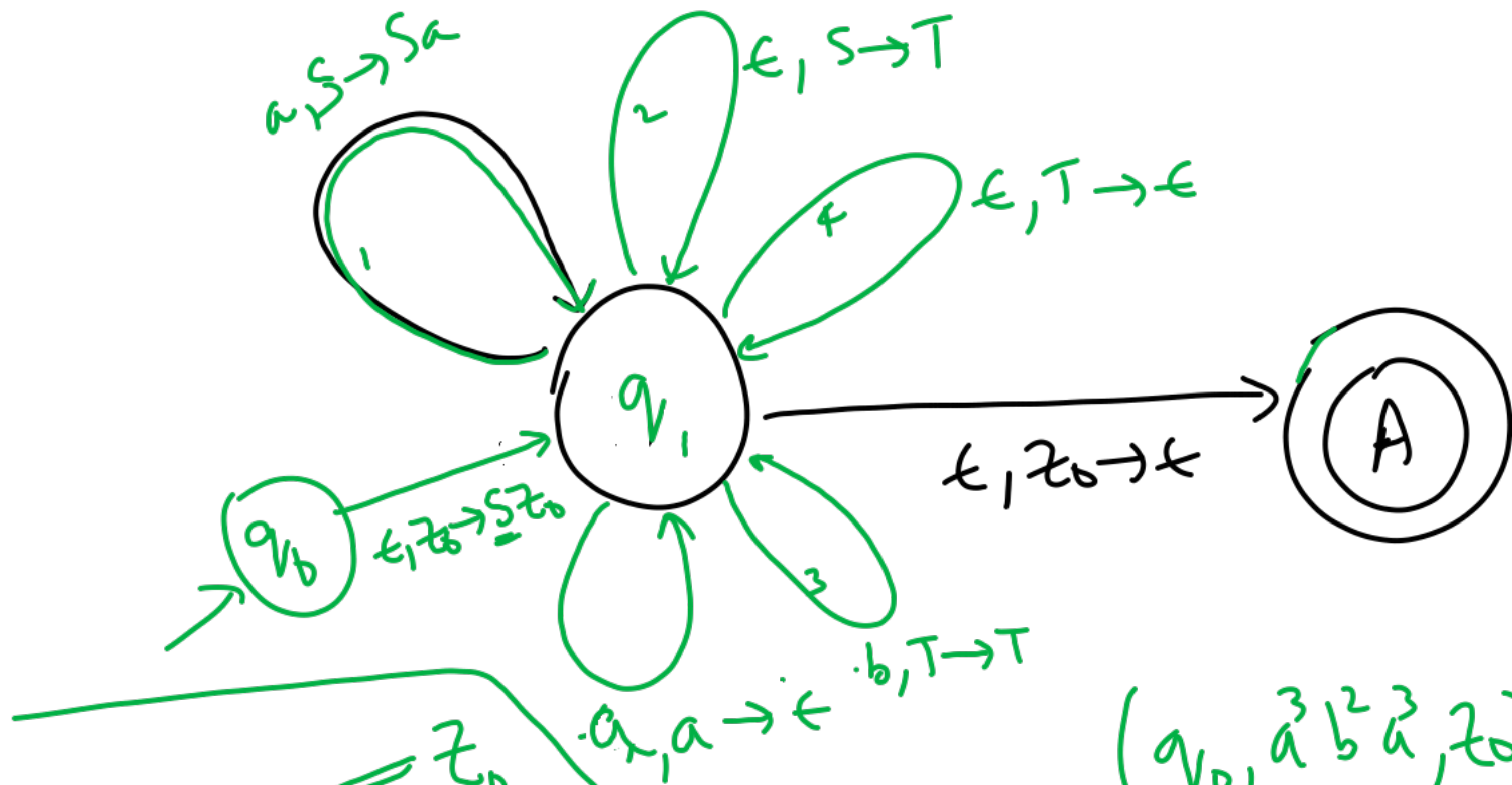
(6-states)

at most one choice



$$(q_0, a^3 b^2 a^3, z_0) \vdash (q_0, a^2 b^2 a^3, a z_0)$$

$$\vdash \vdash (q_0, b^2 a^3, a^3 z_0) \vdash (q_1, b^2 a^3, a^3 z_0)$$



1. $\checkmark S \rightarrow aS \bar{a}$
2. $\checkmark S \rightarrow T$
3. $\checkmark T \rightarrow bT$
4. $\checkmark T \rightarrow \epsilon$

$(q_0, w, \bar{z_0})$
 $\uparrow$
 bottom of stack

$(q_0, \bar{a^3} b^2 \bar{a^3}, z_0)$
 $\vdash$

$\vdash$

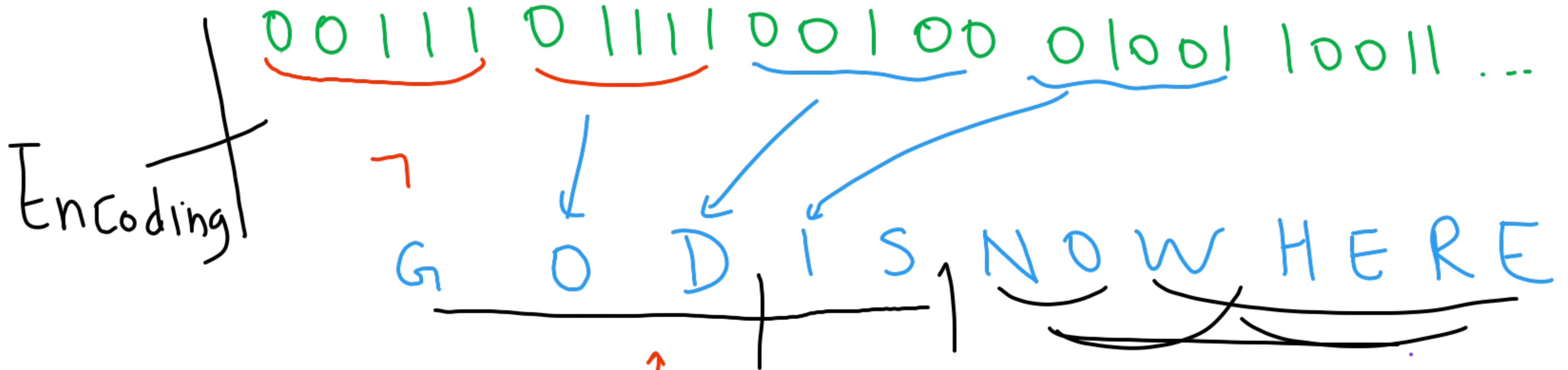
$\vdash (A, \epsilon, \epsilon)$

30.8.2025

Bits / Alphabet

CS 310

→ G. Sivakumar
Siva@iitb.ac.in



EnCoding

"stand on the shoulders of giants"

Natural Language
Formal → we define the Syntax, Semantics

AMBIGUITY

3 models
 Finite State Automata FSA
 Push Down Turing machine PDA TM

What

Computer



Science & Engineering

Turing test

Abacus

Slide rule

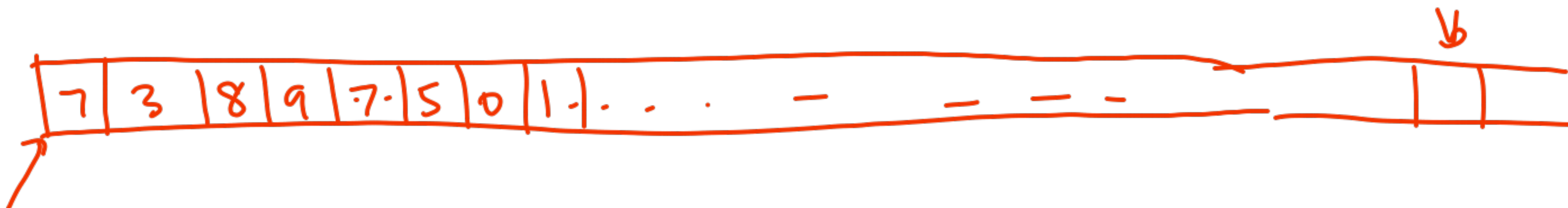
Casio FX

mobile phone

⋮

Hilbert's 10th problem
 $3x^2 - 5x + 7 = 0$
 $\rightarrow 7x^3y^2 - 9x^2y^3 + 13xyz$
 $+ \dots = 0$
 undecidable

Input Type



Is it
→ divisible by 7?

→ Give a solution (diagram)
that any 10-year old
can use to answer
in "real-time"

→ FSA

PDA

Input Tape

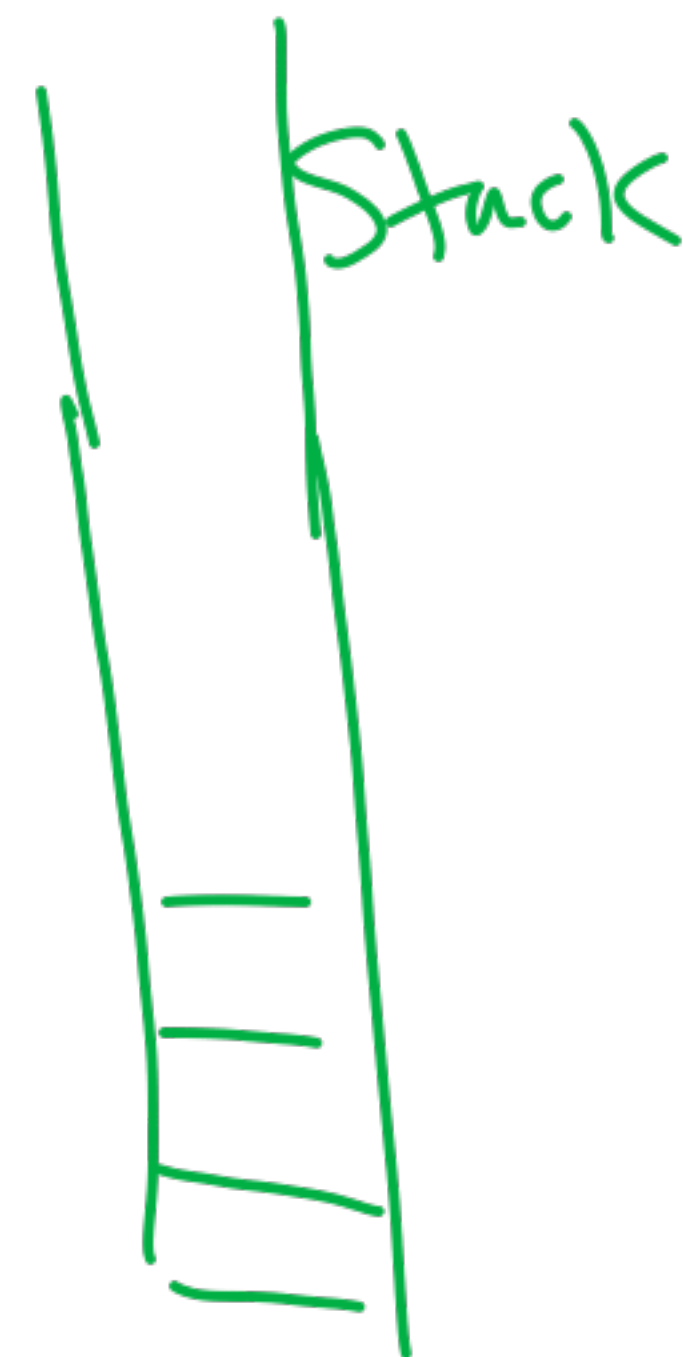
Is the string of the form $7^n 9^n$?

$n \geq 1$



Is it
a palindrome?

→ Non-determinism
(guess)



Turing machine

turingmachinesimulator.io

Input
tape

Read/write



① replace
a digit by some symbol.

② move 1-step L or R q_k

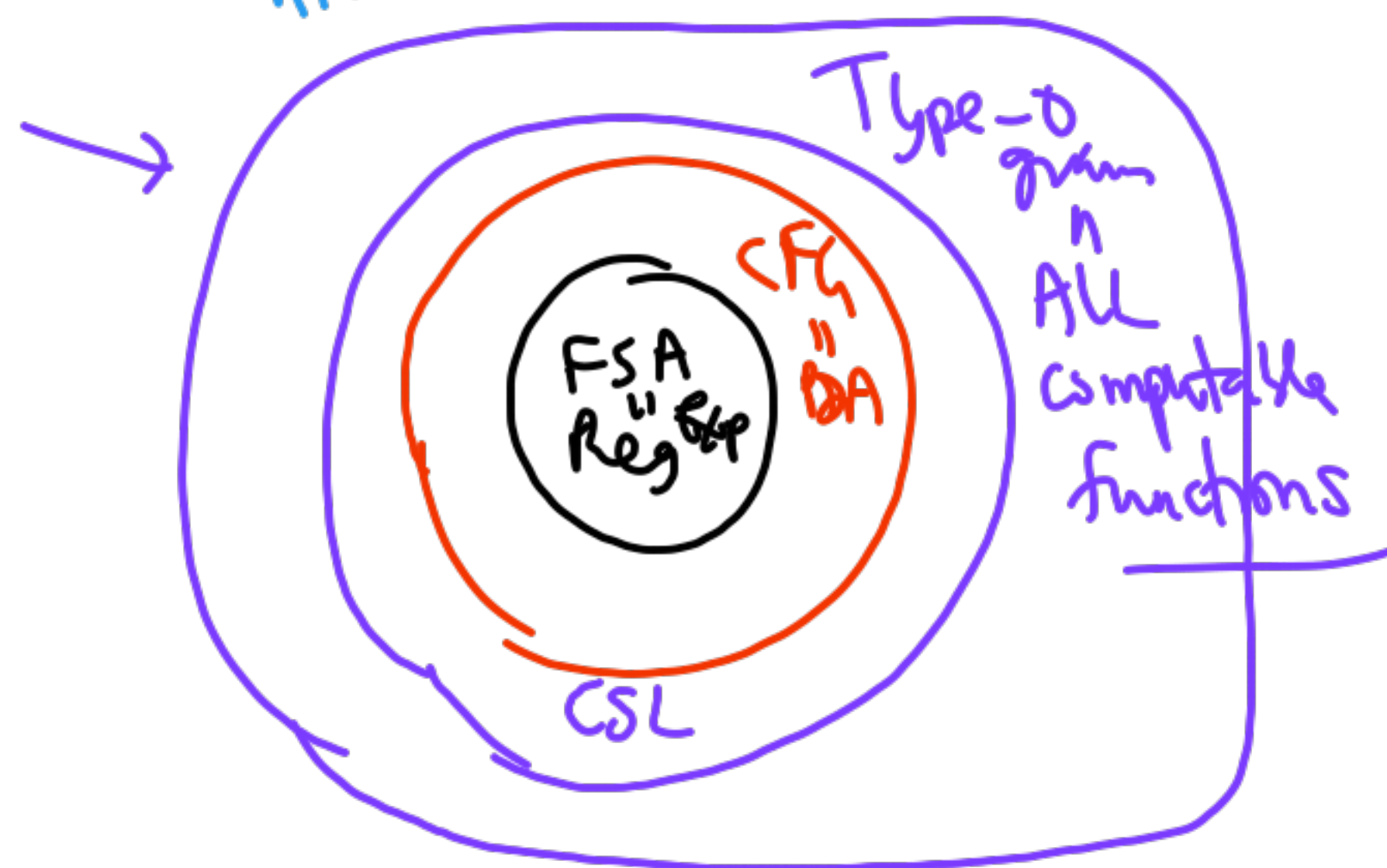
Is n prime?

Chomsky
hierarchy

Machines	Language
FSA	Regular Expression
PDA	Context free grammar
TM	Type 0 grammar

$(0+00)^*1^*$

$S \rightarrow aSb$
 $S \rightarrow ab$
 ~~$B \rightarrow cb$~~



lhs
Can have
any
string

$a^n b^n c^n$
 $aSc \rightarrow Sbc$

a^n
 S^n

12.9.2025

CS 310

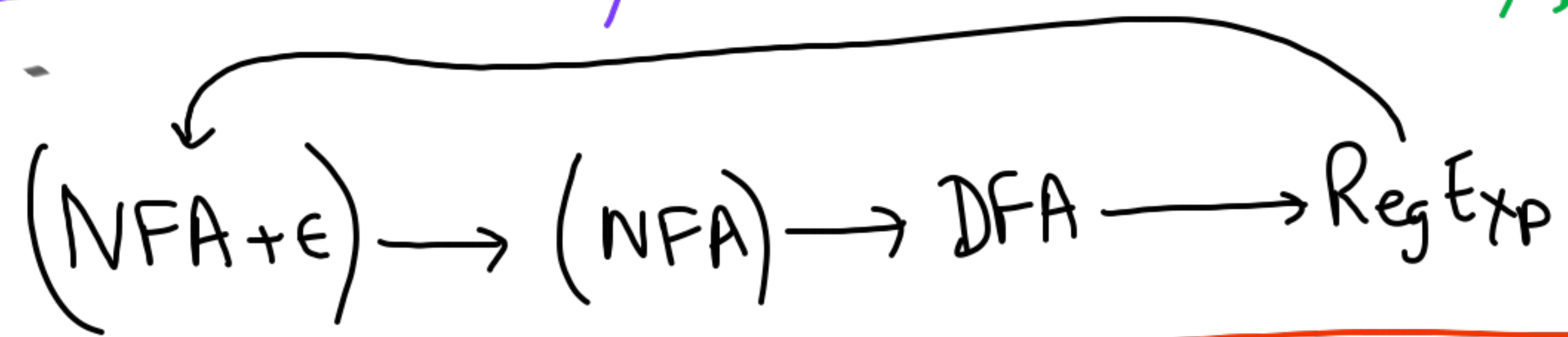
Languages — Machines

Canonical (Normal) Forms

→ Closure Properties
→ Determinism

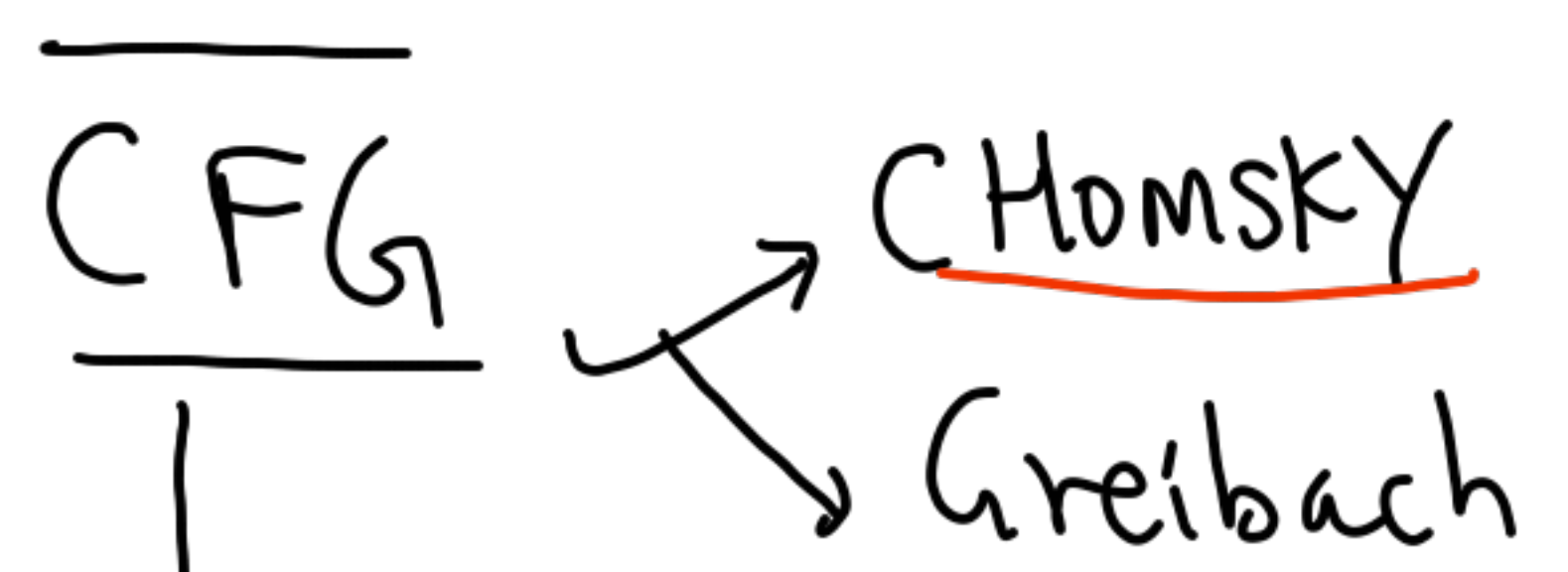
①

10 mins



10 mins

②



50 mins

③

PUMPING Lemma

MIDSEM

→ Sat 20/9

~~LA 01, 02~~
201

Ensemble
Federated

Moodle

NFA + ϵ

NFA

DFA

Reg Exp

ϵ -closure(S_7)

no new state

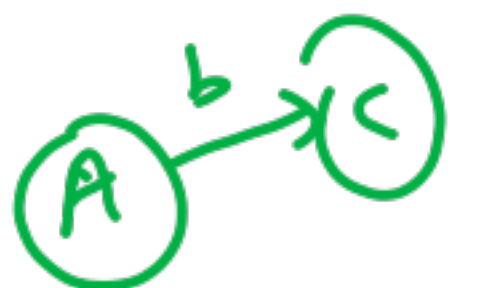
easy

Brute-force

Right Linear Grv

for small examples

Subset 2^R states



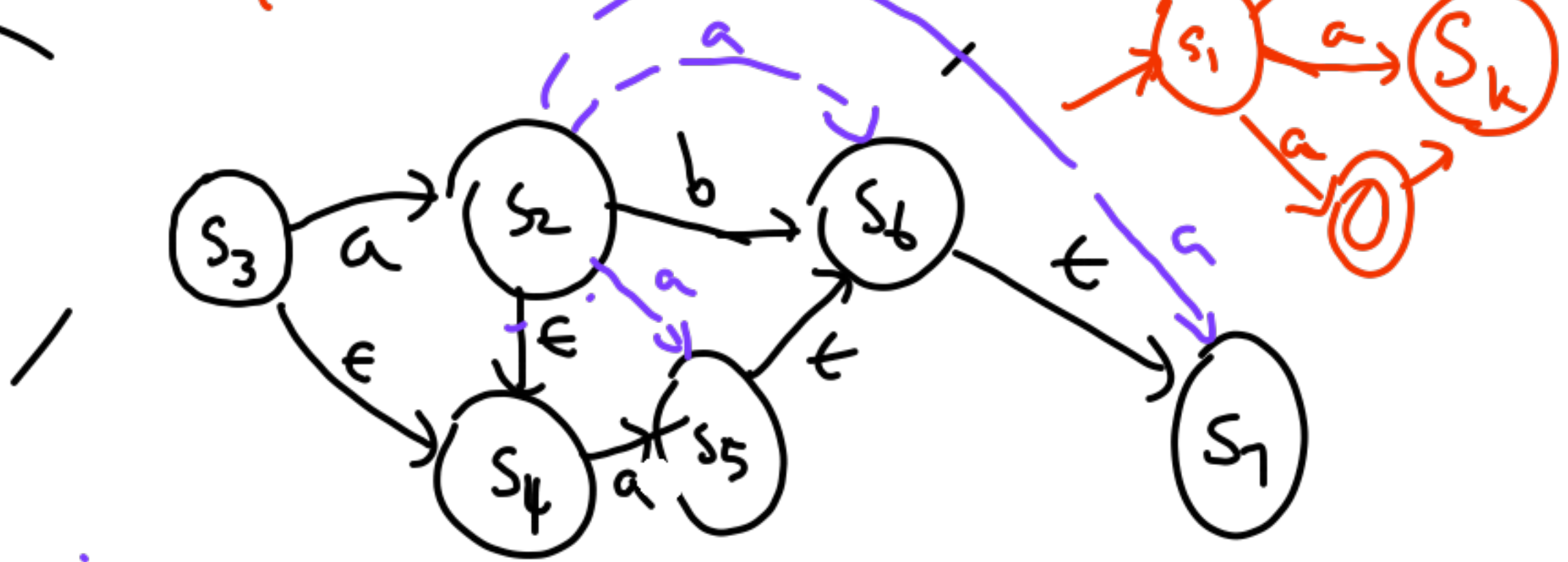
$A \rightarrow bC$

Dynamic programming

Intelligent

generate "as needed"

lazy evaluation



from $S_3 \xrightarrow{a}$

which states $\xrightarrow{a}$ how many



CFG Normal Forms (Σ, V, S, P)

$S \rightarrow ABCD$

Chomsky

$S \rightarrow AX_1$

$X_1 \rightarrow BX_2$

$X_2 \rightarrow CD$

for any $A \in V$

type 2 (T1) $A \rightarrow c$

type 2 (T2) $A \rightarrow BD$

only 2 types of rules



Sheila Grebach

only 1 type of rule

$A \rightarrow \alpha$ anything

$c \in \Sigma, \alpha \in (V \cup \Sigma)^*$

one alphabet

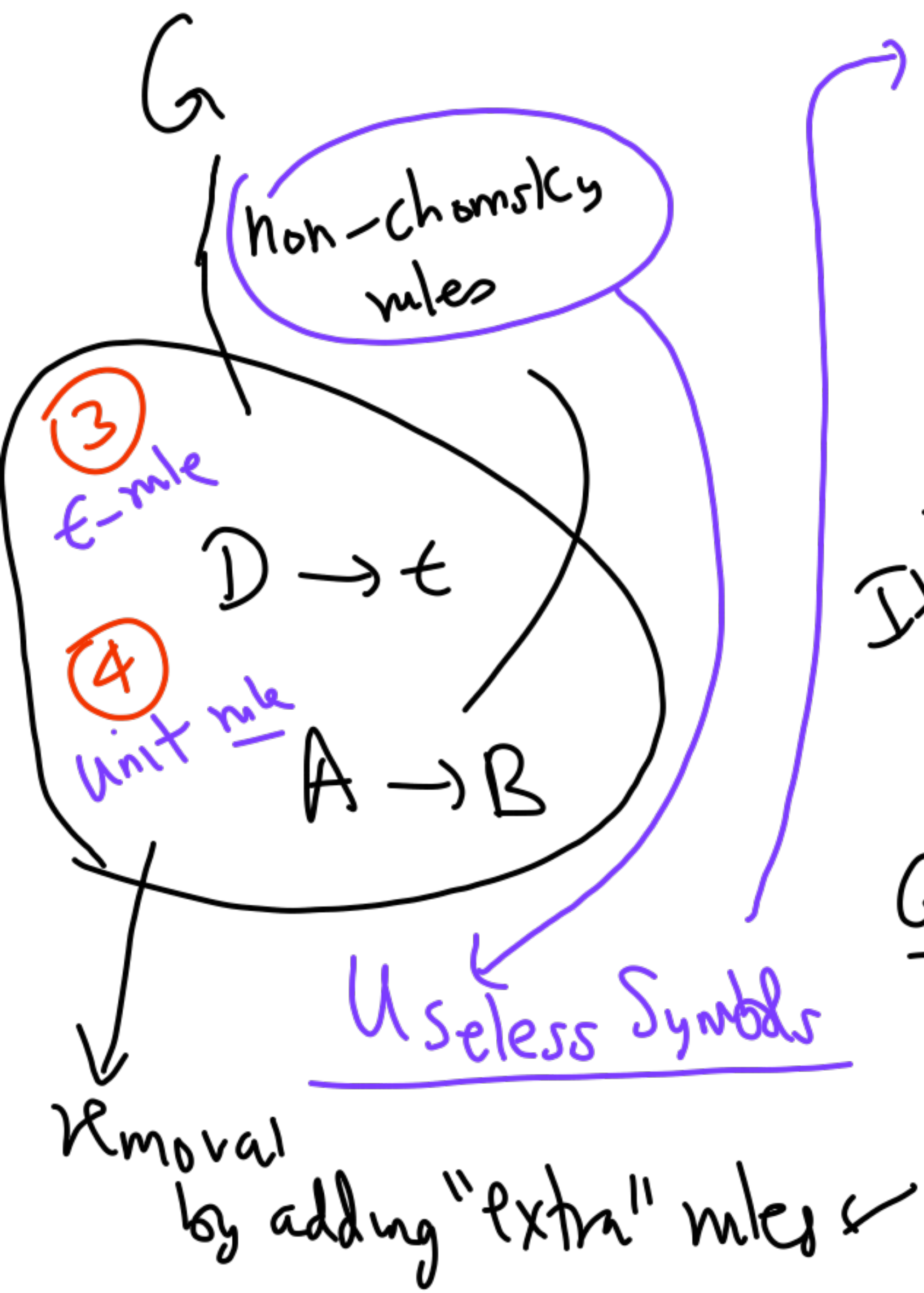
~~$S \rightarrow X_1 S X_2$~~

$S \rightarrow X_1 S_2$

$S_2 \rightarrow S X_2$

$S \rightarrow X_1 X_2$

$S \rightarrow \bar{a} S \bar{b}$
 $S \rightarrow \bar{a} \bar{b}$
 $\bar{x}_i \bar{x}_i$



Useful iff both productive & reachable

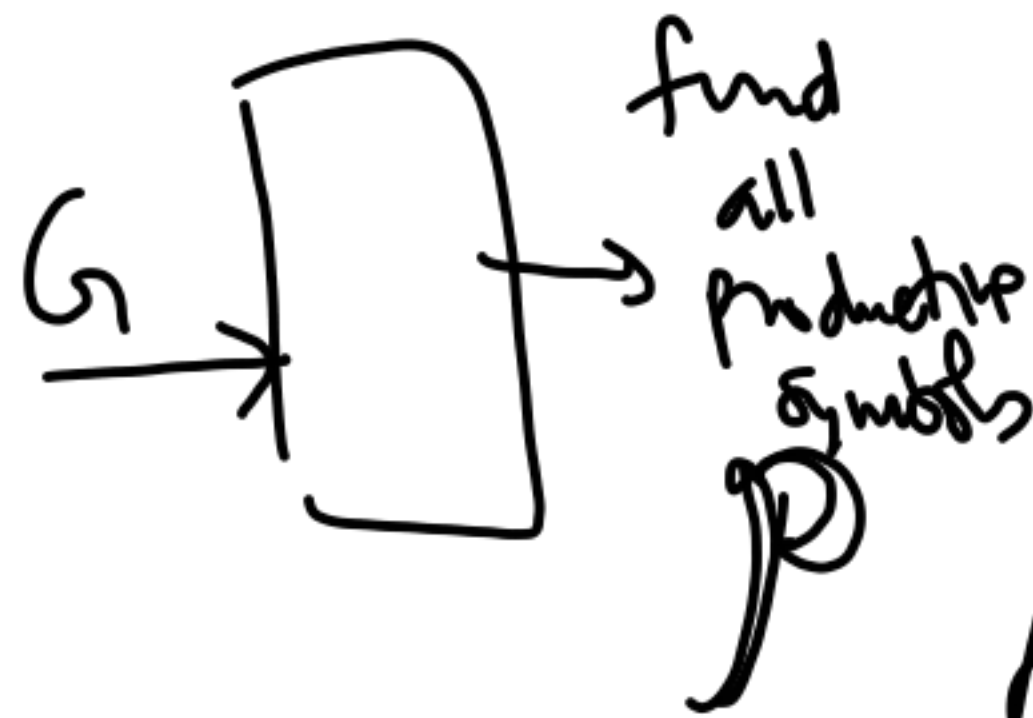
① productive

② reachable

$$A \xrightarrow{*} w \in \Sigma^*$$

$$S \xrightarrow{*} \alpha_L A \alpha_R$$

Iterative Fix-Point Method



① $P_0 = \{ \text{all symbols } D \rightarrow \alpha \in \Sigma^* \}$

② $P_{i+1} = P_i \cup \{ \text{all symbols whose rules have RHS} \in P_i \}$

③ until $P_{i+1} = P_i$

1-step



Pumping lemma

Chomsky
Normal
Form

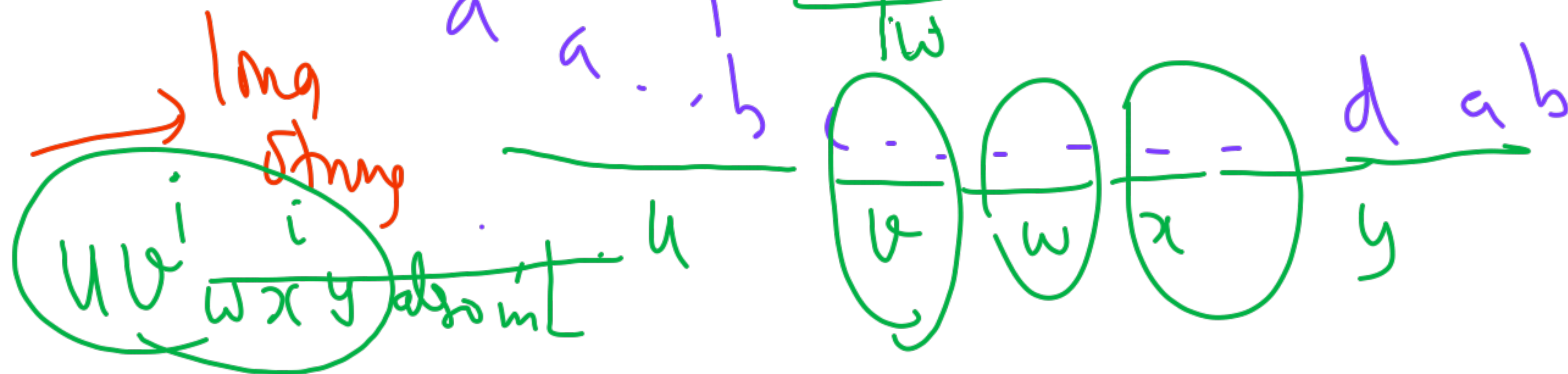
Pigeonhole

"Some variable
repeats"

Derivation tree

S path
longer
than $|V|$

min depth
to derive
string of
length n
is
 $\log(n)$



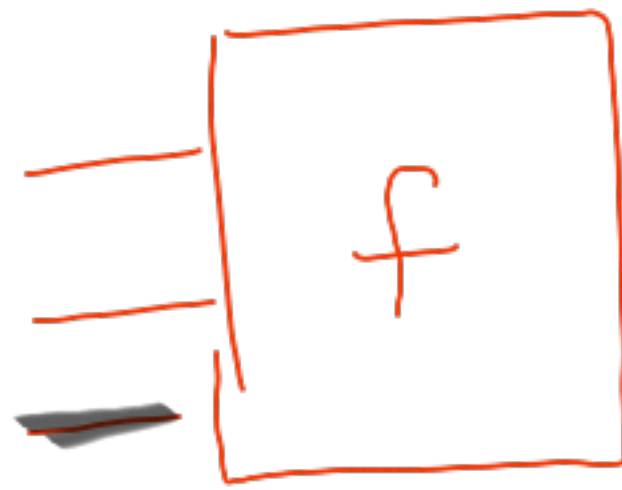
22.9.25

"Computer"

$$N = \{1, 2, \dots\}$$

Inputs

Output



function.

$$N^k \rightarrow N$$

$$N \rightarrow N$$

Language. $\Sigma \Sigma^*$ $\rightarrow \{0,1\}$

there is a language that a TM cannot ACCEPT

Reg } Pumping
CFG } lemma

① define

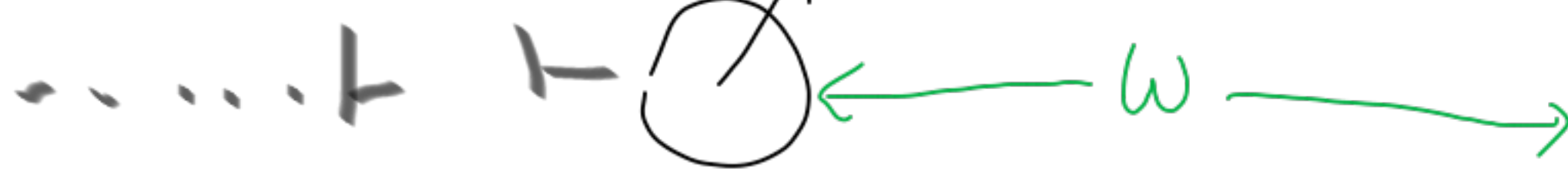
I.D.

T_M

tape
 $\alpha q_k \beta$
right

$M = \langle Q, \Sigma, \Gamma, \delta, q_1, b, F \rangle$

$q_1 w$



Start

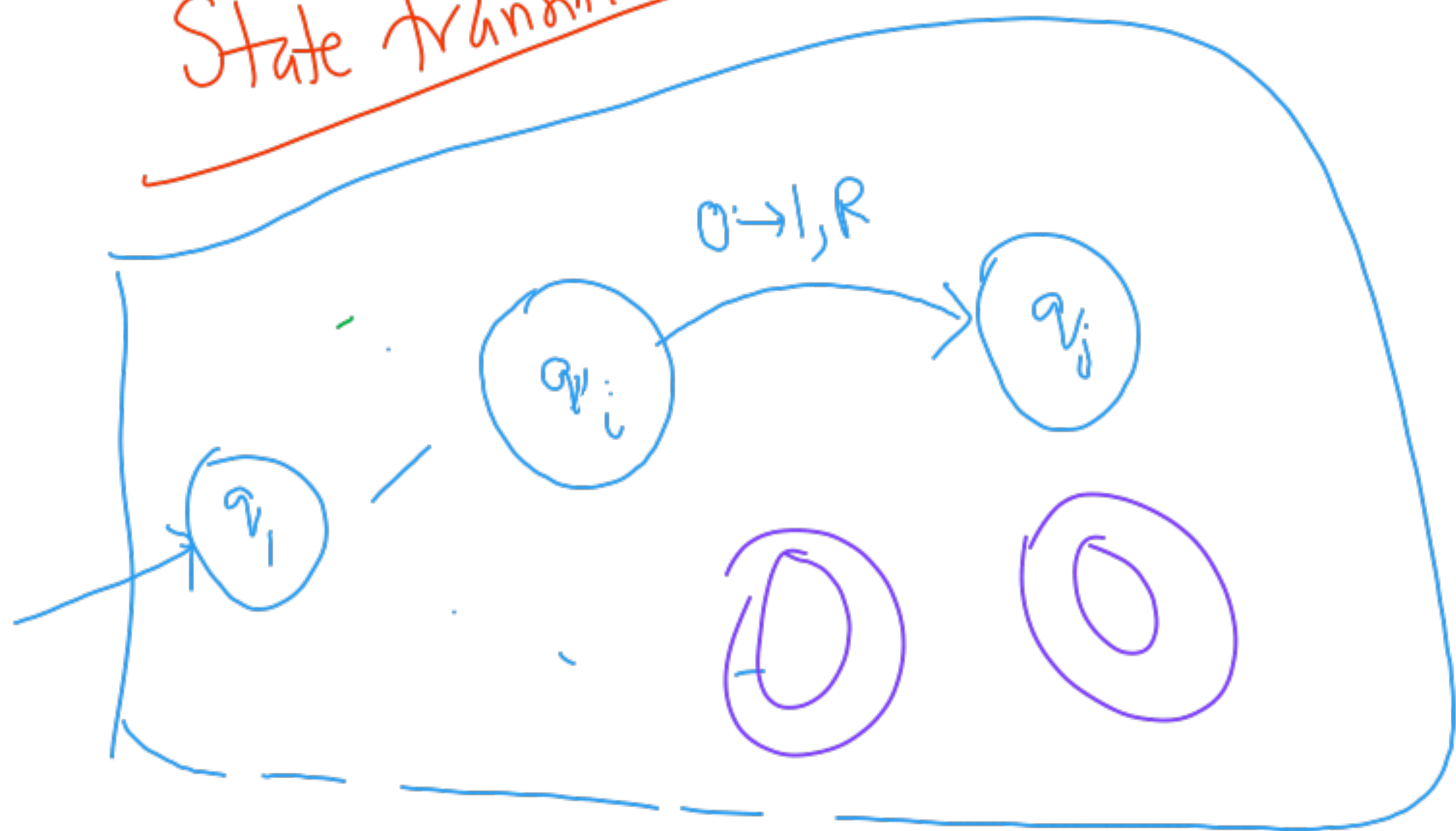


transitions

$Q \times \Gamma \rightarrow Q \times \Gamma \times D$

tape symbol
direction
L, R

State transitions



Accept M accepts w

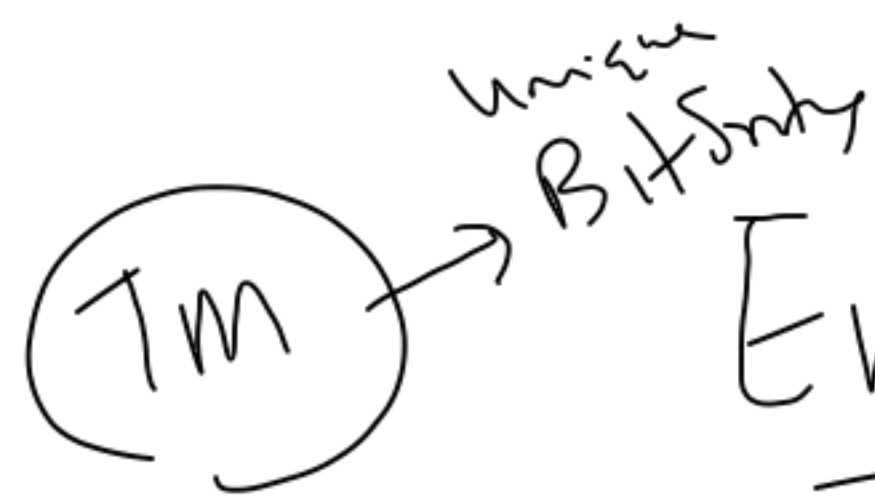
①
Final state

②
machine Halts
 $F = \emptyset$
(no $\textcircled{0}$ states)

$q_1 w \vdash \dots \vdash \alpha q_k \beta$

$q_k \in F$ $\textcircled{q_k}$

$q_1 w \vdash \dots \vdash \alpha q_k \beta$
no further move



Encode a TM as a bitstring

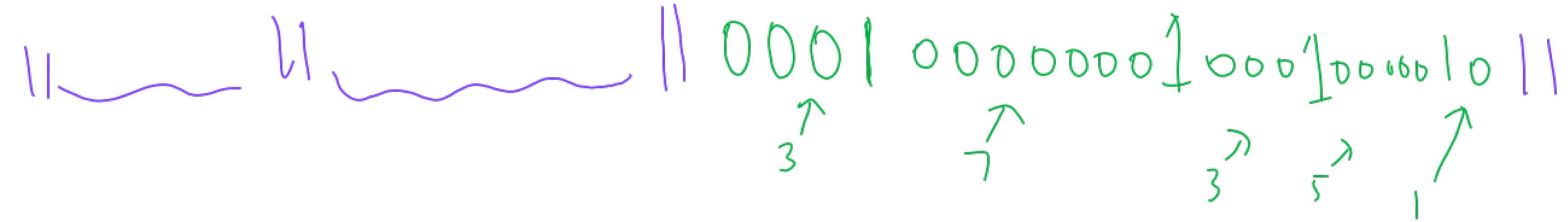
$$\Sigma = \{0, 1\}$$

$\uparrow \quad \uparrow$
 $a_1 \ a_2 \dots a_k$

$$\Gamma = \{X, Y, \dots\}$$

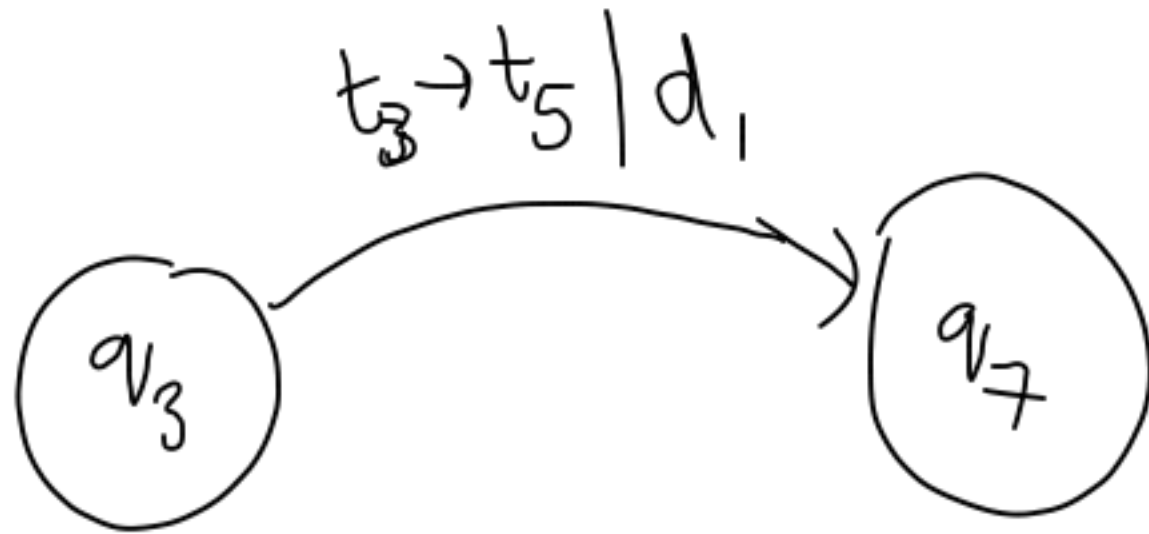
$\uparrow \quad \uparrow \quad \dots$
 $t_1 \ t_2 \ \dots$

///



///

single move



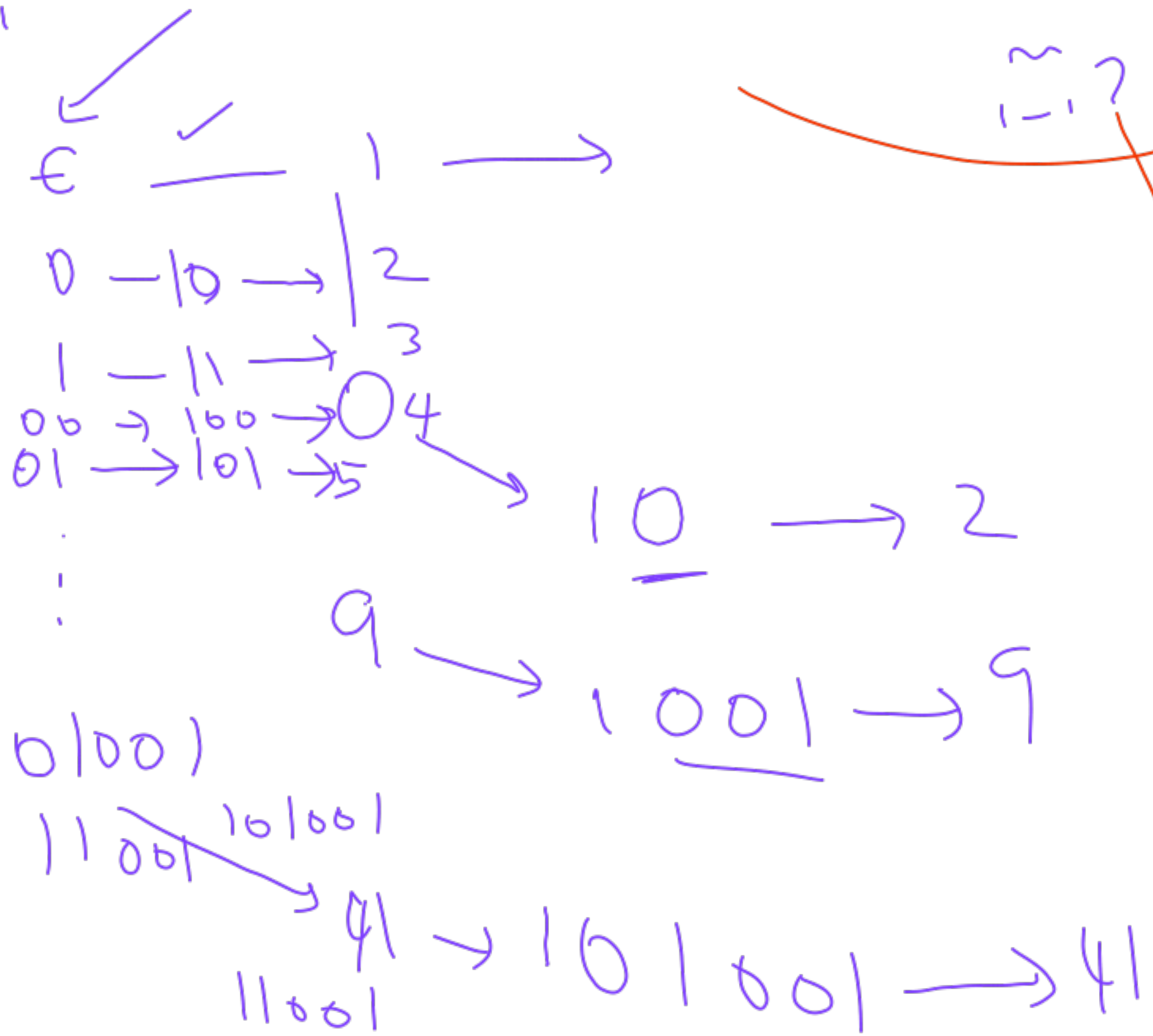
$$D = \{L, R\}$$

$d_1 \ d_2$

Bitstrings $\leftrightarrow N \xleftrightarrow{\{1, 2, \dots\}} N^k$

Unique prime factorization

~~1-1?~~



Binary form of Decimals

1	—	€	1
2	—	0	2
3	—	1	3
4	—	0 0	4
5	—	0 1	5
6	—	1 0	6
7	—	1 1	7
8	—	0 0 0	8
9	—	0 0 1	9
10	—	0 1 0	10
11	—	0 1 1	11
		1 0 0	

leading 1

Decide w.e.l?
 Yes
 No

Languages

Recursive

halts on all inputs $w \in L$
 in accept state
 & halts in non-accept state if $w \notin L$

prime?

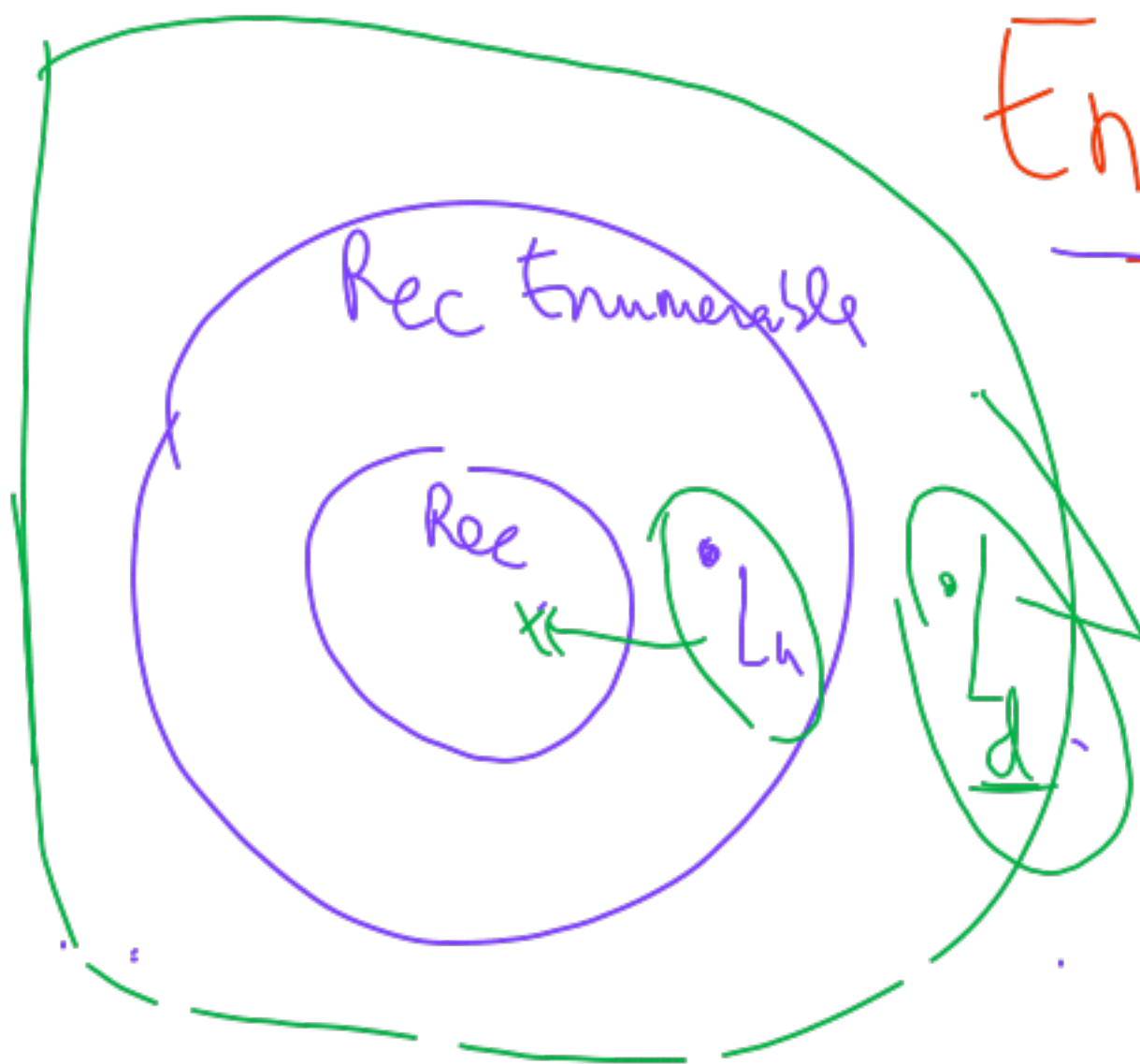
L is

Recursive

Enumerable

if there is a TM M
 such that M accepts L

programming



"no TM can be built"

$$7x^3y^2 + \dots = 0$$

Hilbert's 10th prob

undecidable

Semi-decidable

1.10.2023

(Emily)

Post Correspondence Problem

Sequence
→

$$X = \{ x_1, x_2, x_3 \} = \{ 0, 1, 011 \}$$

→

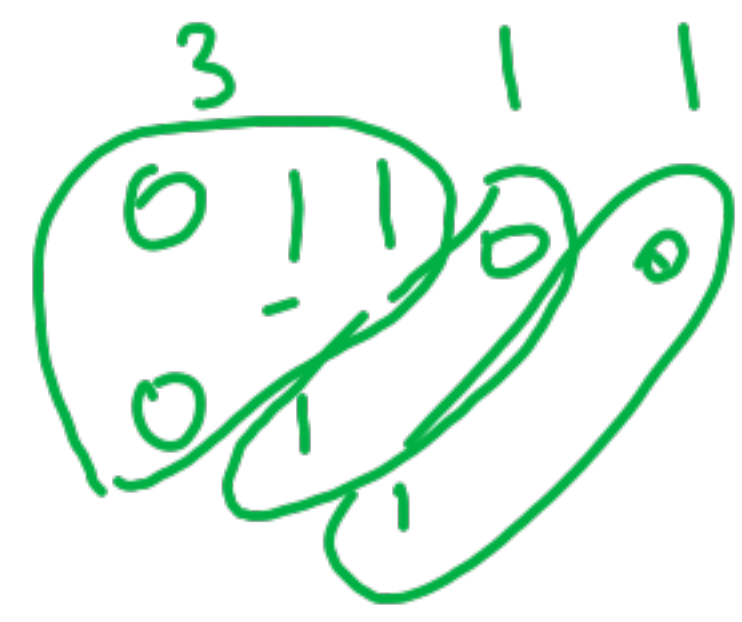
$$Y = \{ y_1, y_2, y_3 \} = \{ 1, 011, 0 \}$$

Corresponding elements

Solution:

$$i_1, i_2, \dots, i_m$$

↑
3
2
1
sequence



Smallest Solution is length 75

Assignment 4 on Moodle

Quiz 2 (Oct 15) Wed

90% Started slides

Future

Identical

x_{i_1}	x_{i_2}	x_{i_m}
y_{i_1}	y_{i_2}	y_{i_m}

- 1 - TM $\equiv$ Type 0
- 2 - Rec. Fn. theory
- 3 - λ -Calculus
- 4 - Term Rewriting
- 5 - NP-completeness

TMs

Variants

tracks

Assembly
lane $\leftrightarrow$

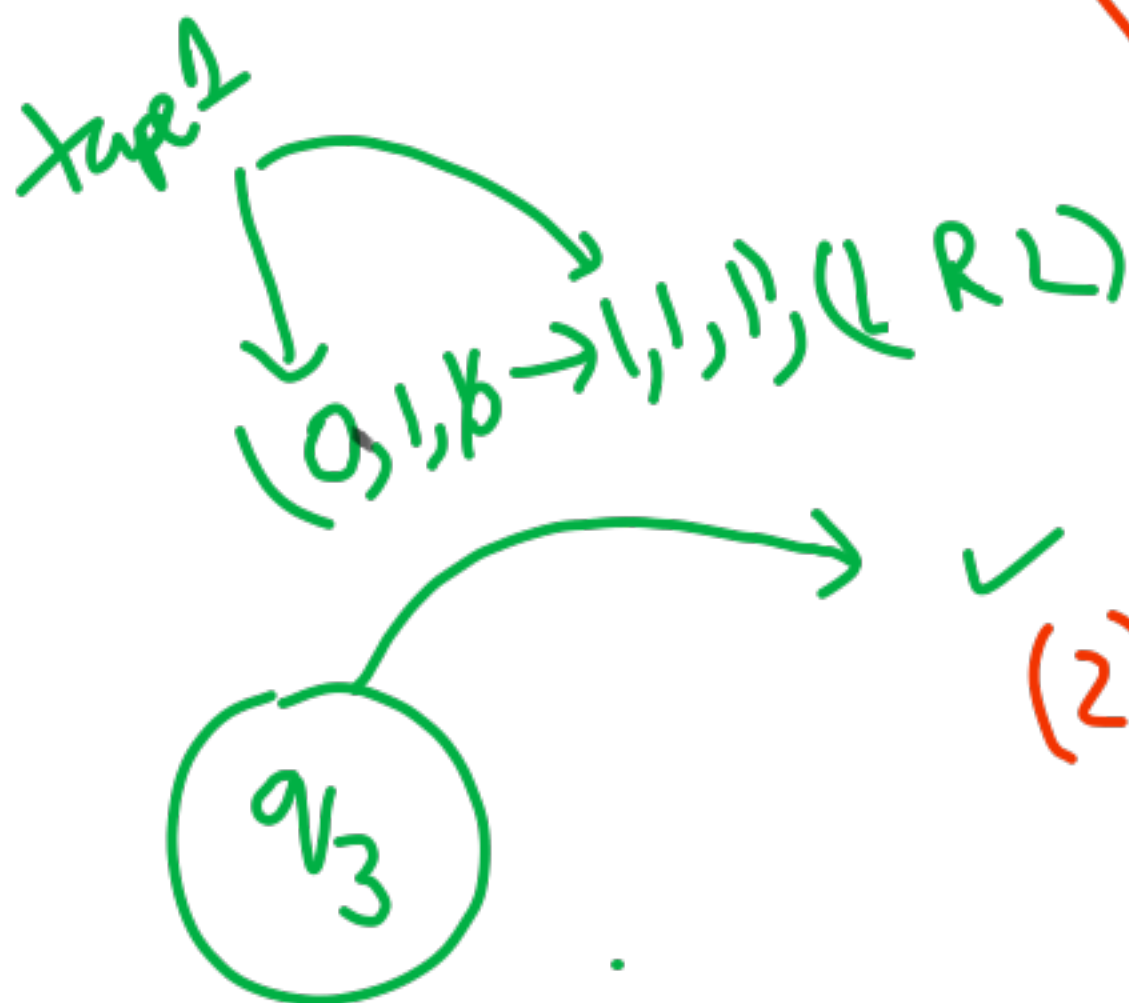
$\langle Q, \Sigma, \Gamma, \delta, q_0, \emptyset, F \rangle$

Variants

③ Non-deterministic



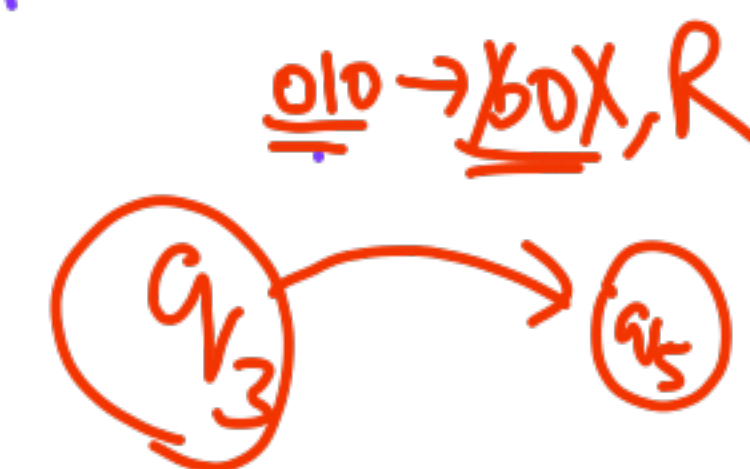
④ 1-way tape ✓



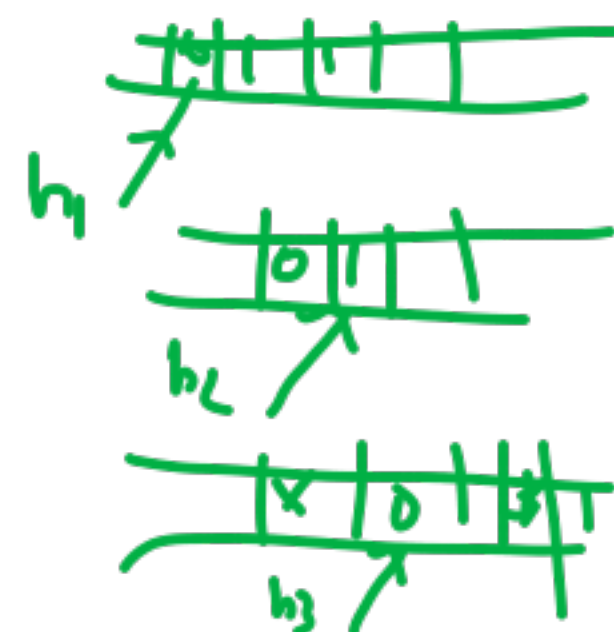
(1) Multi-track

x	0	1
y	1	1
z	0	0

head



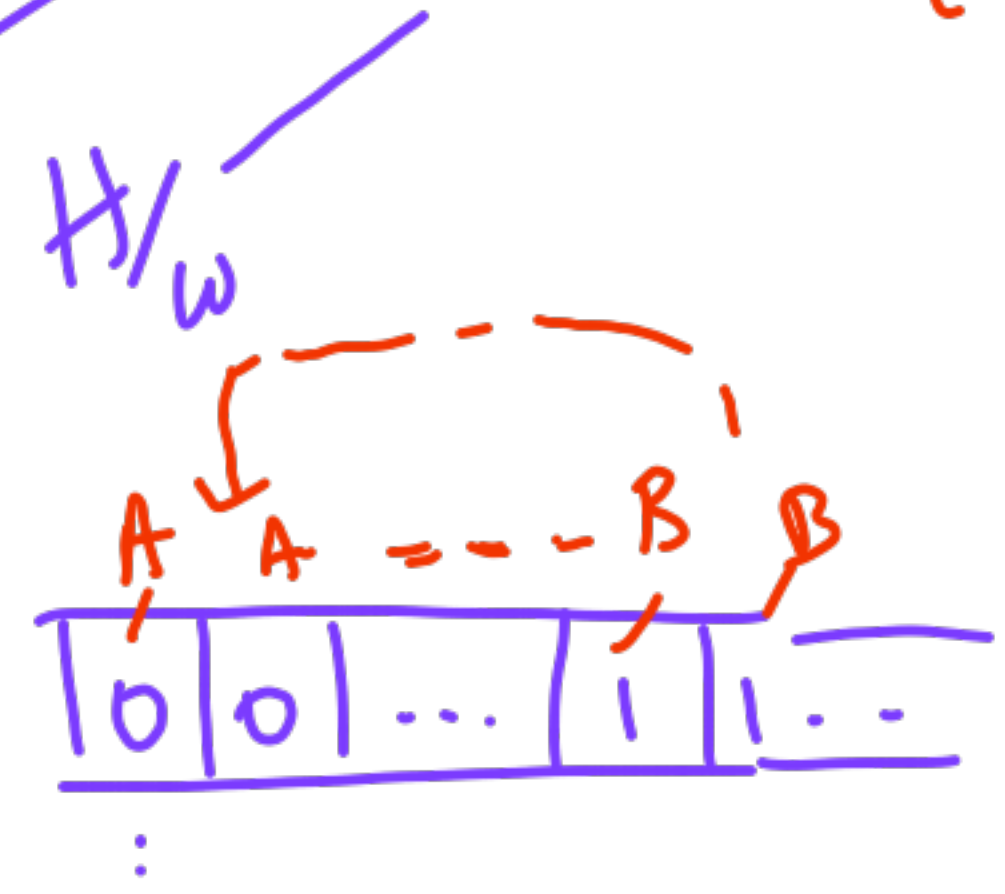
(2) Multi-tape



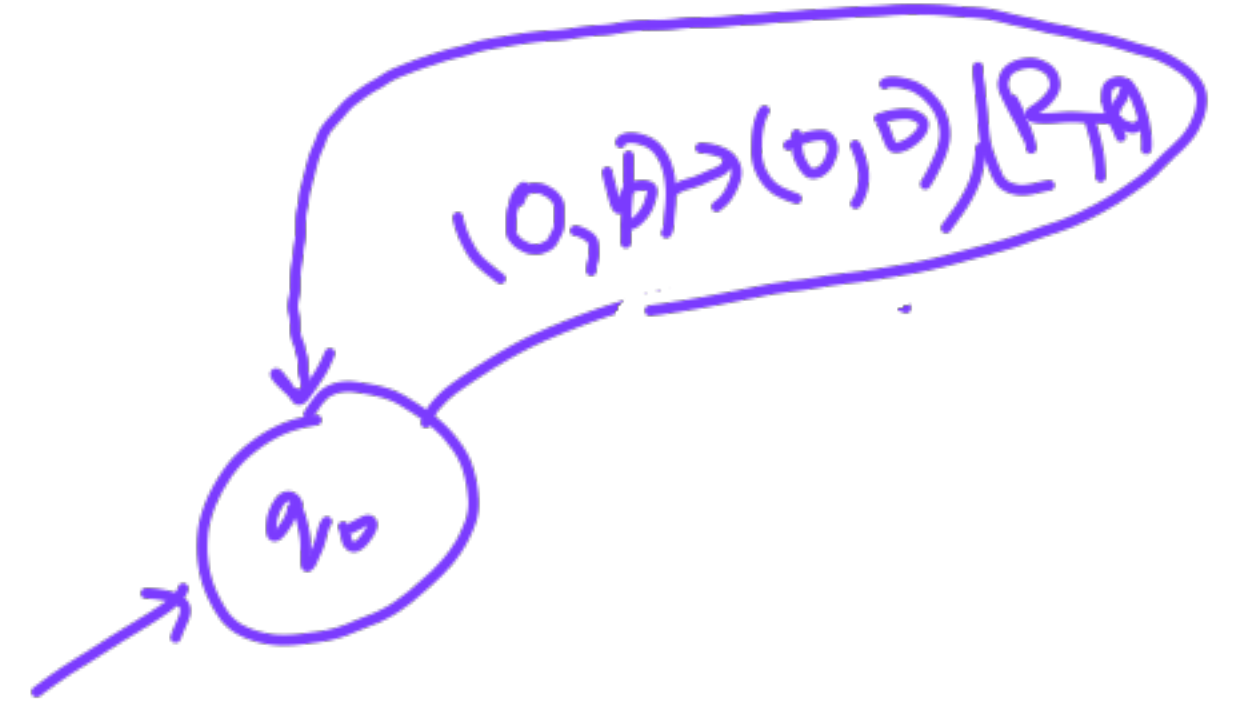
Turing machine. 10

$$L = \{ 0^n 1^n \mid n \geq 1 \}$$

✓ 2-tape



Input



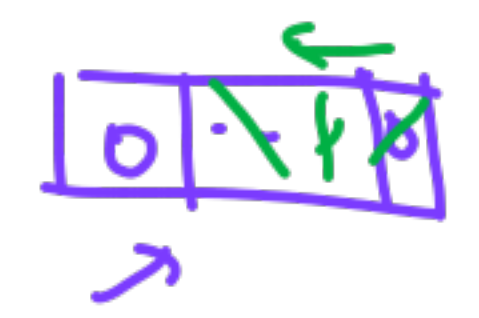
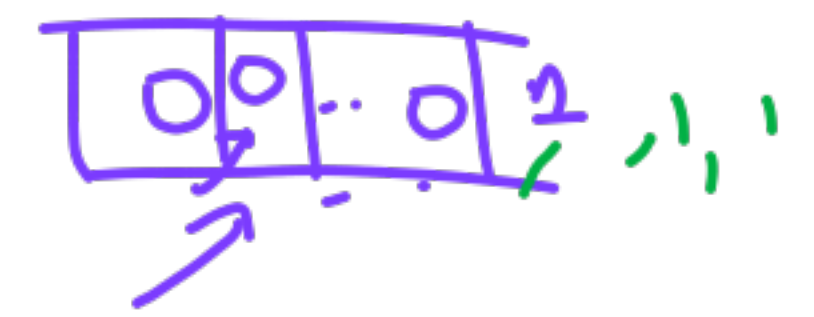
$O(n^2)$

$O(n)$

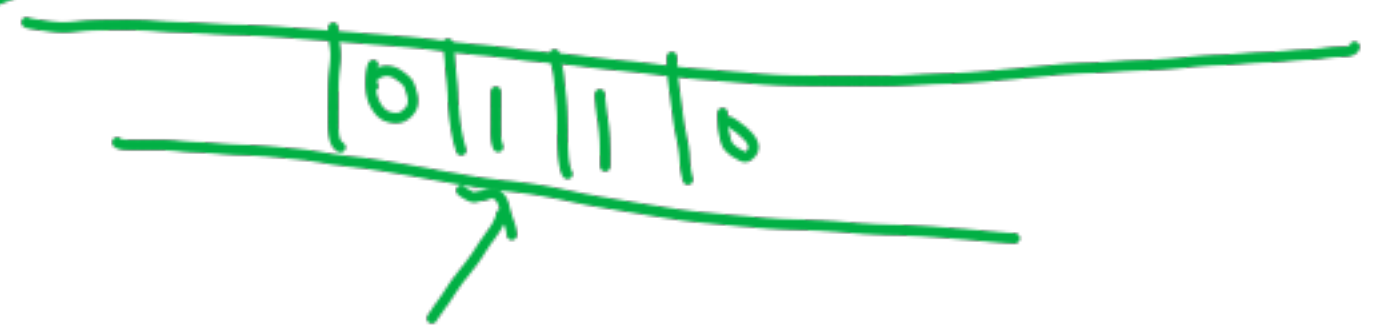
work
easier



Idea



Shift & right



No extra-power
for any of
the variants

Simple-type JM
deterministic

~~prots~~
(very technical)

Can convert
systematically
to equivalent

can
mimic
all
variants

$a_j \rightarrow a_x / d_m$

j -th tape symbol $\rightarrow x$ -th symbol $\rightarrow m$ -th dir $\rightarrow p$



Model of
a (ception)

Encoding
machine begins

$0^i | 0^j | 0^l | 0^m | 0^k ||$

move 2
move 3
||
||

① Accept states
 $q_0 w + \dots + \alpha q_f \beta$

② Halt $q_0 w + \dots + \alpha q_i \beta$ $\rightarrow$ no moves possible

$q_f \in F$

H/w

DFA

Reg

Dyn. Prog

Languages

$L_d \neq$

Rec. Enumerable $\rightarrow$

L_u

Recursive $\rightarrow$

L is rec. $\Leftrightarrow \exists$ TM M

that halts on all w

and is in accept state if $w \in L$
in reject state if $w \notin L$

L

Closure

Union

Inter

Cat

Rev

Difference

Complement

L is r.e. $\Leftrightarrow$ there is a TM M

Such that $L(M) = L$

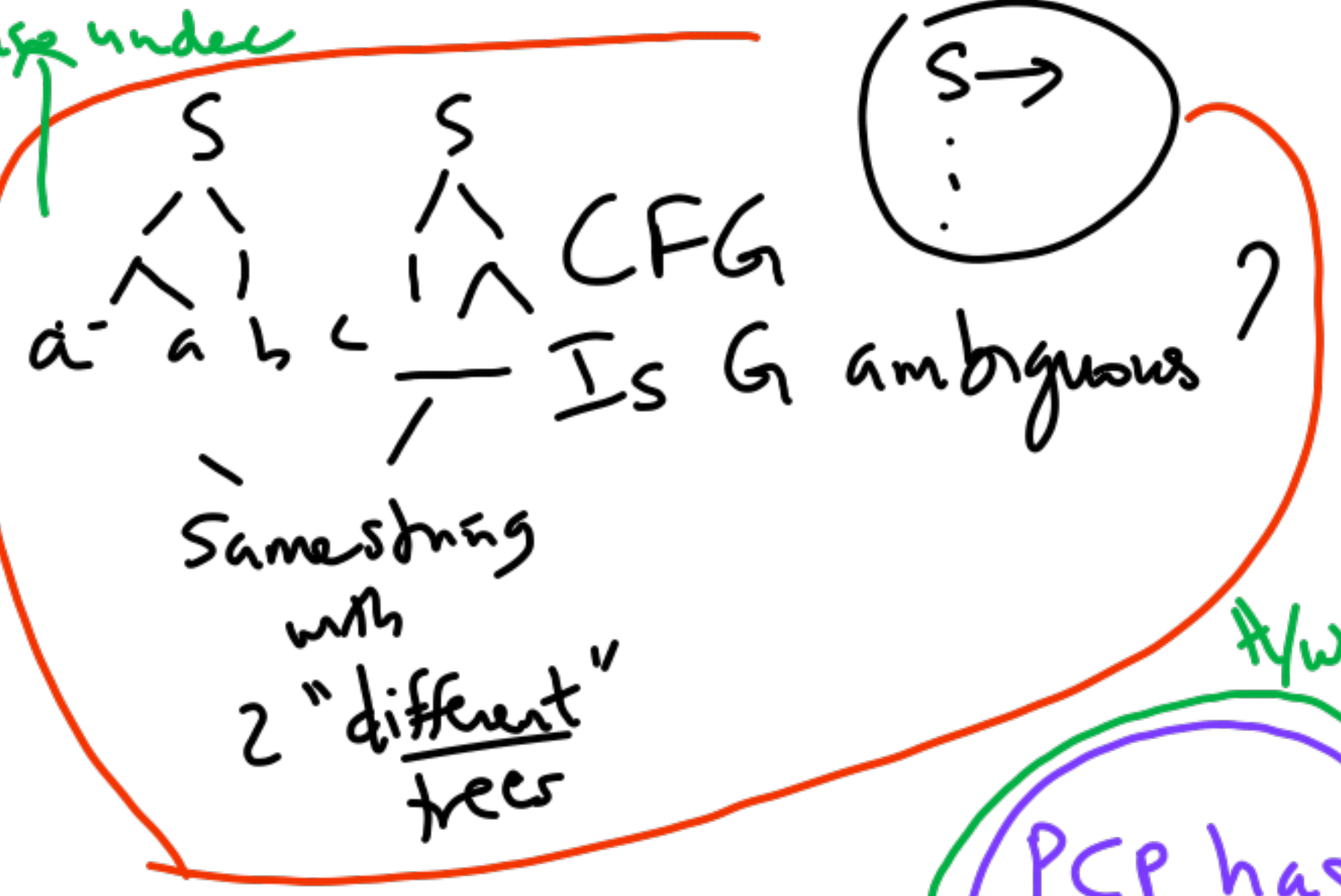
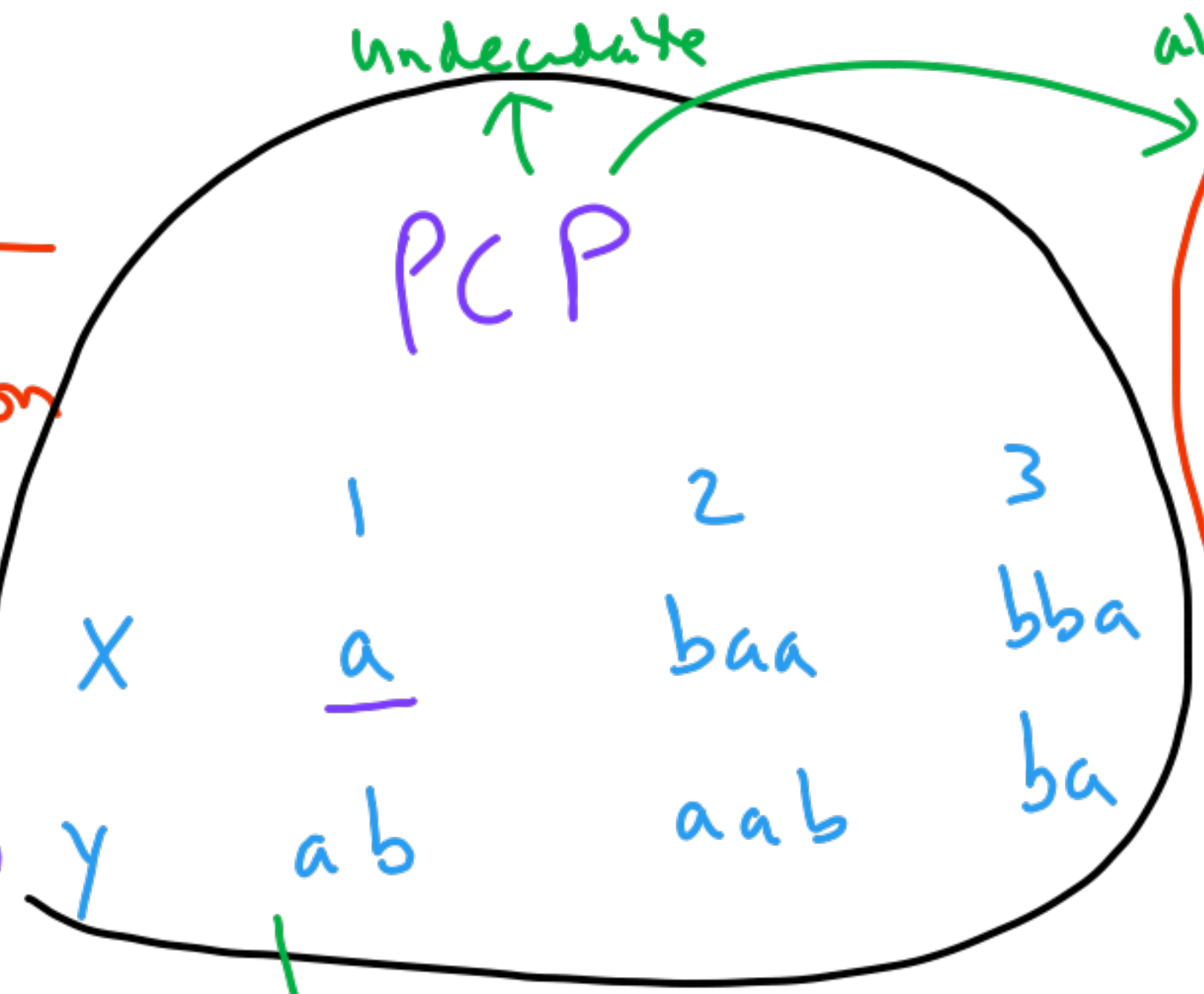
M accepts all $w \in L$

3.10.2025

① Problem Reduction

Example

A →
B →



Soln

1 2* 3
 $\bar{a}$ $\bar{b}$ $\bar{b}$ $\bar{a}$
 $\bar{a}$ $\bar{b}$ $\bar{b}$ $\bar{a}$
 1 3

PCP → "convert" → Gr. Ambiguity
 "cost"

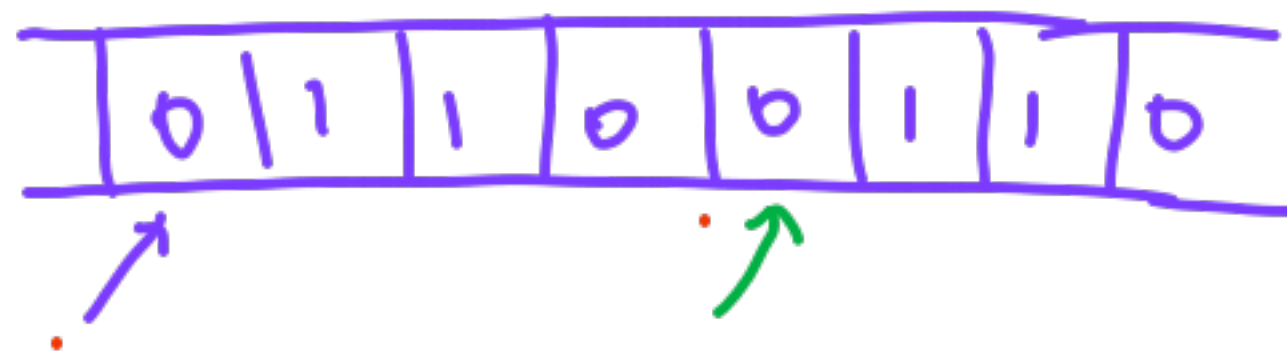
$S \rightarrow A \mid B$ (equivalent for decidability)
 $A \rightarrow aA1 \mid a1 \mid baaA2 \mid baa2 \mid bbaA3 \mid bba3$
 $B \rightarrow abB1 \mid ab1 \mid aabB2 \mid aab2 \mid baB3 \mid ba3$

PCP has a soln
 $\Updownarrow$
 Grammar is ambiguous

2-7 wpe

Non-det TM

for $L = \{ww \mid w \in (01)^*\}$

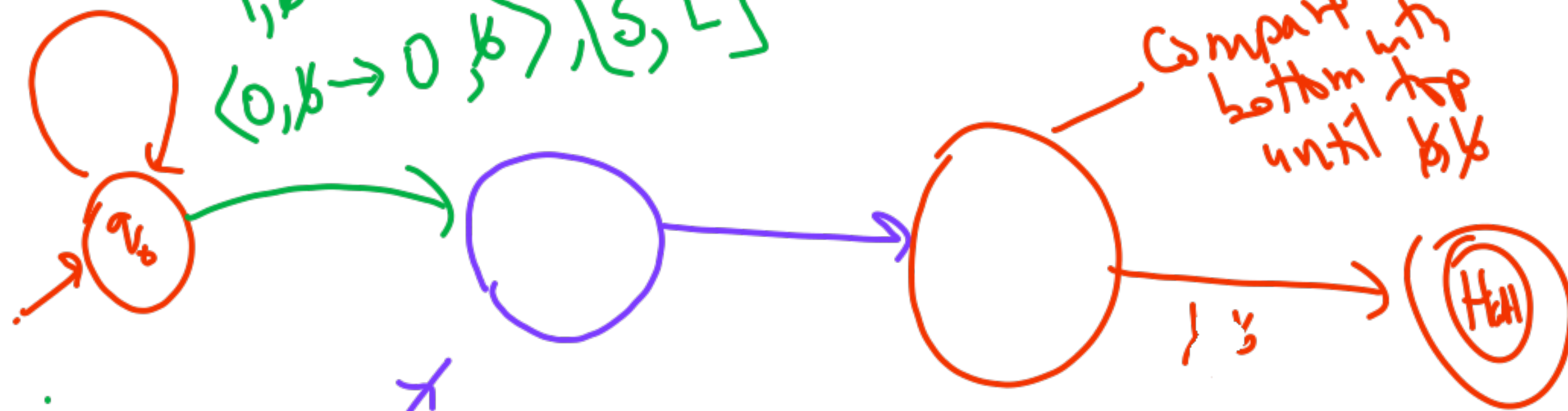


0 1 1 0

$\langle 1, b \rightarrow 1, D \rangle$
 $\langle 0, b \rightarrow 0, 0 \rangle, [R, R]$

$1, b \rightarrow 1, b$
 $\langle 0, b \rightarrow 0, b \rangle, [S, L]$

Compare bottom with top until b/b



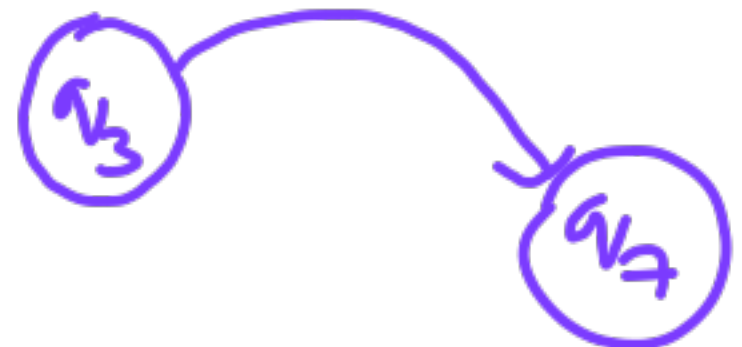
Keep going left on bottom tap until b

Extension $\rightarrow$ Stay

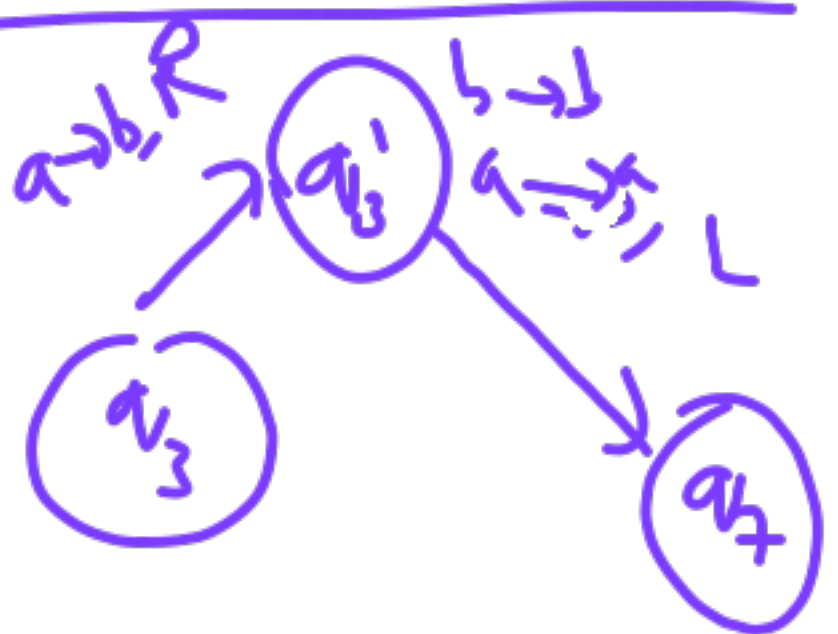
$L, R \rightarrow$ stay $\rightarrow$ non-det

L, R, S stay

$a \rightarrow b, S$



$x \rightarrow x$



Decidability

All languages $L \subseteq \Sigma^*$

Rec. Enumerable
"Semi-Decidable"

Rec. Decidable

L_d

L_d

Antenna

Diagrammed

L is decidable if
 $w \rightarrow M \rightarrow$ Yes if $w \in L$
 No if $w \notin L$
 and M always halts.

L is r.e
 (semi-decidable)
 if there is a TM, M

$w \rightarrow M \rightarrow$ Yes if $w \in L$ & halt
 No & halt } $w \notin L$
 Run forever }

L_d fails here also

Is G ambiguous?

Does PC have a solution

Hilbert's 10th problem

$$7x^3y^2 - 11xy^2z - \dots = 0?$$

not recursive

① TMs are enumerating
 Countable

② there is a language
 L_d no TM can accept.

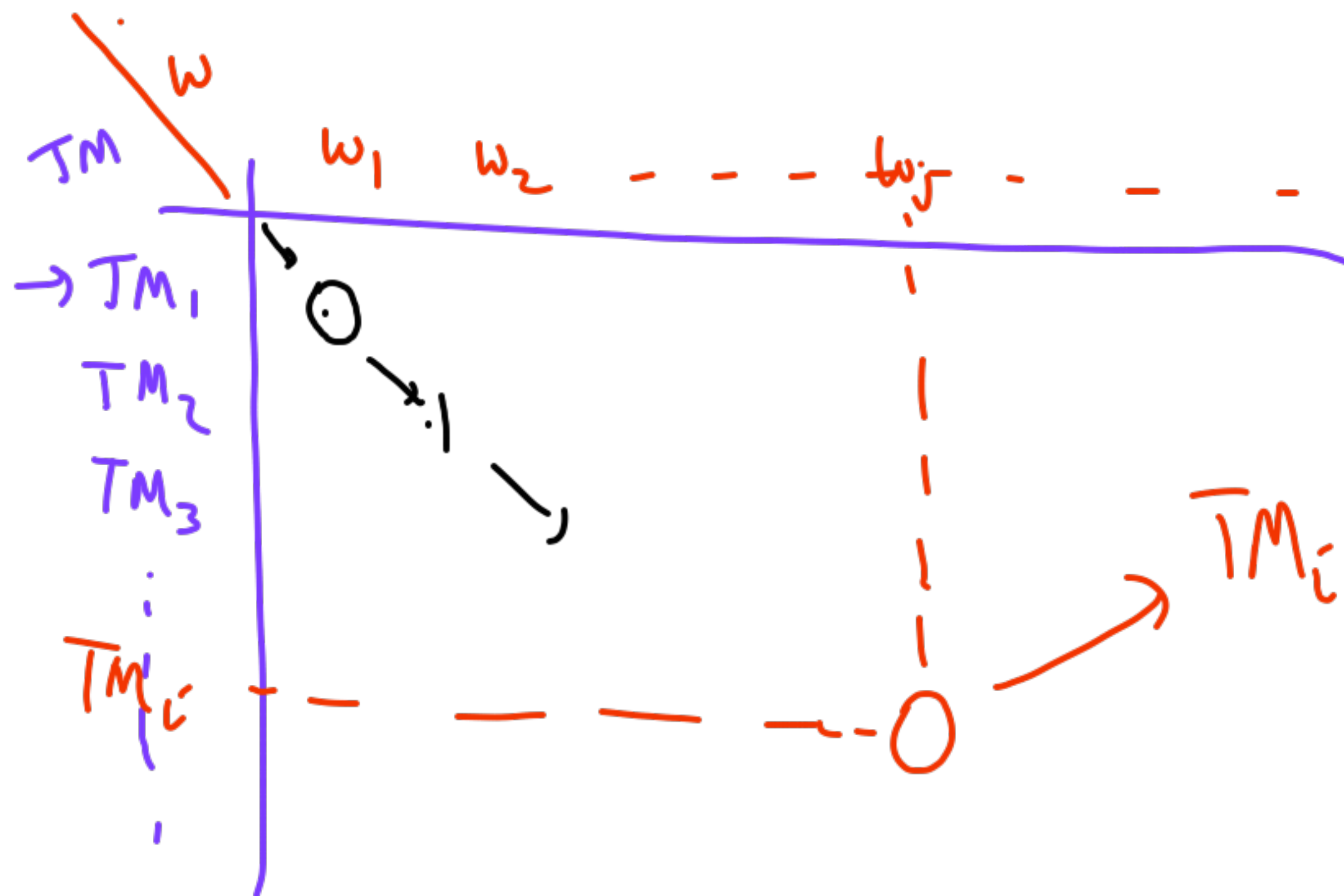
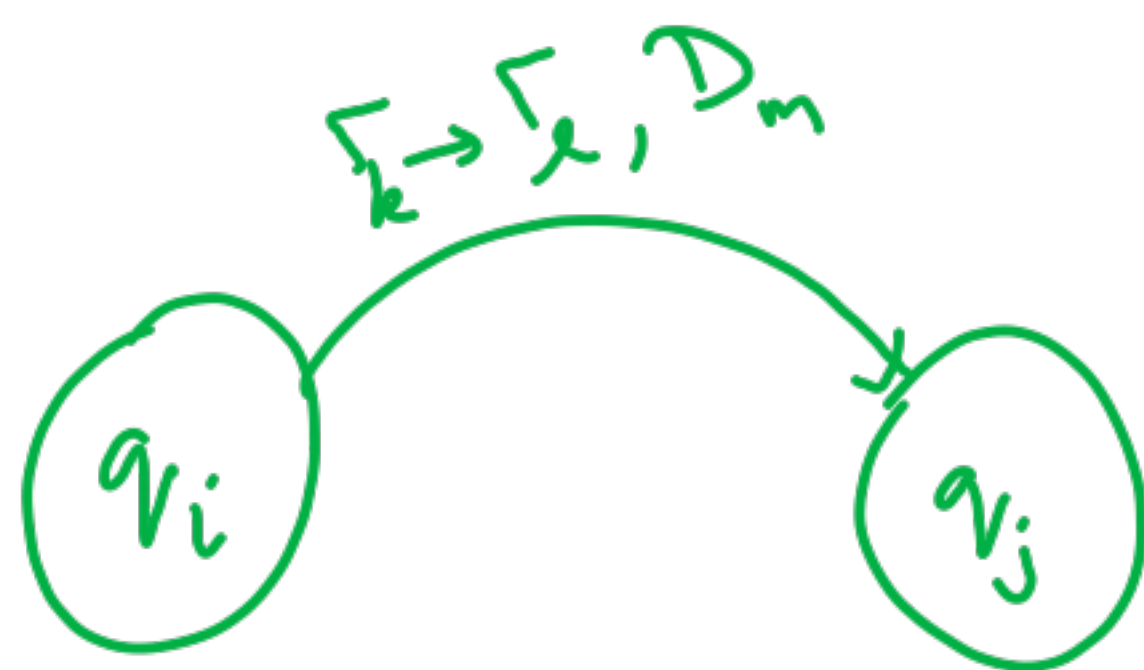
③ there is a language
 L_u that a TM can enumerate
 but cannot "decide"

1-move

$0^i | 0^j | 0^k | 0^l | 0^m$

$\underbrace{\text{move}_1}_{\text{2 ones}} || \text{move}_2 || \dots || \text{move}_n$

2 ones



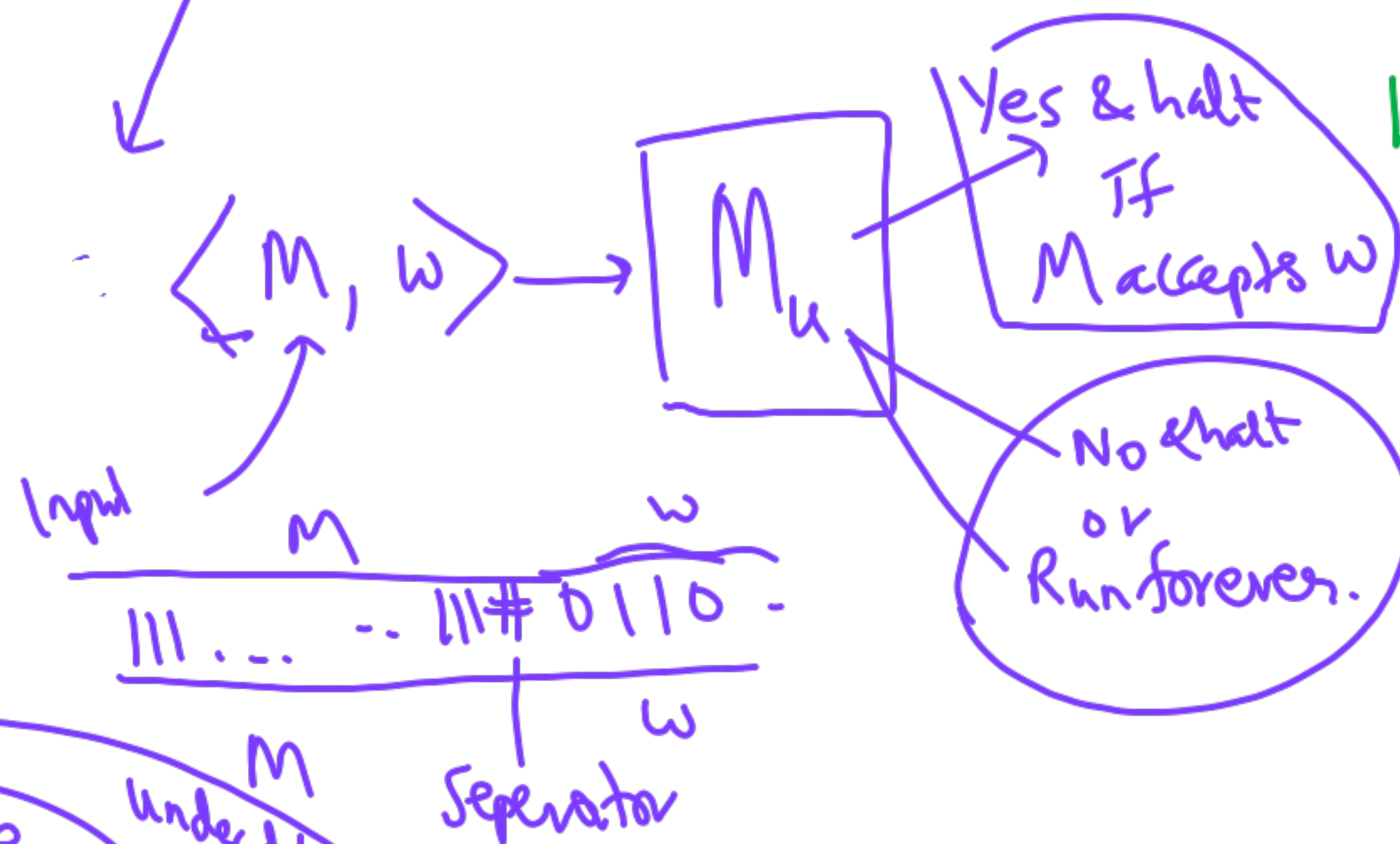
$$L_d = \left\{ w \mid \begin{array}{l} w = w_i \\ \text{and } TM_i \text{ does not} \\ \text{accept } w_i \end{array} \right\}$$

cannot be
accepted by
any
TM

TM_i does not accept w_j

Proof by contradiction - if $\bar{T}_k$ accepts L_d
then $\bar{T}_k$ gives wrong answer
on w_k

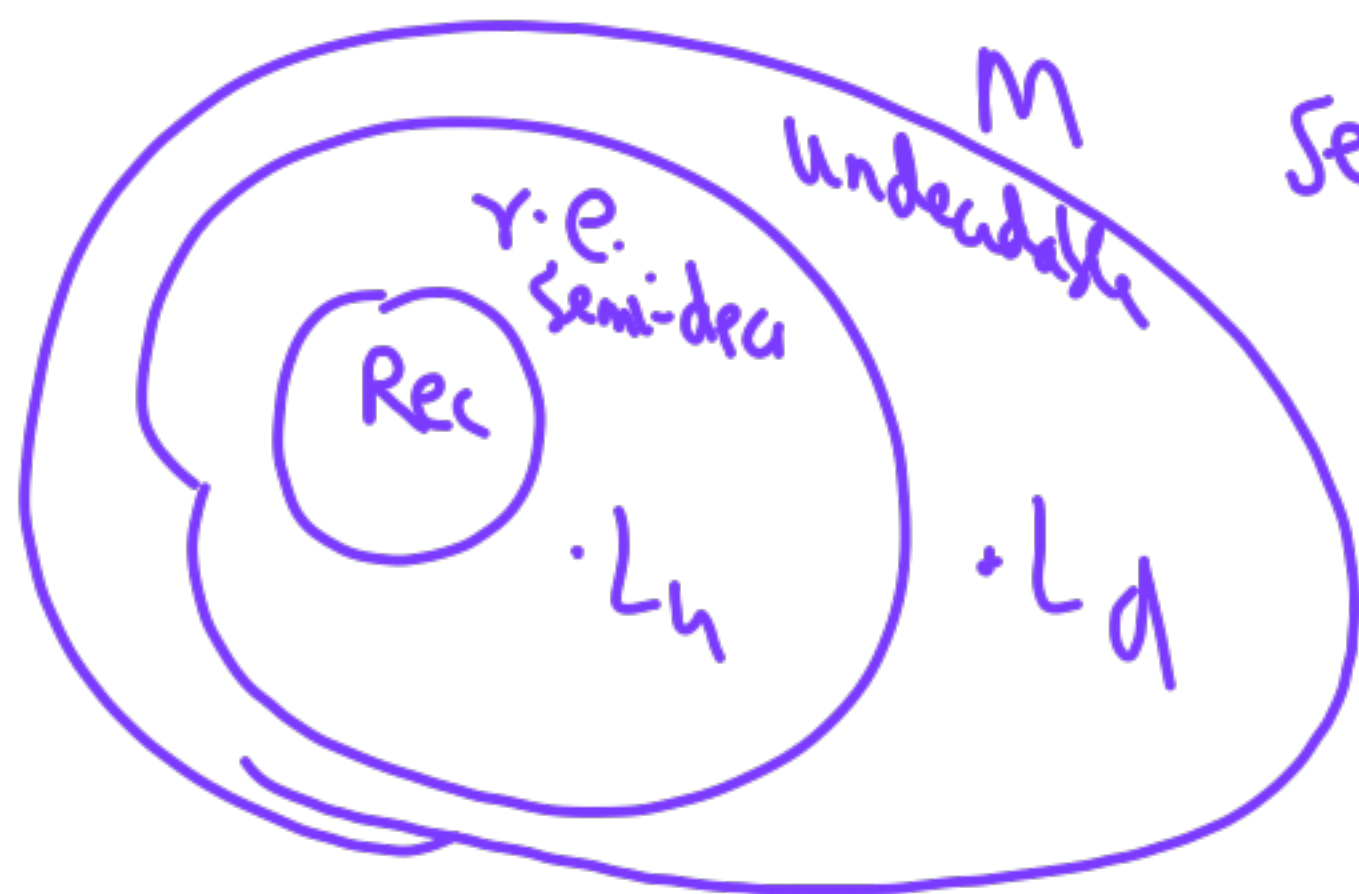
L_u is a language for which we can build a TM M_u universal TM



but M_u will not always halt

Simulation

Can behave like any other TM



8.10.2025

Language

Closure Properties

Models of Computation

TM

- a. λ -calculus
- b. Term Rewriting
- c. Rec. Fn. Theory

Church
Turing
thesis

Quiz 2 on
15/10/25

→ Assign 4.

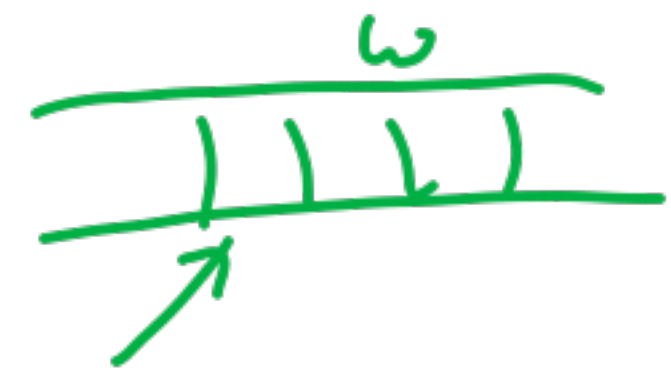
→ NP-completeness -

$$\underline{P \stackrel{?}{=} NP}$$

$\text{r.e. } L \text{ is rec En.} \iff \text{TM } M \text{ accepts } L$

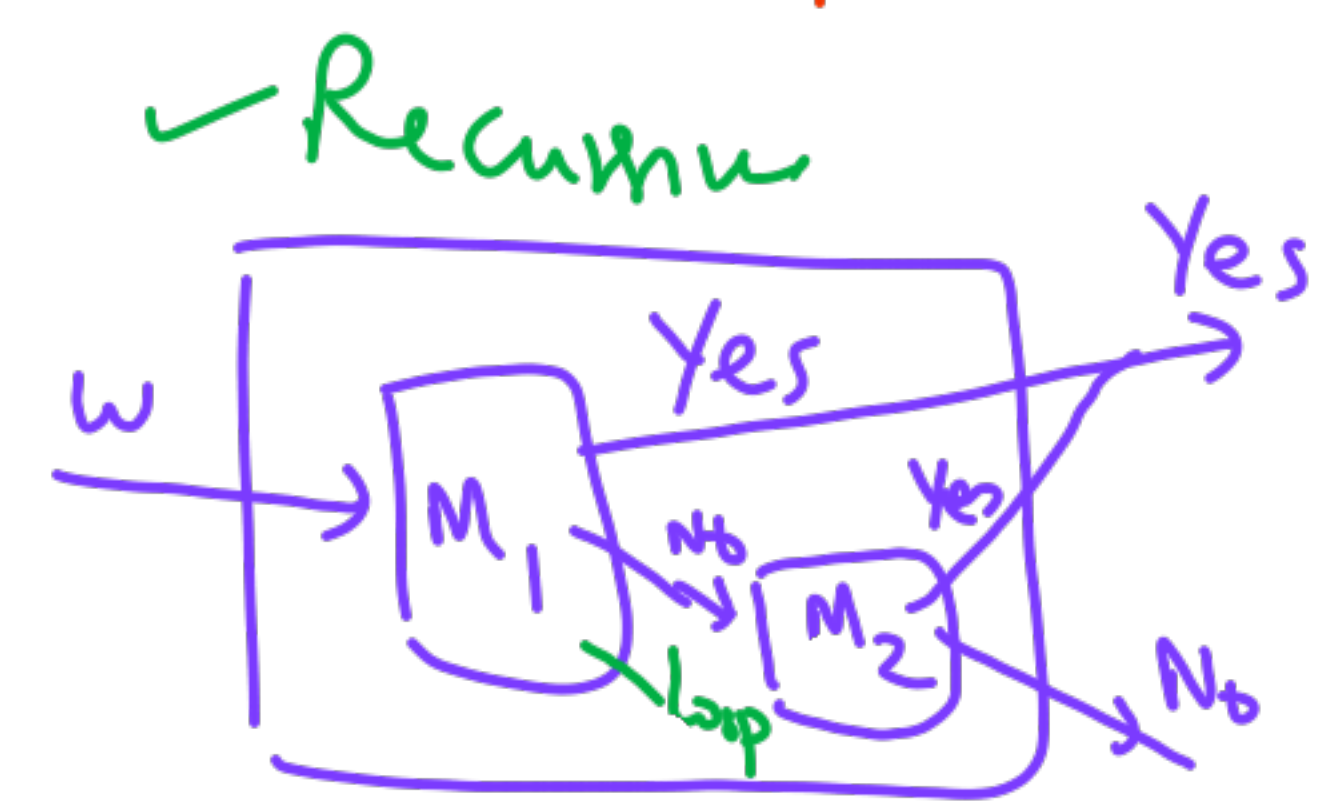
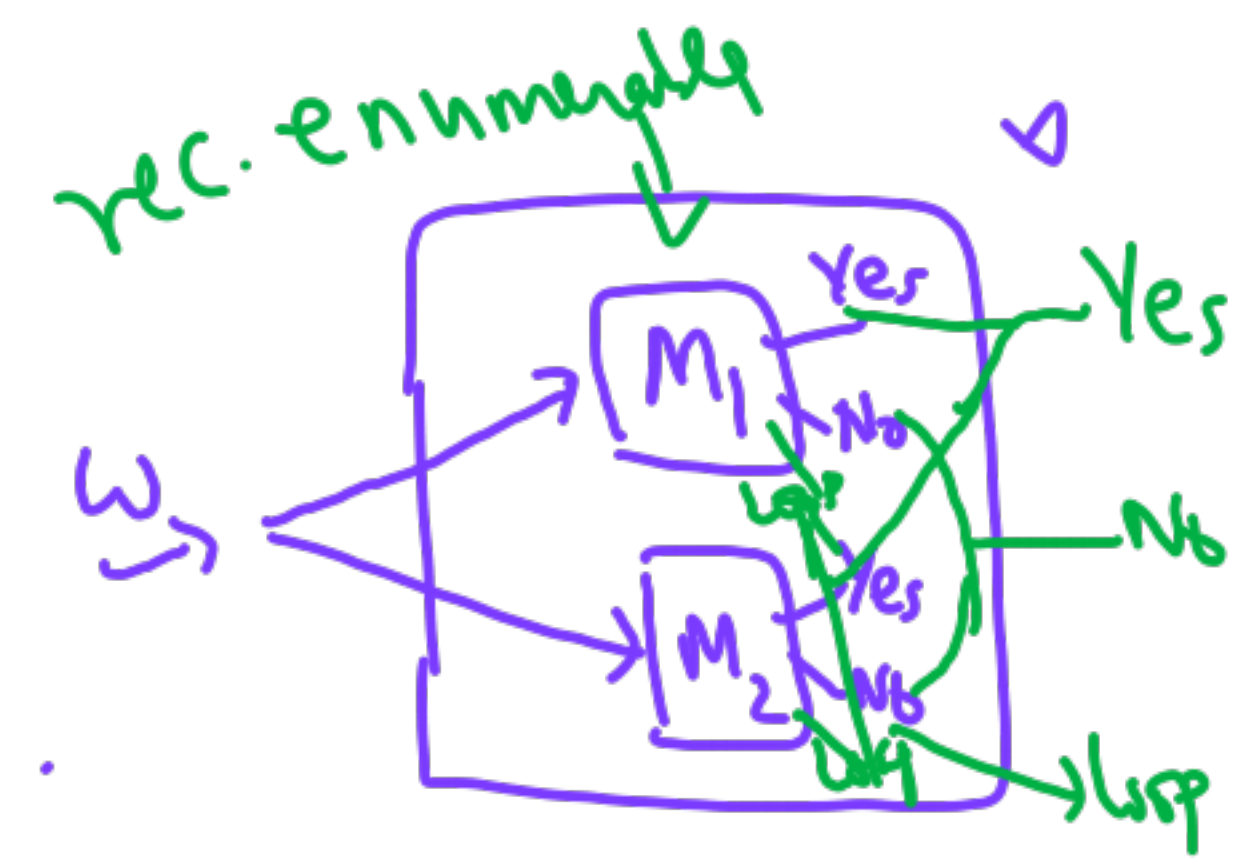
$L = \{ ww \mid w \in (0+1)^* \}$

$\text{rec. Recursive} \iff \text{halts in reject state if } w \notin L$

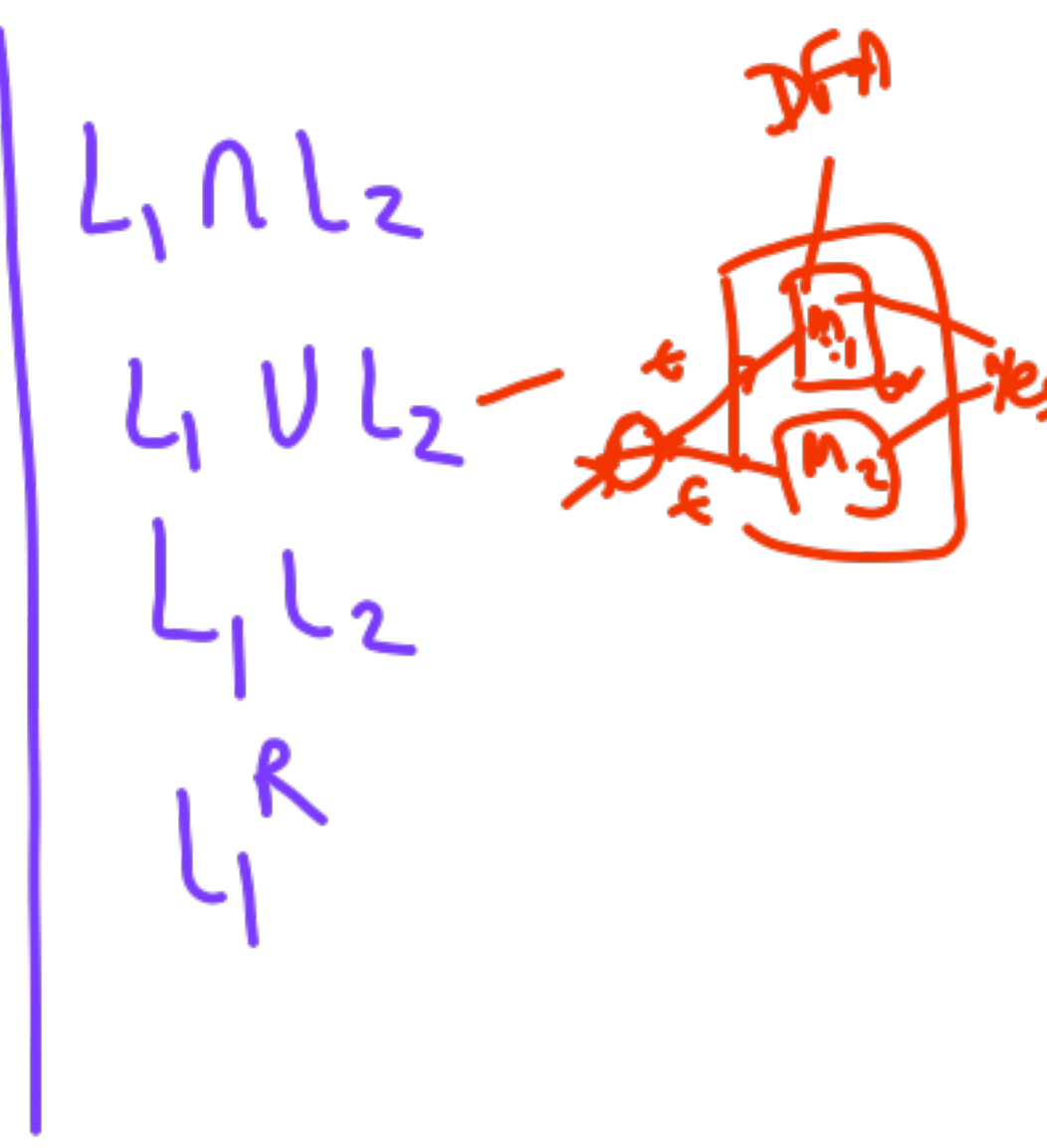


DFA/NFA
 $\uparrow$
 $L_1, L_2 \text{ are regular}$

Union L_1, L_2 are $\begin{cases} \text{r.e.} \\ \text{rec.} \end{cases}$ what about $L_1 \cup L_2$?



allowed 3 behaviours
 only 2 behaviours

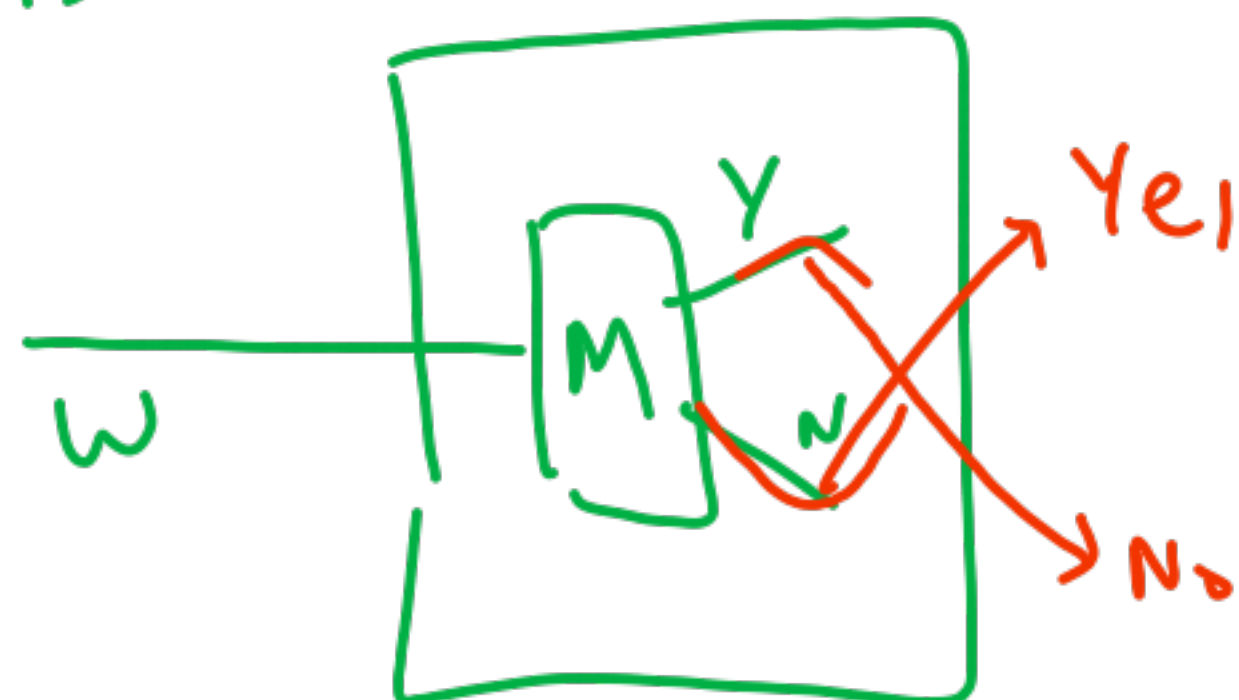


Complement

r.e. h/w

Rec

L is rec - L^c ?



flip answer

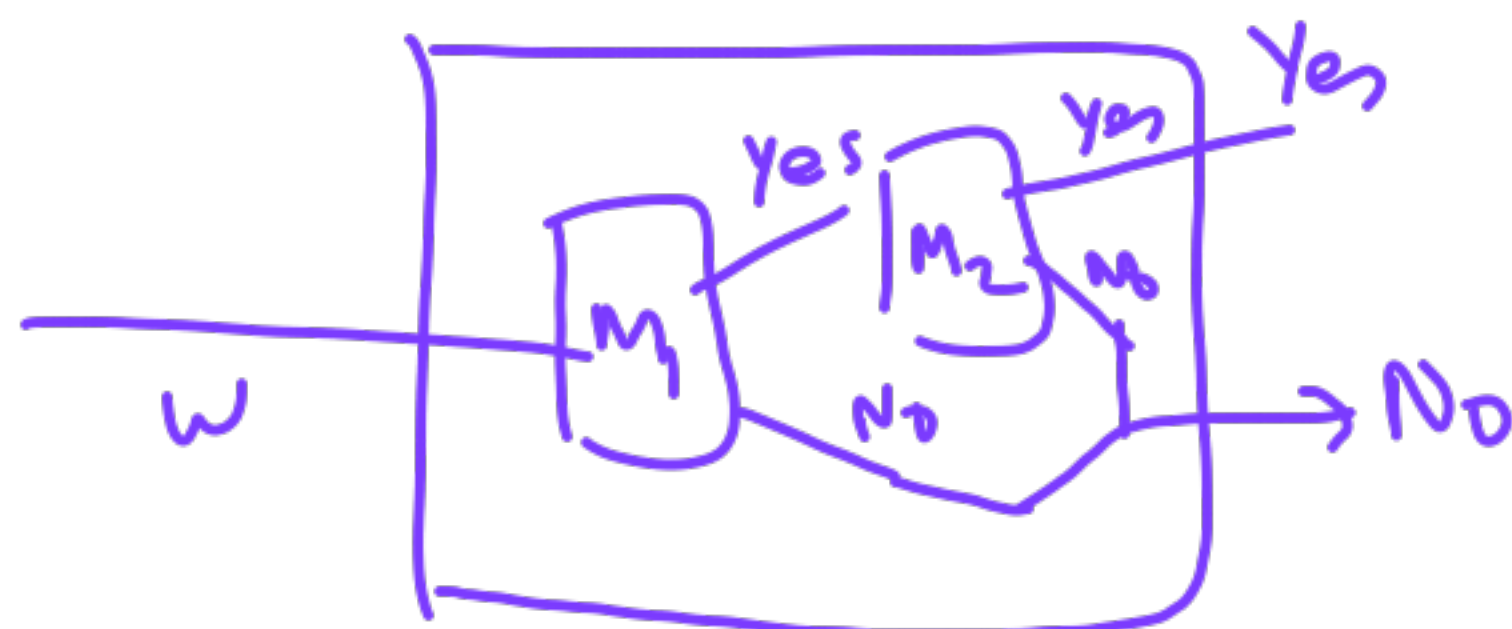
works for r.e.?



Intersection

r.e. $\rightarrow$ h/w

L_1 is rec L_2 is rec
 $L_1 \cap L_2$?



works for r.e.?

H/w $\rightarrow$ If L_1 is r.e. and L_2 is r.e. then $L_1 \cap L_2$ is r.e.

Rec

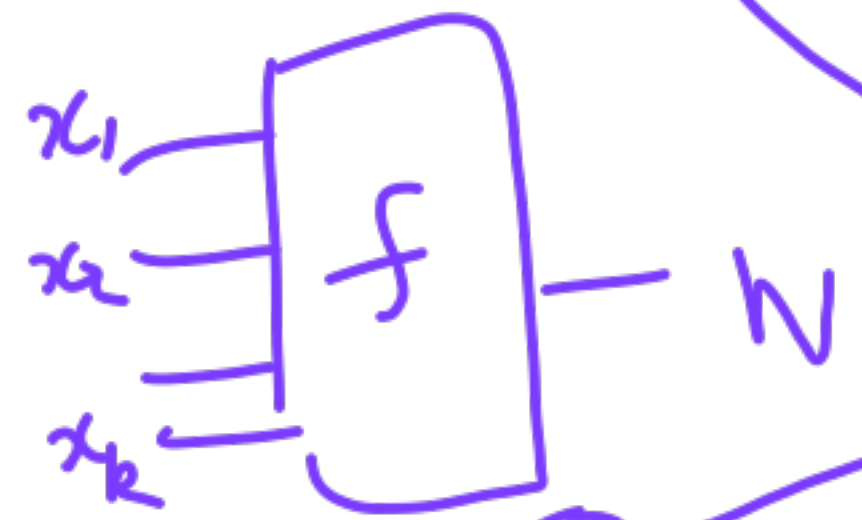
$L_1 L_2$

L_1^*

L_1^{rev}

Computations

Can be embedded as Language.



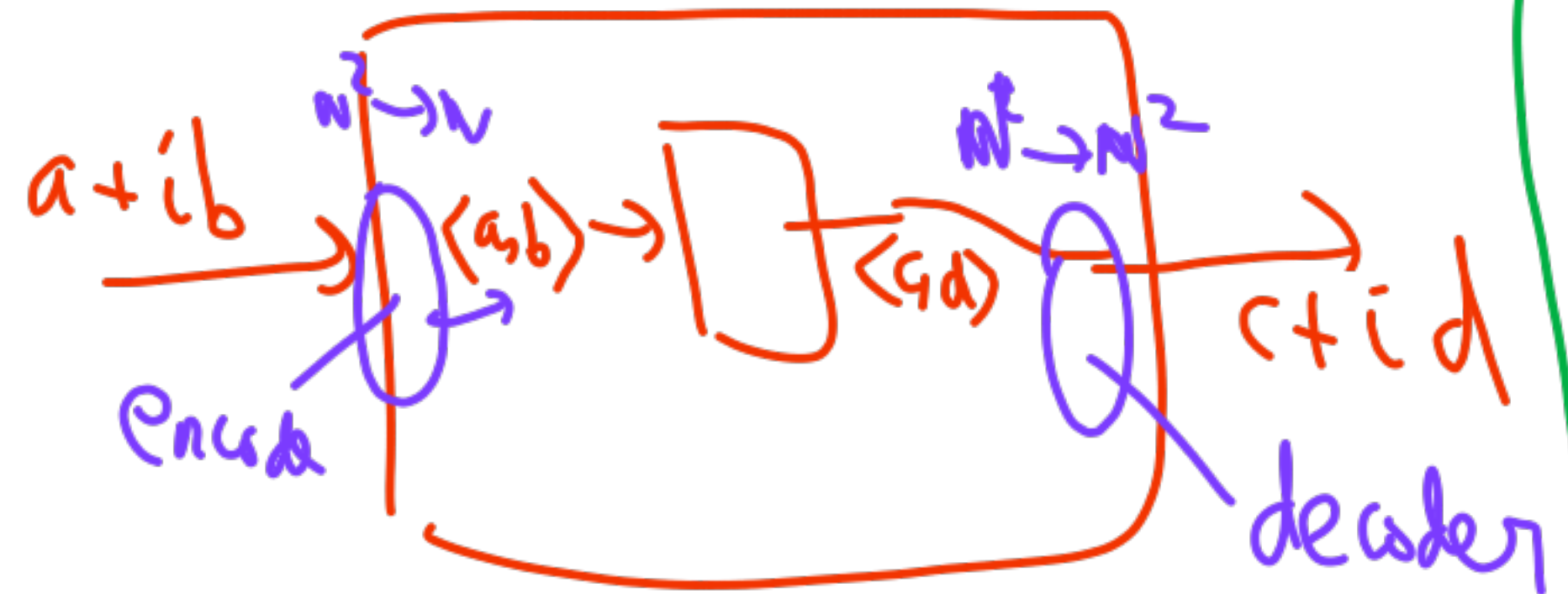
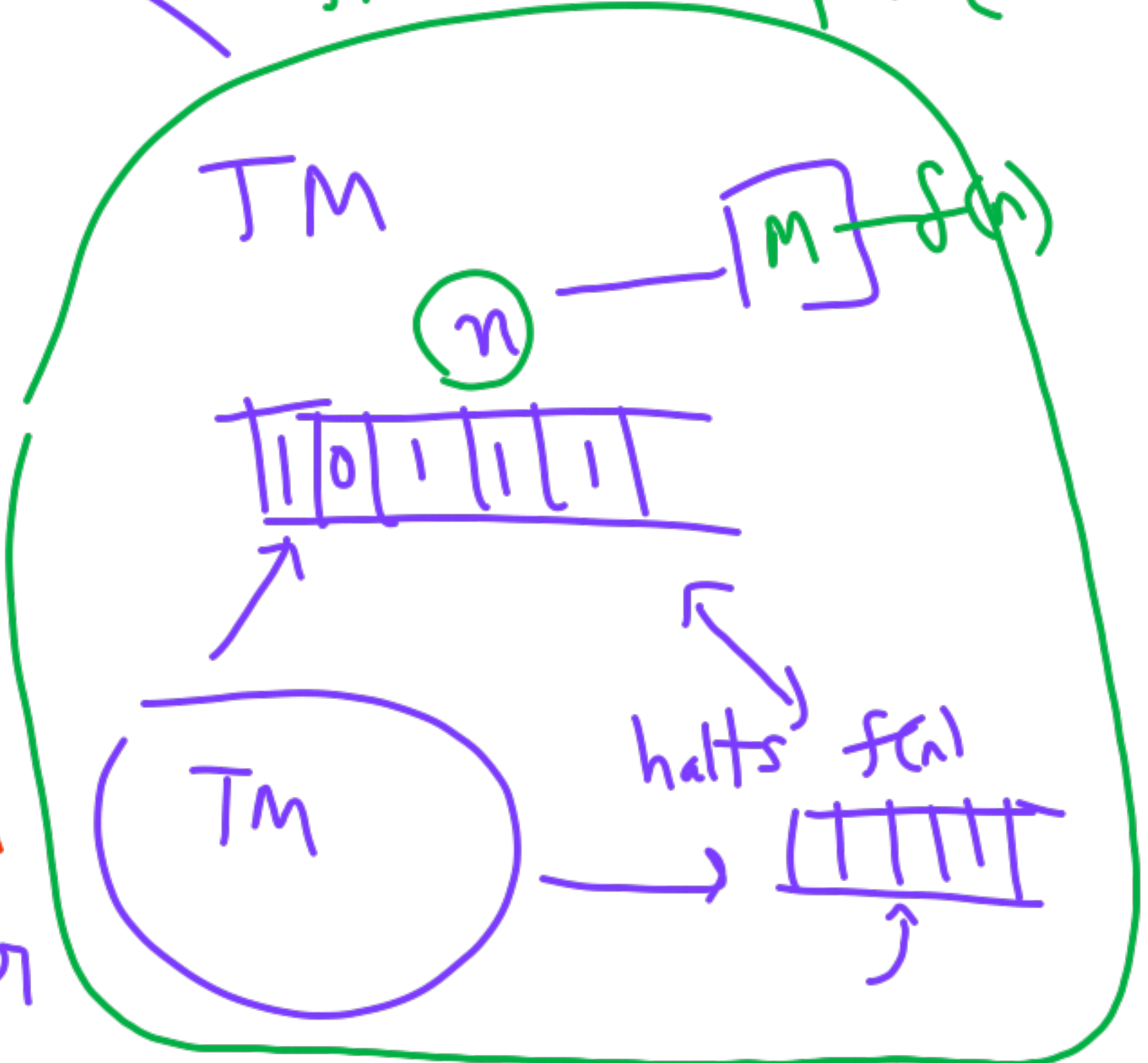
$$a^i b^j c^{i+j}$$

$$a^i b^{2i^2+3i+9}$$

TM as a Computer



$$\langle k, \langle m_1 \dots m_k \rangle \rangle$$



Equivalent computational models.

λ -Expression

λ -calculus

Abstraction

Application

Inductive

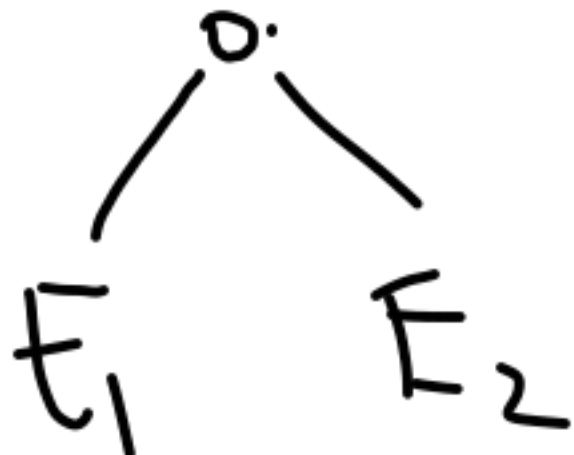
Variable

x, y, z

λx

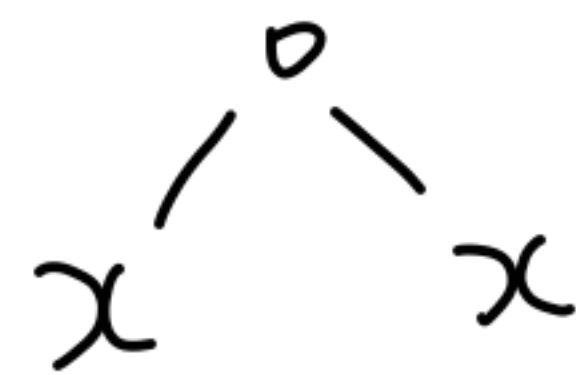
E

any λ -express



β -reductions

Syntax



λx

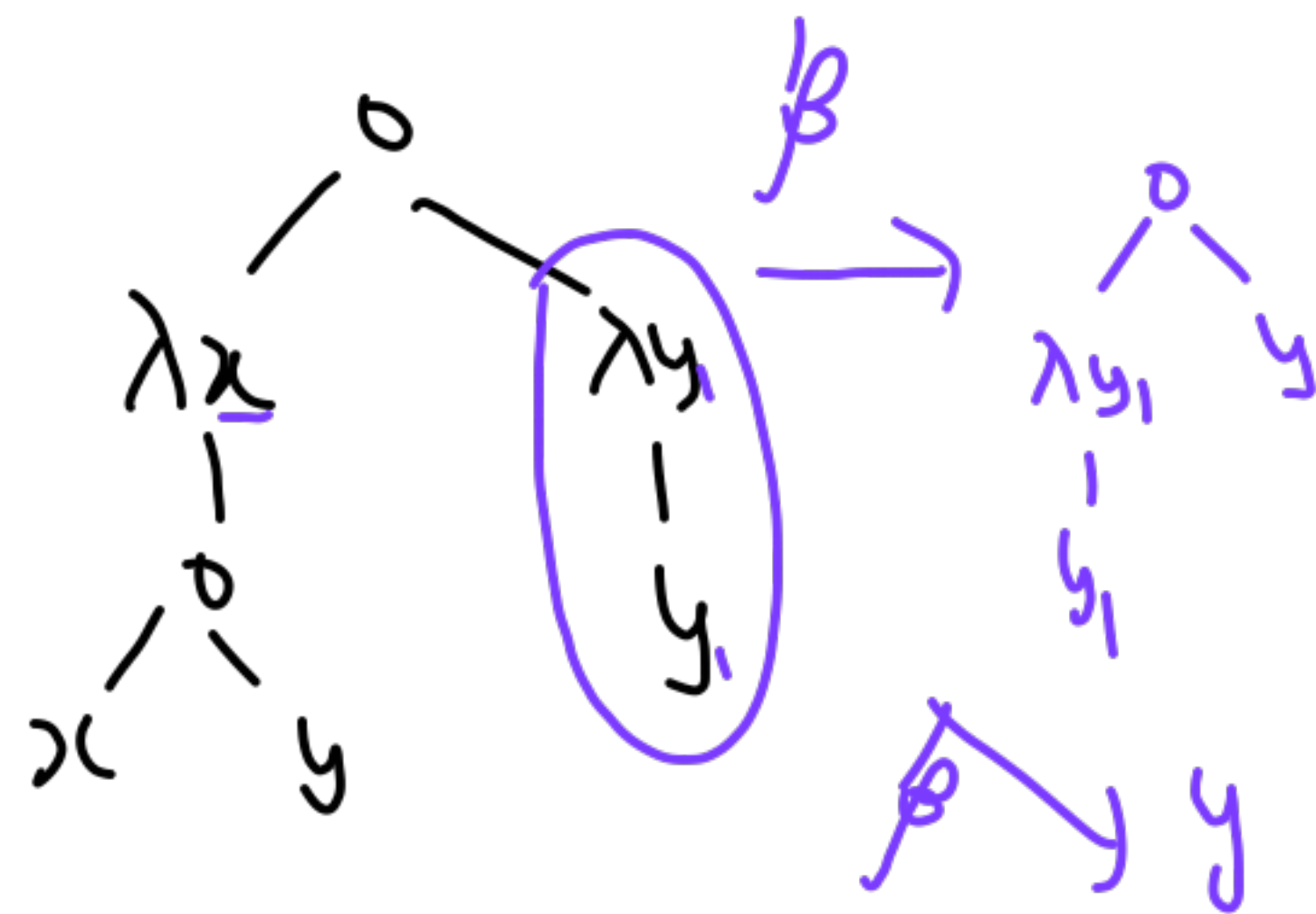
x_1

β

value

(call by name)

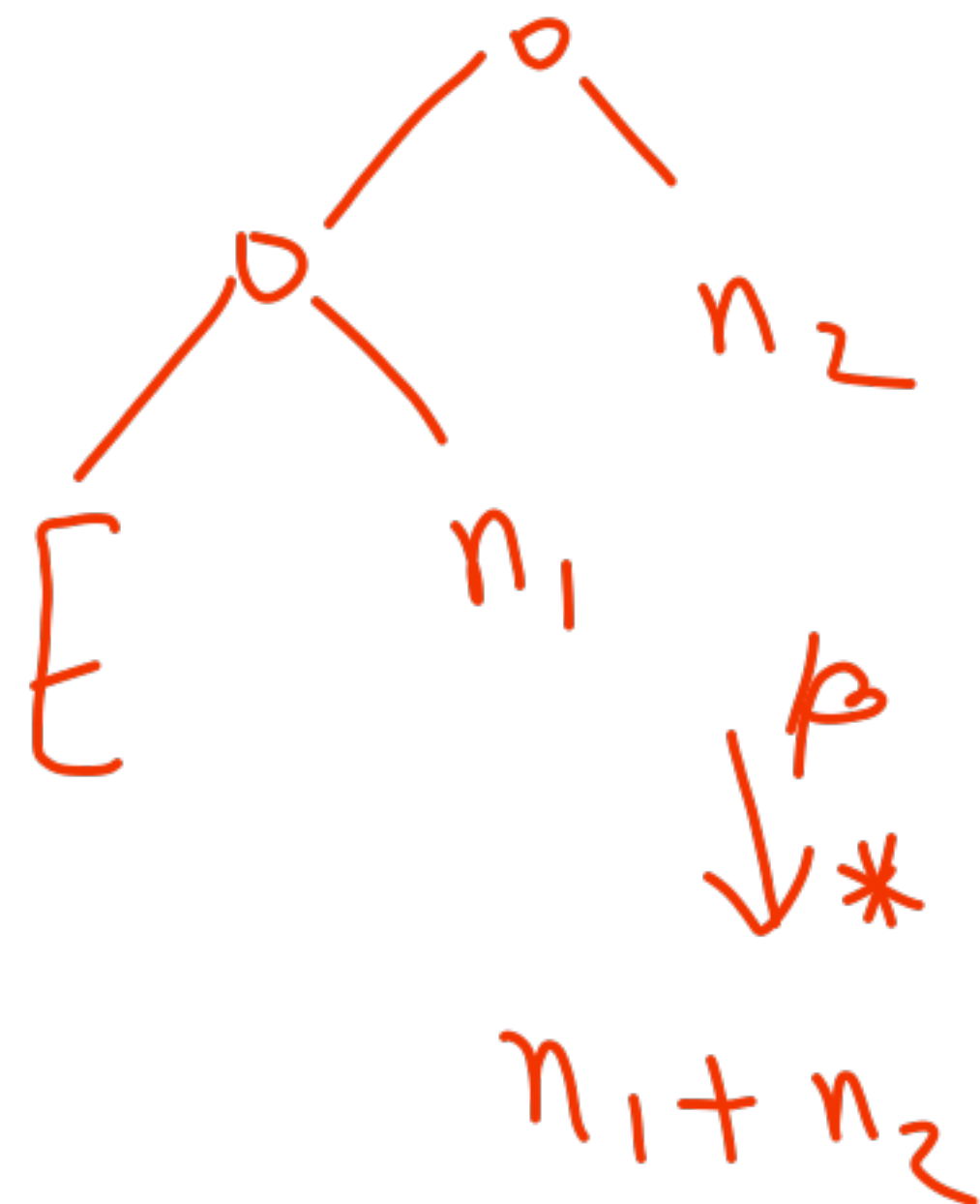
y



Goal

Numbers

Church
numerals

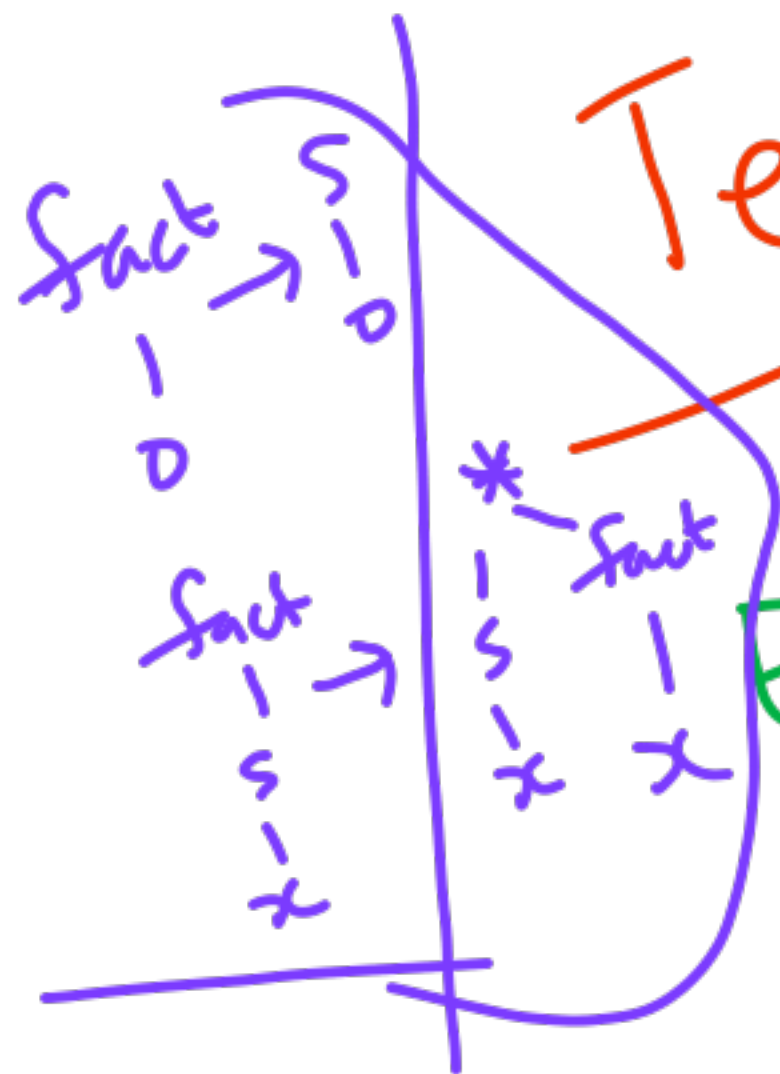


addition
expr

mult
expr



Term Rewriting H/W $\rightarrow$ square

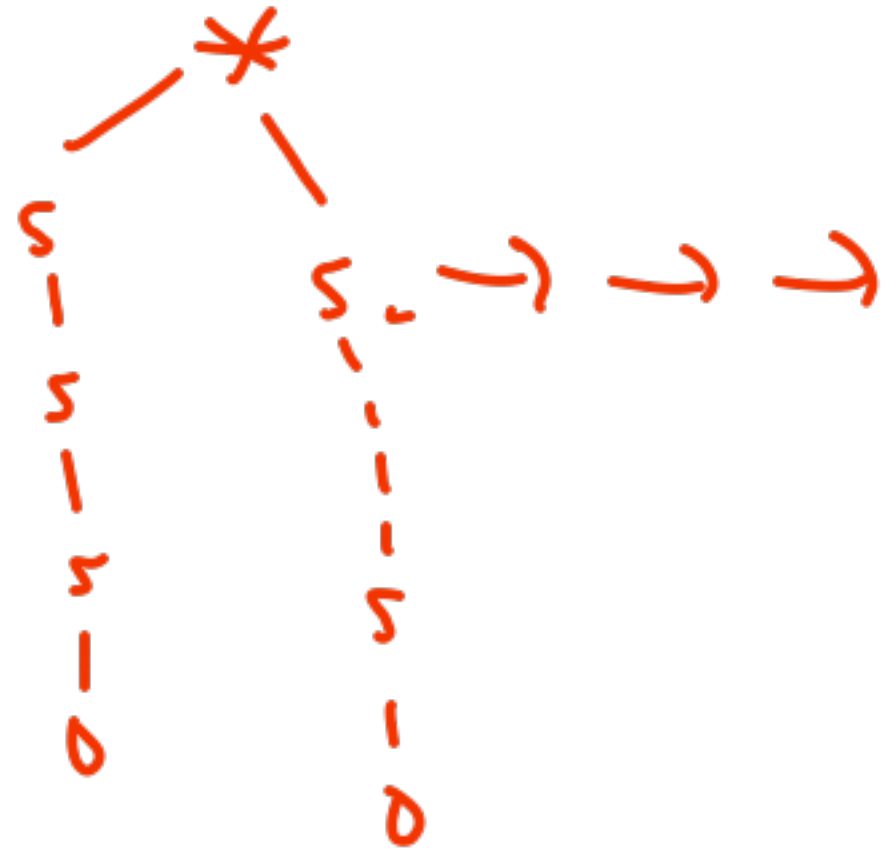


Equality

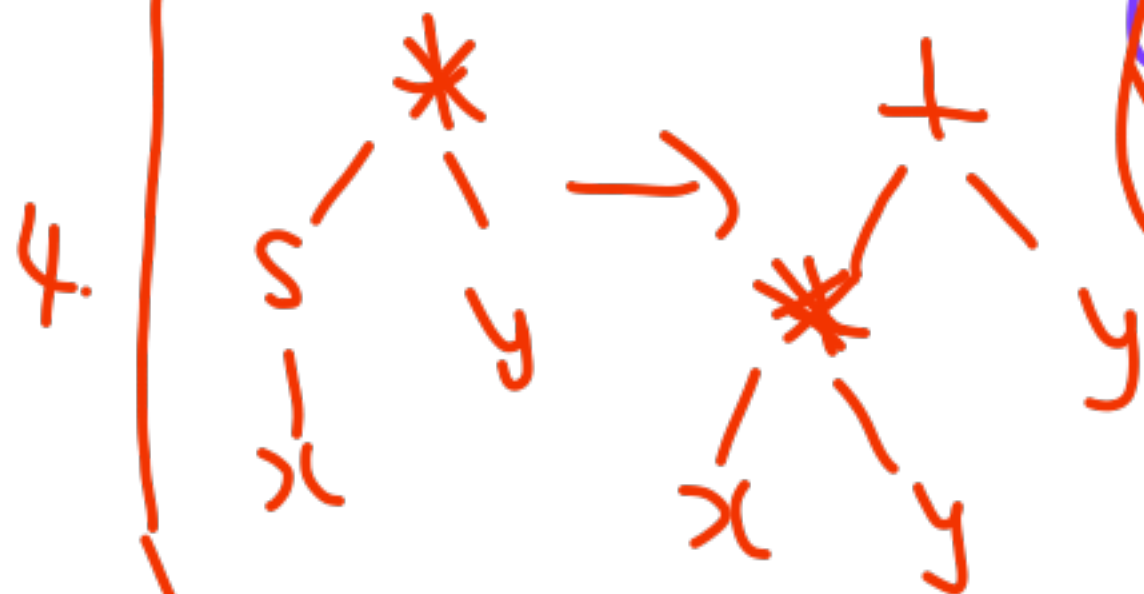
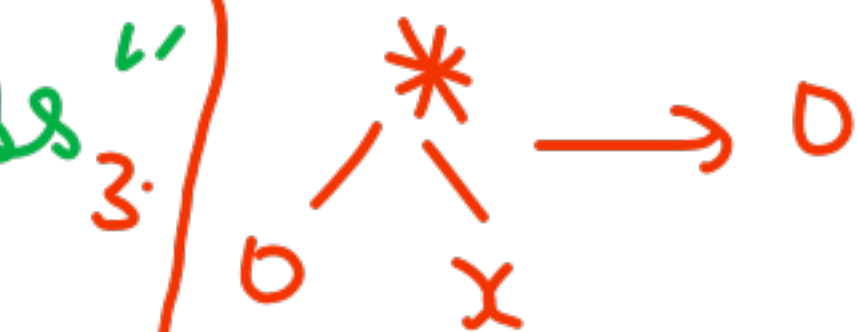
=

"replace equals by equals"

3 * 7 = 21



Terms



Constants

Constructors (domain)

defined by rules

fn. symbol

variable



10.10.2025

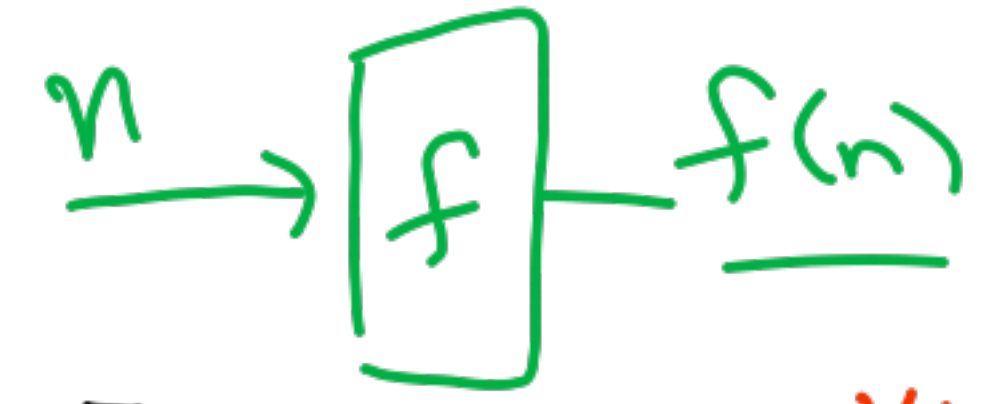
Quiz 2
15/10
11-12:30

① λ -calculus

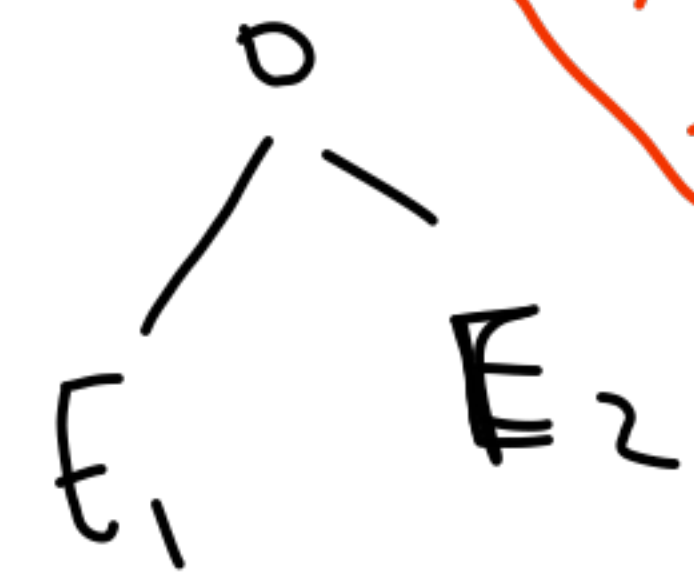
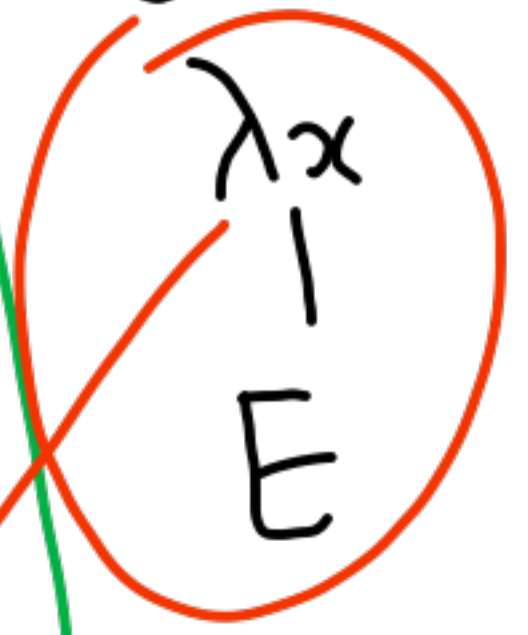
② Term Rewriting

③ Recursive Function Theory

Not in Quiz 2
↓
Quiz 3 &
I will post pointers on moodle



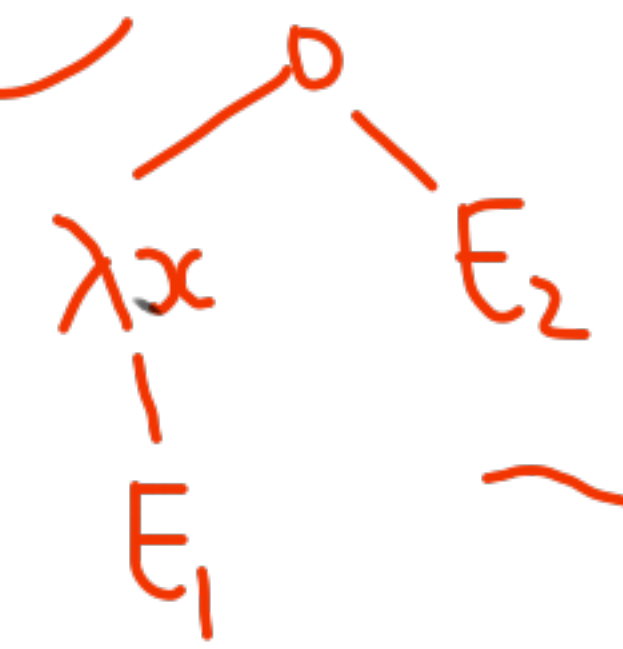
λ -Expression → Variables
 x, y, z
 f, m^n



Abstraction
formal $\approx$ parameter

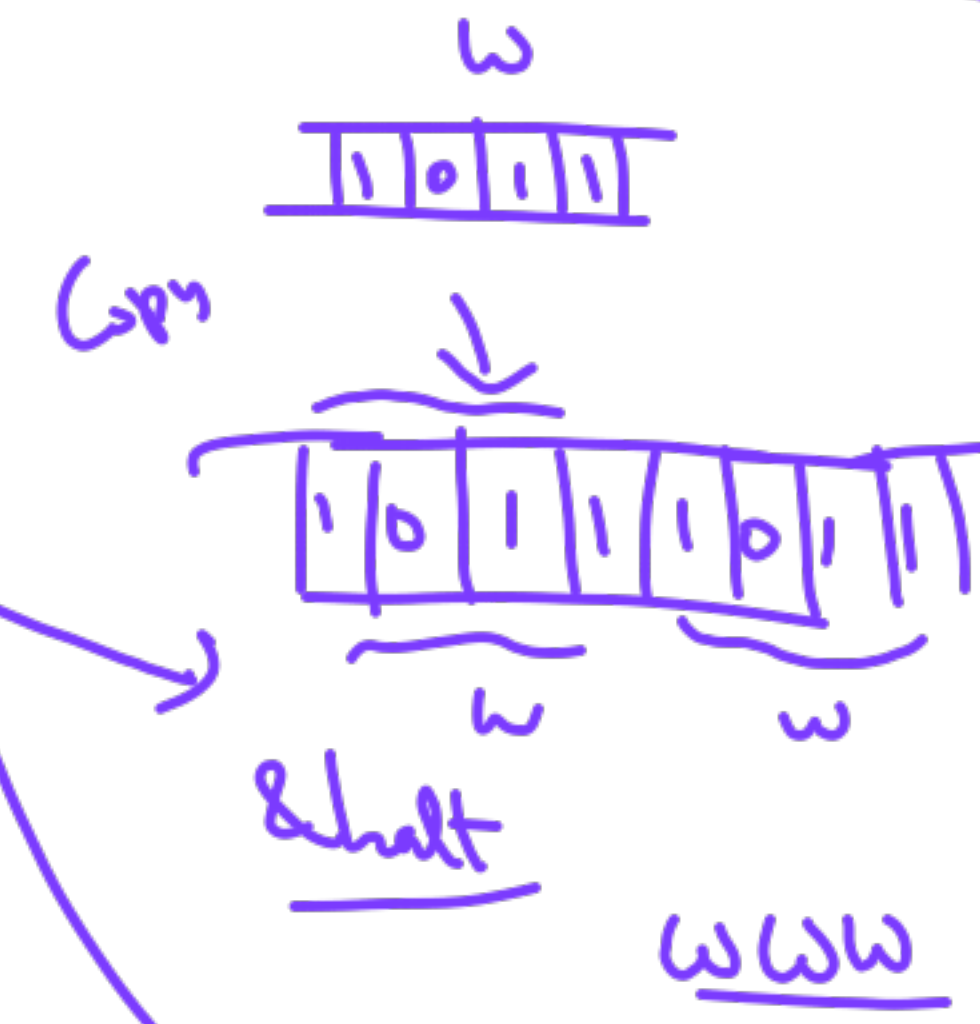
Application

β -redn



$\xrightarrow{\beta} E_1[x \rightarrow E_2]$

Type 0
TM practice



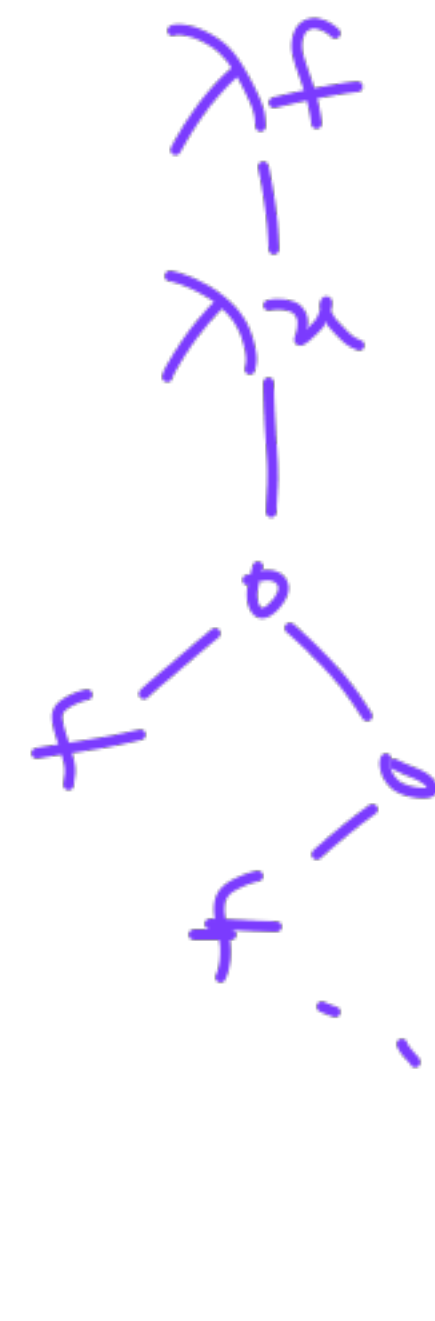
Church numerals

Numbers

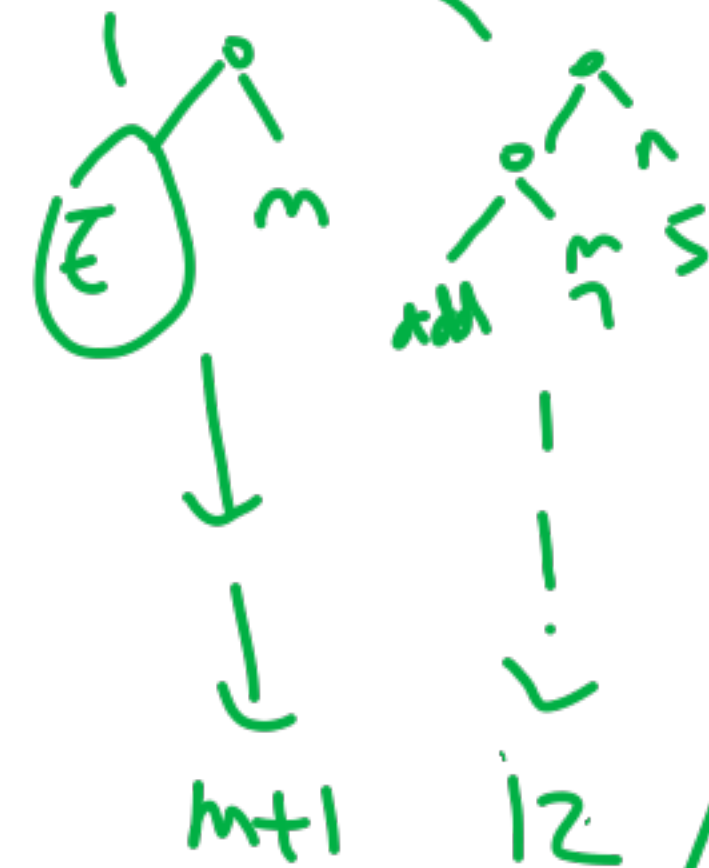
abstraction → 0



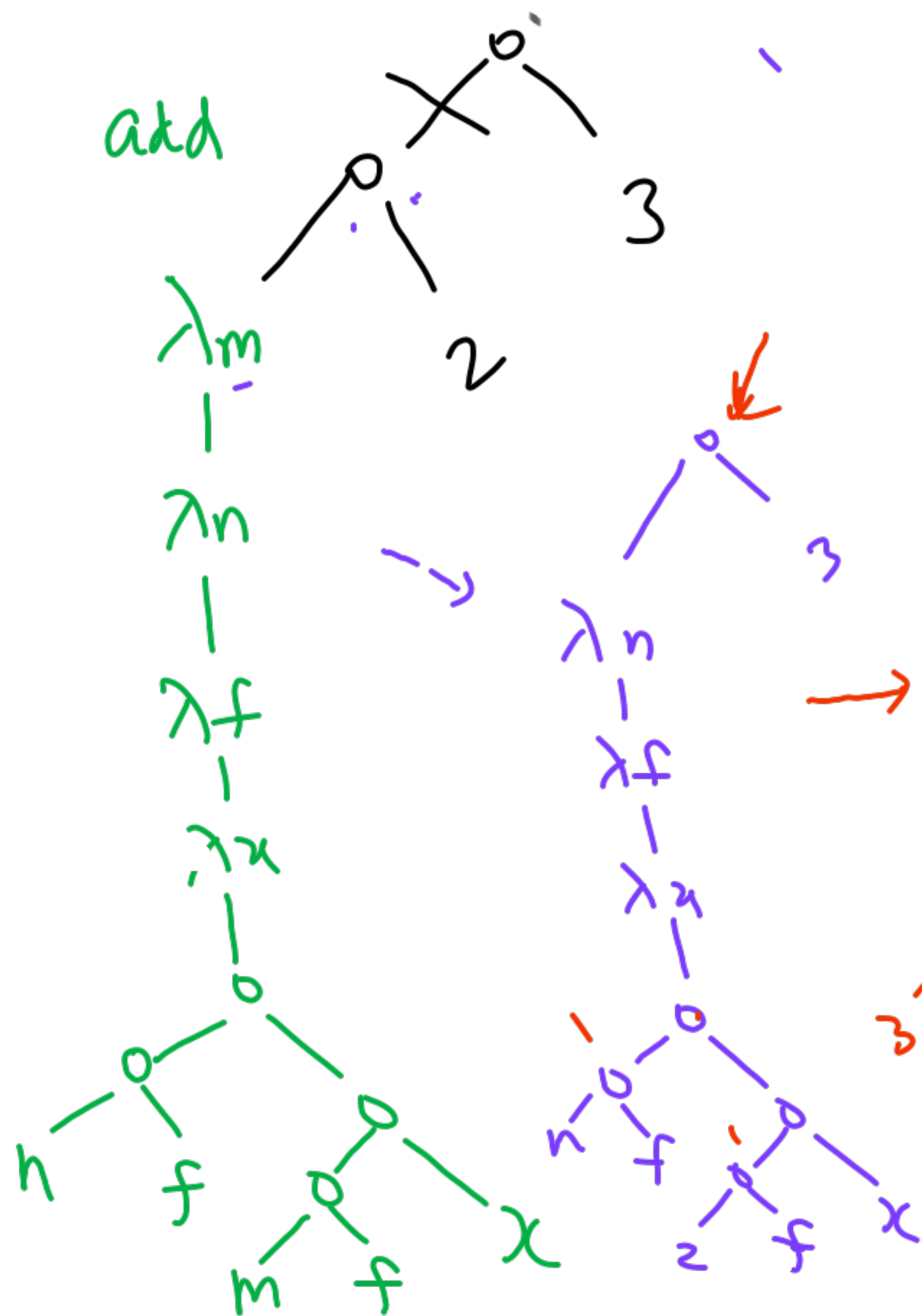
23



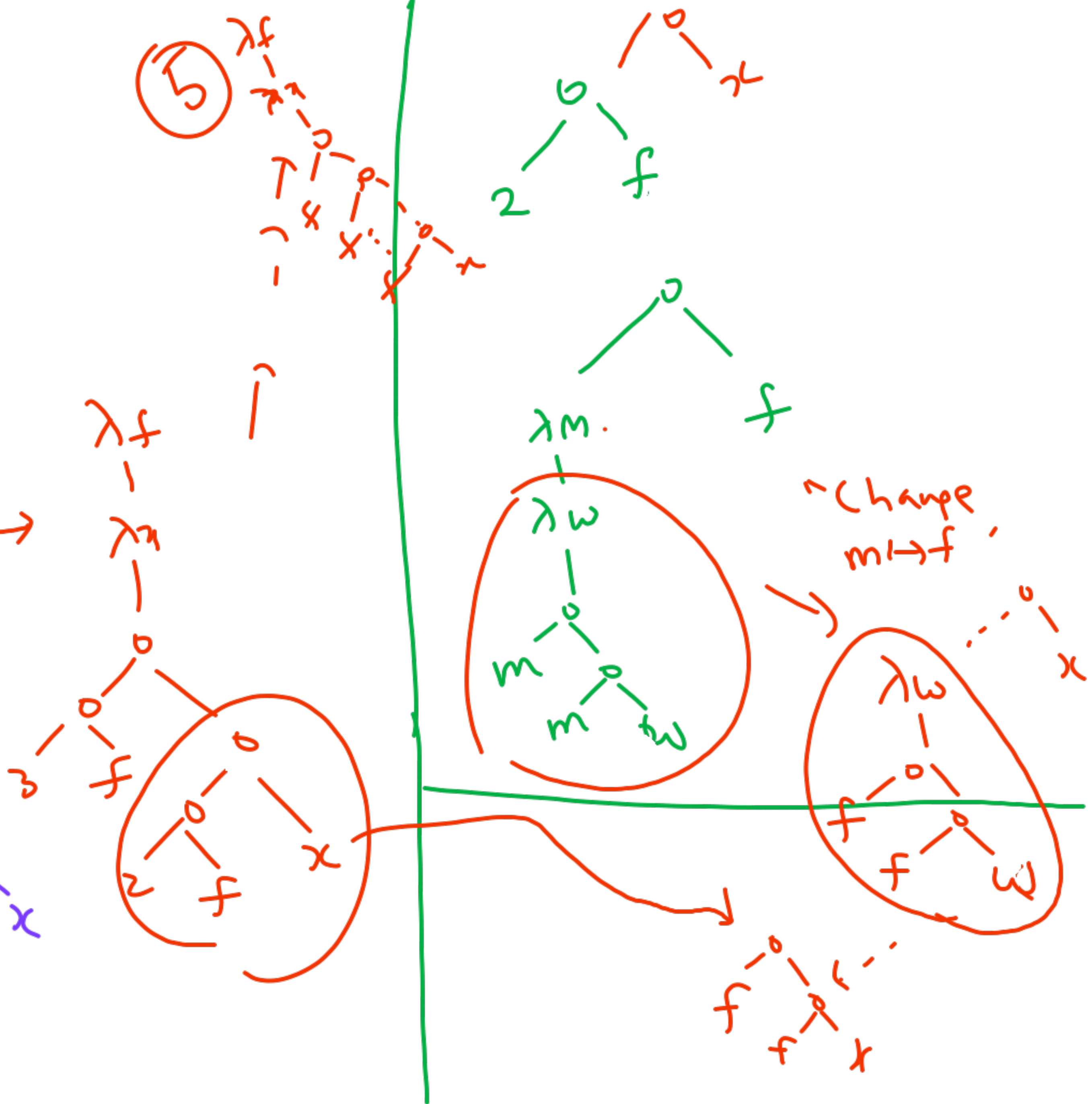
Goal
 Succ add mult exp



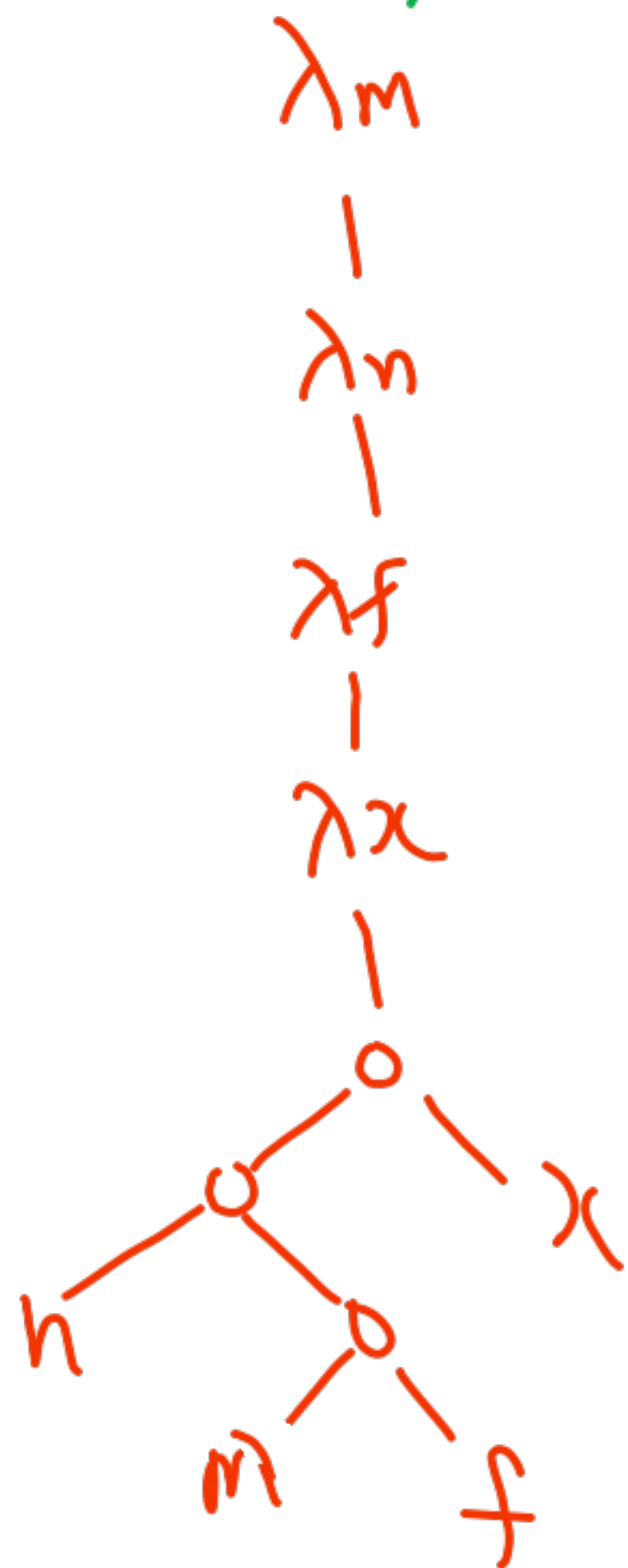
add



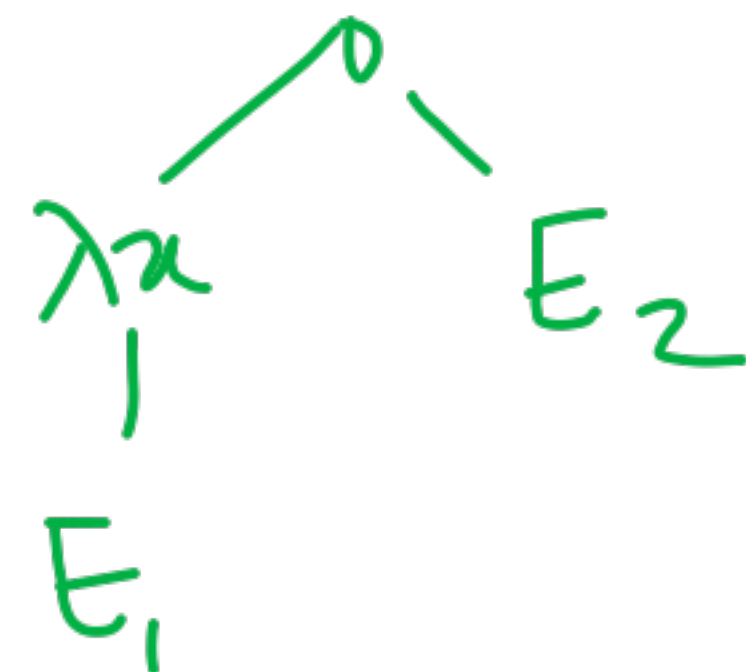
⑤



mult



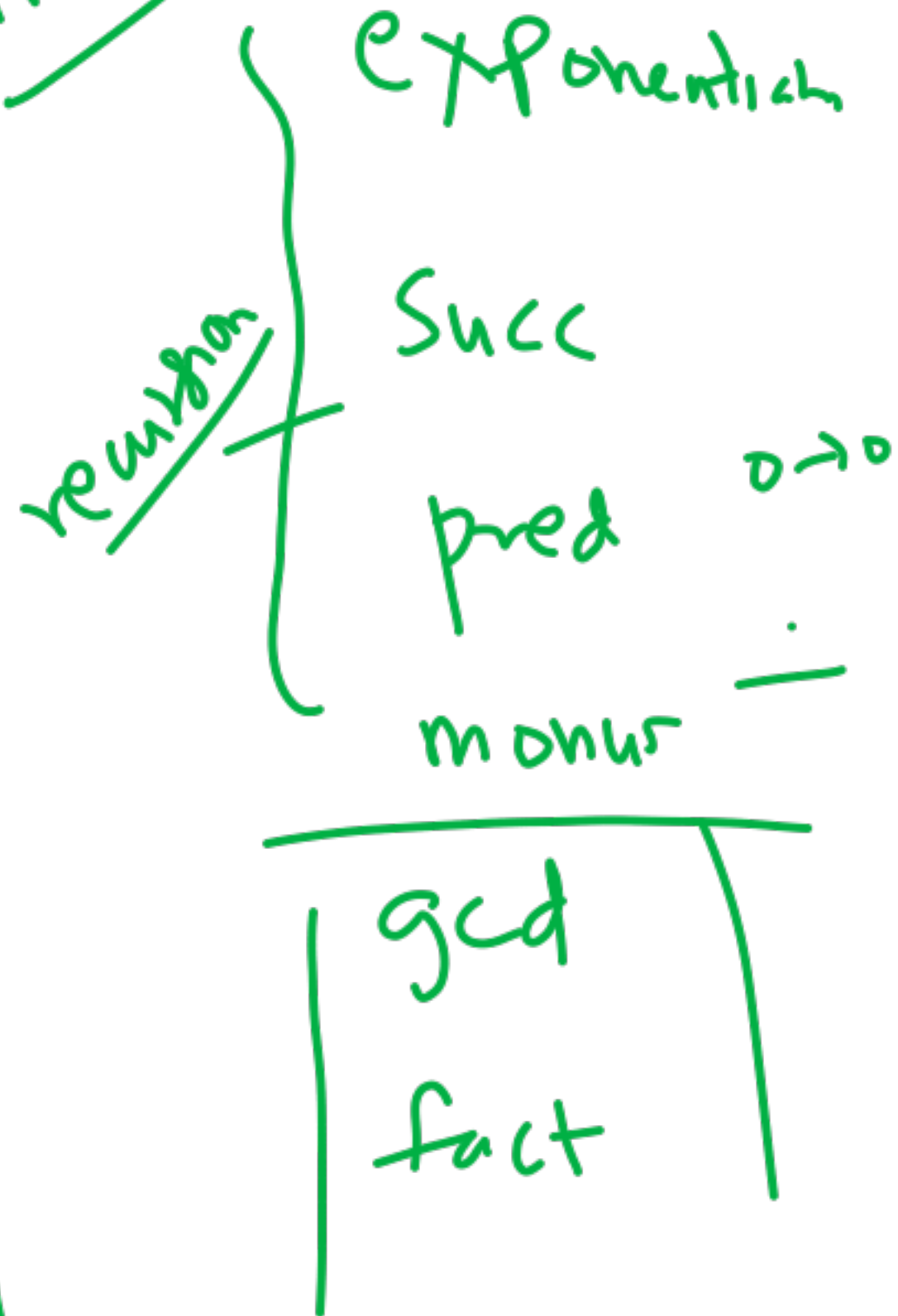
β-reduction



$E_1[x \mapsto E_2]$

"replace x by E_2 in E_1 "

Arithmetik



Boolean

False

True

λx
|
 λy
|
 y

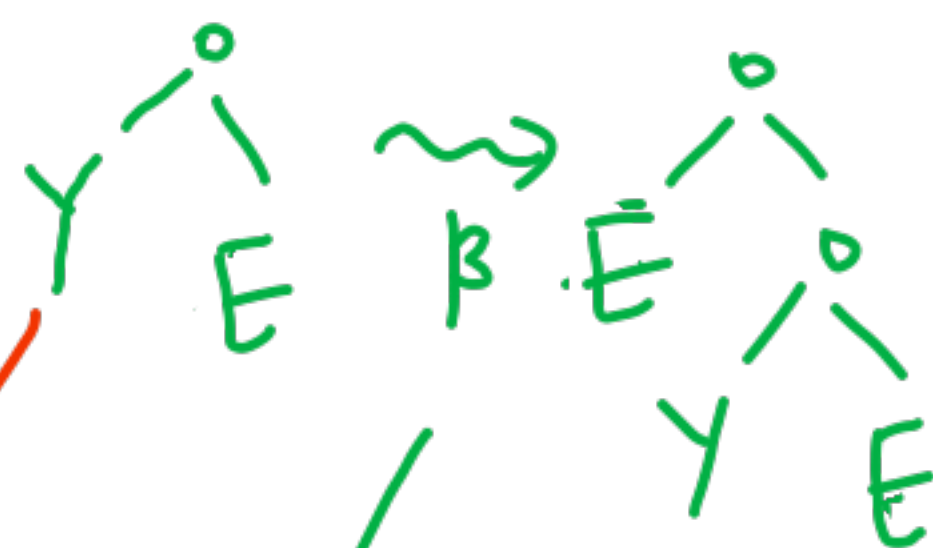
λx
|
 λy
|
 x

and

λp
|
 λa

or $\Rightarrow$ not

Reduction



1-step unrolling of reduction

Y is



"Lazy" eval

if-then-else

test Exp
==

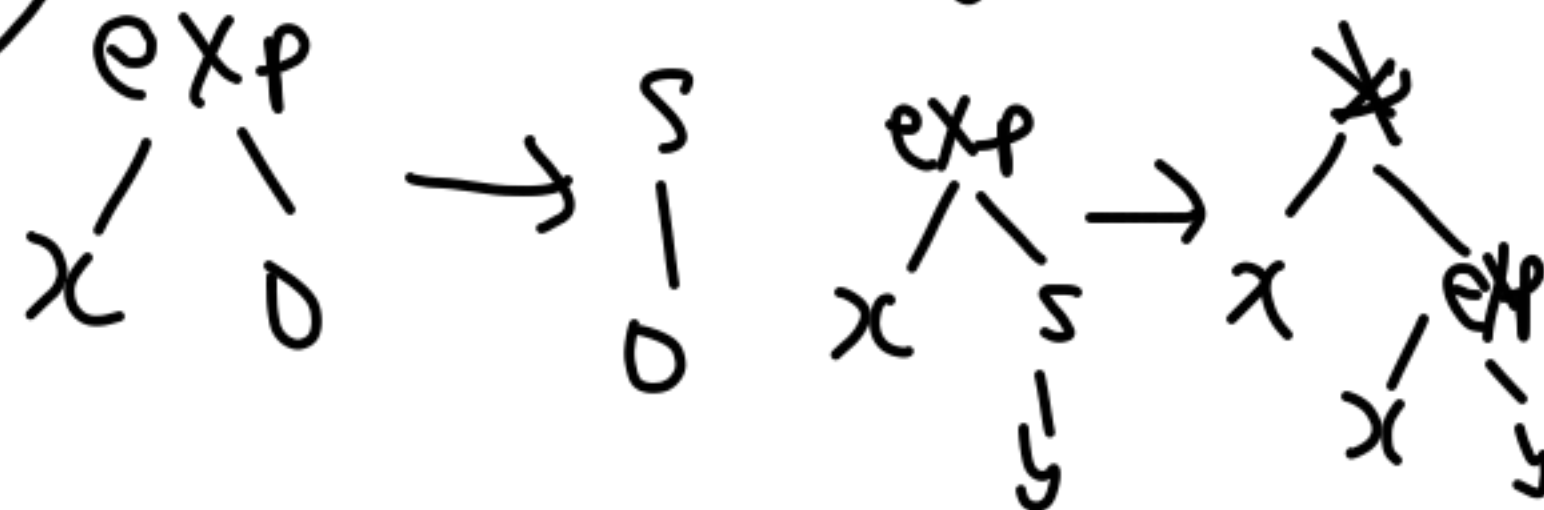
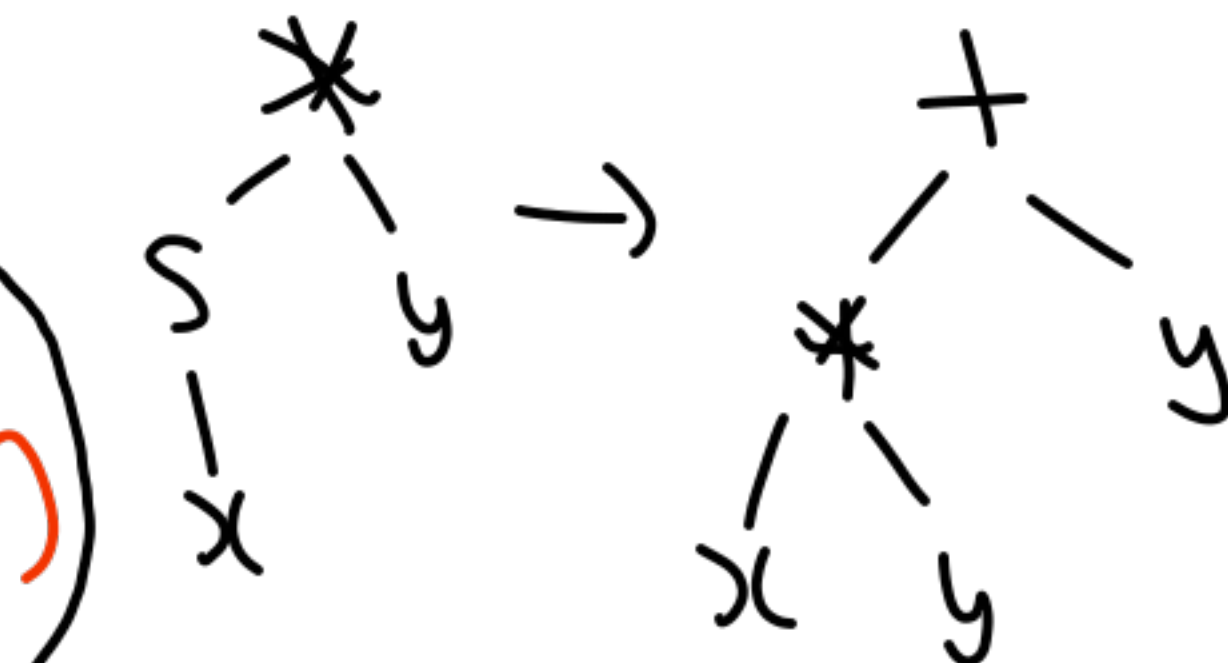
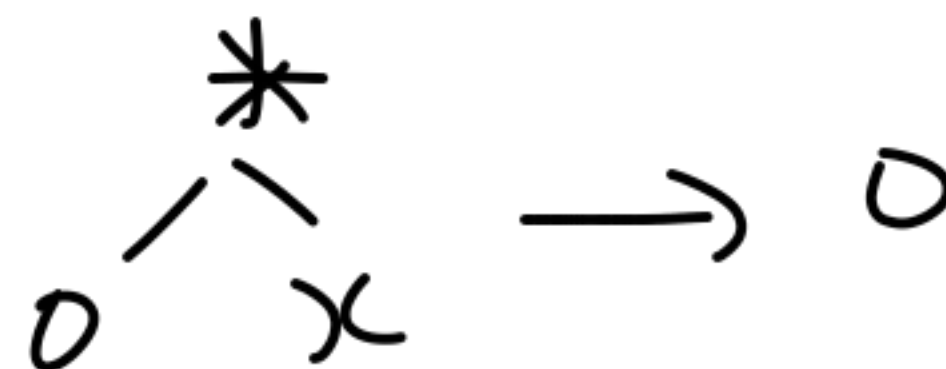
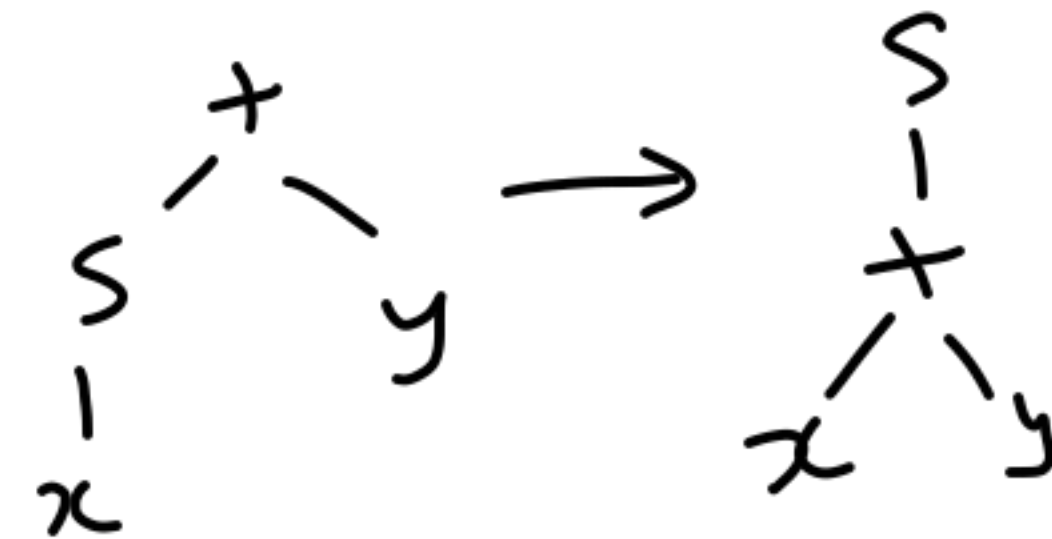
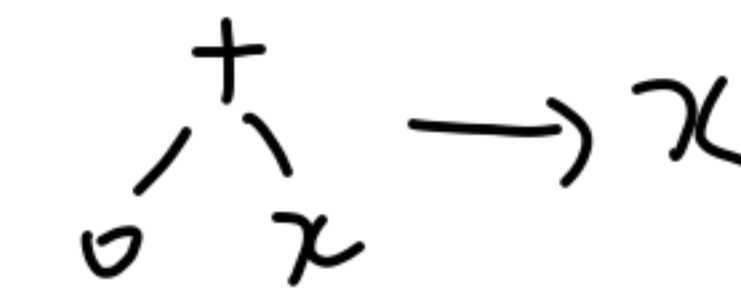
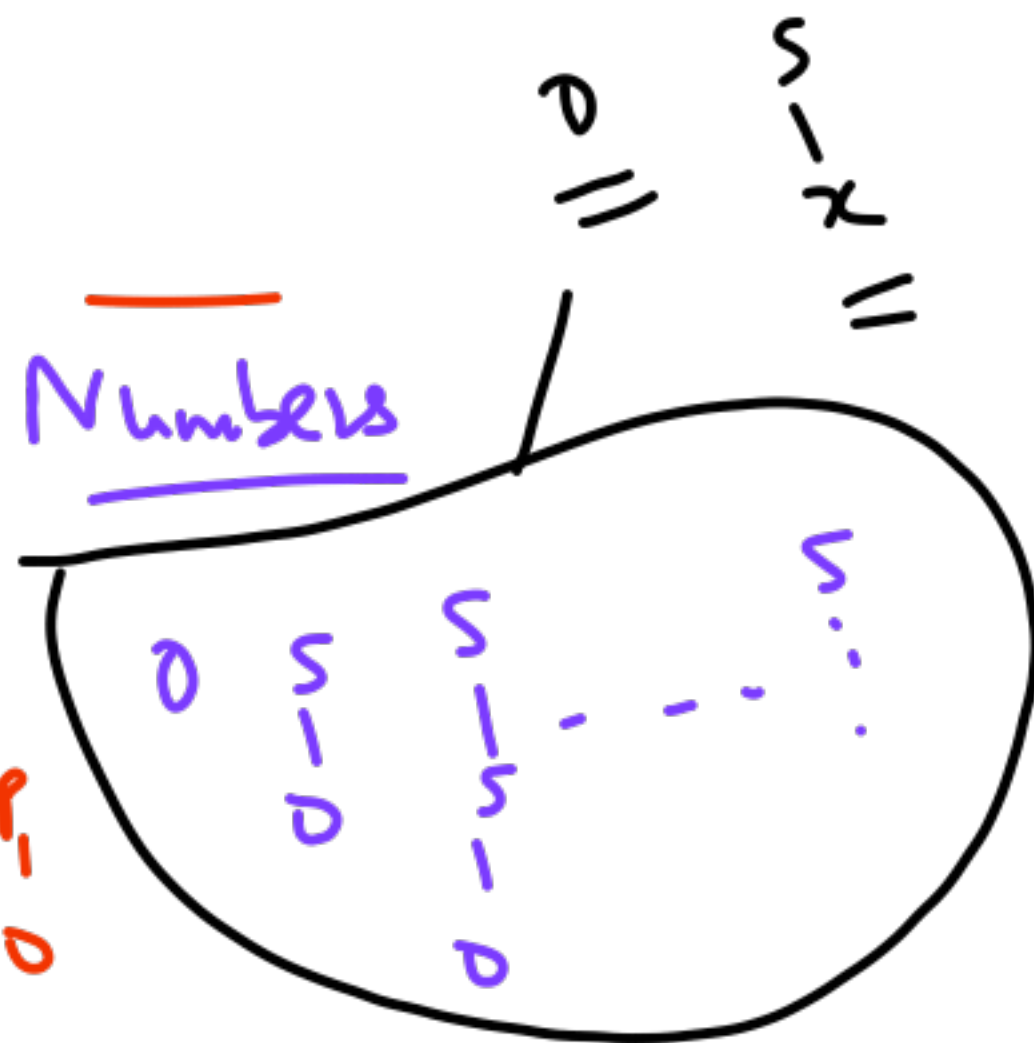
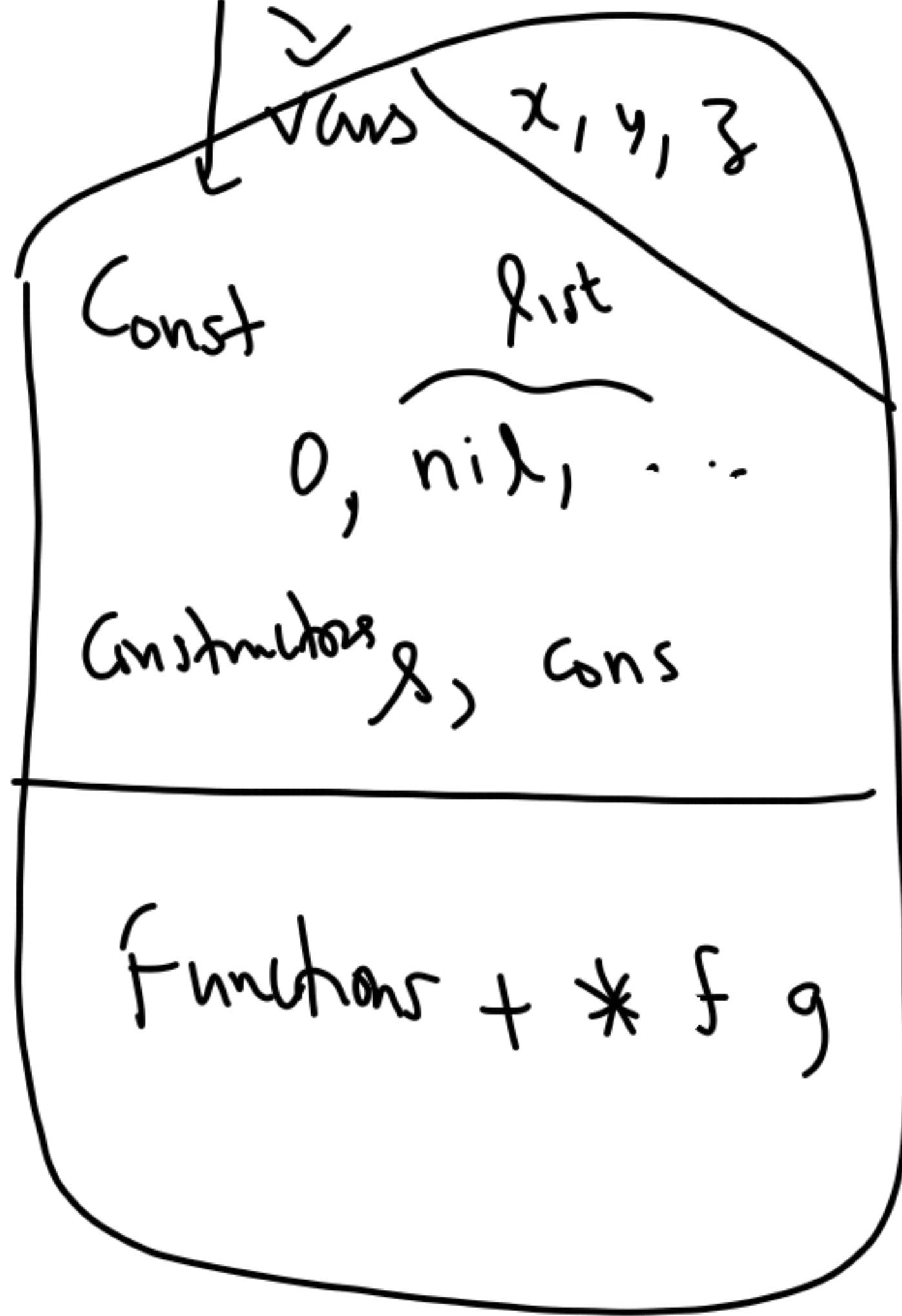
then Exp
==

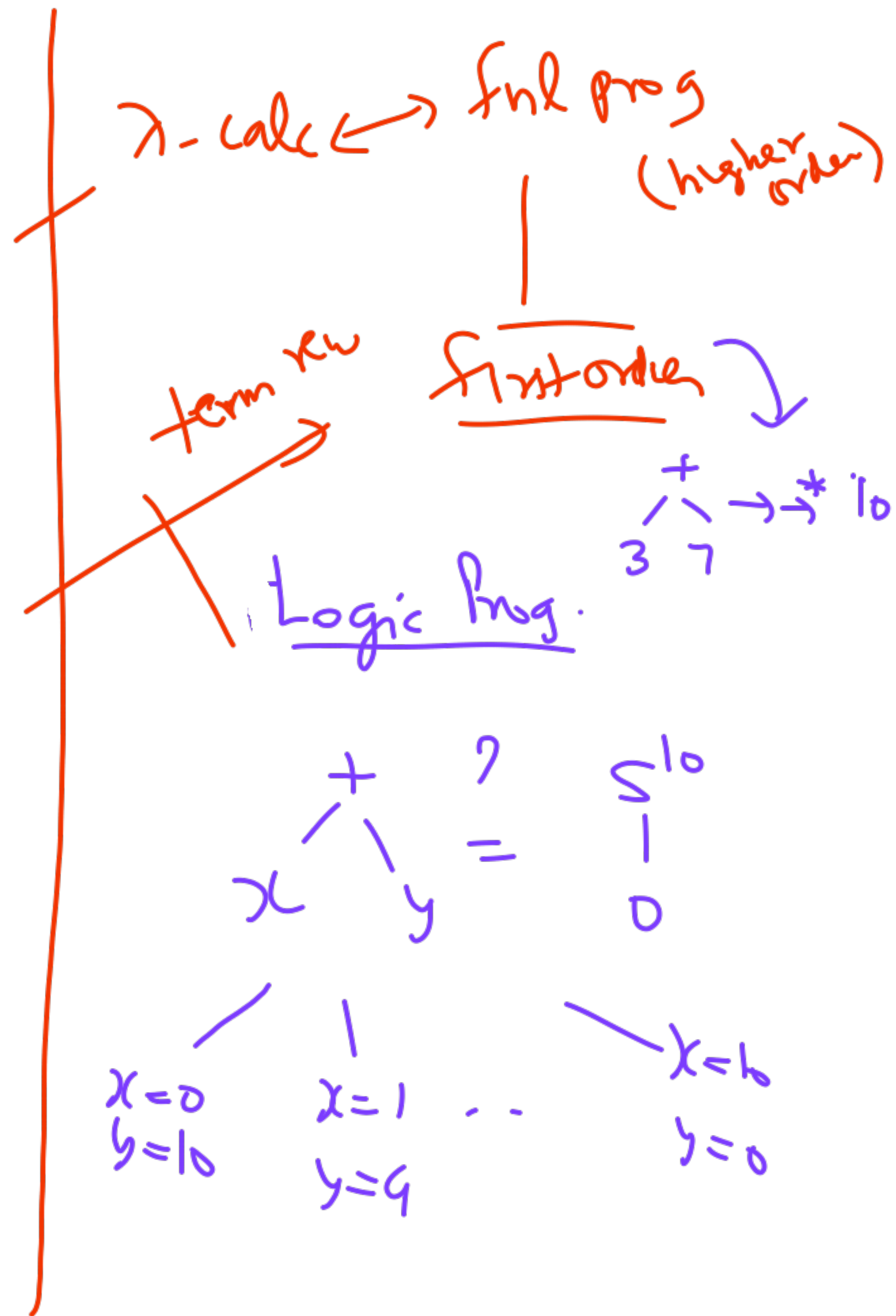
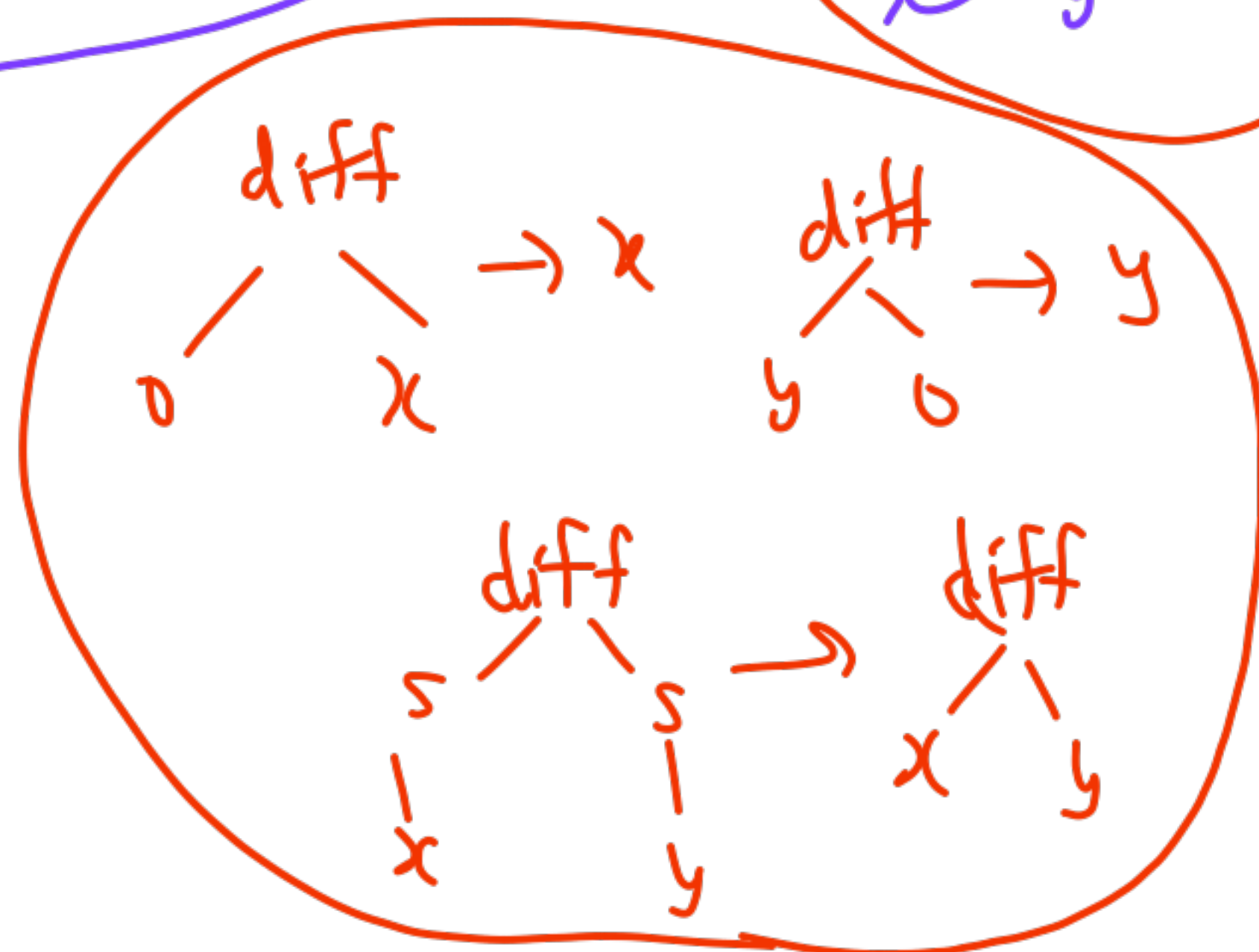
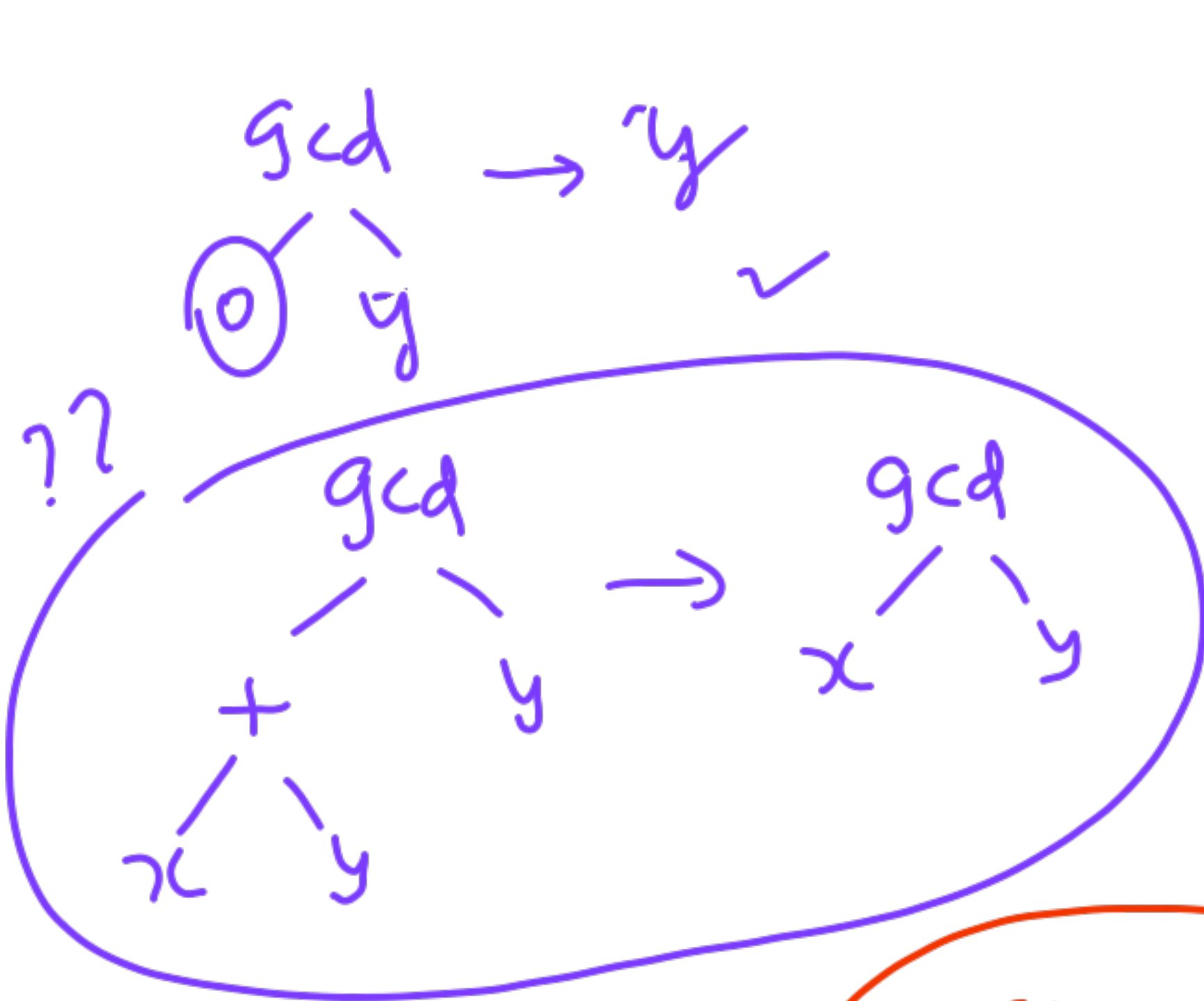
else Exp
==



Term Rewriting Systems

rules
replace equals by equals
 $l \rightarrow r$





22.10.25

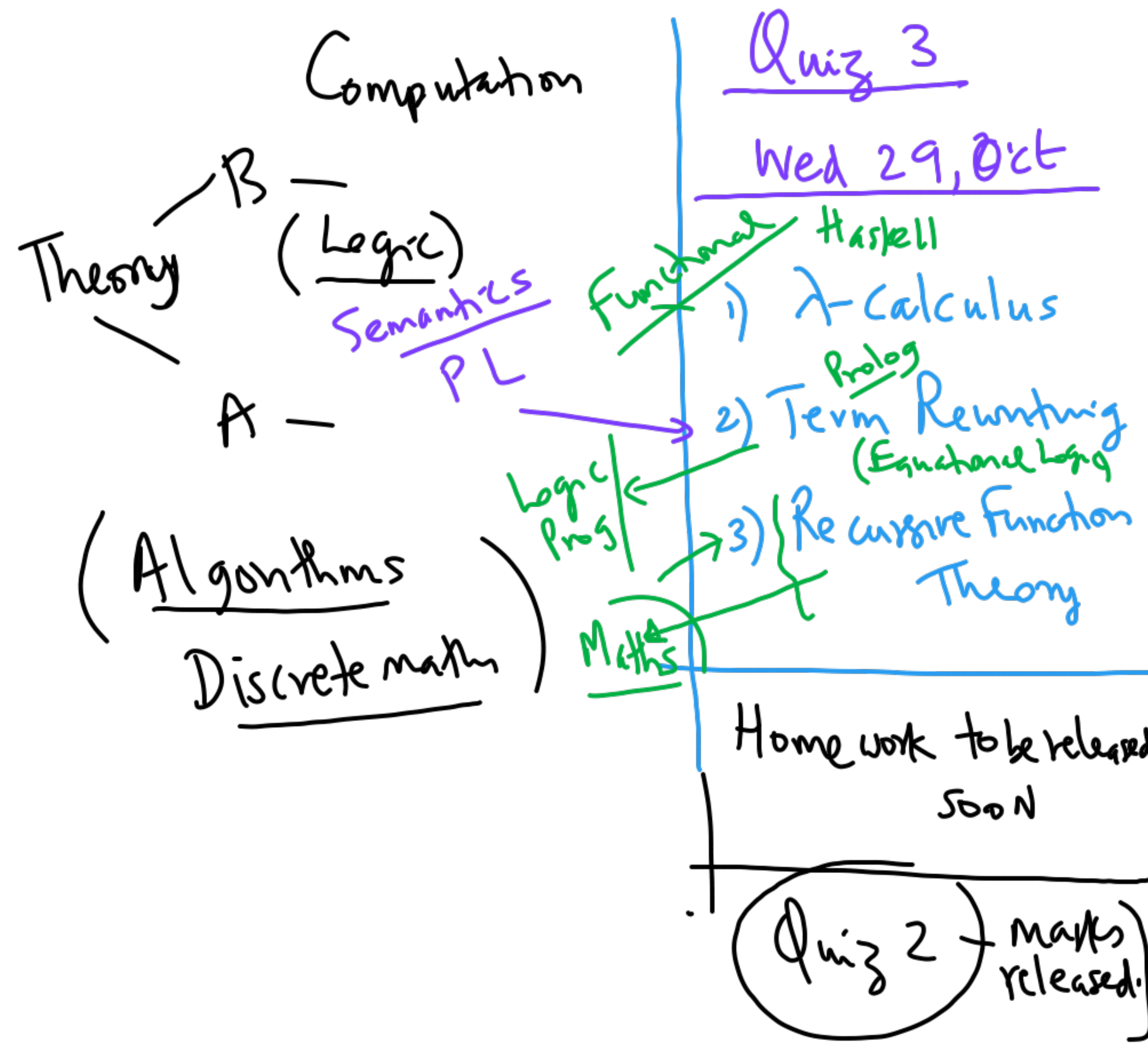
Rec. Fn. Theory



(Building block)

How to define functions f

0
 $g(n)$ successor $\rightarrow n+1$
in a formal mathematical way



Basic Functions:

1. Zero $Z(n) = 0$
2. Succ $S(n) = n + 1$

3. Projection $P_i^k(n_1, \dots, n_k) = n_i$

$k \geq i$

$P^{32} \rightarrow \# \text{ inputs}$

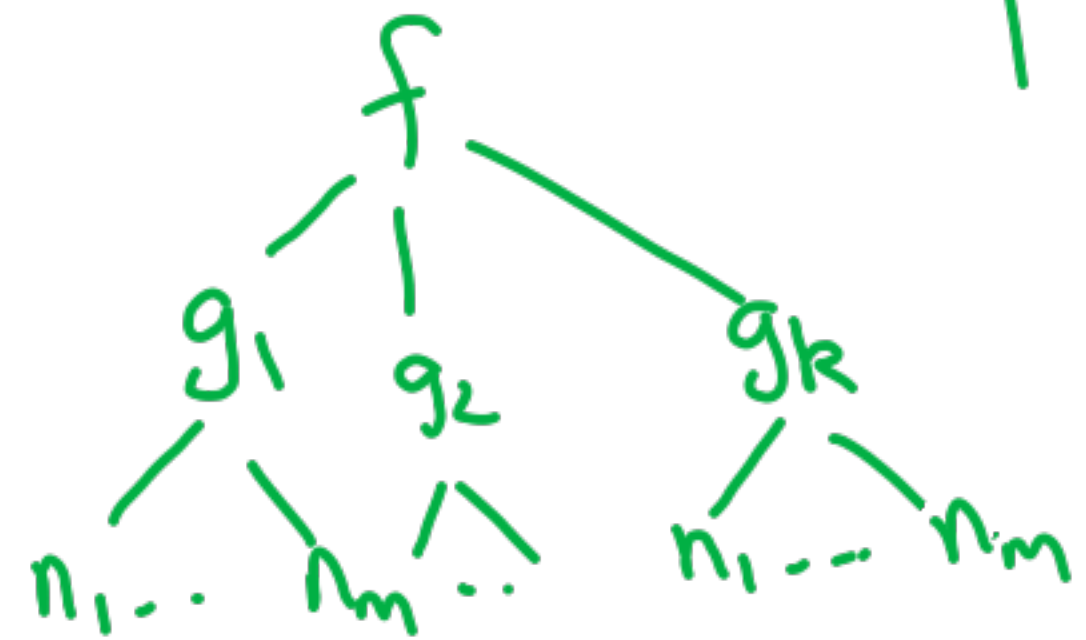
$P_7 \rightarrow \text{projected output}$

4. Composition

$\begin{matrix} \text{arity} & f & k \\ \downarrow & & \end{matrix} f: \mathbb{N}^k \rightarrow \mathbb{N}$

$\begin{matrix} g_i & m \\ \downarrow & \end{matrix} g_1, \dots, g_k: \mathbb{N}^m \rightarrow \mathbb{N}$

$f \circ (g_1, \dots, g_k) =$



Zero, Succ, Proj

Primitive Recursive Functions

given $g: N^k \rightarrow N$ $h: N^{k+2} \rightarrow N$

$f: N^{k+1} \rightarrow N$

define

Base Case →

$$0 \underset{\substack{| \\ n_1}}{\overset{f}{\swarrow}} \dots \underset{n_k}{\searrow} = \underset{\substack{| \\ n_1}}{\overset{g}{\swarrow}} \dots \underset{n_k}{\searrow}$$

Rec Case

$$s \underset{\substack{| \\ n}}{\overset{f}{\swarrow}} \dots \underset{n_k}{\searrow} = \underset{\substack{| \\ n}}{\overset{h}{\swarrow}} \underset{\substack{| \\ f}}{\swarrow} \dots \underset{n_k}{\searrow}$$

$$\underset{\substack{| \\ n}}{\overset{f}{\swarrow}} \dots \underset{n_k}{\searrow} = \underset{\substack{| \\ n}}{\overset{h}{\swarrow}} \underset{\substack{| \\ f}}{\swarrow} \dots \underset{n_k}{\searrow}$$

$$\underset{\substack{| \\ 0}}{\overset{\text{add}}{\swarrow}} \underset{n_2}{\searrow} = \underset{\substack{| \\ n_2}}{\overset{P_1}{\swarrow}}$$

$$s \underset{\substack{| \\ n}}{\overset{\text{add}}{\swarrow}} \underset{n_2}{\searrow} = \underset{\substack{| \\ n}}{\overset{h}{\swarrow}} \underset{\substack{| \\ \text{add}}}{\swarrow} \dots \underset{n_2}{\searrow}$$

where

$$\underset{\substack{| \\ x_1}}{\overset{h}{\swarrow}} \underset{\substack{| \\ x_2}}{\swarrow} \dots \underset{x_3}{\searrow} = s \underset{\substack{| \\ P_2^3}}{\swarrow} \underset{x_1}{\swarrow} \dots \underset{x_3}{\searrow}$$

$$\begin{array}{c} \text{mult} \\ / \quad \backslash \\ 0 \quad n_2 \end{array} = \begin{array}{c} \text{Zero} \\ | \\ n_2 \end{array}$$

$$\begin{array}{c} \text{mult} \\ / \quad \backslash \\ s \quad n_2 \\ | \\ n \end{array} = \begin{array}{c} h \\ / \quad | \quad \backslash \\ n \quad \text{mult} \quad n_2 \\ \quad \wedge \\ \quad n \quad n_2 \end{array}$$

where

$$\begin{array}{c} h \\ / \quad | \quad \backslash \\ x_1 \quad x_2 \quad x_3 \end{array} = \begin{array}{c} \text{add} \\ / \quad \backslash \\ p_2^3 \quad p_3^3 \\ / \quad \backslash \quad \backslash \quad / \quad \backslash \\ x_1 \quad x_2 \quad x_3 \quad x_1 \quad x_2 \quad x_3 \end{array}$$

$\text{add}_0(p_2^3, p_3^3)$

H/w $\rightarrow$ monus $x \dot{-} y = \begin{cases} 0 & \text{if } x \leq y \\ x - y & \text{otherwise} \end{cases}$

$\searrow$ exp $\searrow$ fact $\searrow$ fib

Ackermann
fn.

$$a(3,3) \rightarrow 61$$

$$a(4,2) \rightarrow 2^{2^{16}} - 3$$

$$a(0, n) = n + 1$$

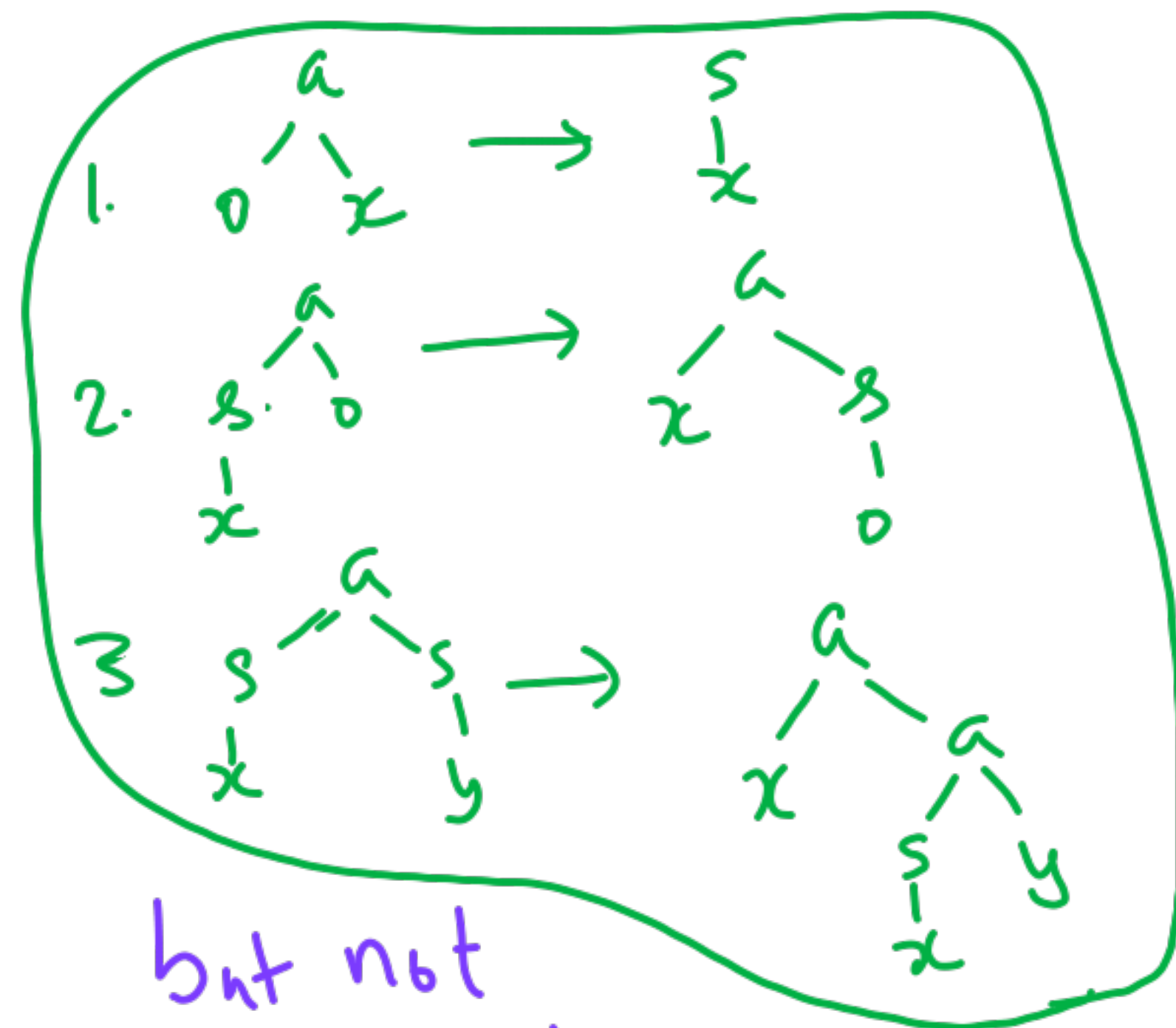
$$a(m, 0) = a(m-1, 1)$$

$$a(m, n) = a(m-1, a(m, n-1))$$

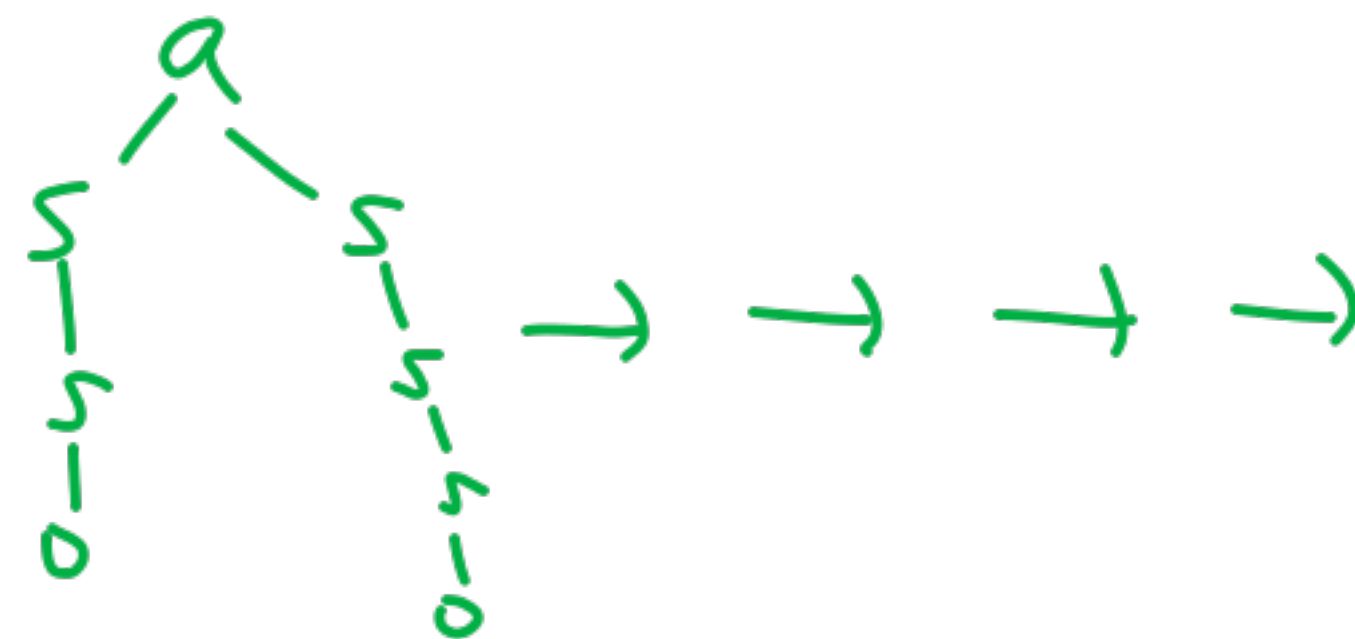
$$a(2,3) \rightarrow a(1, \underline{a(2,2)}) \rightarrow a(1, \underline{a(1, \underline{a(2,1)})})$$

$$\rightarrow \rightarrow 9$$

Term Rewriting



but not
primitive recursion



Total
Partial
Functions

Minimization

Given

define

$$g : N^{k+1} \rightarrow N$$

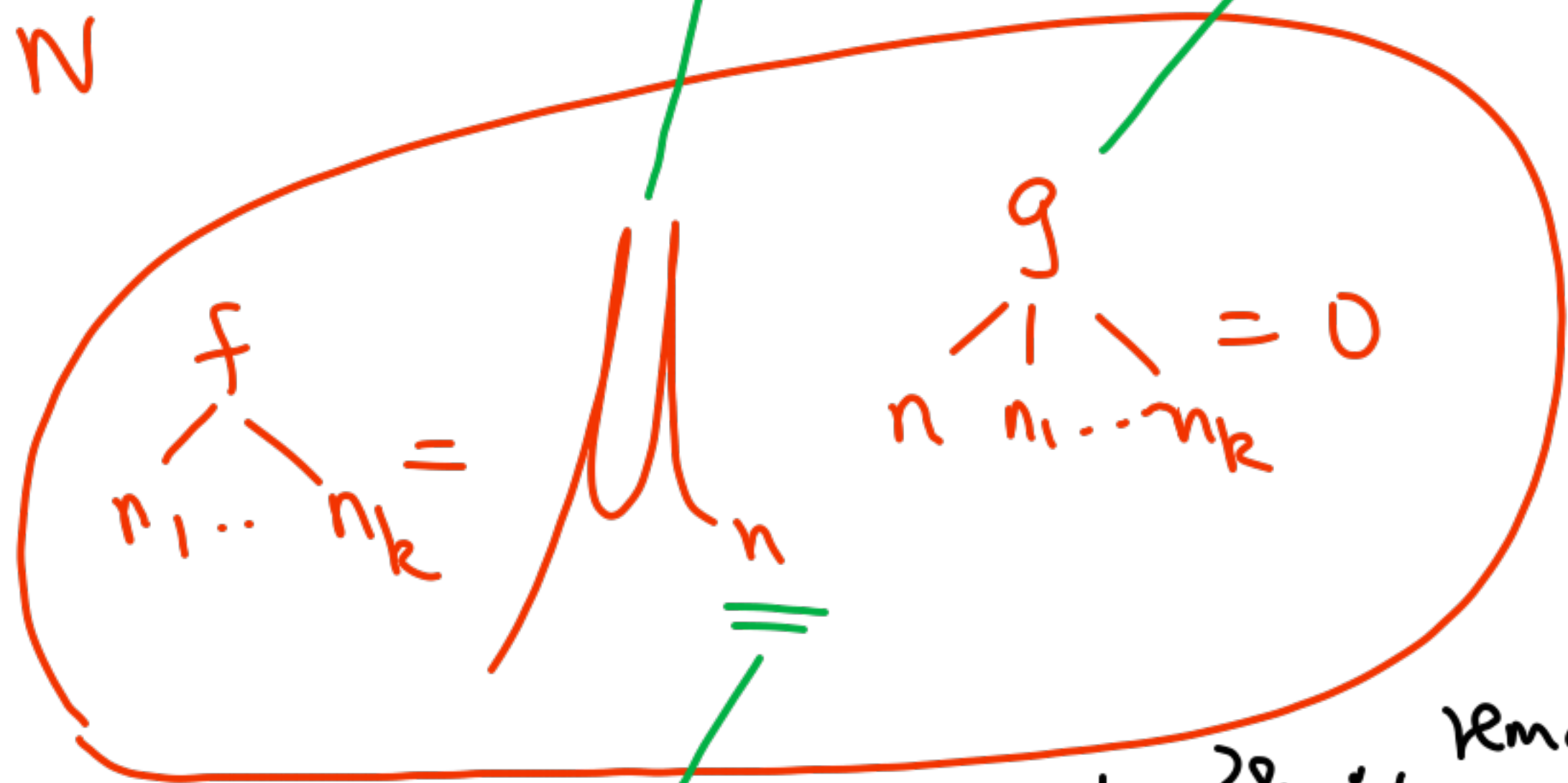
$$f : N^k \rightarrow N$$

Smallest
value of n
for which
becomes
0

using
minimization

$$f(13, 45)$$

$$= \begin{array}{c} 9 \\ \swarrow \downarrow \searrow \\ 0 \quad 13 \quad 45 \end{array} \begin{array}{c} 2 \\ \swarrow \downarrow \searrow \\ 1 \quad 13 \quad 45 \end{array} \rightarrow \begin{array}{c} 9 \\ \swarrow \downarrow \searrow \\ 2 \quad 13 \quad 45 \end{array} \begin{array}{c} 2 \\ \swarrow \downarrow \searrow \\ 2 \quad 13 \quad 45 \end{array}$$



$$f(n_1 \dots n_k) \equiv n$$

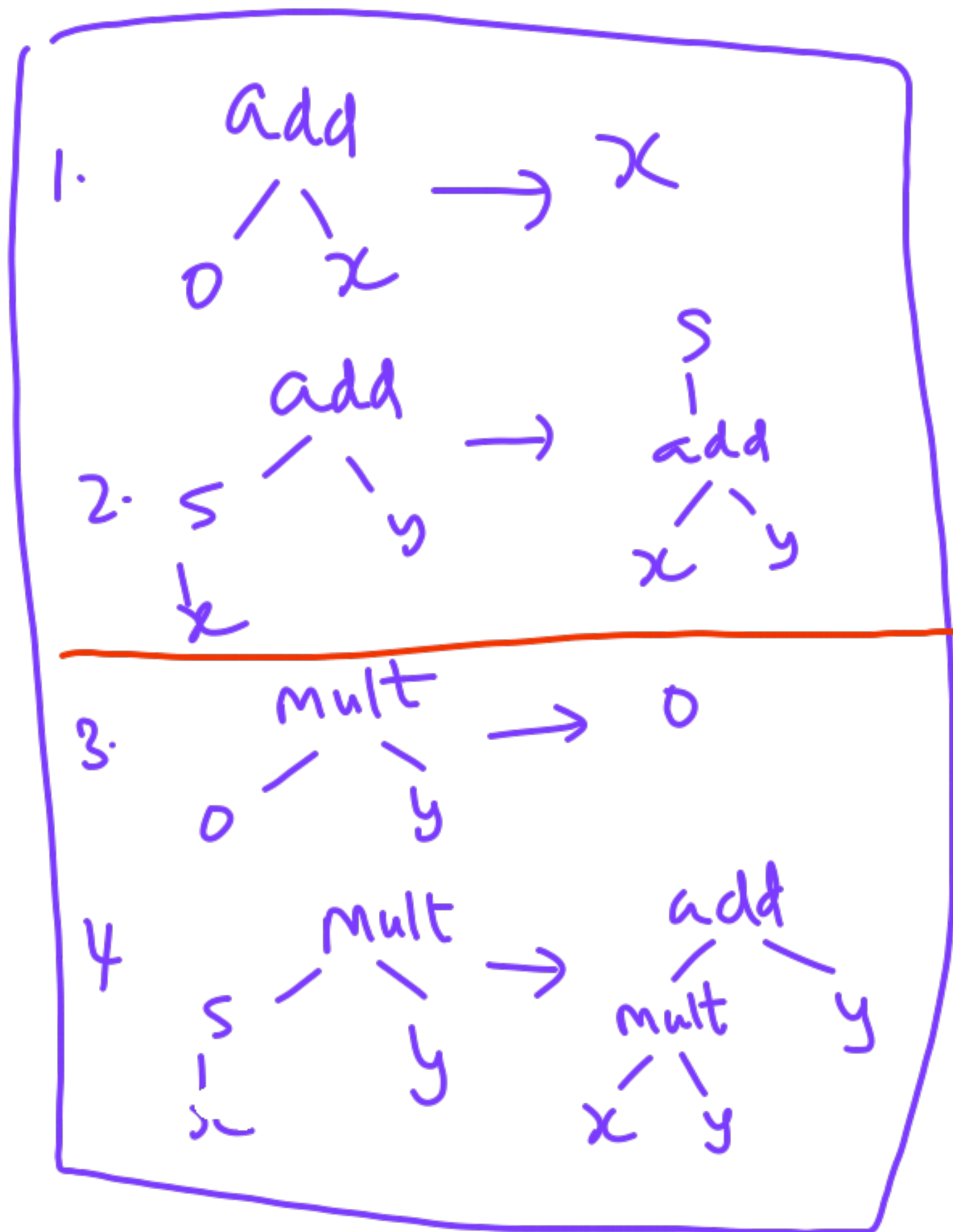
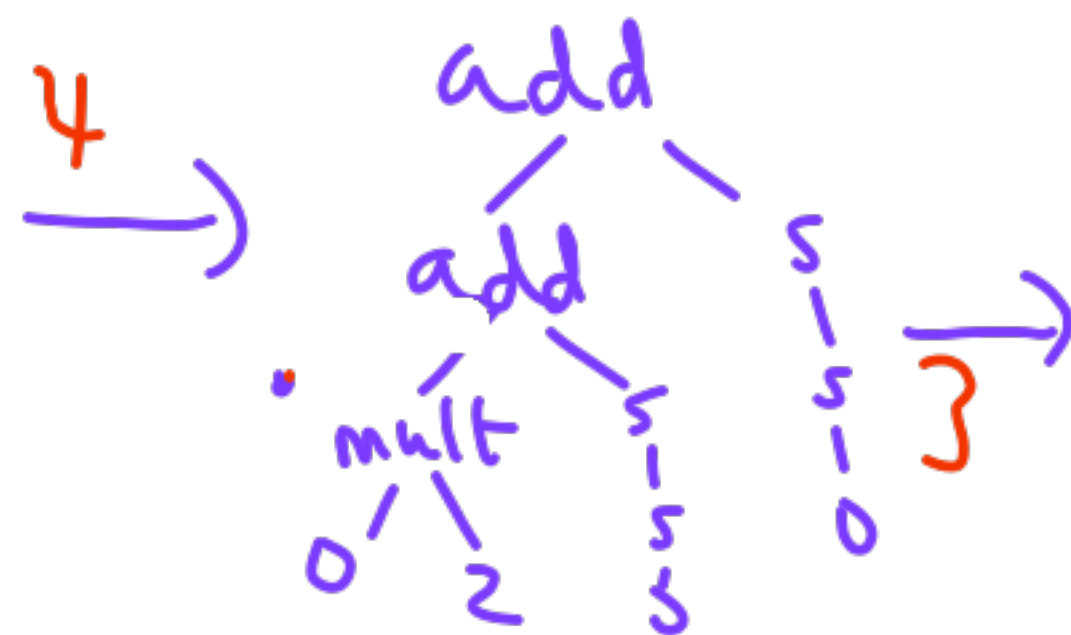
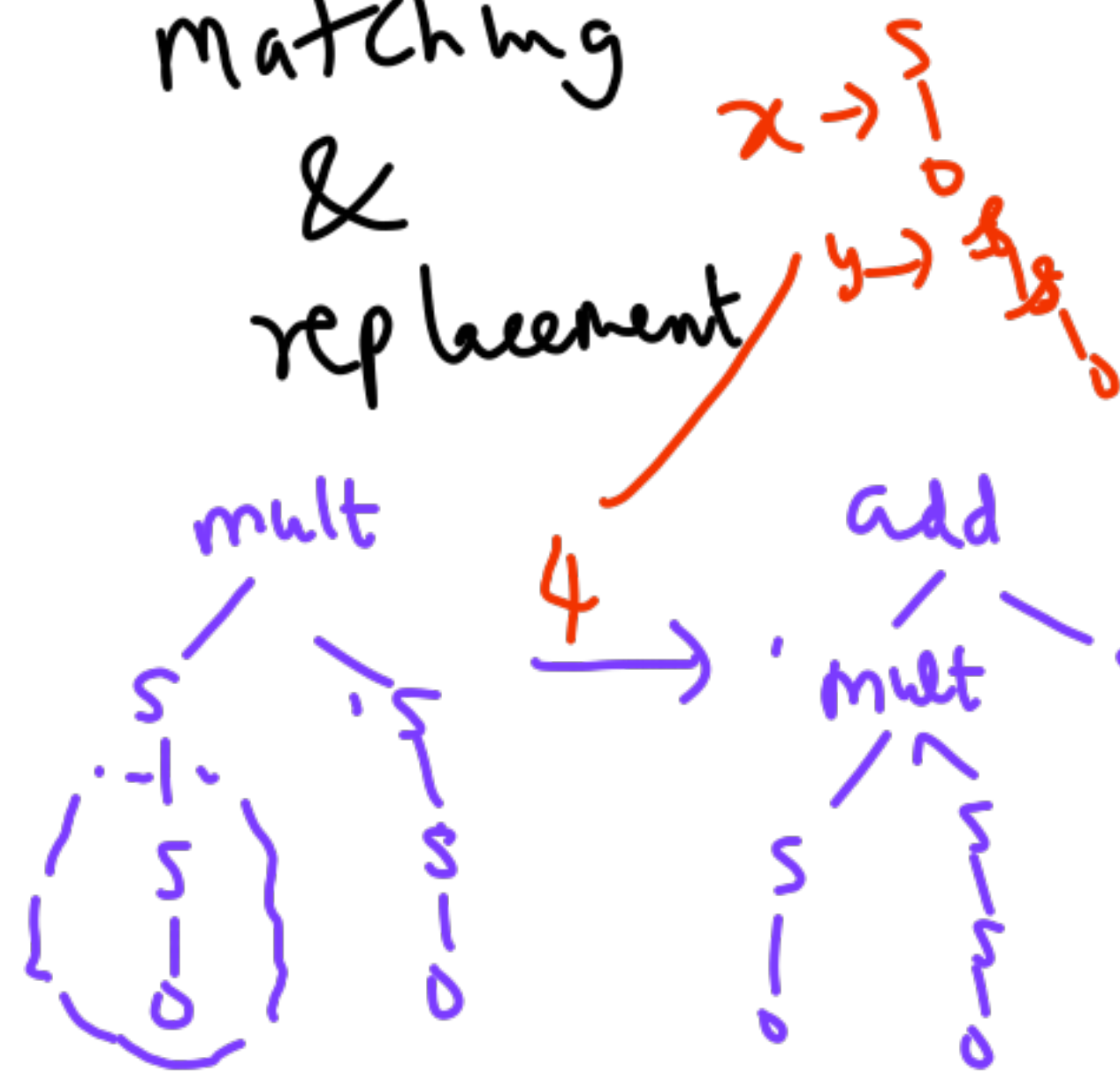
$$\begin{array}{l} H/w \rightarrow 38 \% 7 = 3 \text{ remainder} \\ 38 / 7 = 5 \text{ quotient} \end{array}$$

λ -calc Rec. fn theory

Term Rewriting

Ackerman

matching
&
replacement



make the rule apply

add

u v

?

s
s
0

u = s
u

~~s~~
add
u₁ v

=

~~s~~
s
0

u = s
0

v = s
0

1
u₁ = 0

v = s
0

2
u = s
s
0

v = 0

①

②

③

25.10.25

Quiz 3
Wed 29.

Lazy Evaluation $\rightsquigarrow$ λ expr

Prog Lang

Term

β

β
Input
output

 n_1, n_2

0, 8
Numbers

+ * $\rightarrow$ defined fns

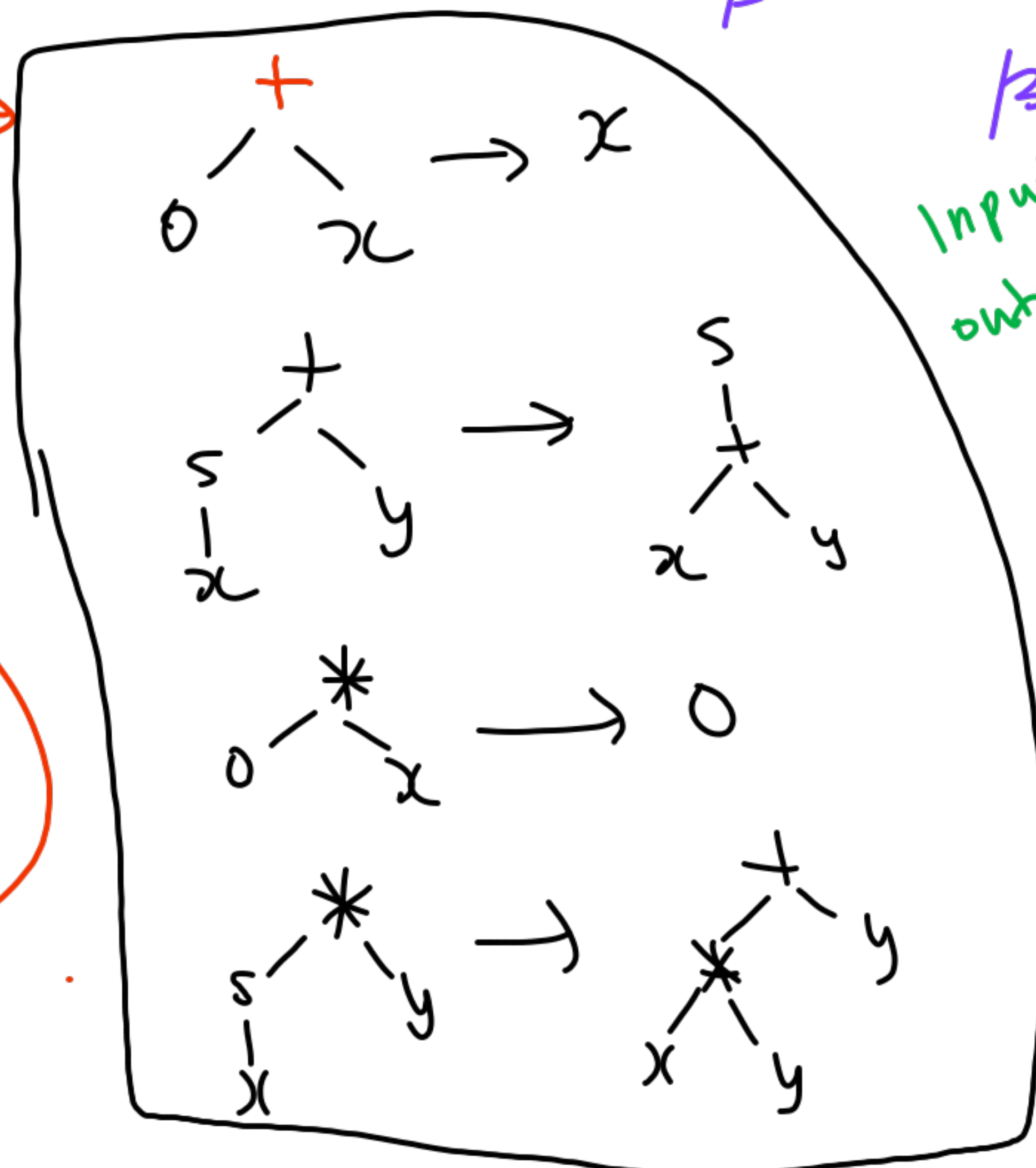
"Complete"

n_1 * $n_2 \rightarrow \dots \rightarrow n_3$

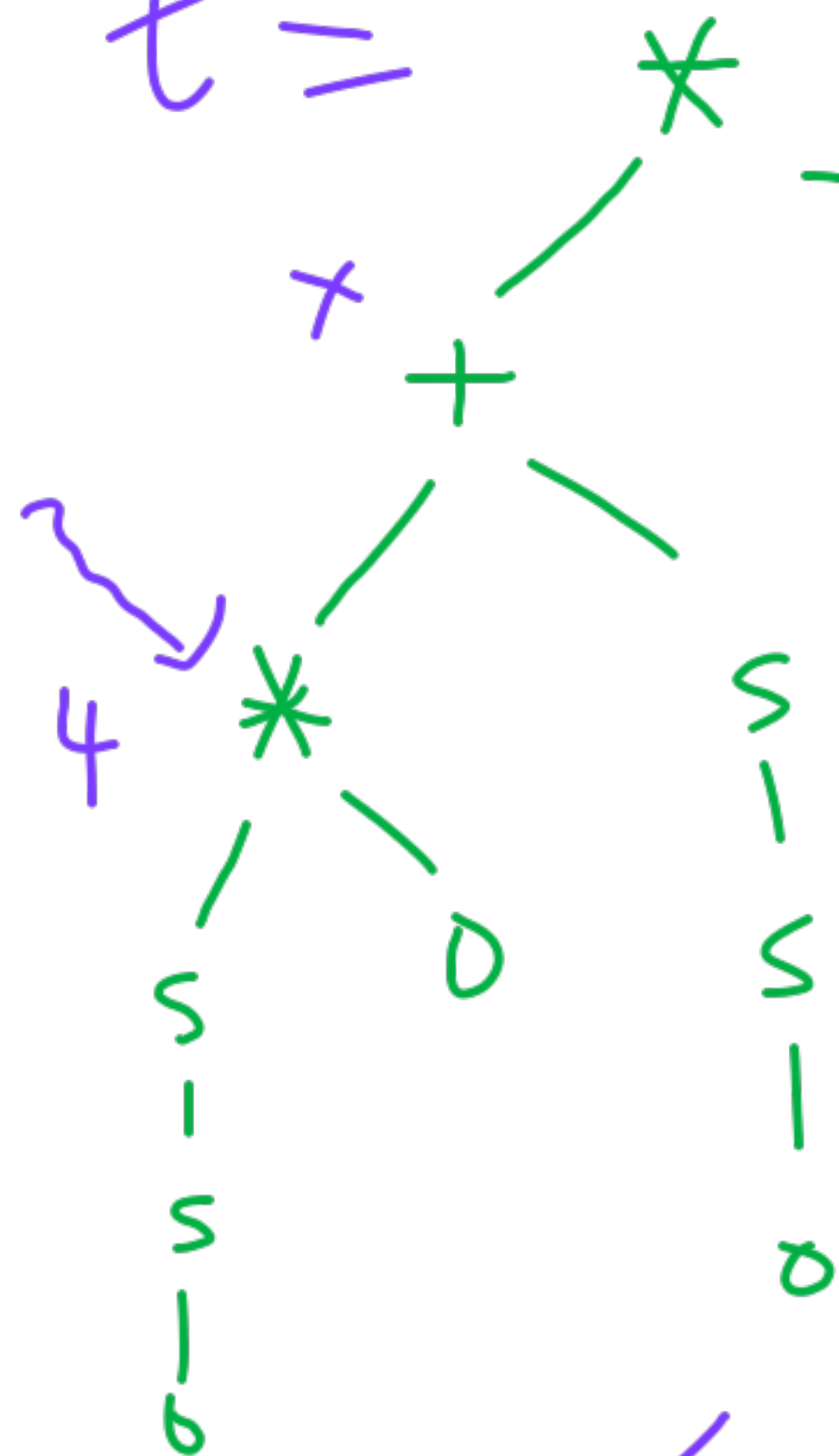
if-then-else
cond then else

def fact(x):
 if x = 0 then 1
 else x * fact(x-1)

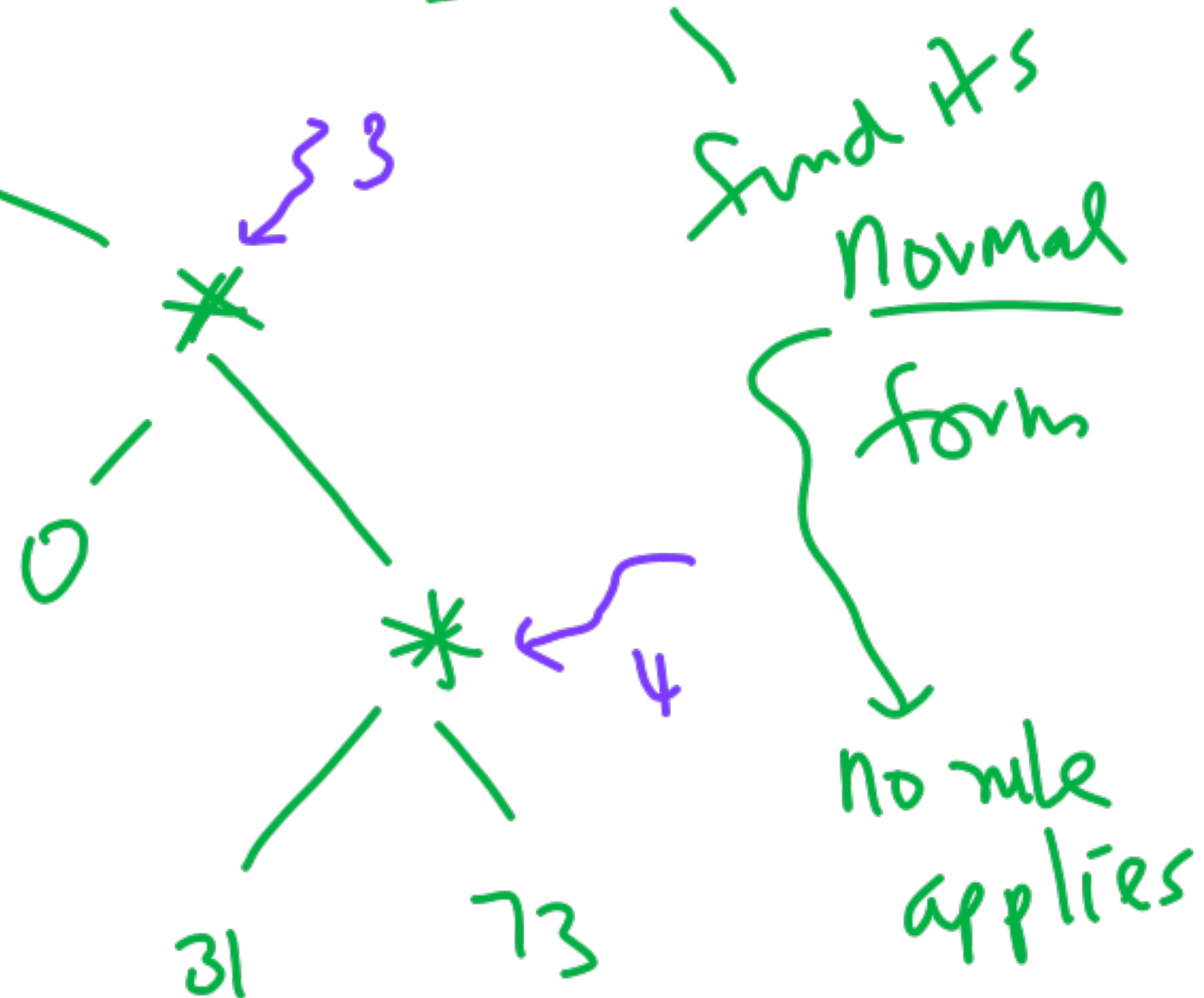
Spectre



t =



evaluate



Find its
Normal
form

no rule
applies

Redex

is a position
where a rule
can "apply"

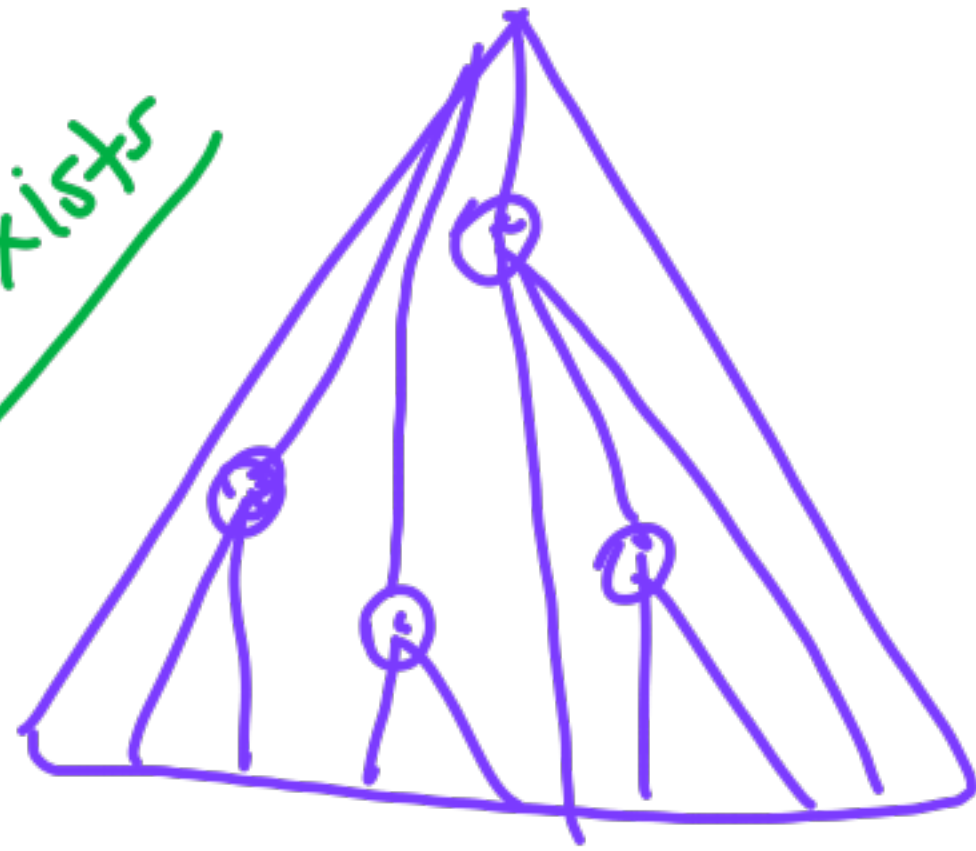
t_1

t_2

t_3

Strategies for evaluating a term

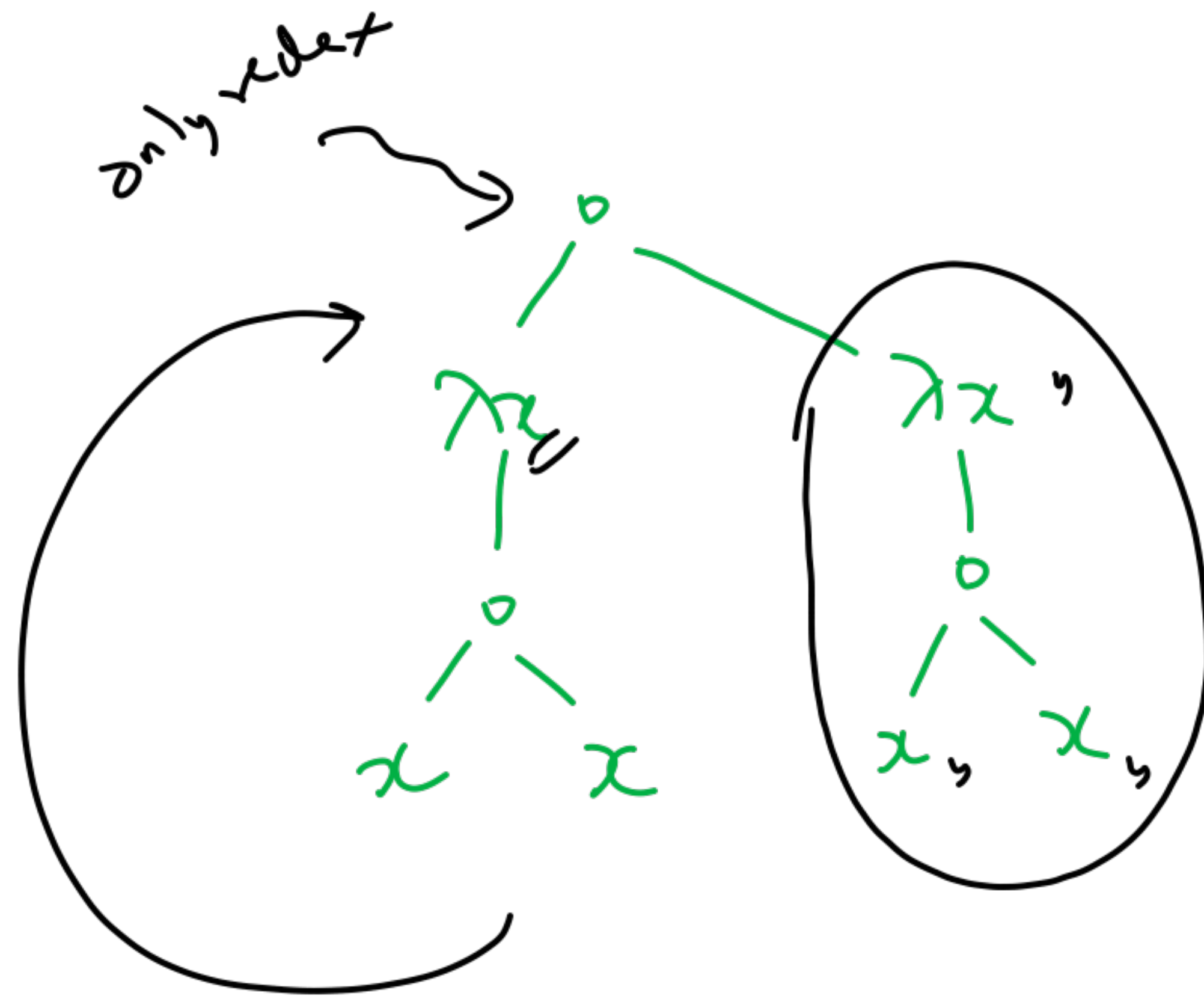
guaranteed
to find normal
form if it exists



Outermost
(Leftmost)
(Lazy)

Innermost $\times$
(bottom-most)
Eager (strict)

Mixed
(any)



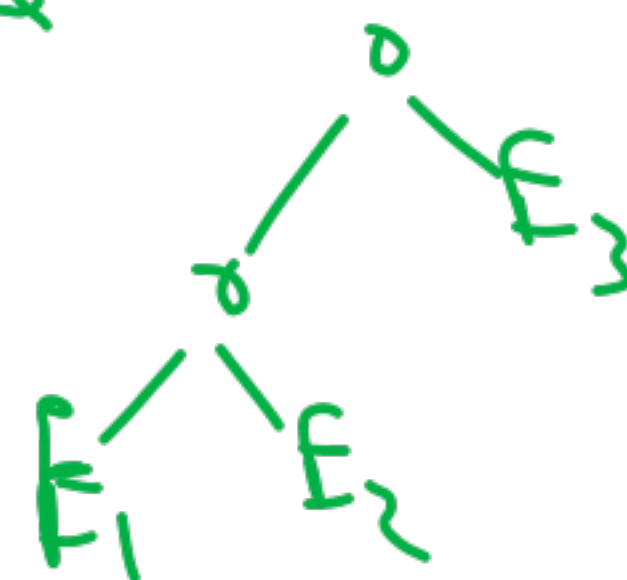
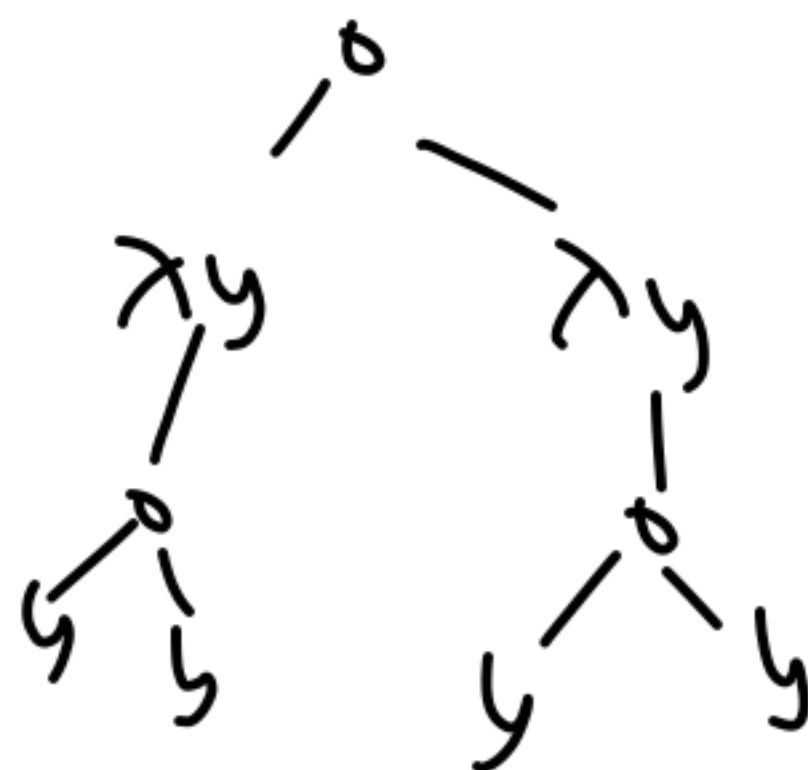
$$(\lambda x. x x) . (\lambda x x x)$$

$E_1 E_2 E_3$

$$((E_1 E_2) E_3)$$

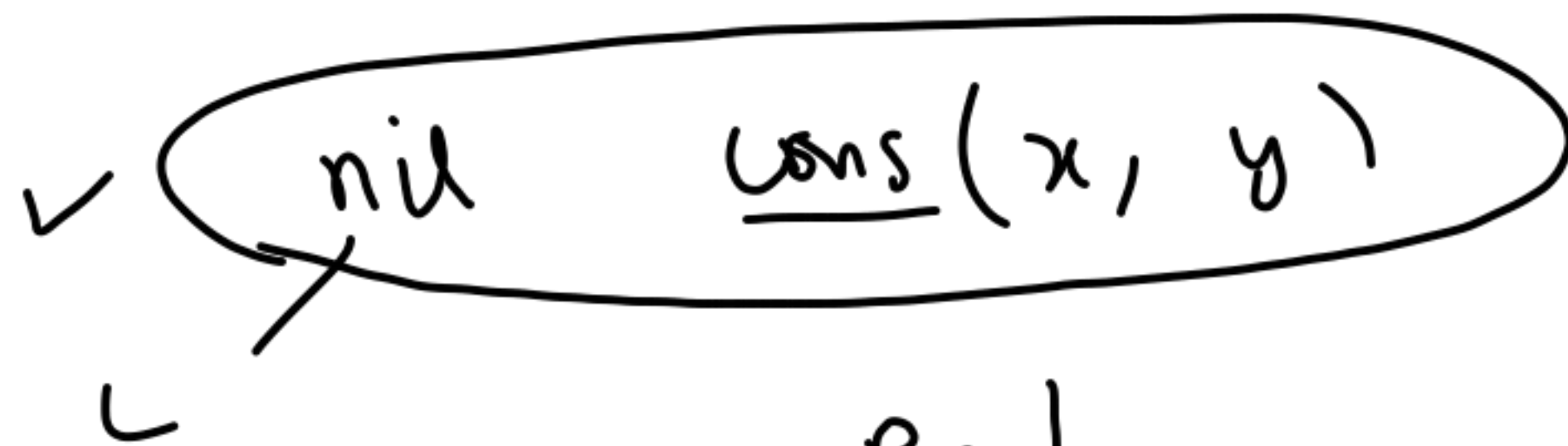
left-associative

$$f(x) \rightarrow f(g(x))$$





$[]$ $[e : L]$



$[]$ $e \circ L$



meaningful term



"x is a member of L" $\rightarrow$ 0 not there
 $\rightarrow$ S if it is there
 0

1. $\text{mem}(x, \overset{\text{nil}}{[]}) \rightarrow 0$

2. $\text{mem}(x, \underbrace{[x : y]}_{\text{cons}(x, y)}) \rightarrow \begin{matrix} S \\ | \\ 0 \end{matrix}$

3. $\text{mem}(x, \underbrace{[y : z]}_{\text{cons}(y, z)}) \rightarrow \text{mem}(x, z)$

$D \times \text{ful}$
 $S \rightarrow \text{true}$
(6)

① $mem(x, nil) \rightarrow 0$

② mem(x, cons(y, z))

$$\rightarrow \text{ite}(\overline{\text{eq}(x, y)}, 0, \text{mem}(x, z))$$

③ $\text{ite}(0, x, y) \rightarrow y$

④ $\text{ite} \left(\begin{smallmatrix} s \\ 0 \end{smallmatrix}, x, y \right) \rightarrow x$

⑤ $e_v(0,0) \rightarrow \sum_0 e_v\left(\begin{smallmatrix} s \\ x, y \end{smallmatrix}\right) \rightarrow e_v(x,y)$

⑥ $Q_x(0, s_1) \rightarrow 0$

⑦ $e_v(s, 0) \rightarrow 0$

⑧-

Tail-Recursion

$$f(b) \rightarrow \begin{pmatrix} s \\ 1 \\ 0 \end{pmatrix}$$

$$f\left(\begin{pmatrix} s \\ 1 \\ x \end{pmatrix}\right) \rightarrow \begin{pmatrix} s \\ 1 \\ x \end{pmatrix} * \begin{pmatrix} f \\ 1 \\ x \end{pmatrix}$$

① $f \rightarrow \begin{matrix} f_1 \\ / \quad \backslash \\ x \quad s \\ \quad \quad | \\ \quad \quad 0 \end{matrix}$

② $f_1 \rightarrow y$
 $\swarrow \searrow$
 $0 \quad y$

③

```
graph TD
    f1_1[f1] --- s_1[s]
    f1_1 --- y_1[y]
    s_1 --- x_1[x]
    f1_2[f1] --- x_2[x]
    f1_2 --- star[*]
    star --- s_2[s]
    star --- y_2[y]
    s_2 --- x_2_sub[x]
```

The diagrams show the recursive calls and returns for $\text{fib}(5)$:

- Diagram 1:** Shows the initial call $\text{fib}(5)$ with an upward arrow, and a return path from $\text{fib}(4)$ to $\text{fib}(5)$.
- Diagram 2:** Shows the call $\text{fib}(4)$ with an upward arrow, and a return path from $\text{fib}(3)$ to $\text{fib}(4)$.

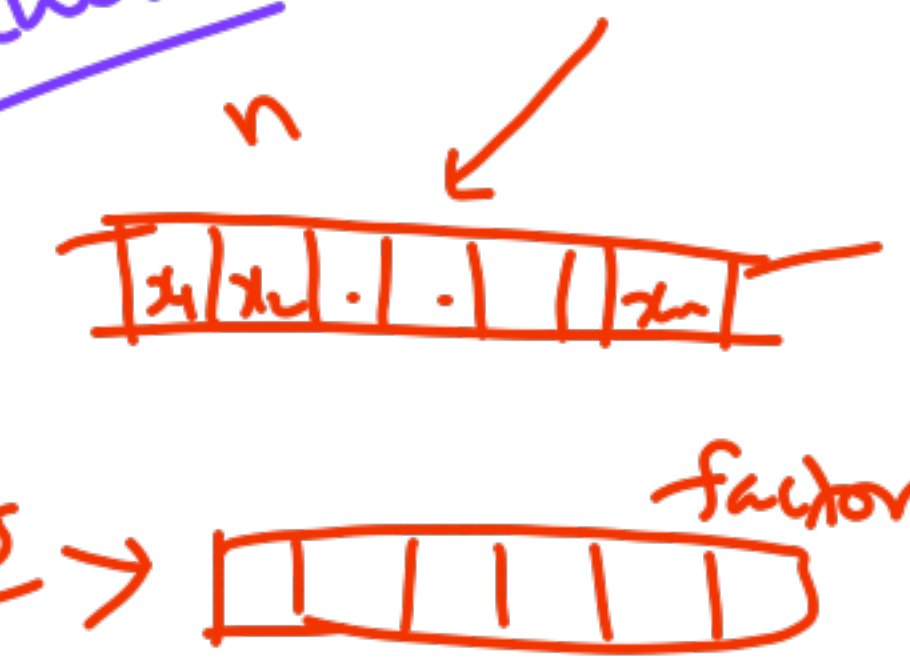
(3)

$$\begin{array}{c} f_{1,3} \\ | \\ s \\ | \\ s \\ | \\ x \end{array} \rightarrow \begin{array}{c} + \\ / \quad \backslash \\ f_{1,6} \quad f_{1,6} \\ | \quad \quad | \\ s \quad \quad x \\ | \\ x \end{array}$$

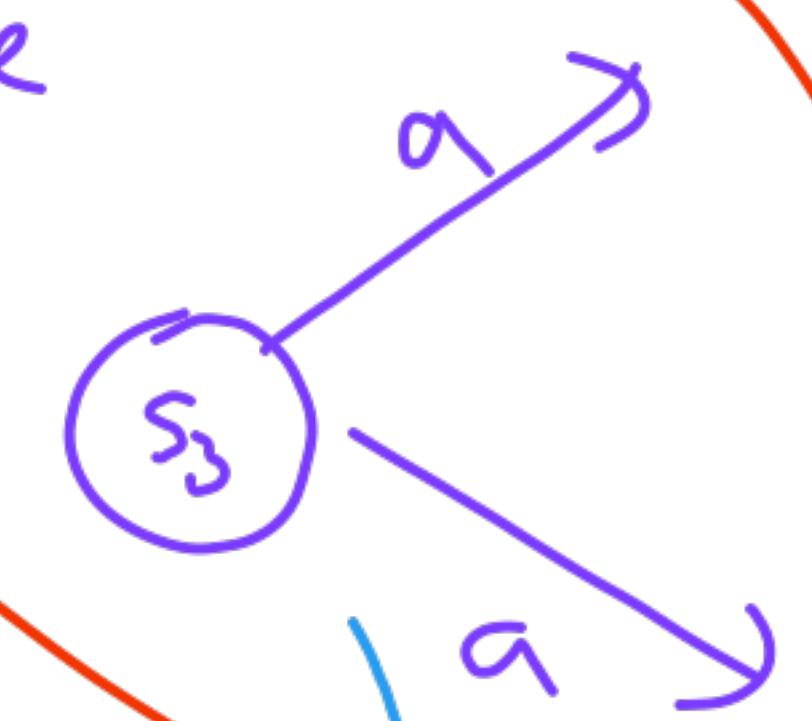
31.10.25

NP-Completeness

Polynomial # steps —
Turing Machine



Non-deterministic — choice



Verify in polynomial time

Verifier "given by Oracle"

"Guess"
one correct choice $\Rightarrow$ accept
Search
don't know
find a solution

- Quiz 3
- Lectures
 - 1. 3/10
 - 2. 5/11
 - 3. 7/11
- (Assignment - 6)
- Final Exam (15/11)

Polynomial Time

Deterministic

DTM

Count # of steps

Input Size

graph

$n_1 \# n_2 \# n_3 \# n_4$
edge 1 edge 2

Knapsack

x unary

numbers

log

binary

Standard
↓ site
3 lectures

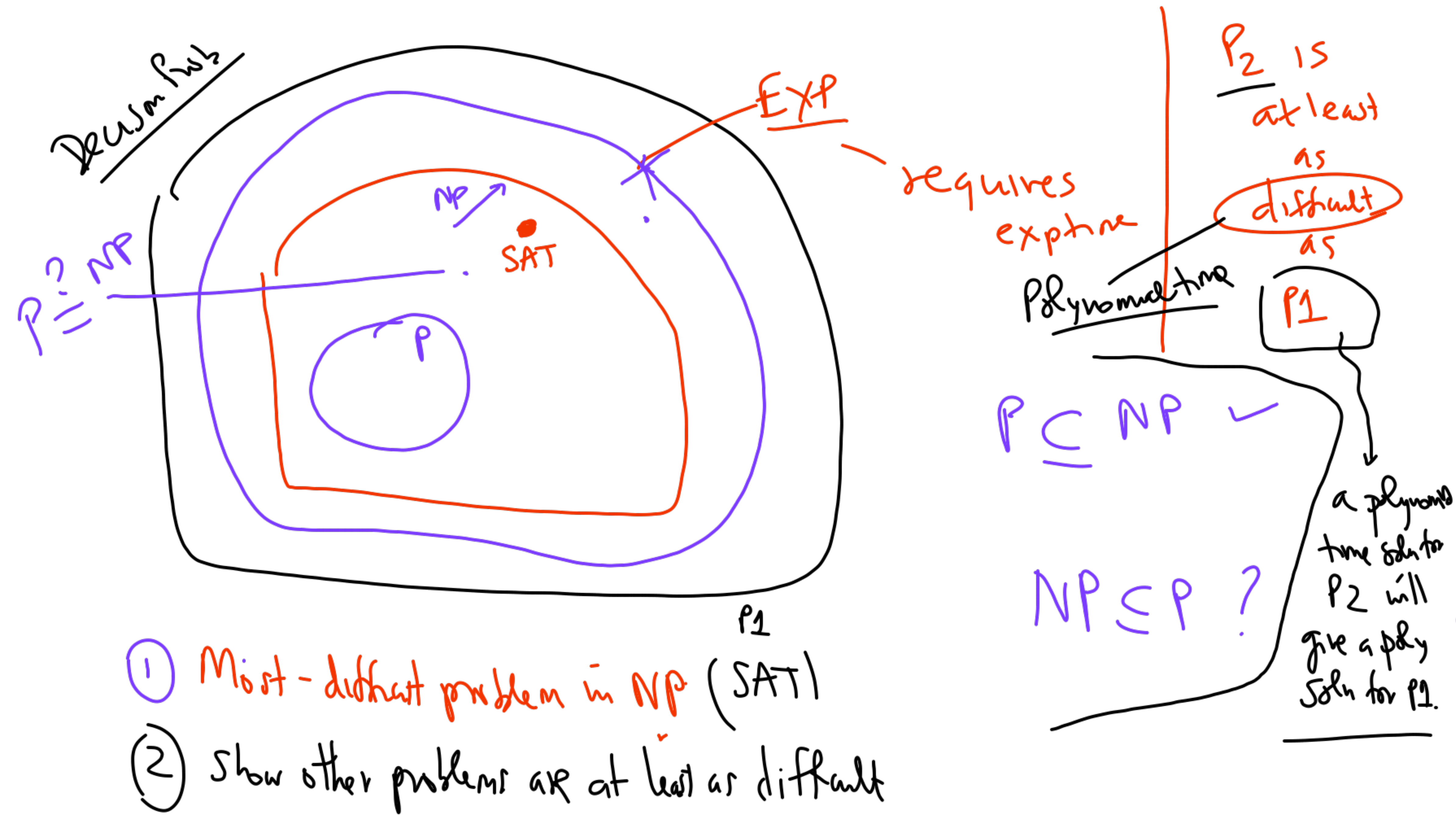
Decision Problems

vs
Computations
or
Optimizations

Polynomial time
algorithms

$ID_0 \rightarrow ID_1 \rightarrow ID_2$

$1 q_0 x_1 x_2 \dots$



Show that 3-SAT is at least as difficult as SAT ✓

Any prop logic formula is a CNF formula

Literal	$P, \neg P$
Clause	$P \vee Q \vee \neg R \vee \neg S \dots$
CNF	$C_1 \wedge C_2 \wedge \dots \wedge C_k$

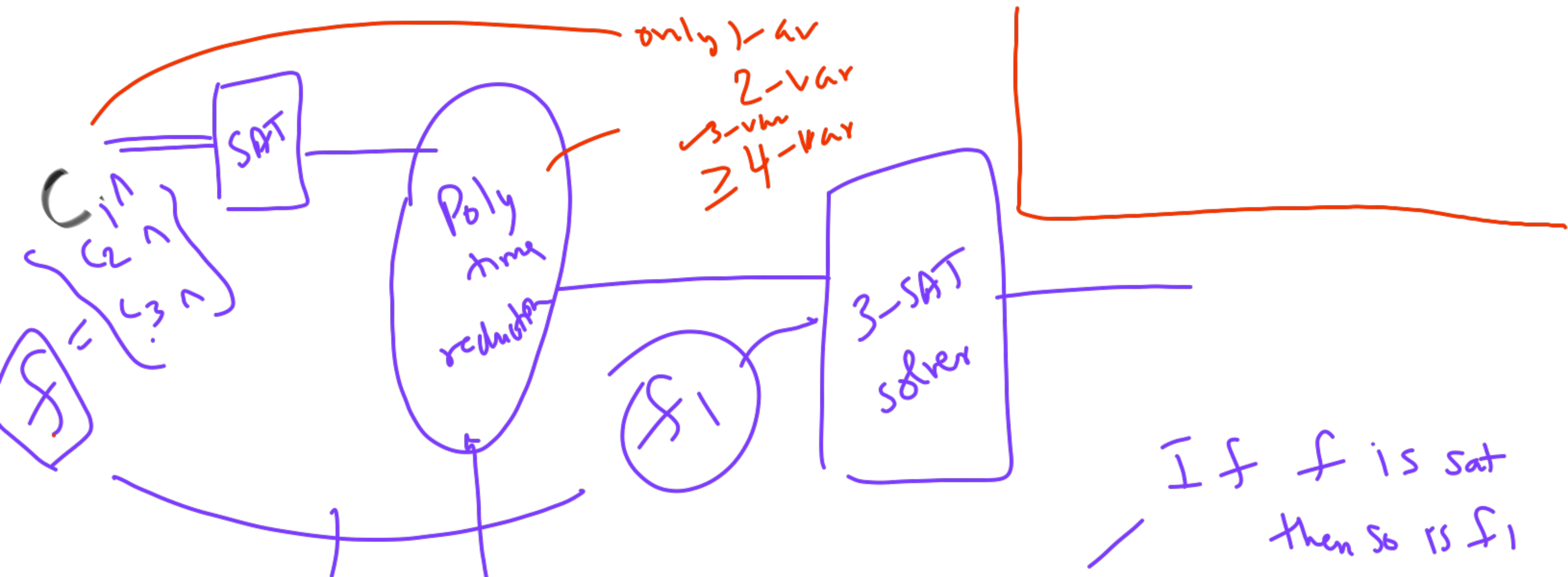
C_1	$x_1 \vee \bar{x}_2 \vee x_4$
C_2	$\bar{x}_3 \vee x_5 \vee x_1 \vee \bar{x}_7 \vee \dots$
$\vdots$	
C_k	

every clause exactly 3-literals

is false

Vars	Truth
x_1	T
x_2	F
x_3	F
$\vdots$	

model



Convert f to f_1
 (SAT) (3-SAT)
 $\rightarrow$ Equi Satisfiable

f
 $a \vee b$

Equivalent f_1 ~~not same~~
 $(a \vee c) \wedge (\neg c \vee b)$ $\left\{ \begin{array}{l} a, c = f \\ b = \text{true} \end{array} \right.$

$\swarrow$ If f_1 is sat then so is f

SAT $\longrightarrow$ 3-SAT

$x = l_1$
 $\bar{y} =$

$\cancel{x_{\text{new vars}}}$

$(l_1 \vee x_1 \vee x_2)$
 $(l_1 \vee \bar{x}_1 \vee x_2)$
 $(l_1 \vee x_1 \vee \bar{x}_2)$
 $(l_1 \vee \bar{x}_1 \vee \bar{x}_2)$

$l_1 \vee l_2 \rightsquigarrow (l_1 \vee l_2 \vee x) \wedge (l_1 \vee l_2 \vee \bar{x})$

new variable

$\rightarrow \underline{l_1 \vee l_2 \vee \dots \vee l_{100}}$

97 new vars
 $x_1 \dots x_{97}$

$(l_1 \vee l_2 \vee x_1)$
 $(\bar{x}_1 \vee l_3 \vee x_2)$
 $(\bar{x}_2 \vee l_4 \vee x_3)$
 $\dots$
 $(\bar{x}_{97} \vee l_{99} \vee l_{100})$

98 clause

Recap

any { NTM
polynomial
verifier. } ✓

Stanford
lecture slides

SAT

H/W
2/11

is the
most
difficult
(NP-complete)

3-SAT ✓

a

Subset-sum

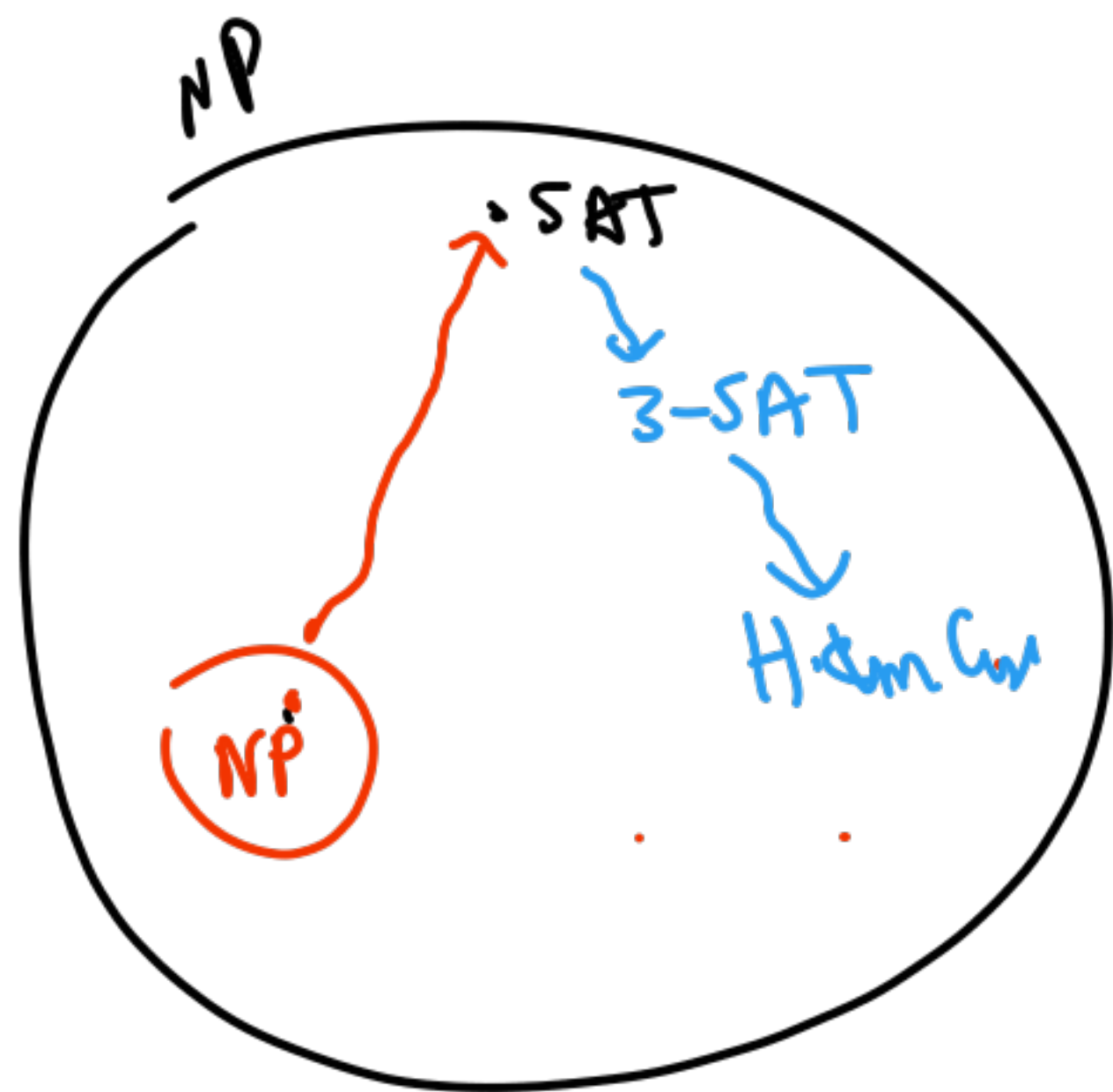
Knapsack

Hamiltonian
Circuit

(TSP)

clique

Vertex-cover



Yes ✓ 3SAT

known NP-complete

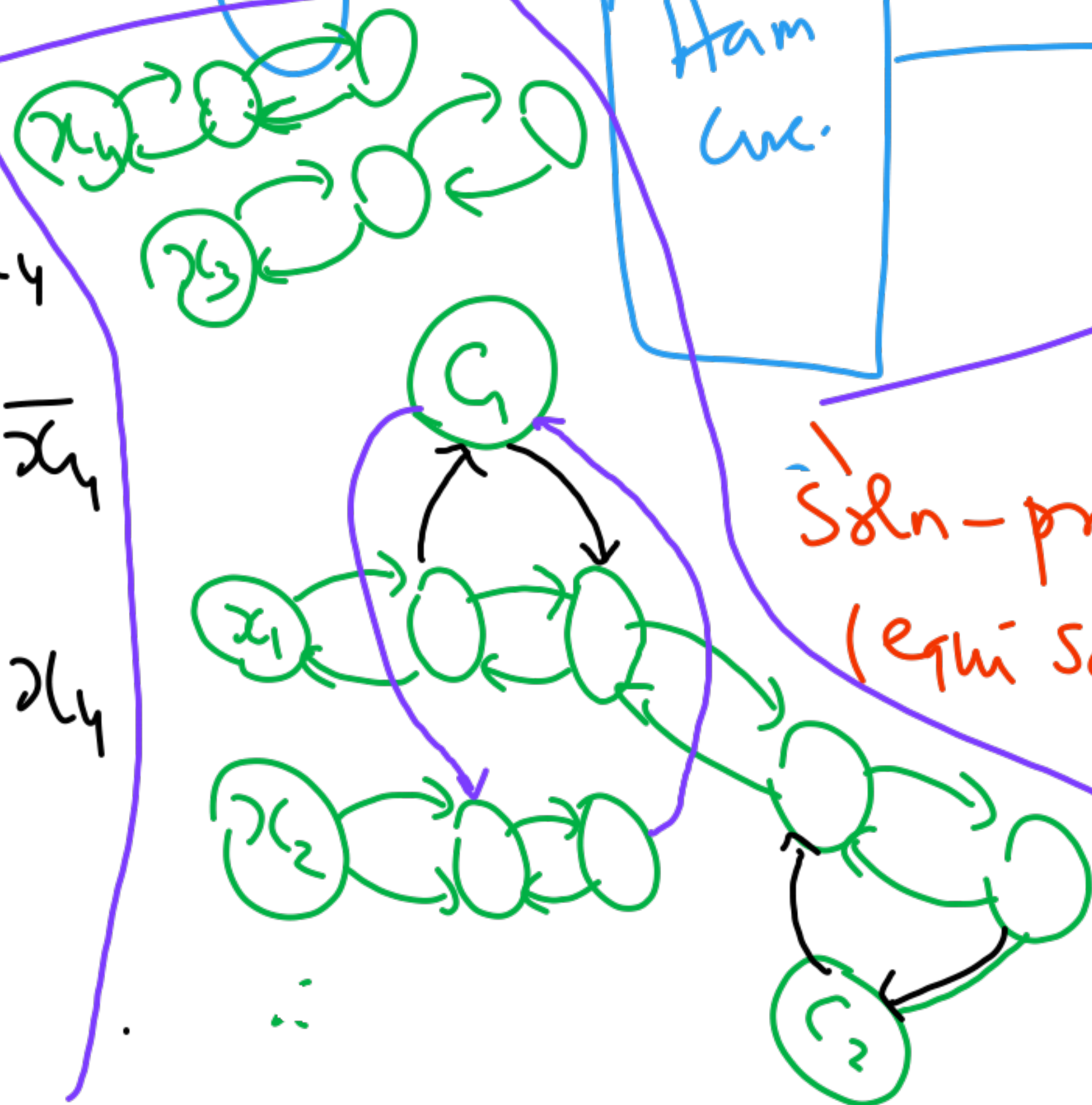
Polynomially reducible

Ham. Circ.

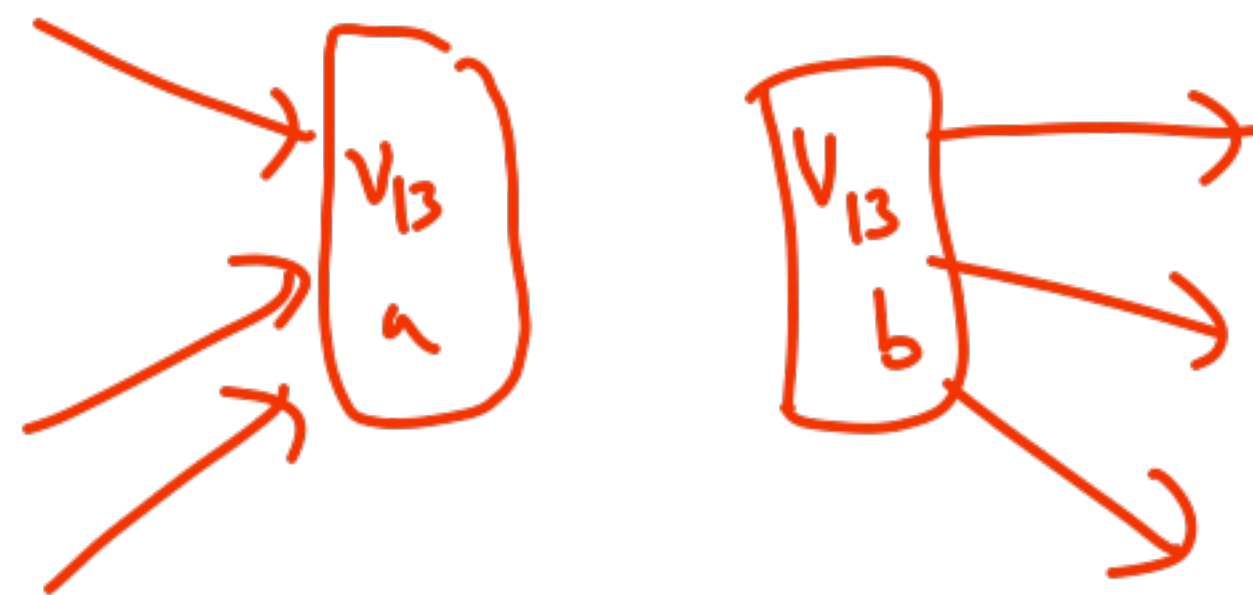
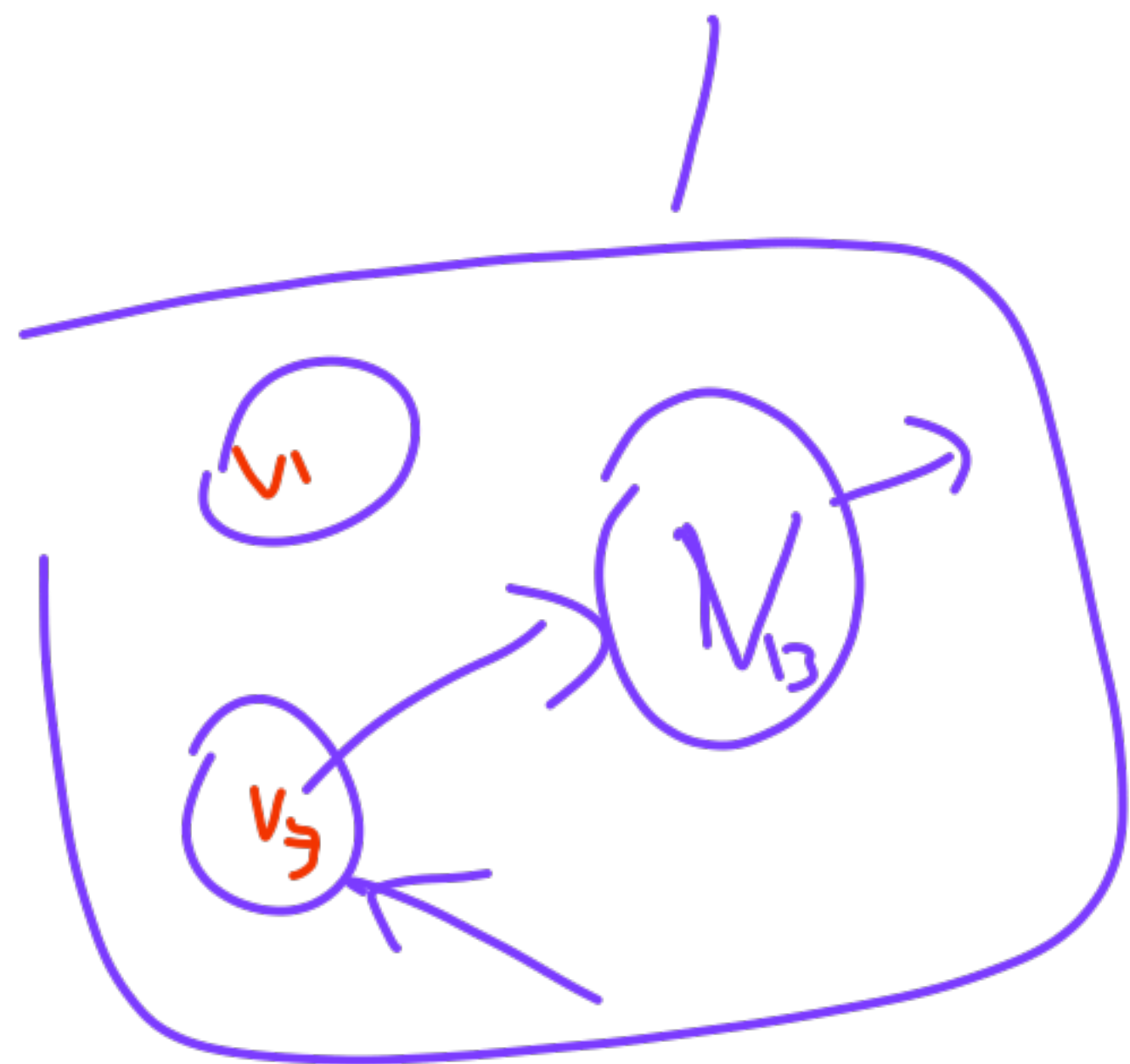
3 pairs of arrows
one for each literal

Soln-preserving.
(equi-satisfiable)

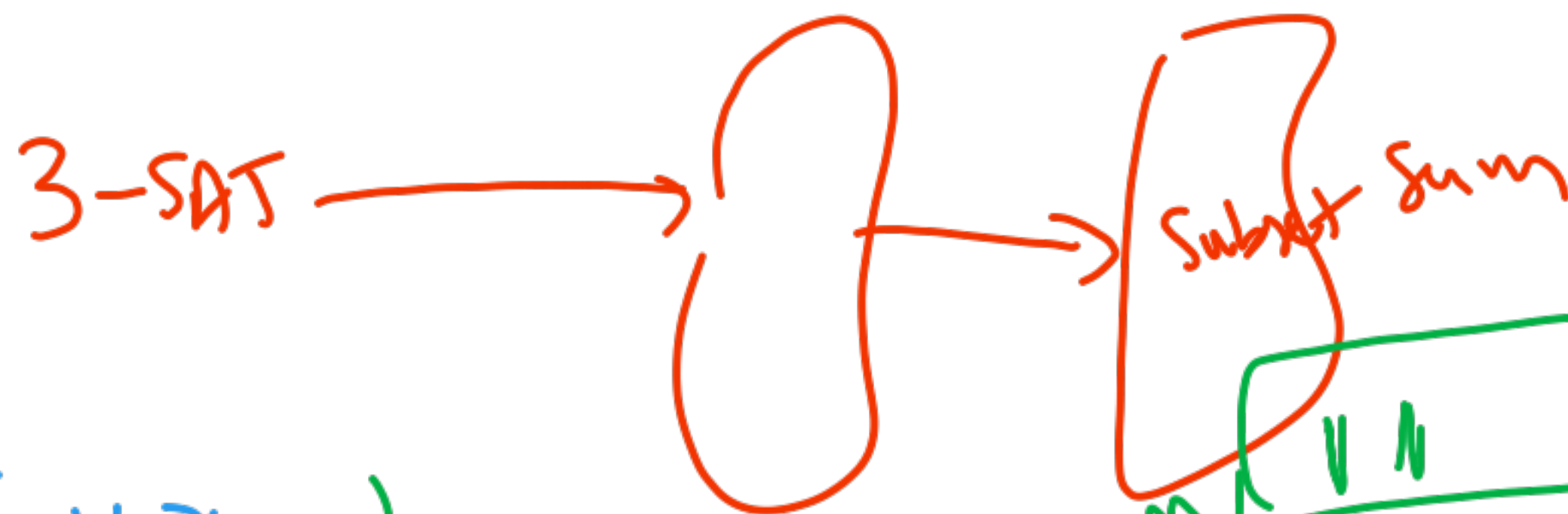
- C1. $\underline{x_1} \vee \bar{x_2} \vee x_4$
- C2. $\bar{x_1} \vee x_3 \vee \bar{x_4}$
- C3. $\bar{x_2} \vee \bar{x_3} \vee x_4$



3SAT $\rightarrow$ HC is NP-complete $\rightarrow$ LIP-path is NP-complete



Subset-Sum



$C_1 \quad x_1 \vee \bar{x}_2 \vee x_3$
 $C_2 \quad x_2 \vee \bar{x}_3 \vee \bar{x}_4$

} S has 12 numbers

$S = \{n_1, \dots, n_k\}, M$

Subset $Q \subseteq S \rightarrow$ sum of numbers in $Q = M$
slack

M

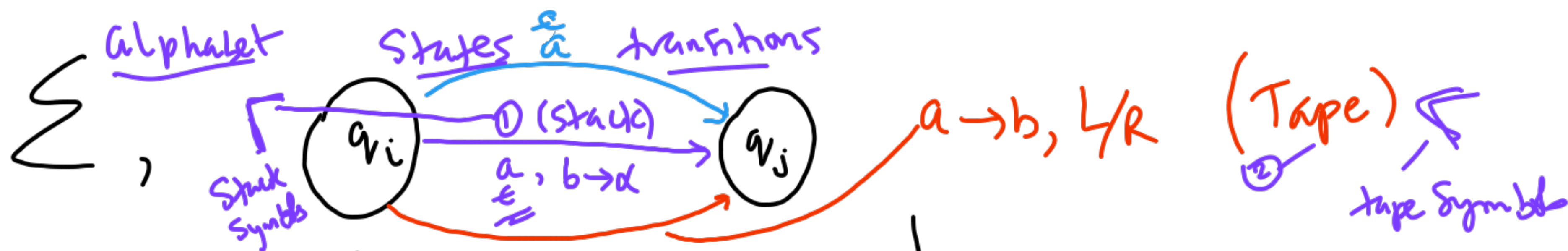
	x_1	x_2	x_3	x_4	C_1	C_2
$x_1 \rightarrow p_1$	1	0	0	0	1	0
$x_1 \rightarrow n_1$	1	0	0	0	0	0
$x_2 \rightarrow p_2$	0	1	0	0	0	1
$x_2 \rightarrow n_2$	0	1	0	0	1	0
$x_3 \rightarrow p_3$	0	0	1	0	0	0
$x_3 \rightarrow n_3$	0	0	1	0	0	0
$x_4 \rightarrow p_4$	0	0	0	1	0	0
$x_4 \rightarrow n_4$	0	0	0	1	0	0
C_1	0	0	0	0	1	0
C_2	0	0	0	0	0	1

M

1 1 1 3 3 ✓

7.11.25

CS 310



FSA	PDA	T M L
Final state q_f (F) ID $\langle q_0; w \rangle$ Accept (Run): 1. Construct $\langle q_0, w \rangle \vdash \dots \vdash \langle q_f, \epsilon \rangle$ 2. Convert NFA to DFA 3. Minimize DFA 4. Not possible move (1-step) DFA $\rightarrow$ Reg Exp	$\langle q_0, w, z_0 \rangle$ Run $\langle q_0, w, z_0 \rangle \vdash \dots \vdash \langle q_f, \epsilon, \alpha \rangle$ (accept by final state) $\langle q_0, w, z_0 \rangle \vdash \dots \vdash \langle q, \epsilon, \epsilon \rangle$ (accept by empty stack)	$\langle q_0, w \rangle$ Run $\langle q_0, w \rangle \vdash \dots \vdash \langle q_f, \alpha \rangle$ halt/crash accept by Halt $\langle q_0, w \rangle \vdash \dots \vdash \langle q, \alpha \rangle$ no move possible
Varia NFA, NFA + ϵ (choice)	DPDA $\rightarrow$ no choice at most 1 choice NPDA $\rightarrow$ no more $\rightarrow$ halt/crash	DTM, NTM 2-tape multi-track; stay S

Grammar $\langle \Sigma, V, S, P \rangle$ rules $l \rightarrow r$

FSA = Reg

PDA $\nwarrow$ Context free
 PDA $\swarrow$

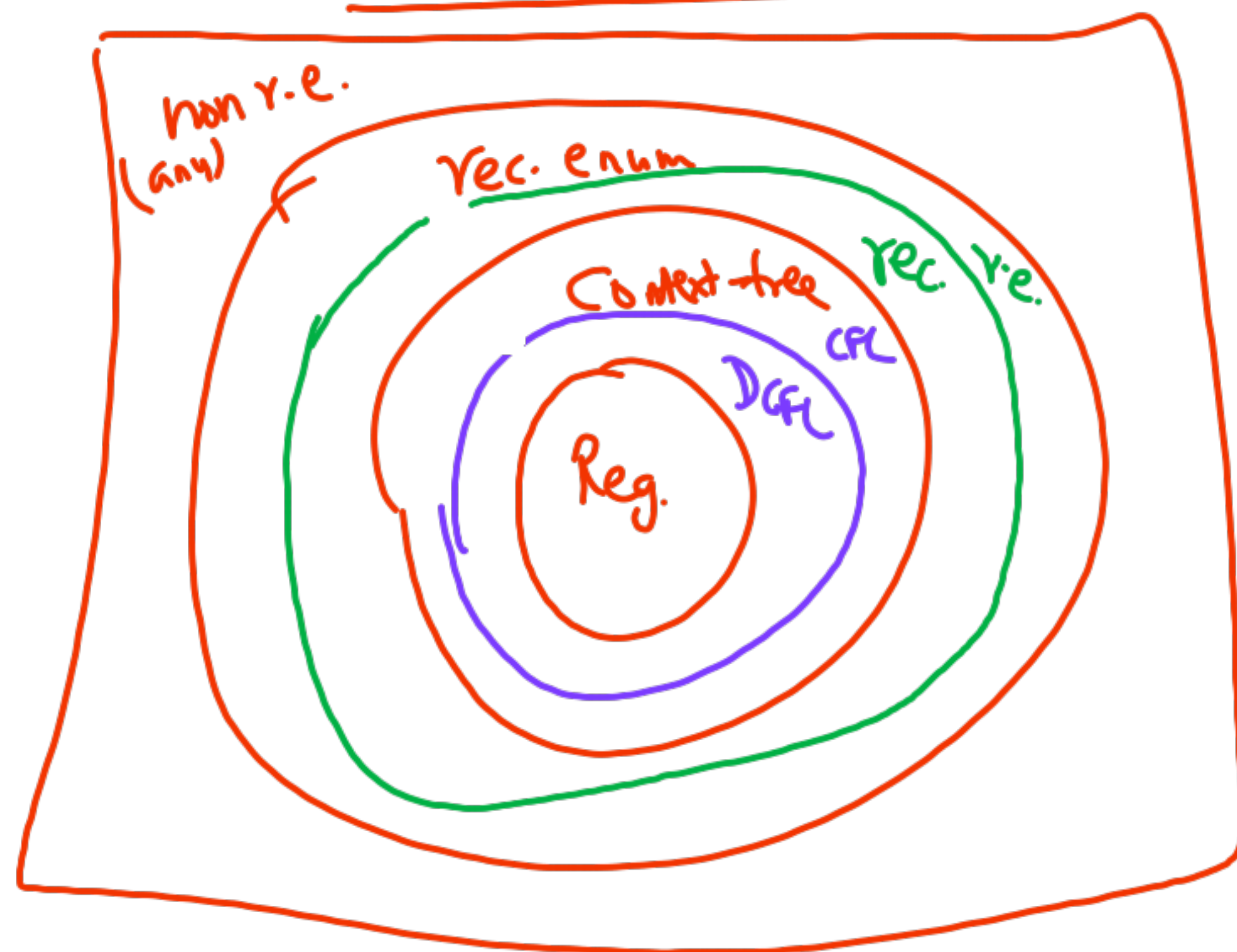
TM Type-0
 r.e.

$l \in V$ (simple var)	left-linear $l \rightarrow XaTc$ right-linear $l \rightarrow abcY$
$l \in V$ (simple var)	any string $(\Sigma \cup V)^*$
any	any

CSL $\nearrow$

reg-exp
 $(0+1)^* 10^*(\dots)$

CHOMSKY-Hierarchy



Exam.

Counting

Enumeration

λ -calc

Equational Prog.

Recursive Function Theory

Topics

Closure Properties

$L_1 \cup L_2, L_1 \cap L_2$
 $\vdots$
 $L_1 - L_2$

Decision Problems

Is $w \in L$?

Normal Forms

Chomsky /
Greibach

not r.e. —
Undecidability

NP-completeness.

Post Cor Problem

Time Complexity
of

Decision Problems

Recursive

TM always answers Yes/No

$P \stackrel{?}{=} \textcircled{1} NP$

Verify guessed soln
in polynomial # of steps

2nd tape

$w \quad |w| = n$

$\frac{\text{|||||}}{\leftarrow n \rightarrow}$

DTM

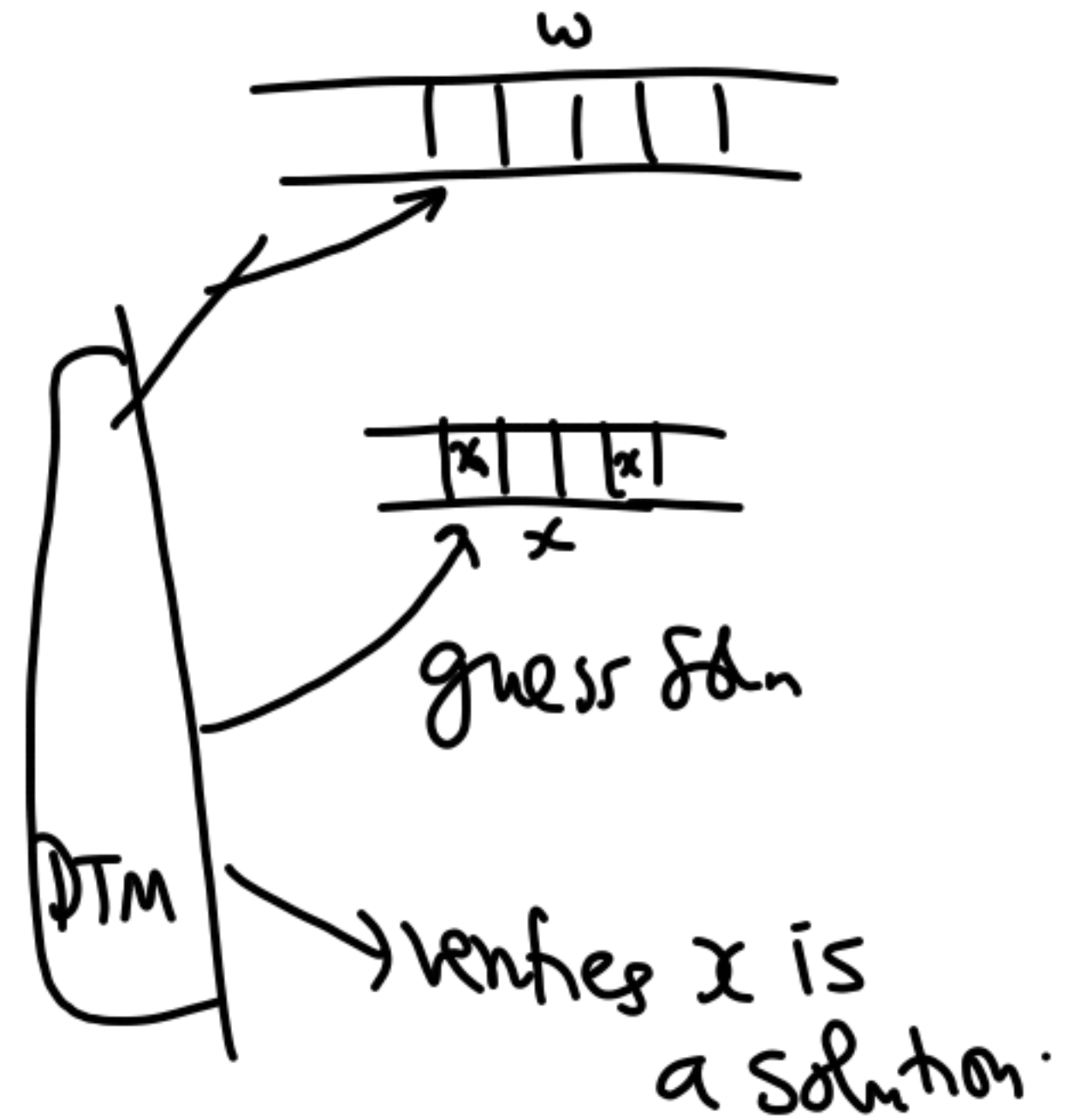
in polynomial number of steps

$\langle q_0, w \rangle \vdash \dots \vdash \text{Yes}$

$\vdash \vdash \vdash$ or No

$n^3 \quad n^8$

$\underline{7n^3 + 9n^2 + \dots}$



$P \stackrel{?}{=} NP$ NP-complete

SAT

— most difficult problem in NP.

Polynomial time reduction from any other problem

"Stanford" Lecture 2

Polynomial-reduction

$SAT \leq_p 3-SAT$ — Vertex-Cover

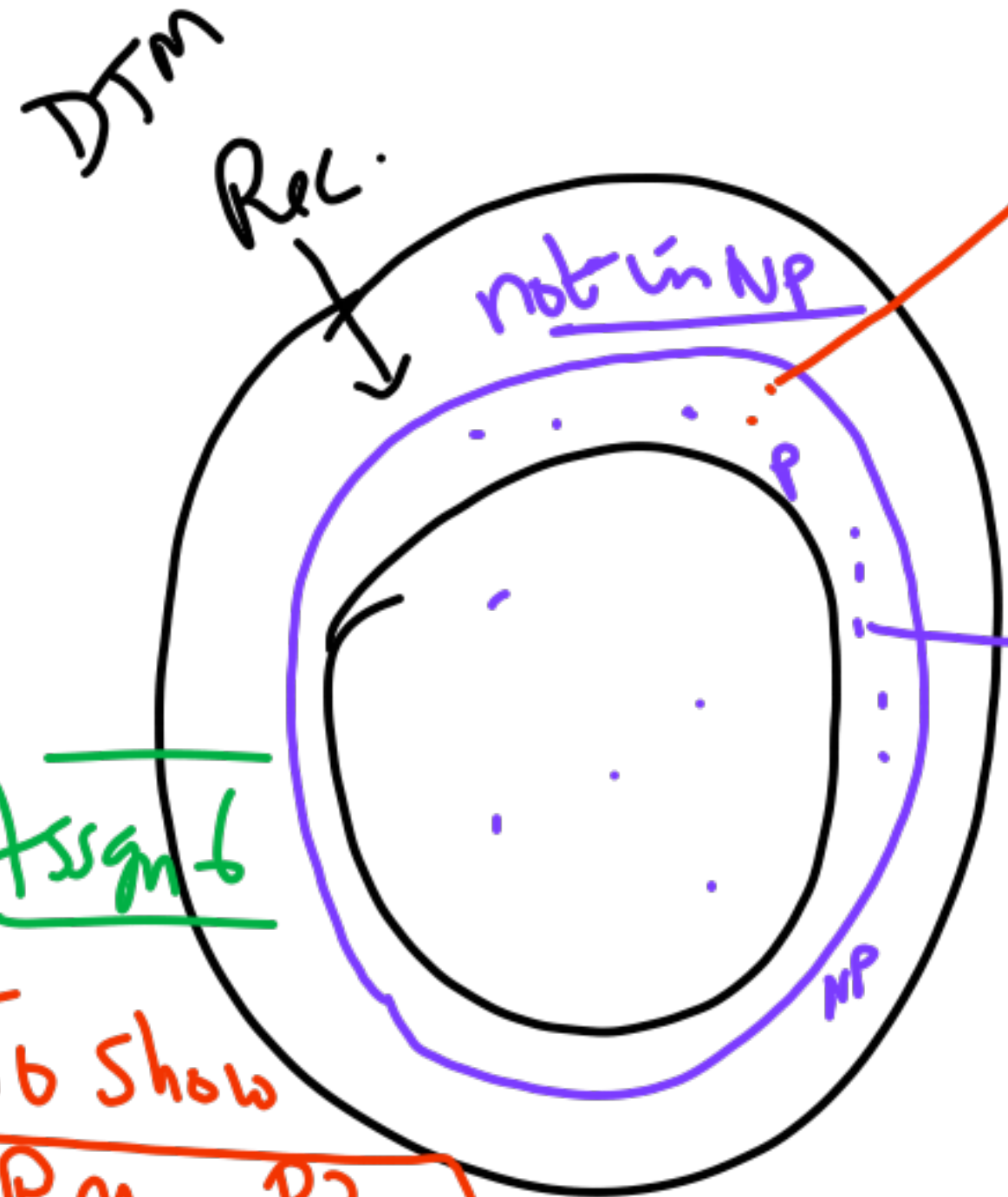
Knapack

Solution preserving

Known to be in NP
but not known to be in P

$P \subseteq NP$

Strict inclusion
reduces in poly-time to P3.



Assign 6

To show
Problem P3
is NP-complete

① Show $P3 \in NP$

② Show known
NP-complete problem
reduces in poly-time to P3.

